

51单片机应用从零开始

展现在大家面前的是一本中学生都读得懂的单片机基础和应用教程。这本教程凝结了国内几所重点大学里站在科研、教学第一线教师们的心理血，也得到了英国剑桥大学、牛津大学、帝国理工、伦敦大学、加尔夫大学等世界著名大学多位博士生导师的指导意见。经过多位学者的精心裁剪，本书的脉络、线索、内容才真正符合读者学习单片机的需要。

在学习过程中如果遇到什么问题，可以访问电路飞翔网，从中能找到大量翔实的帮助，也可以下载到本课件，网址为：

<http://www.circuitfly.com>

欢迎读者给笔者发电子邮件

EEdesign@163.com

来共同探讨问题和参与电路飞翔网的建设。

本书主要作者



杨欣，男，满族，1982年11月生，英国加的夫大学博士生，本书作者之一。曾获英国政府海外留学生奖学金(ORS)、加的夫大学工程奖学金、伦敦大学工程奖学金、Drapers学术成就奖学金。2005年毕业于北京交通大学生物医学工程系，随后取得英国伦敦大学玛丽女皇学院电子物理医学硕士学位。原北京交通大学国家工科物理教学基地“创新实践实验室”总负责人，北京交通大学《创新实践报》主编。参与过北京交通大学国家工科物理教学基地“物理演示与探索实验室导学系统”项目。编著有《电子设计从零开始》、《电路设计与仿真——基于Multisim 8与Protel 2004》、《51单片机应用从零开始》等图书。发表关于无创血管硬化程度检测、3维超声软组织成像等方面的论文数篇。涉足电子电路设计、计算机程序设计、多普勒超声成像、生物医学仪器、运动软件组织损伤诊断、医学图像处理等多方面的研究。



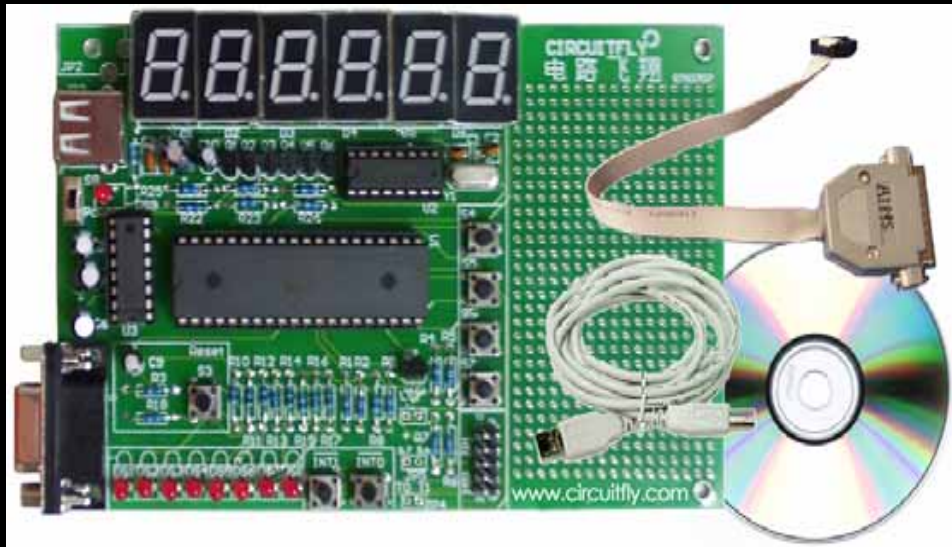
王玉凤，女，满族，1944年11月生，教授，本书作者之一。1967年毕业于南开大学物理系，1977年开始调入北京交通大学工作。30年以来，一直站在基础课教学第一线，2006年被评为国家级教学名师。长期担任物理系副主任，主管教学工作，为北京交通大学应用物理专业的建设和光信息科学与技术专业的建立作了大量开创性的工作。先后主持建设了“无损检测实验室”和国家工科物理教学基地“物理演示与探索实验室”。主编、参编了8本教材、专著3本；公开发表教学改革文章十余篇。主持了国家级和铁道部级教学改革子课题研究和国家工科物理教学基地子项目建设。曾长期担任教育部高等学校工科物理课教学指导委员会委员，曾获国家教学成果特等奖一项、二等奖一项、北京市教学成果一等奖二项、二等奖一项；获全国优秀教师、北京市优秀教师称号；获北京市三八红旗奖章、首都劳动奖章、北京市经济技术创新工程优秀成果奖；享受政府特殊津贴；现为校重点岗特聘教授、示范课教师、女教授联谊会会长。曾当选为北京市海淀区第十一届、第十二届、第十三届人大代表。



刘湘黔，男，汉族，1971年5月生，副教授，本书作者之一。2000年毕业于清华大学自动化系，同年进入北京交通大学工作。现为北京交通大学计算机与信息职业技术学院副教授，为本科生和研究生开设《电路分析》、《模拟电子与数字电子技术》和《计算机控制等课程》。主要从事网络控制、鲁棒控制等方面的研究。曾主持横向课题两项，参与863重大项目等课题多项。近年来主要从事电子技术基础、信号与信息处理等方面的教学与科研。

虽然本书尽力用浅显活泼的文字讲述51单片机的各种功能，但对于与实践结合紧密的工程学科，如果没有亲自动手练习，就如同纸上谈兵，难以掌握其精髓。

为加速广大读者入门，<http://www.circuitfly.com> 提供相应的低价位学习套件——STK-0707型单片机学习板，该板紧密结合书中的实例，能帮助读者快速、深刻地掌握单片机的基础知识和开发技能，迅速提高入门者应对实际问题的能力。



欢迎广大读者登陆网站
获取详细信息。

第1章 身边的单片机

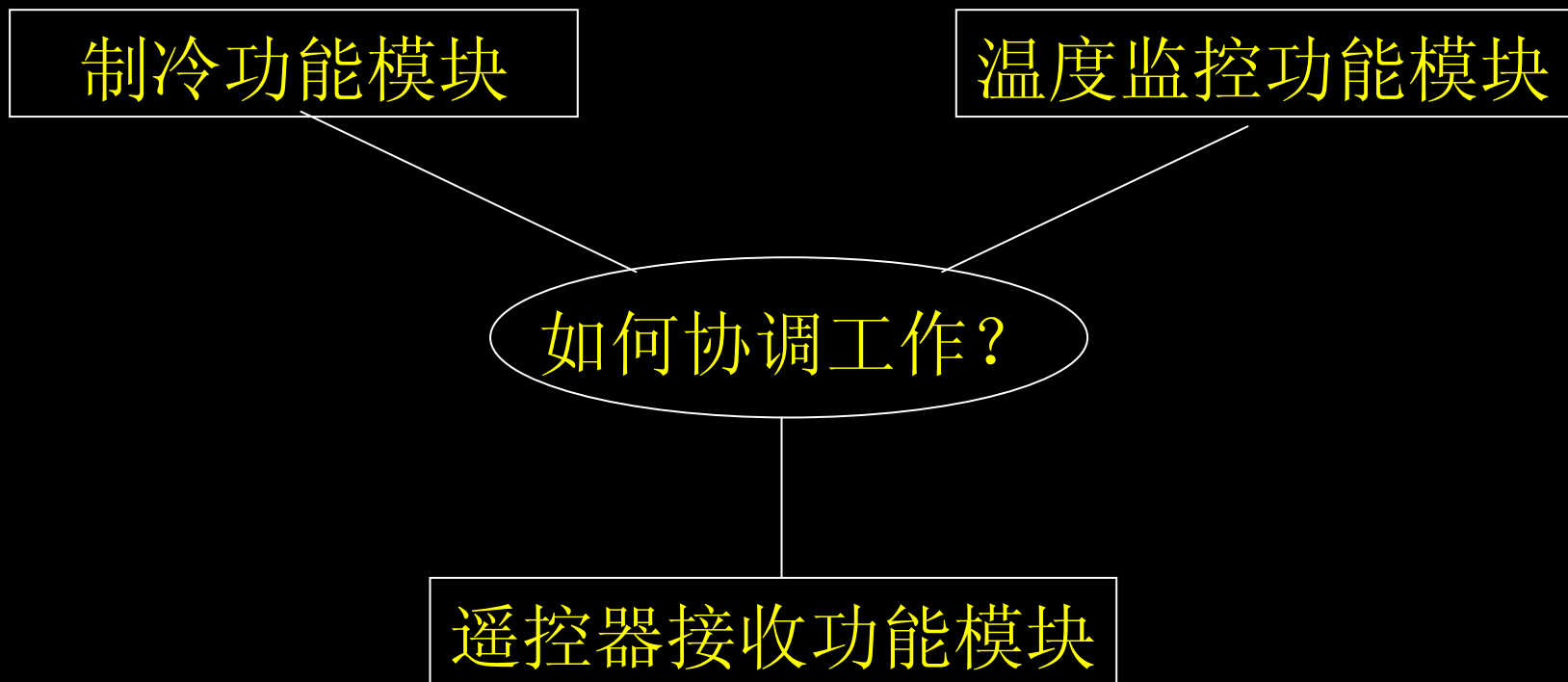
欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 1.1 单片机在哪里
- 1.2 单片机是什么样子的
- 1.3 单片机如何控制其他器件
- 1.4 如何使用单片机
- 1.5 有哪些单片机
- 1.6 实例点拨——电子万年历

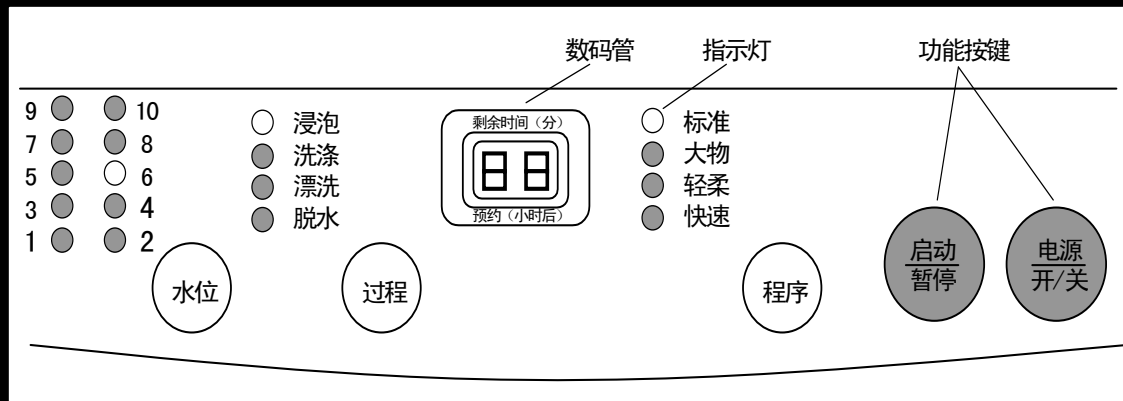
1.1 单片机在哪里

例1: 空调机



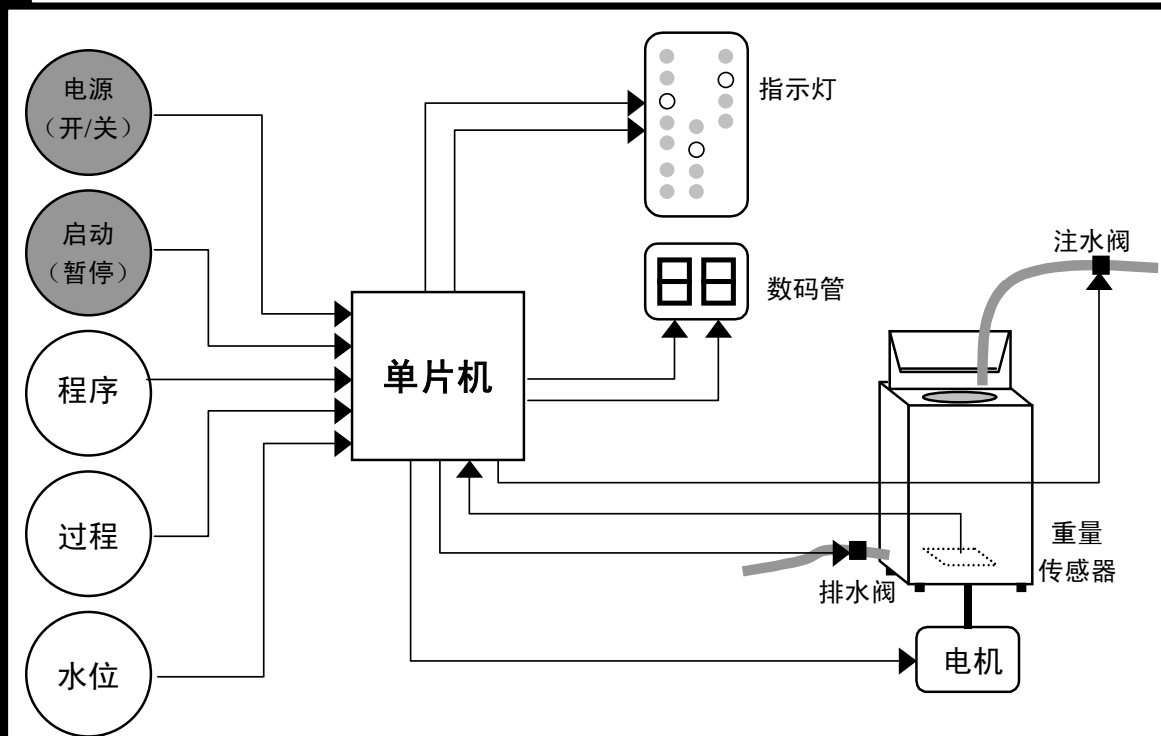
1.1 单片机在哪里

操作
面板



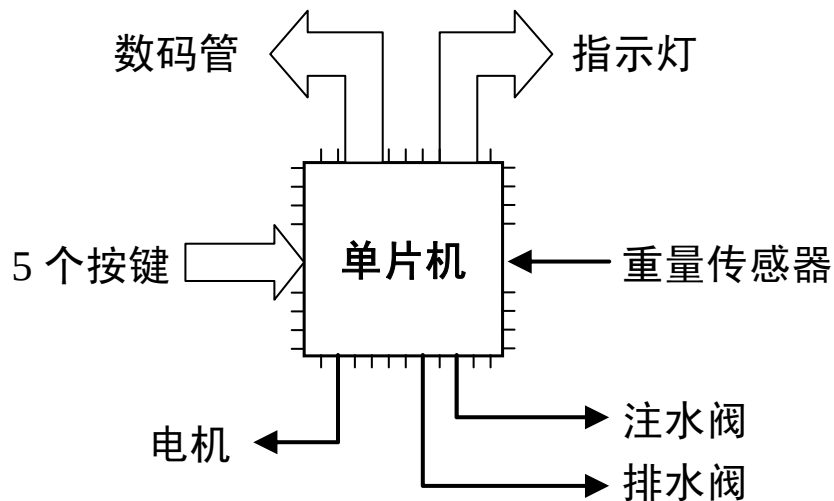
例2:
全自
动洗
衣机

洗衣
机简
化系
统示
意图

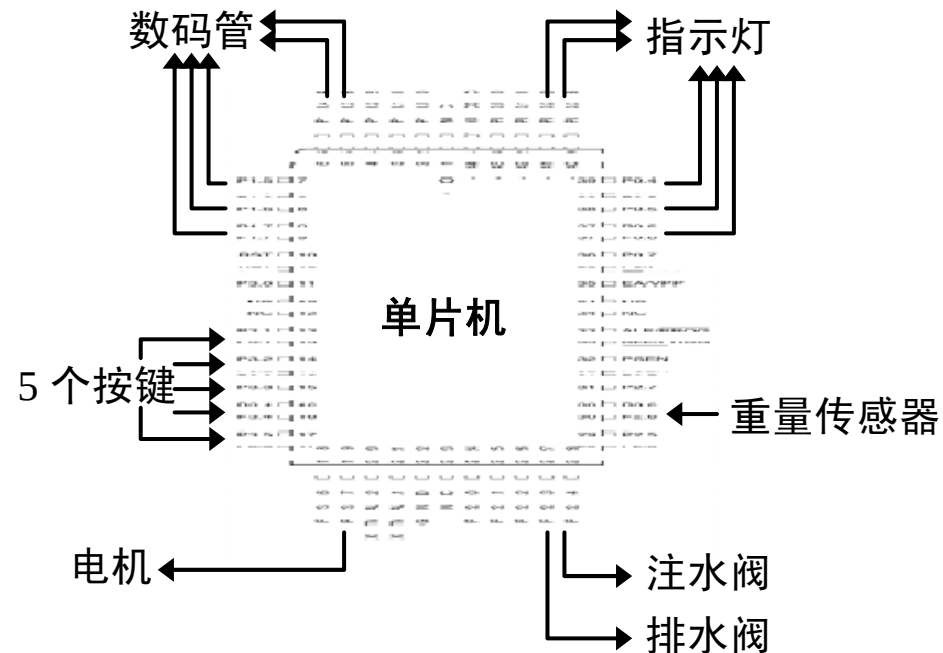


1.2 单片机是什么样子的

全自动洗衣机的单片机和外围控制部件

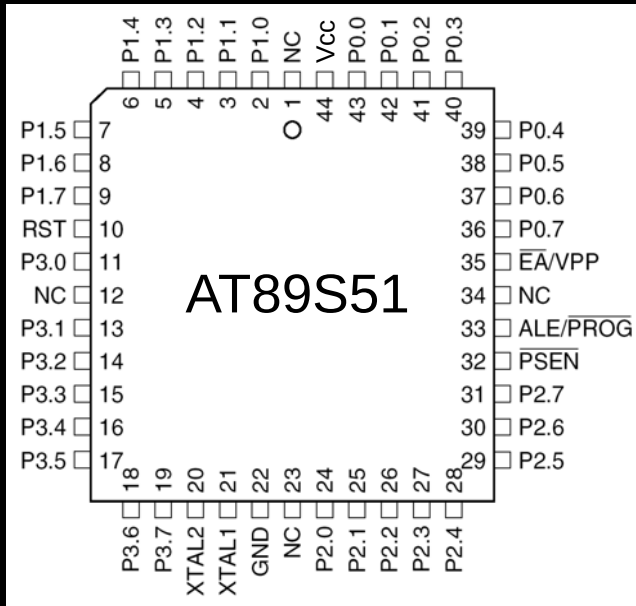


单片机模型

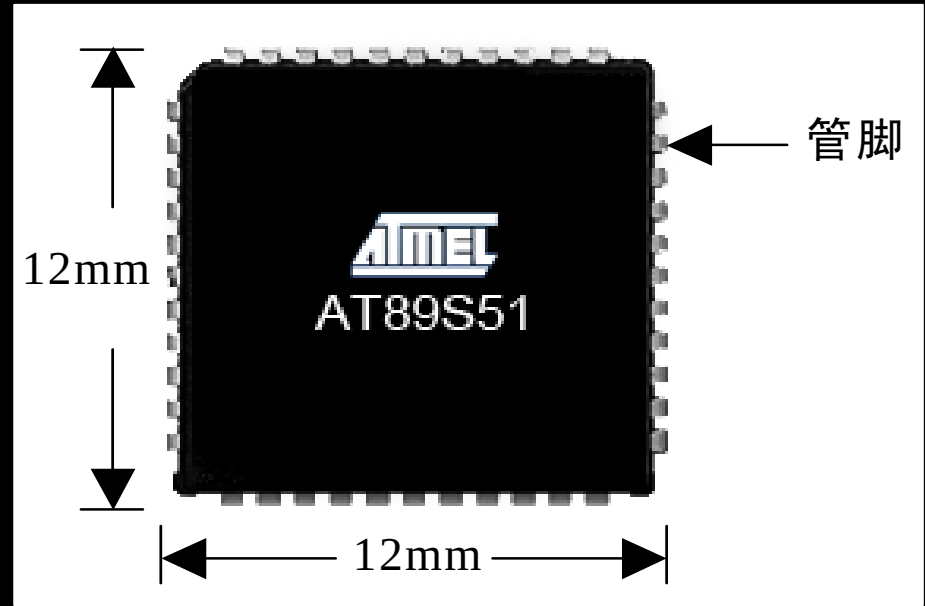


单片机电路符号

1.2 单片机是什么样子的



电路符号



实际单片机器件

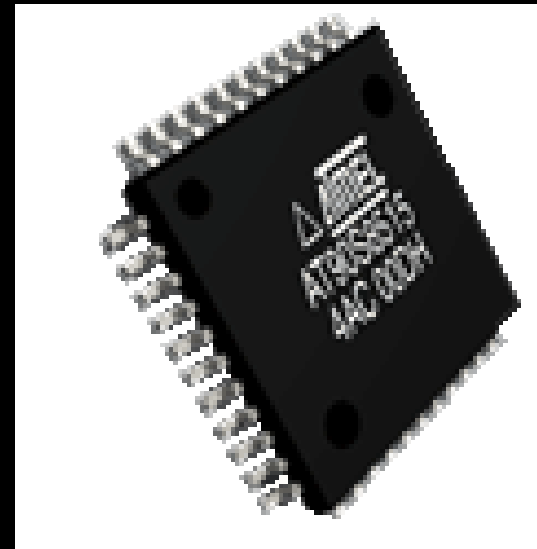
1.2 单片机是什么样子的

方形的单片机有两种常见的形式



PLCC封装

管脚向内侧折起



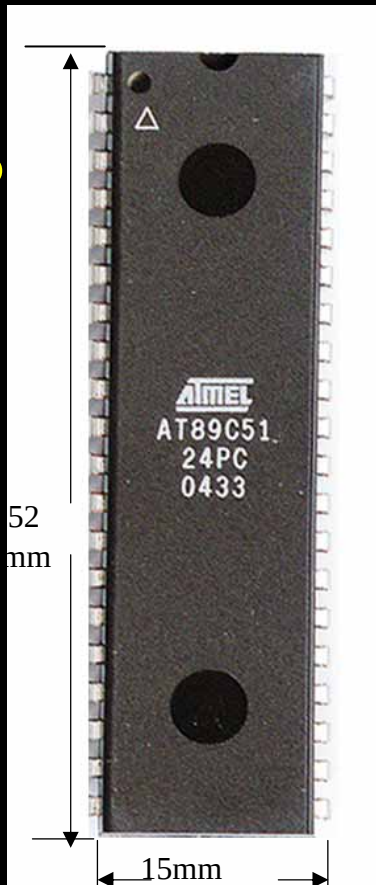
TQFP封装

管脚向外侧伸展

1.2 单片机是什么样子的

双列直插式DIP封装

▷ DIP封装的单片机外形



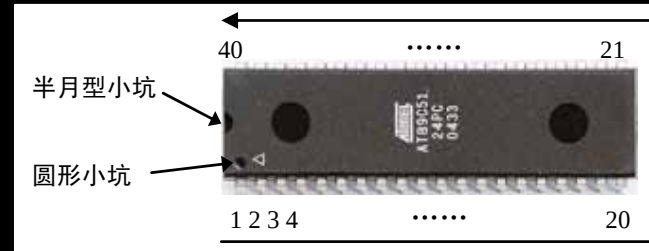
▷ 电路符号1

P1.0	1	40	Vcc
P1.1	2	39	P0.0
P1.2	3	38	P0.1
P1.3	4	37	P0.2
P1.4	5	36	P0.3
P1.5	6	35	P0.4
P1.6	7	34	P0.5
P1.7	8	33	P0.6
RST	9	32	P0.7
P3.0	10	31	$\overline{EA}/VPP$
P3.1	11	30	ALE/ $\overline{PROG}$
P3.2	12	29	$\overline{PSEN}$
P3.3	13	28	P2.7
P3.4	14	27	P2.6
P3.5	15	26	P2.5
P3.6	16	25	P2.4
P3.7	17	24	P2.3
XTAL2	18	23	P2.2
XTAL1	19	22	P2.1
GND	20	21	P2.0

▷ 电路符号2 (省略供电管脚)

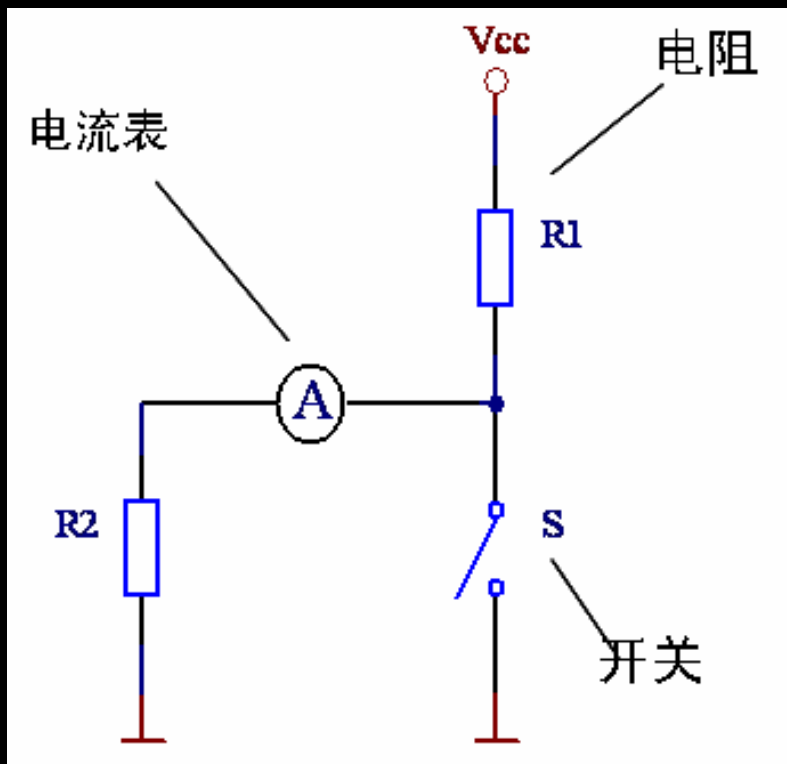
31	$\overline{EA}/VPP$	P0.0	39
19	XTAL1	P0.1	38
18	XTAL2	P0.2	37
9	RST	P0.3	36
		P0.4	35
		P0.5	34
		P0.6	33
		P0.7	32
10	P3.0/RXD	P2.0	21
11	P3.1/TXD	P2.1	22
12	P3.2/INT0	P2.2	23
13	P3.3/INT1	P2.3	24
14	P3.4/T0	P2.4	25
15	P3.5/T1	P2.5	26
		P2.6	27
1	P1.0	P2.7	28
2	P1.1		
3	P1.2		
4	P1.3	$\overline{PSEN}$	29
5	P1.4	ALE/ $\overline{PROG}$	30
6	P1.5		
7	P1.6	P3.6/ $\overline{WR}$	16
8	P1.7	P3.7/ $\overline{RD}$	17

▷ DIP封装单片机的管脚序号

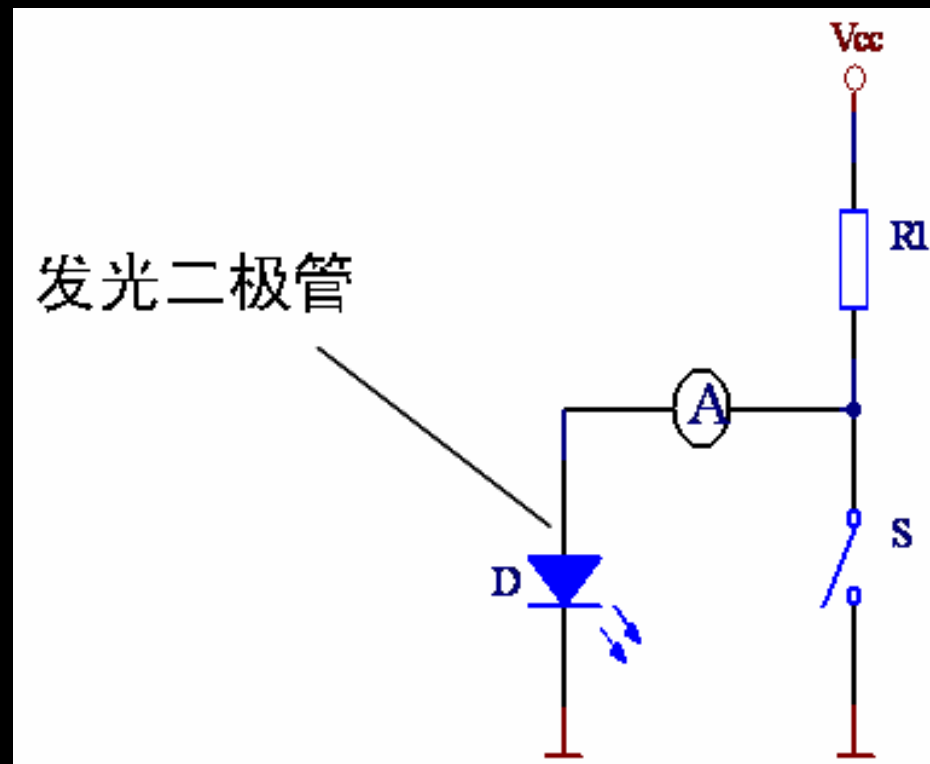


1.3 单片机如何控制其他器件

- 一个简单实验 ---- 点亮小灯



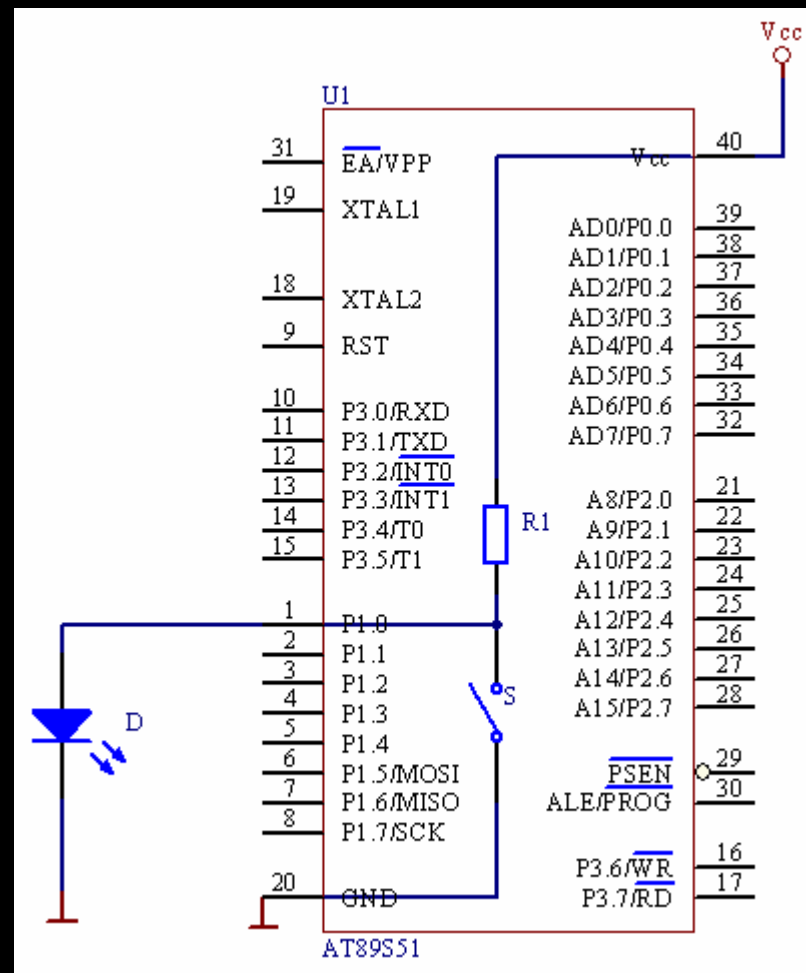
类比电路



端口控制模型

1.3 单片机如何控制其他器件

- 中文指令：
向P1.0管脚输出Vcc
- 单片机能接受的语句：
SETB P1.0

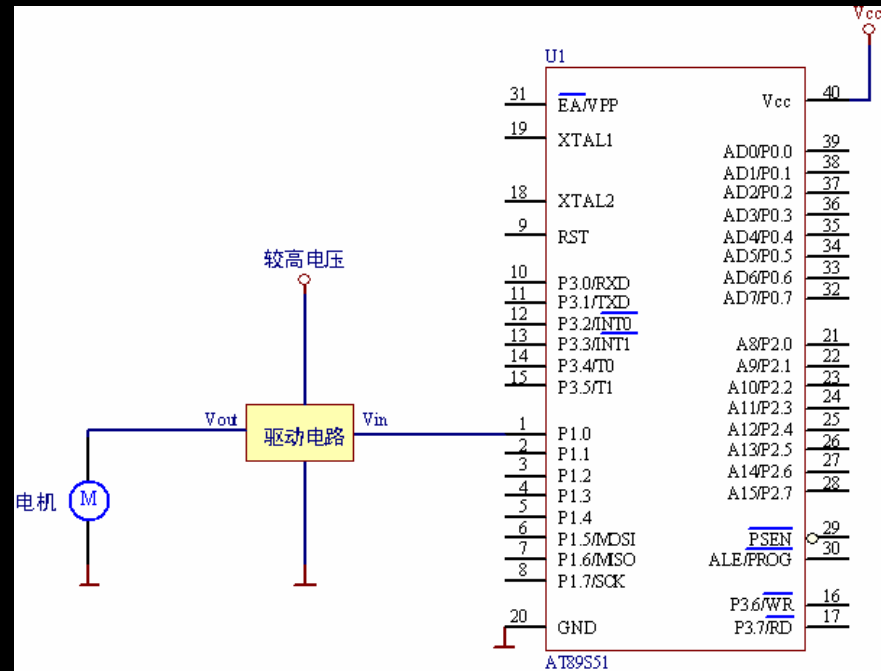


单片机的端口控制

1.3 单片机如何控制其他器件

- 单片机控制洗衣机中的电机

电机属于大功率器件，所以需要用到驱动电路，把单片机管脚输出的低压Vcc转变成能使电机工作的较高电压。

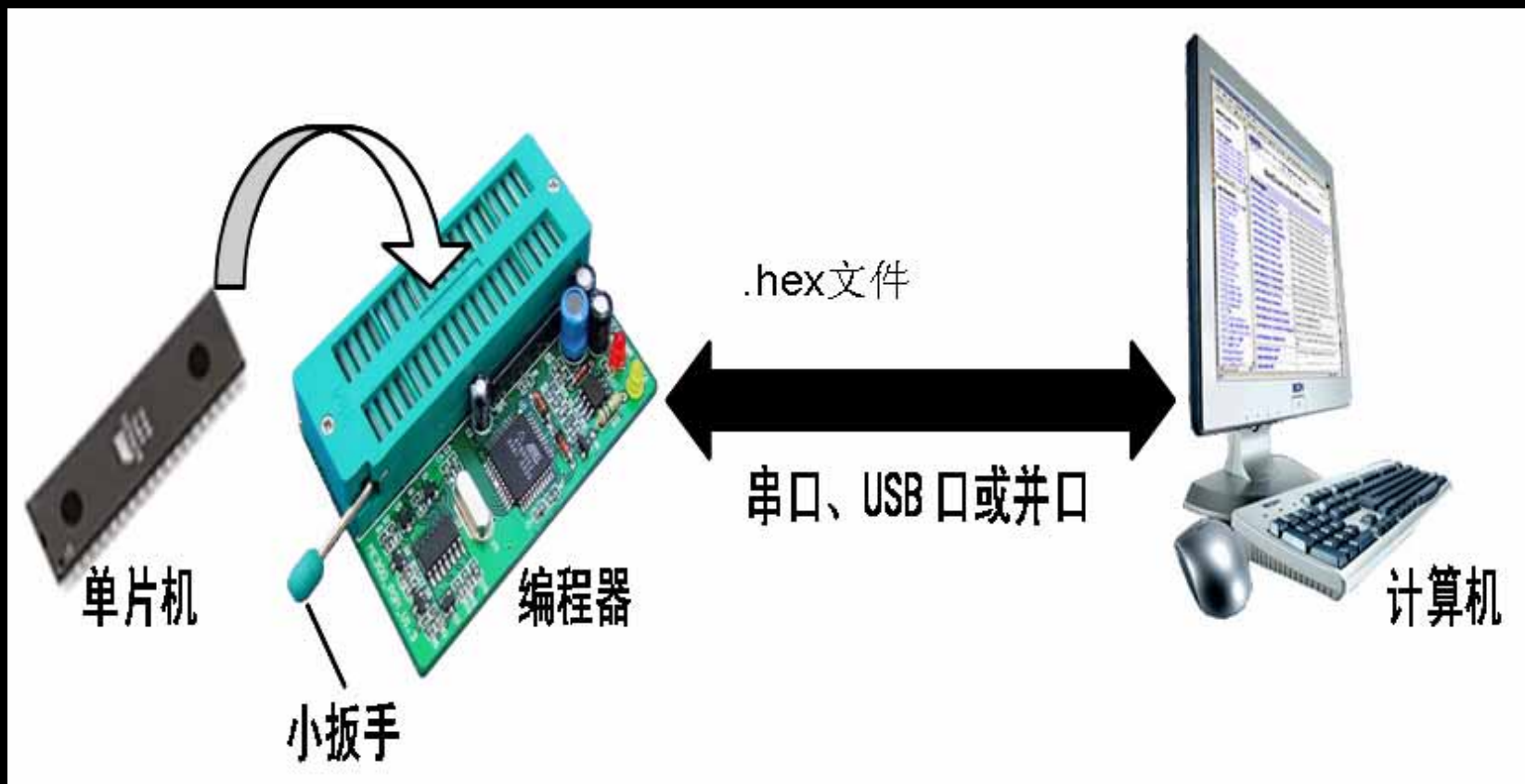


◁ 单片机控制电机

向P1.0管脚输出高电平 → SETB P1.0
持续一段时间 → CALL DELAY
向P1.0管脚输出低电平 → CLR P1.0

1.4 如何使用单片机

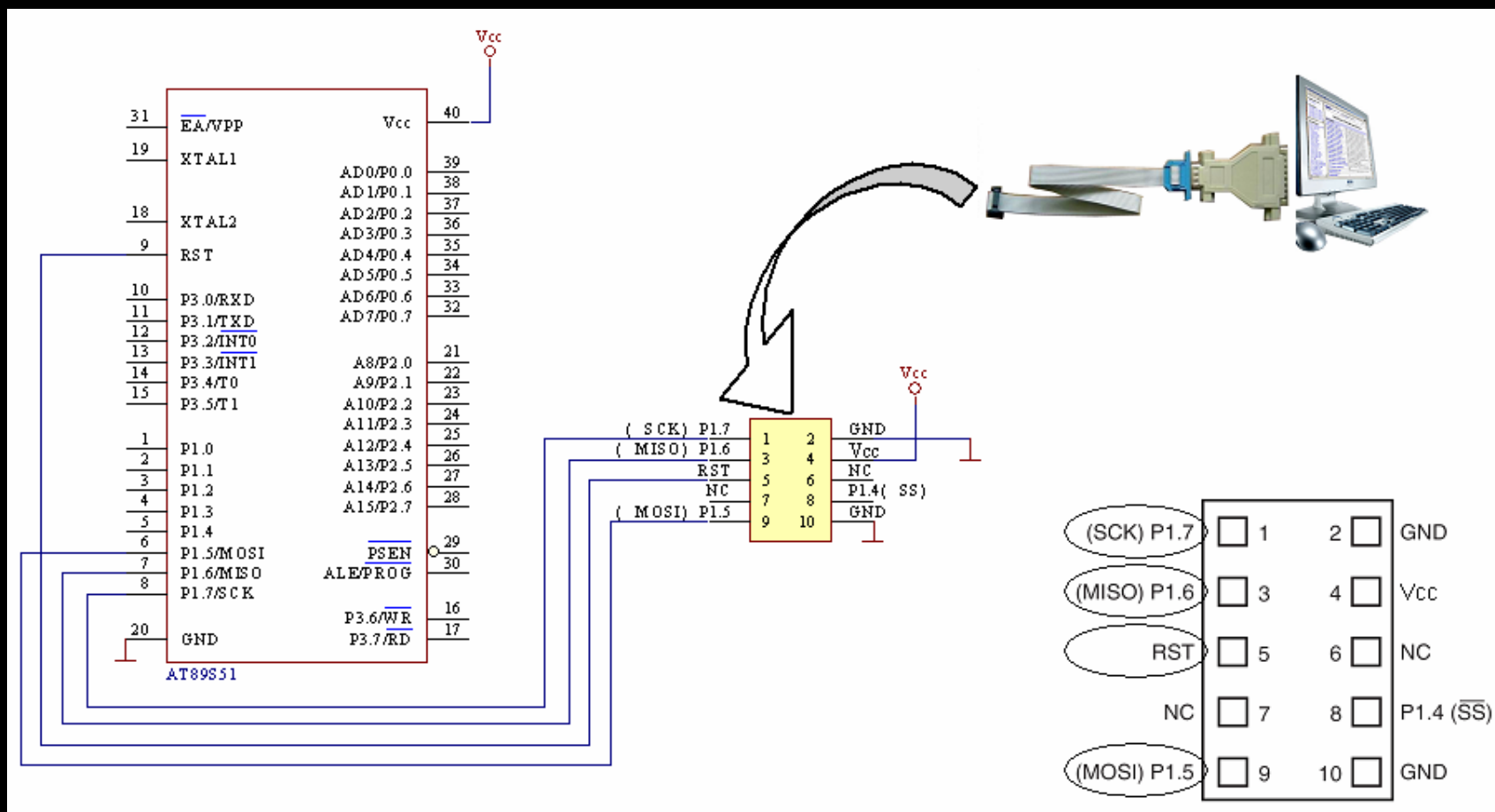
1.4.1 利用编程器烧写单片机



编程器的工作过程

1.4 如何使用单片机

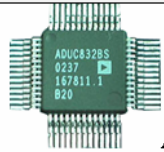
1.4.2 利用下载线烧写单片机



1.5 有哪些单片机

不同厂商不同型号的单片机产品有不同的特点：
存储器的容量、管脚数、内部结构、工作电压、运算速度、指令等.....

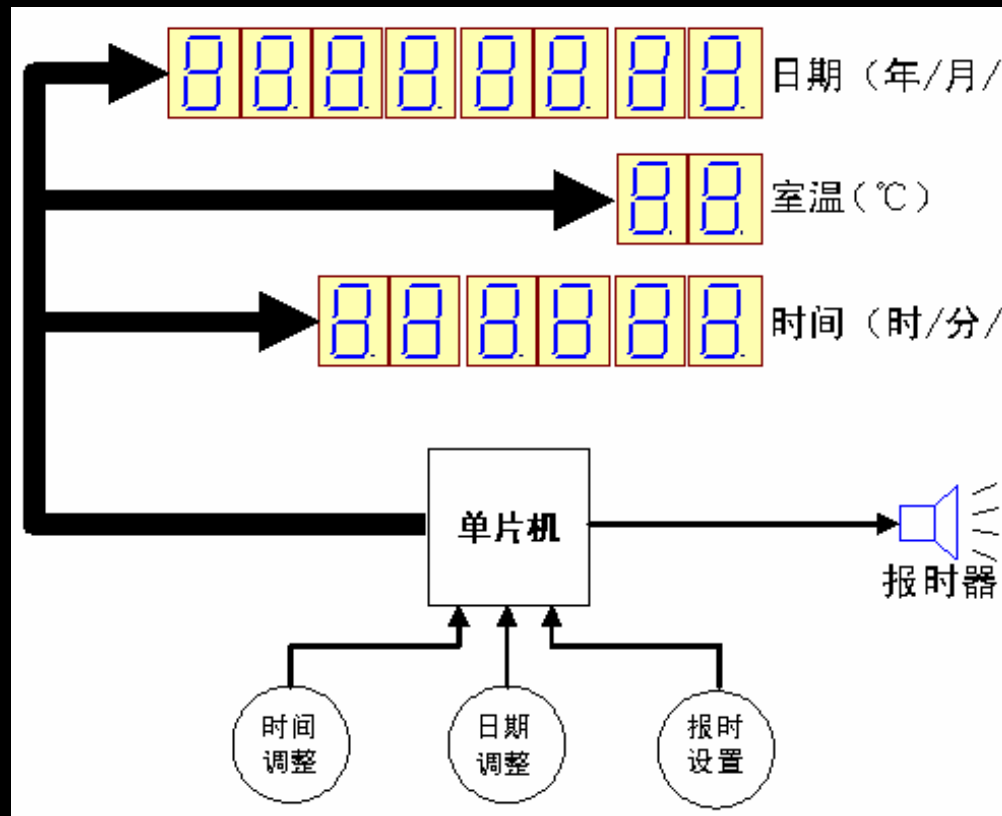
表 1-1 不同厂商的单片机

公 司 标 志	名 称	代表单片机
	Analog Devices (模拟器件)	 ADUC832
	Atmel (爱特梅尔)	 ATMEGA16
	Dallas (达拉斯)	 DS80C390
	Freescale Semiconductor (飞思卡尔)	 MC68000
	Infineon (英飞凌)	 C1645V
	Intel (英特尔)	 P8051AH
	Microchip (微芯)	 PIC24
	National Semiconductor (国家半导体)	 COP8CBR9HVA8

1.6 实例点拨——电子万年历

- 电子万年历的功能模块有：

▷
数码
管显
示模
块



▷
报时
器模
块

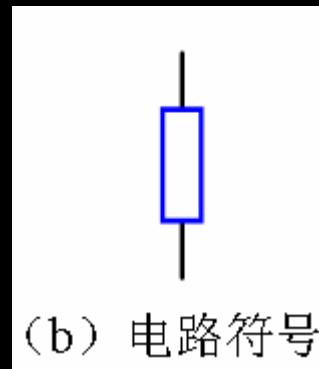
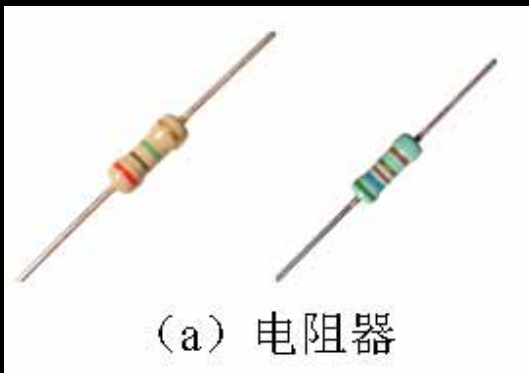
▷
单片机模块

▷
调整按钮模块

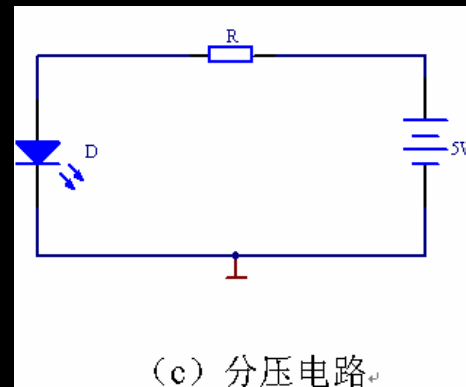
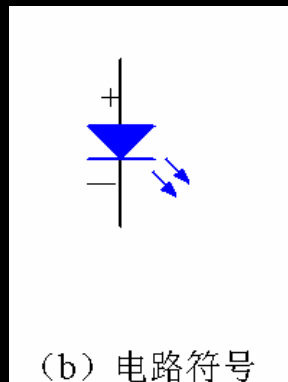
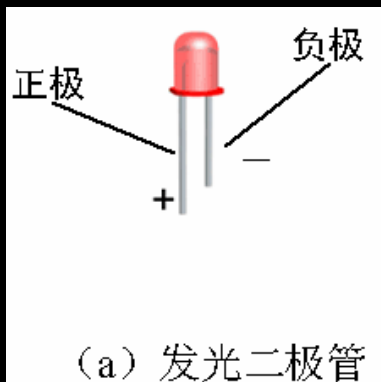
系统中单片机的作用：控制外围器件和处理内部计时

器件介绍

1. 电阻器

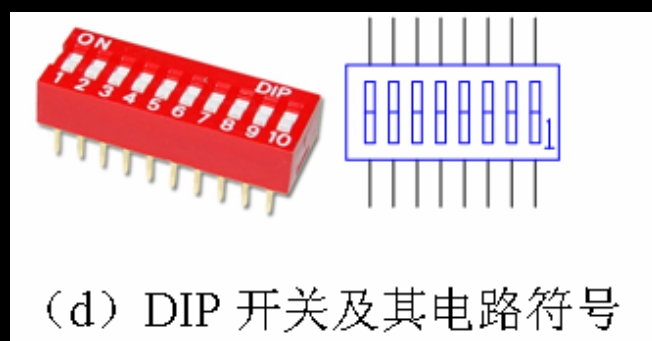
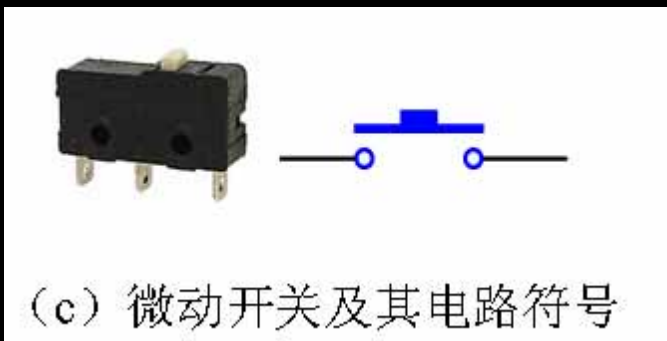
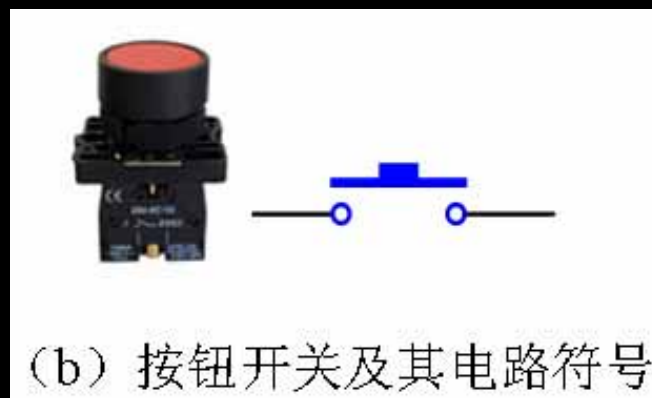
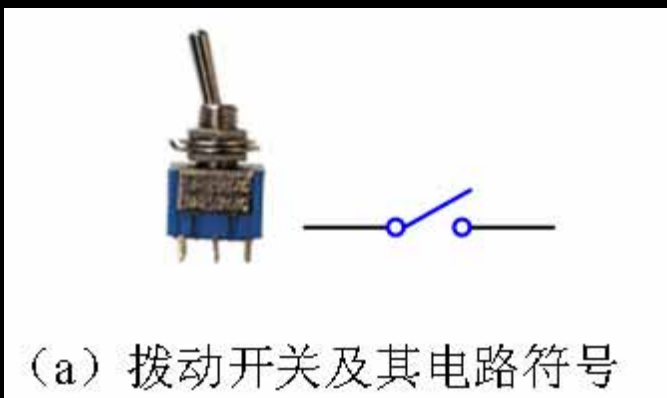


2. 发光二极管



器件介绍

3. 开关



第2章 第一个单片机系统

欢迎访问 电路飞翔网

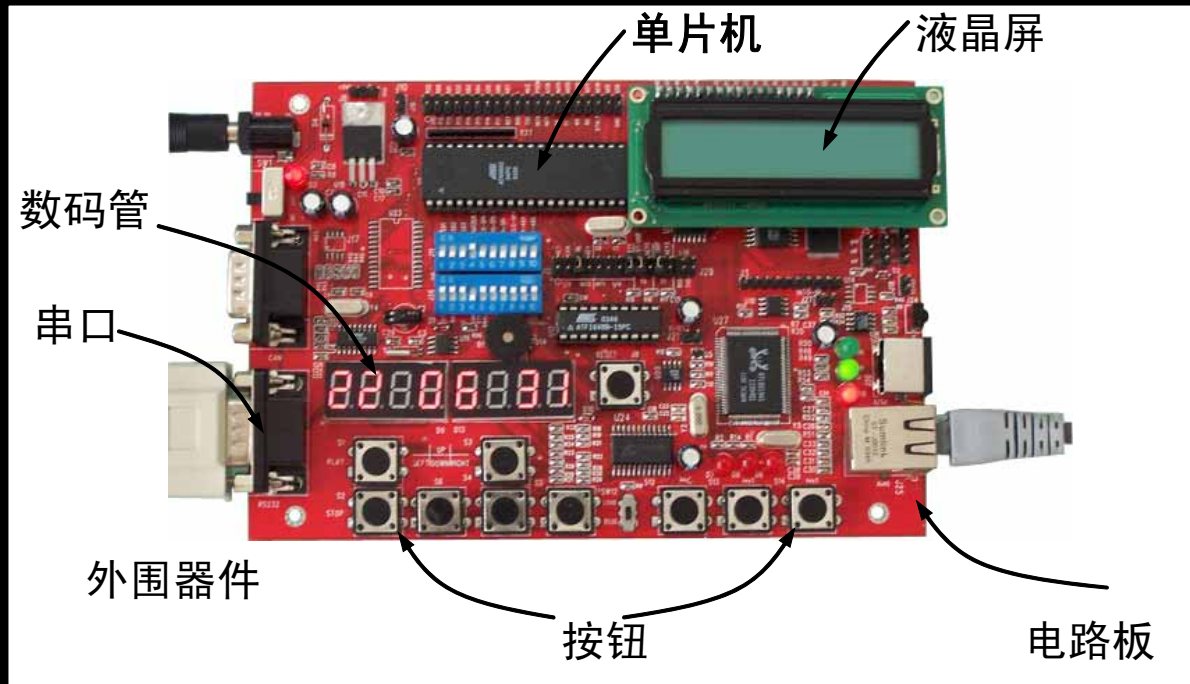
<http://www.circuitfly.com> 获取更多信息

- 2.1 什么是单片机系统
- 2.2 如何控制一个发光二极管
- 2.3 单片机系统开发过程
- 2.4 实例点拨——闪烁5次的发光二极管

2.1 什么是单片机系统

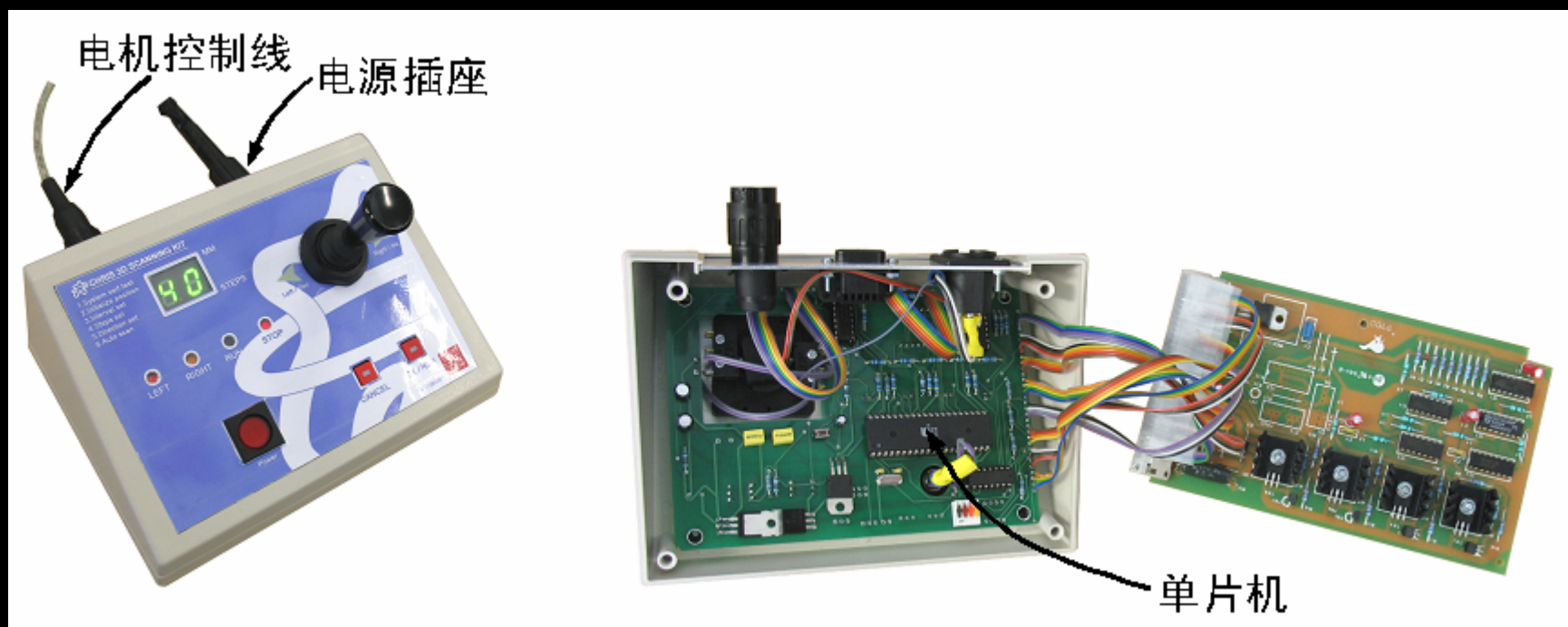
• 2.1.1 单片机本身

单片机本身就是一个电子器件，可以根据指令的要求控制外围器件，它的功能可以通过指令进行控制。



2.1 什么是单片机系统

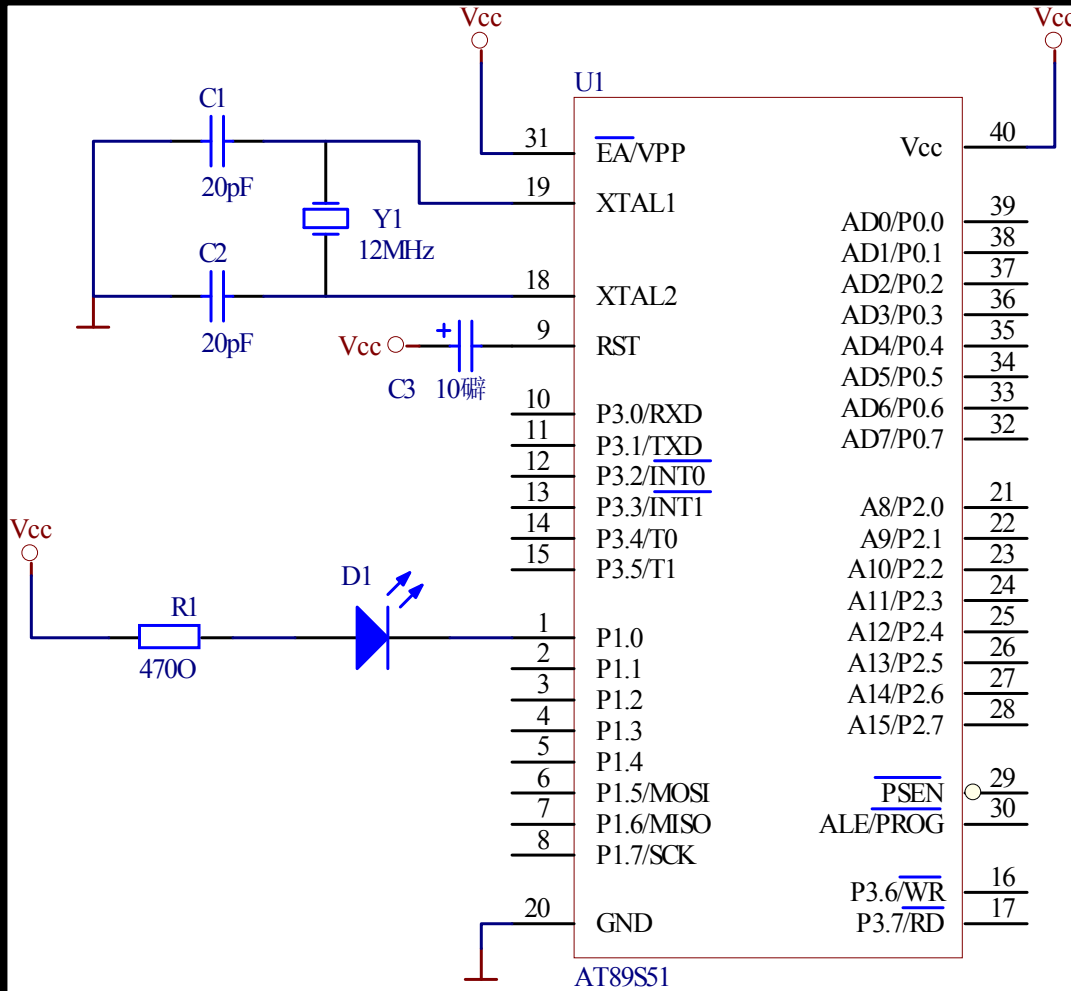
• 2.1.2 构成单片机系统——单片机+外围器件



电机控制盒实例

2.2 如何控制一个发光二极管

• 2.2.1 硬件设计（系统电路图）



电容 C1



电容 C2



电容 C3



电阻 R1



发光二极



管 D1

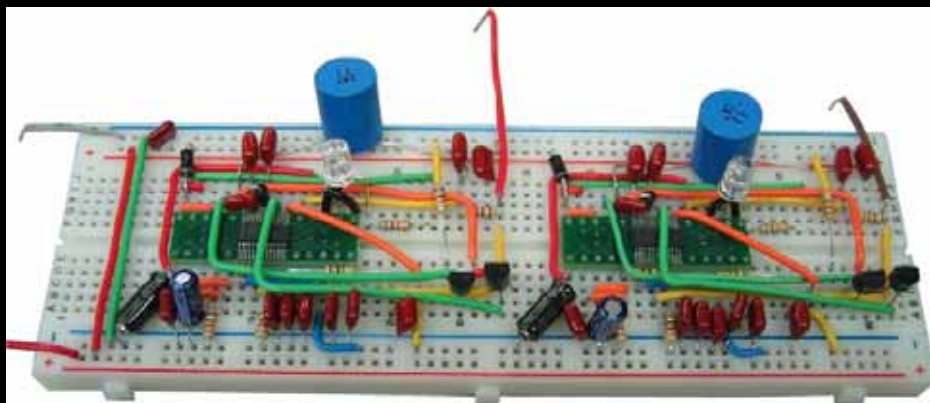


晶振 Y1



2.2 如何控制一个发光二极管

• 2.2.1 硬件设计（搭建硬件电路的器材）



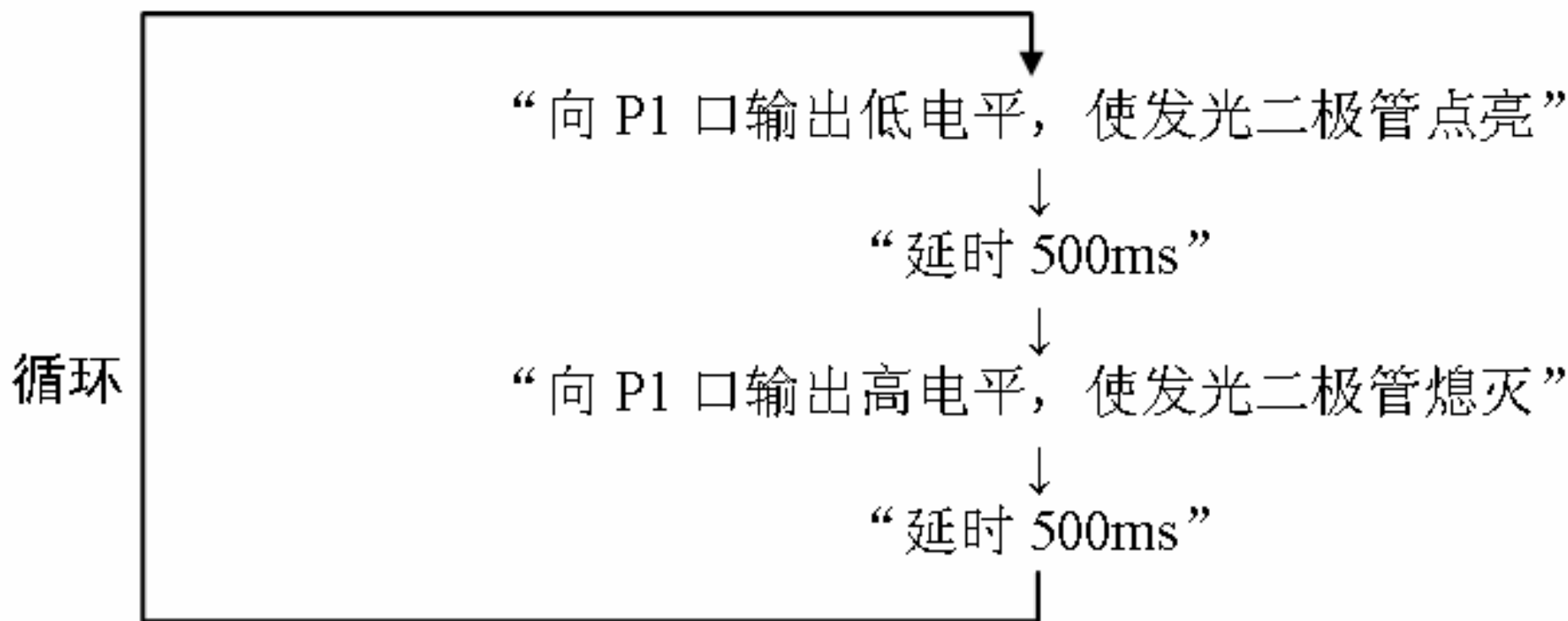
◁ 面包板用于电子技术实验



◁ 万用板用于电子技术实验

2.2 如何控制一个发光二极管

• 2.2.2 软件设计（中文描述的程序设计思路）



2.2 如何控制一个发光二极管

- 2.2.2 软件设计（翻译成单片机的语言）

START:

```
MOV    P1, #00H      ; 向P1口输出低电平
CALL   DELAY          ; 调用延时子程序
MOV    P1, #0FFH     ; 向P1口输出高电平
CALL   DELAY          ; 调用延时子程序
JMP     START         ; 跳回START, 循环执行
```

2.2 如何控制一个发光二极管

• 2.2.2 软件设计（指令解析）

- 标号——可以是以英文字母开头的字母、数字或某些特殊符号的序列。
- 助记符——用来表达指令的操作功能。
- 目的操作数——指令最终作用的对象。
- 源操作数——参与指令的操作。
- 注释——分号后面的文字描述

START:	MOV	P1,	#00H	;向P1口输出低电平
↑	↑	↑	↑	↑
标号	:	助记符	目的操作数, 源操作数	;注释

2.2 如何控制一个发光二极管

• 2.2.2 软件设计（完整的程序1/2）

```
ORG    00H                ; 设置起始地址  
START:                ; 标号  
MOV     P1, #00H          ; 向P1口输出低电平，点亮二极管  
CALL    DELAY             ; 调用延时子程序  
MOV     P1, #0FFH         ; 向P1口输出高电平，熄灭二极管  
CALL    DELAY             ; 调用延时子程序  
JMP     START             ; 跳回START，循环执行
```

2.2 如何控制一个发光二极管

2.2.2 软件设计（完整的程序2/2）

DELAY:

```
        MOV    R3, #50          ; 延时子程序（500ms）  
D1:     MOV    R4, #20  
D2:     MOV    R5, #248  
        DJNZ   R5, $  
        DJNZ   R4, D2  
        DJNZ   R3, D1  
        RET                      ; 返回主程序  
        END                      ; 汇编程序结束
```

2.2 如何控制一个发光二极管

2.2.2 软件设计（指令简析）

ORG 00H —— 伪指令,其功能是设定经汇编后的执行代码存储在单片机中的起始地址为00H。

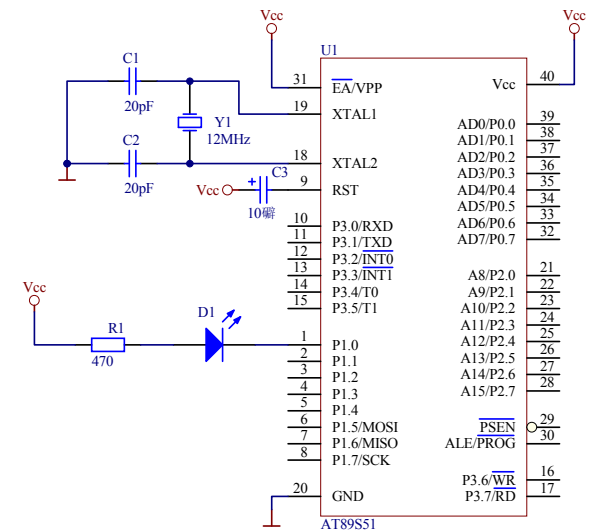
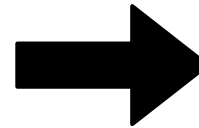
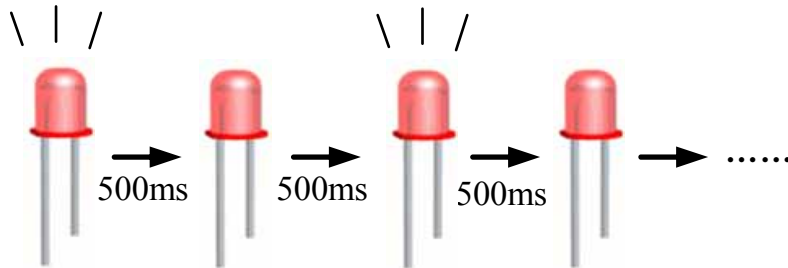
END —— 伪指令,提示汇编器程序结束于此,当汇编遇到END后,就不会再继续进行。

伪指令 (pseudo opcode) —— 汇编程序所提供的帮助汇编器进行汇编的指令,并非单片机指令的一部分,因此伪指令不占存储器空间,只是协助程序的汇编工作。

2.3 单片机系统开发过程

1. 明确系统功能:
闪烁, 间隔500ms

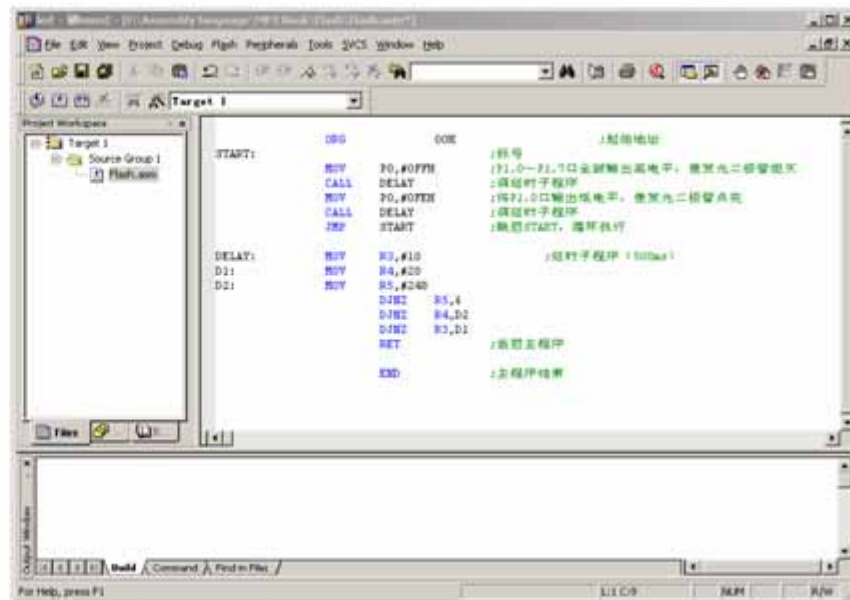
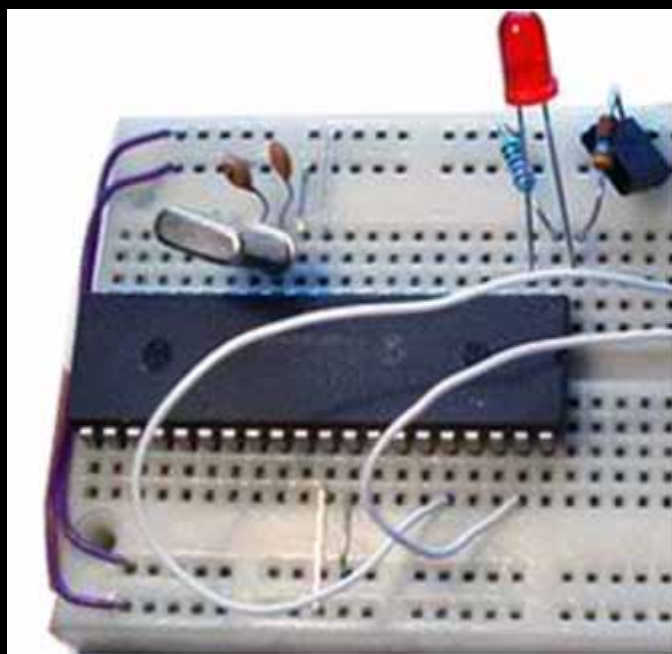
2. 硬件设计:
系统电路图



2.3 单片机系统开发过程

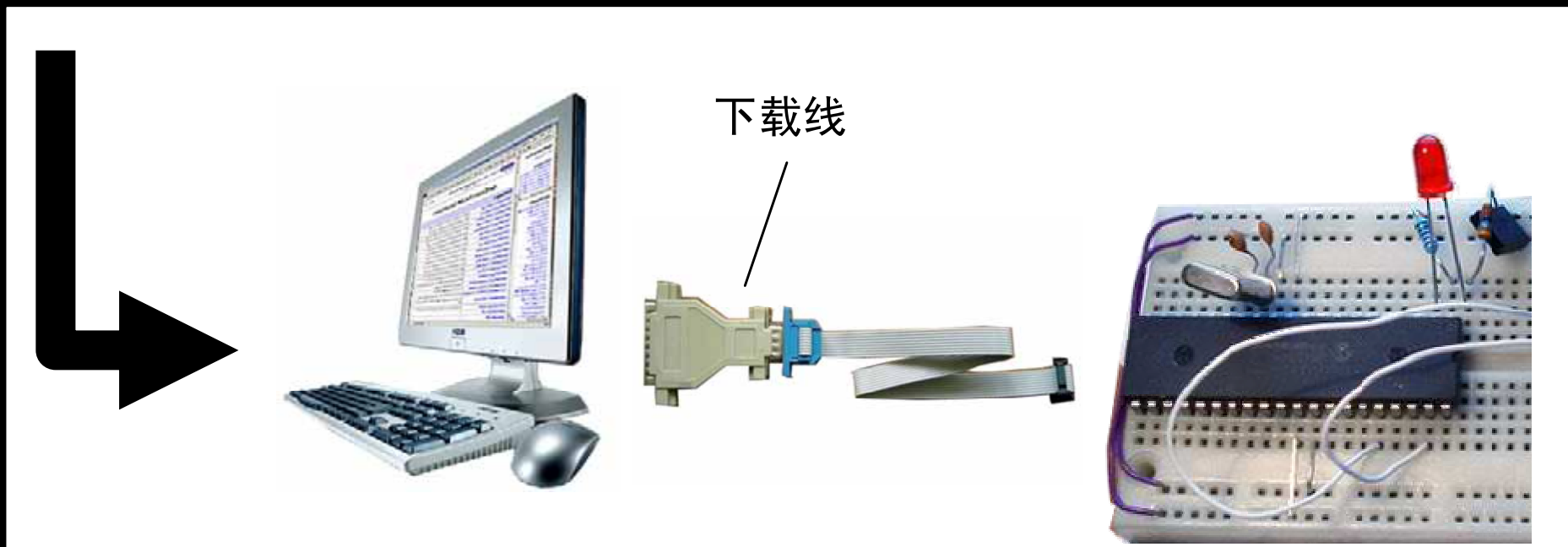
3. 搭建硬件平台：
在面包板、万用板
或实验板上

4. 软件设计：
在汇编语言开发环境中编
写汇编程序



2.3 单片机系统开发过程

5. 下载程序到单片机并调试



2.4 实例点拨——闪烁5次的发光二极管

• 2.4.1 明确系统功能

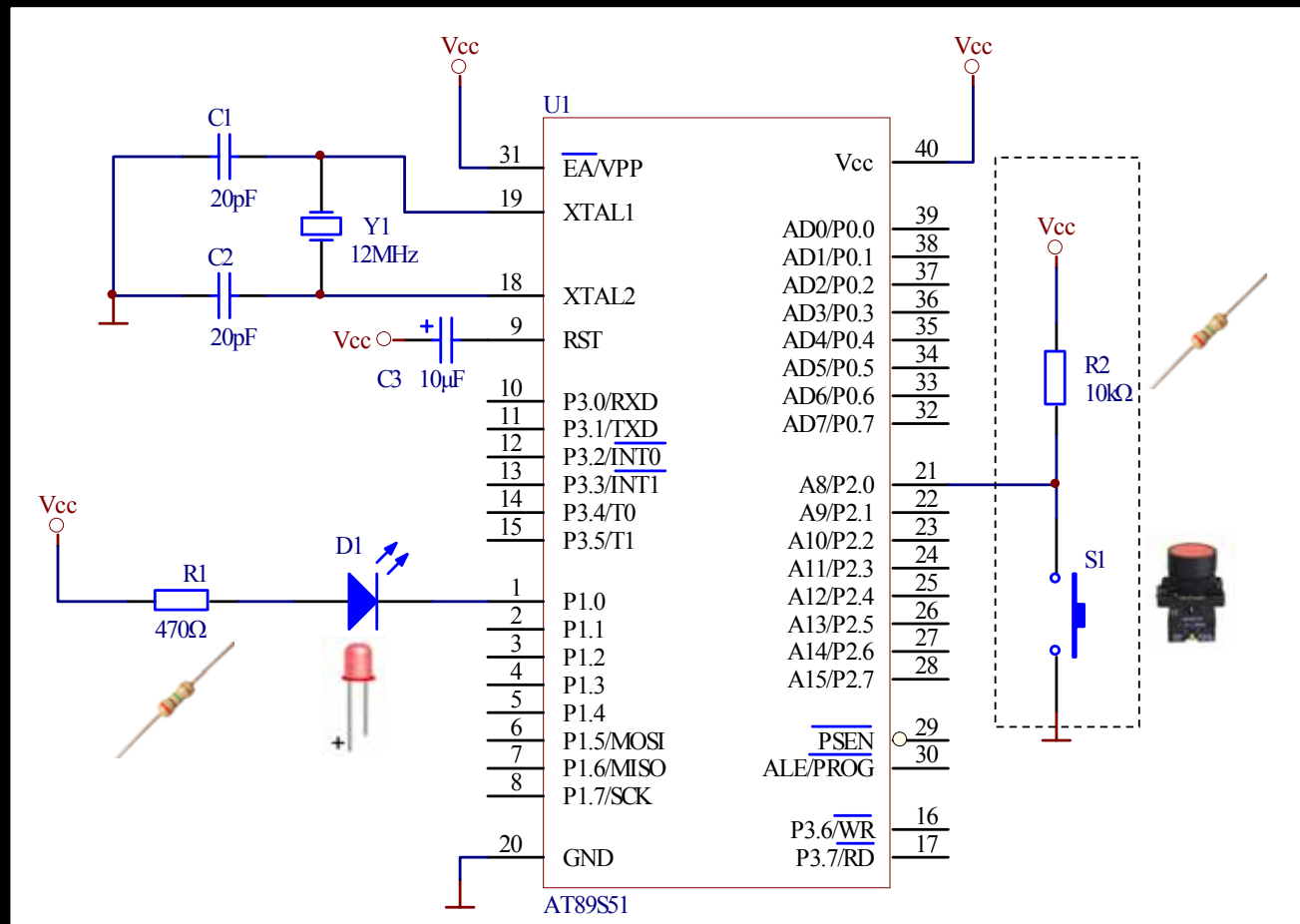
系统启动后，一支发光二极管闪烁5次后熄灭，

当按下一个按钮开关后再闪烁5次后熄灭。

以后每次按下按钮开关，发光二极管都会闪烁5次。

2.4 实例点拨——闪烁5次的发光二极管

• 2.4.2 硬件设计(单片机+外围器件)



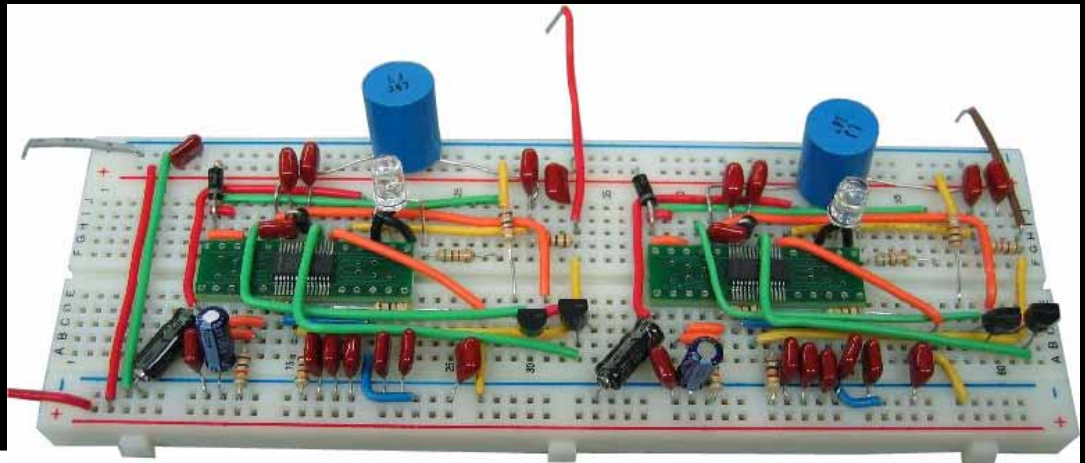
外围器件：

一个发光二
极管、

一个按钮开
关

2.4 实例点拨——闪烁5次的发光二极管

• 2.4.3 搭建硬件平台



在面包板或万用板上插入器件，用导线完成电气连接

2.4 实例点拨——闪烁5次的发光二极管

2.4.4 软件设计

```
ORG    00H           ; 设置起始地址
START:                ; 标号
MOV     R0, #5        ; 循环闪烁5次, R0为计数器
LOOP:                ; 标号
MOV     P1, #00H      ; 向P1口输出低电平, 点亮二极管
CALL    DELAY         ; 调用延时子程序
MOV     P1, #0FFH     ; 向P1口输出高电平, 熄灭二极管
CALL    DELAY         ; 调用延时子程序
DJNZ    R0, LOOP      ; R0减1, 如果不等于0跳到LOOP标号处
```

2.4 实例点拨——闪烁5次的发光二极管

2.4.4 软件设计

```
JB    P2.0, $    ; 如果P2.0为高电平则循环执行本行
JMP   START     ; 跳到START标号处重新开始执行
DELAY:
.....          ; 延时子程序（500ms）
RET                    ; 返回主程序
END                  ; 汇编程序结束
```

2.4 实例点拨——闪烁5次的发光二极管

• 2.4.4 软件设计（指令解析）

✓ MOV R0, #5

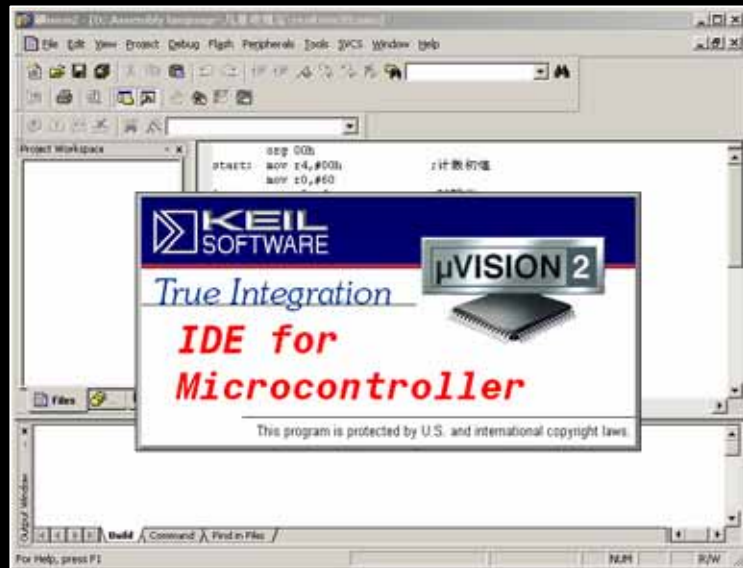
✓ DJNZ R0, LOOP

✓ JB P2.0, \$

✓ JMP START

2.4 实例点拨——闪烁5次的发光二极管

• 2.4.5 下载程序到单片机并调试

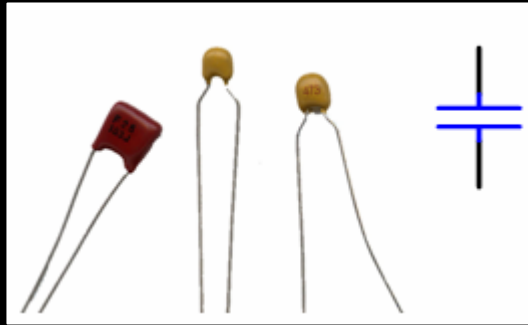


- 在 μ Vision中把目标程序编辑好,
- 生成单片机可执行的代码,
- 通过下载线下载到单片机系统板中,
- 然后运行该程序

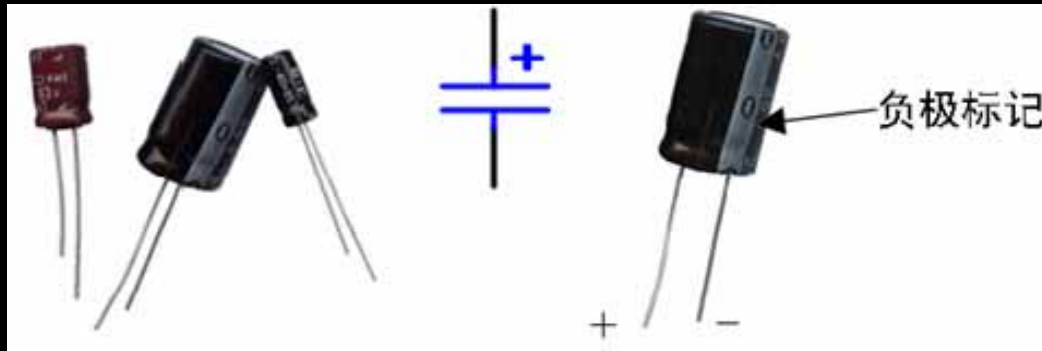
开发环境 μ Vision

器件介绍

1. 电容器

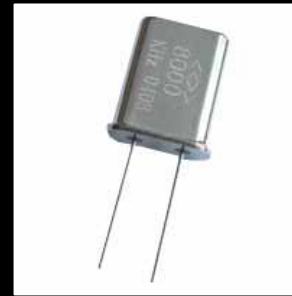


△ 独石电容



△ 电解电容

2. 晶体振荡器

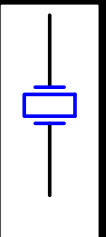


◁ 无源晶振



◁ 有源晶振

晶振的电路符号 ▷



第3章 一开始遇到的问题

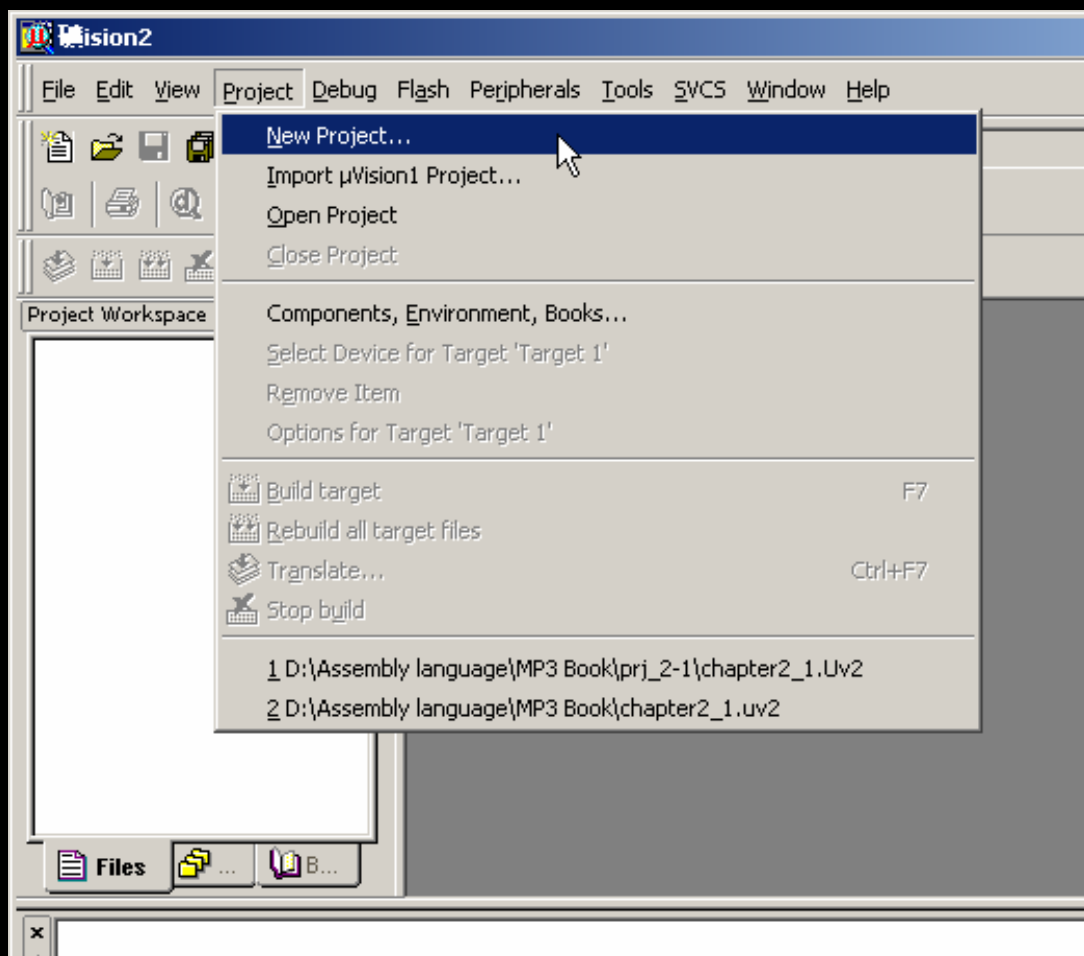
欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 3.1 μ Vision软件的使用
- 3.2 开发的线索小结与软件仿真方法
- 3.3 单片机最简（小）系统分析
- 3.4 实例点拨——流水灯

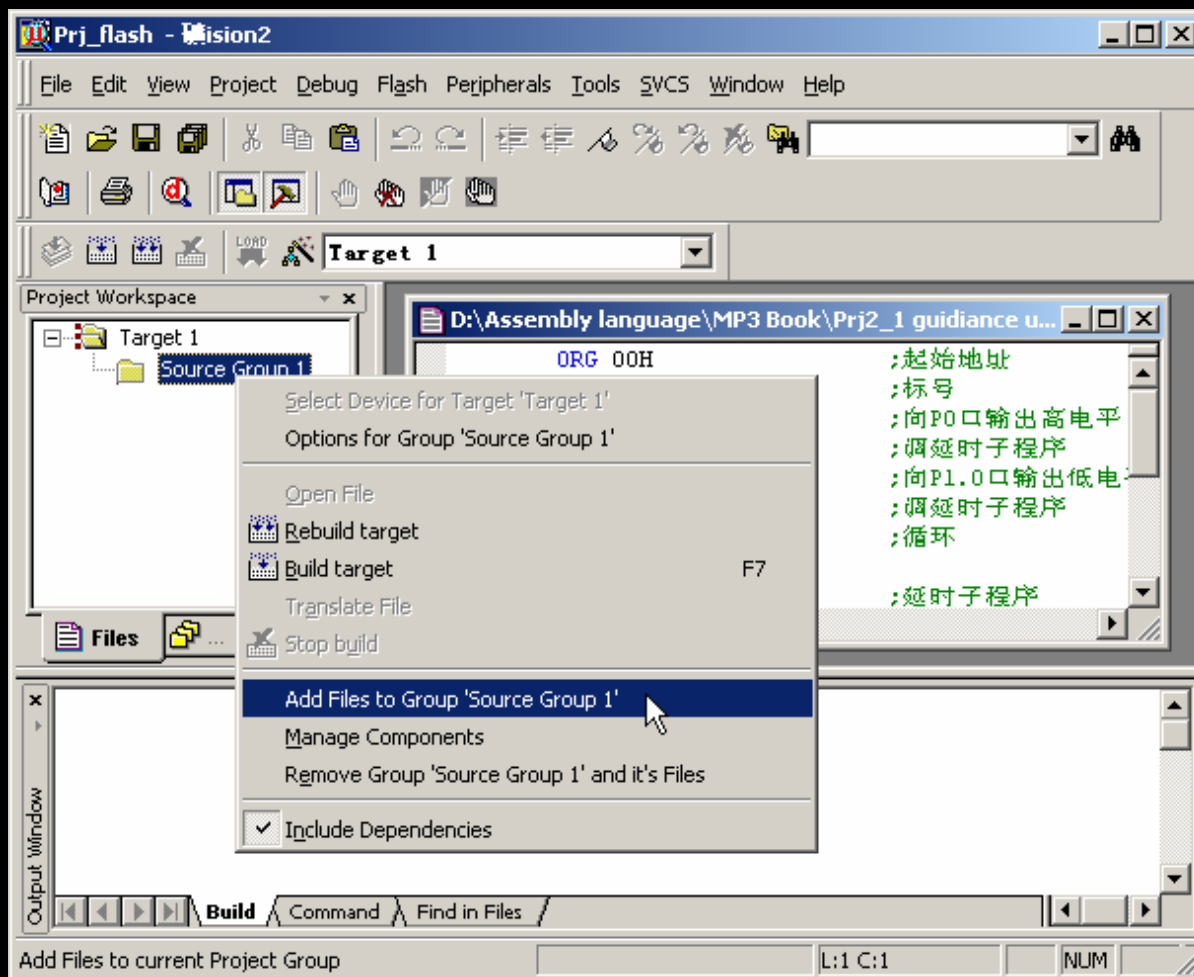
3.1 μ Vision软件的使用

① 新建和保存工程——建一个鸡蛋壳



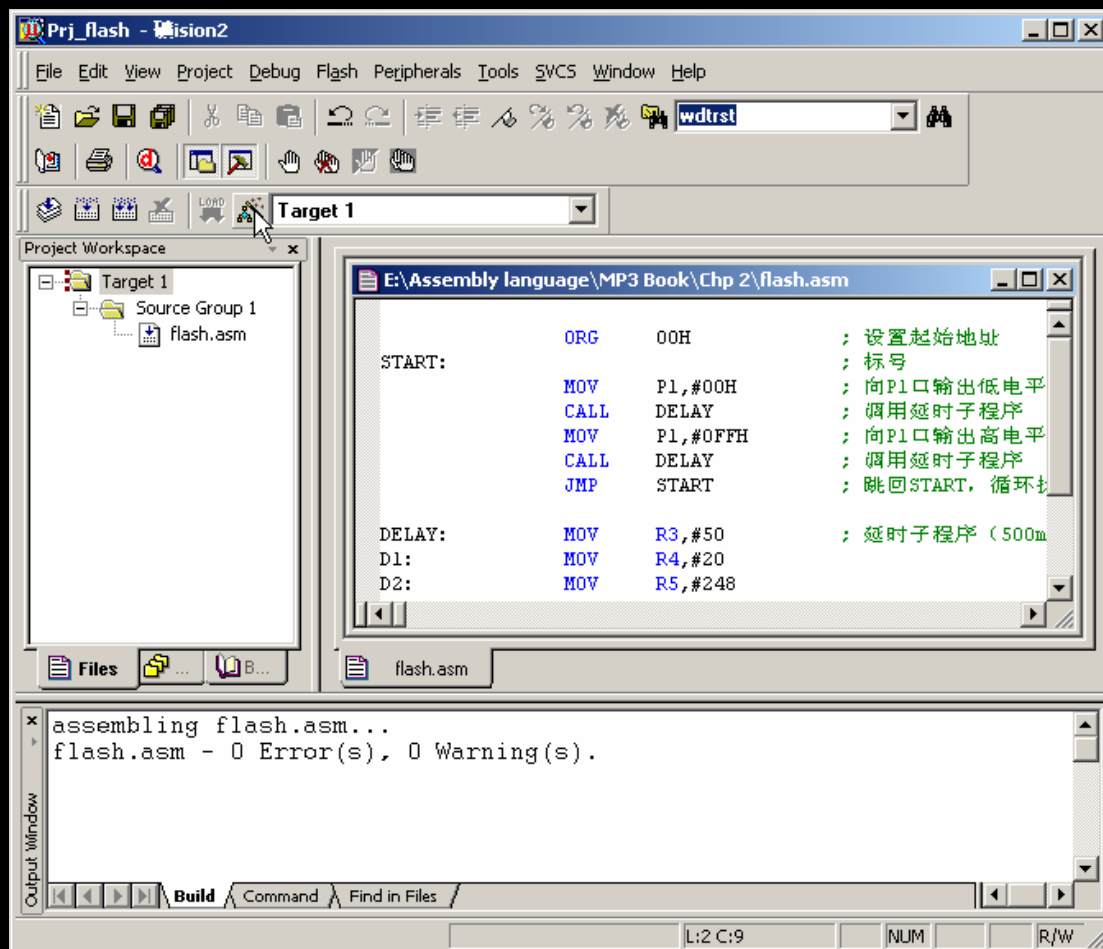
3.1 μ Vision软件的使用

② 往“蛋壳”中添加源程序文件.asm



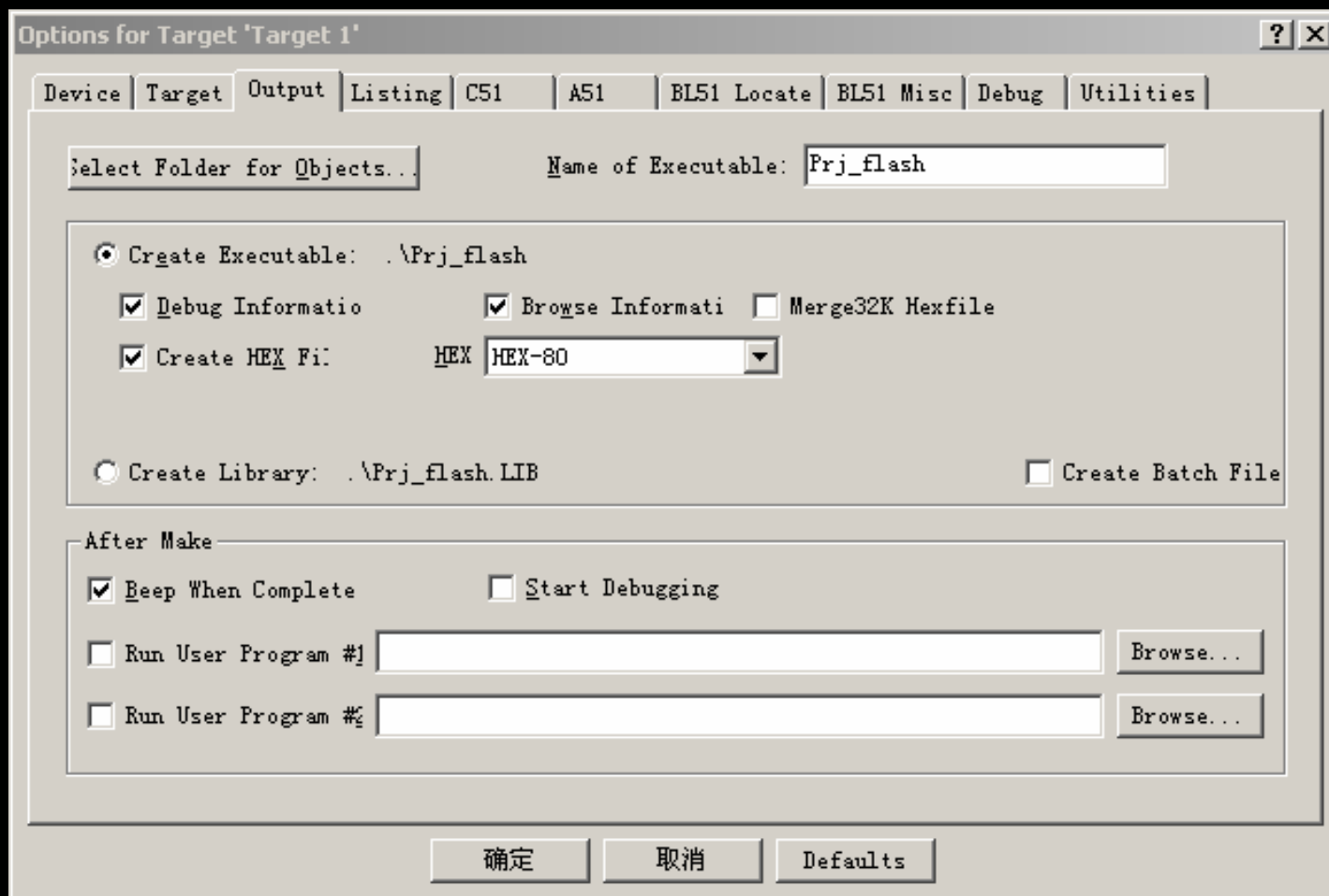
3.1 μ Vision软件的使用

③ 汇编程序



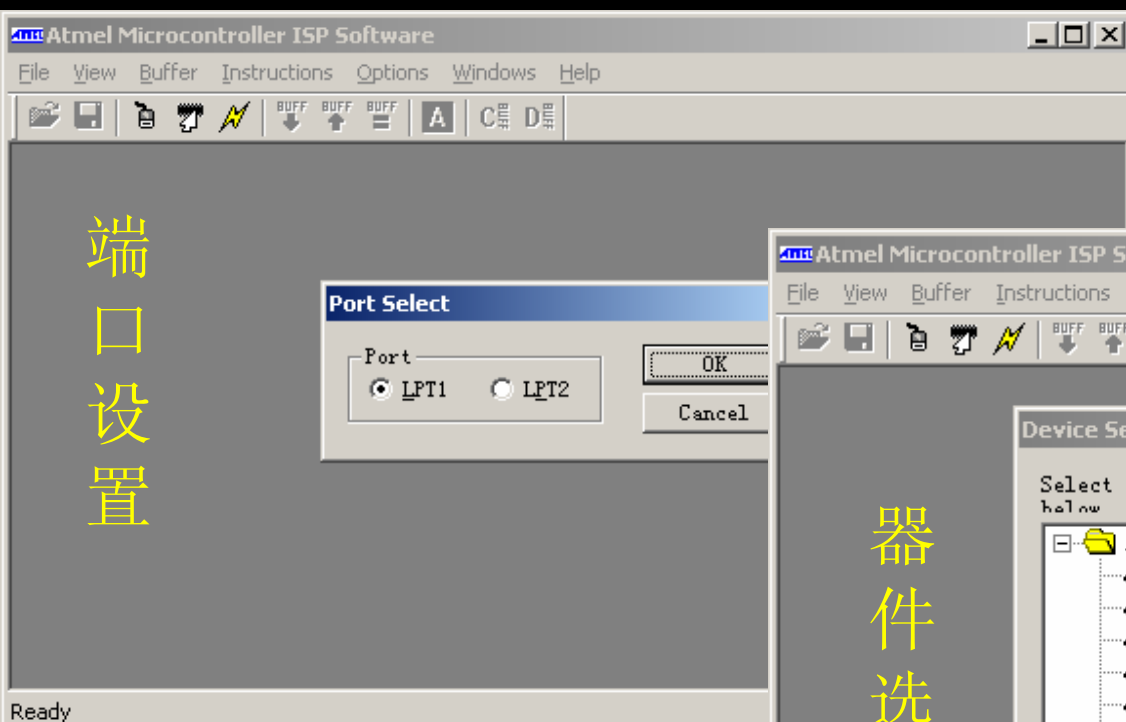
3.1 μ Vision软件的使用

④ 生成下载到单片机中的文件

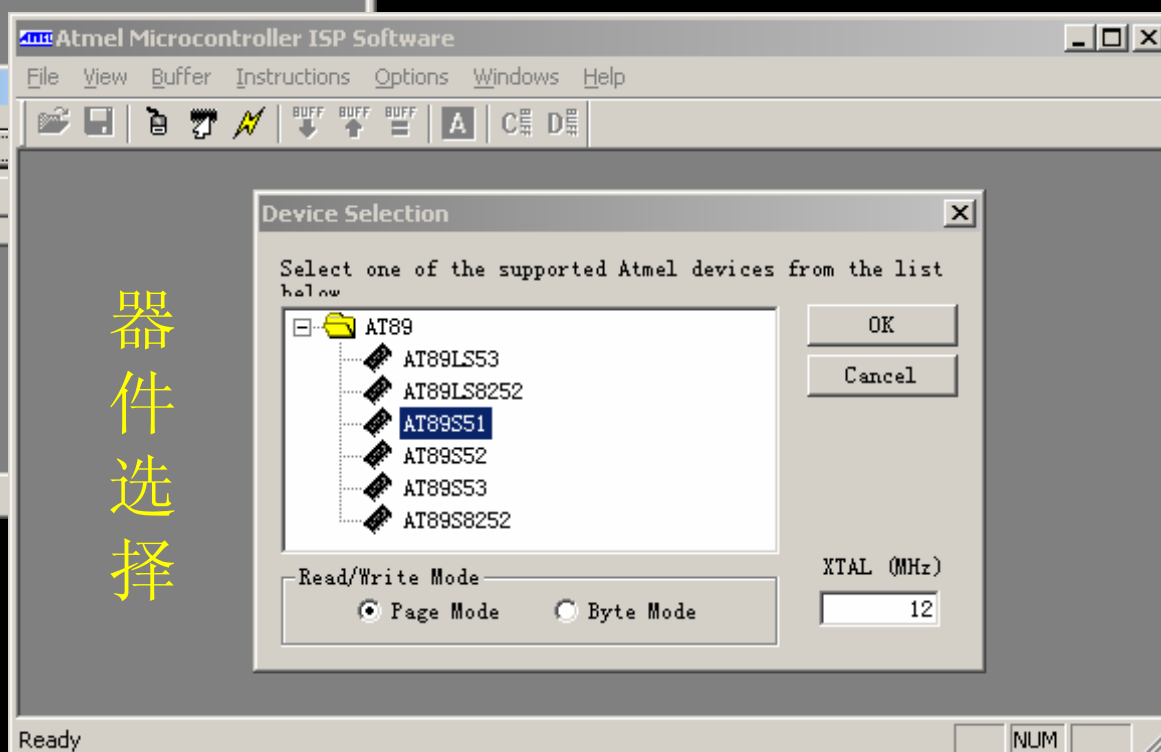


3.1 μ Vision软件的使用

⑤ 下载代码到单片机中



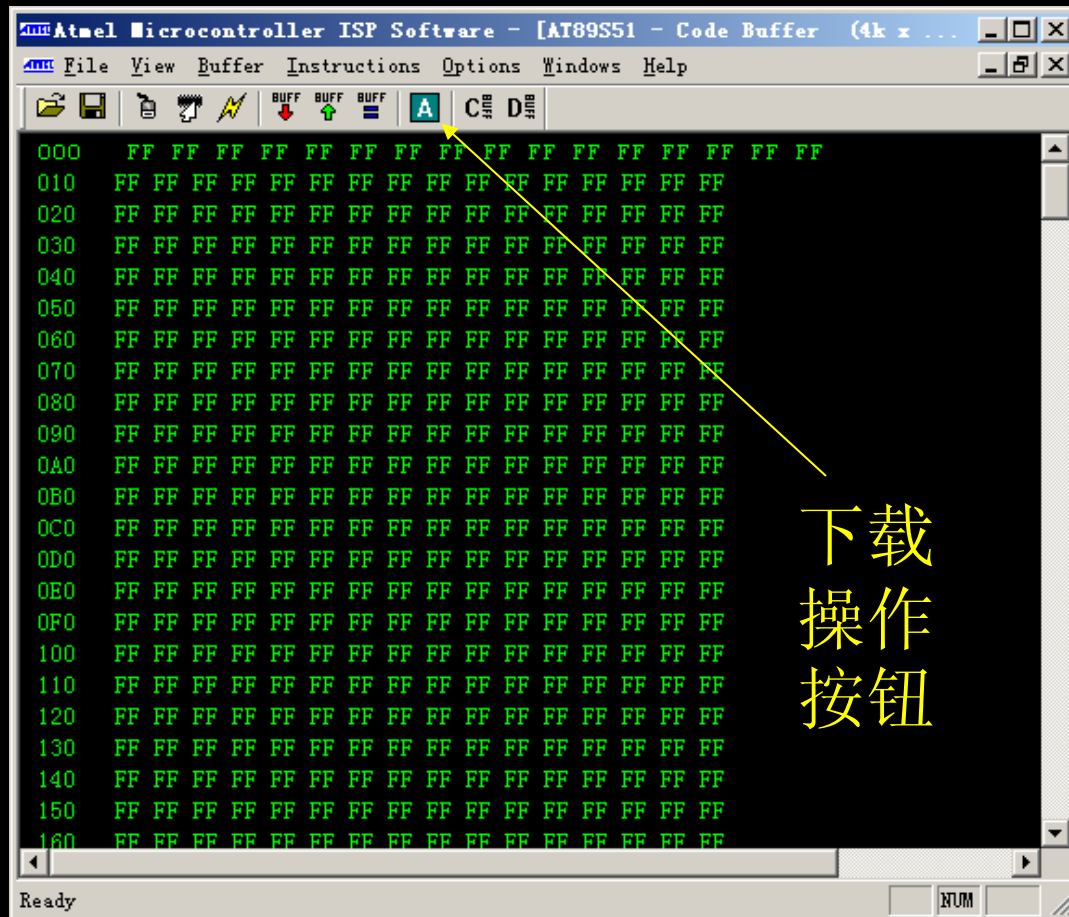
端口设置



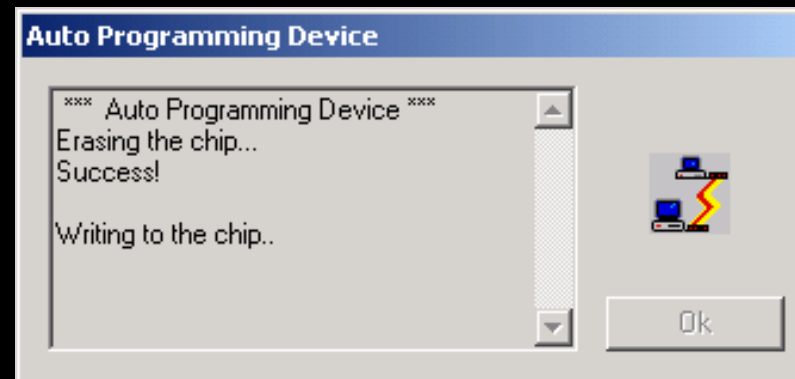
器件选择

3.1 μ Vision软件的使用

⑤ 下载代码到单片机中



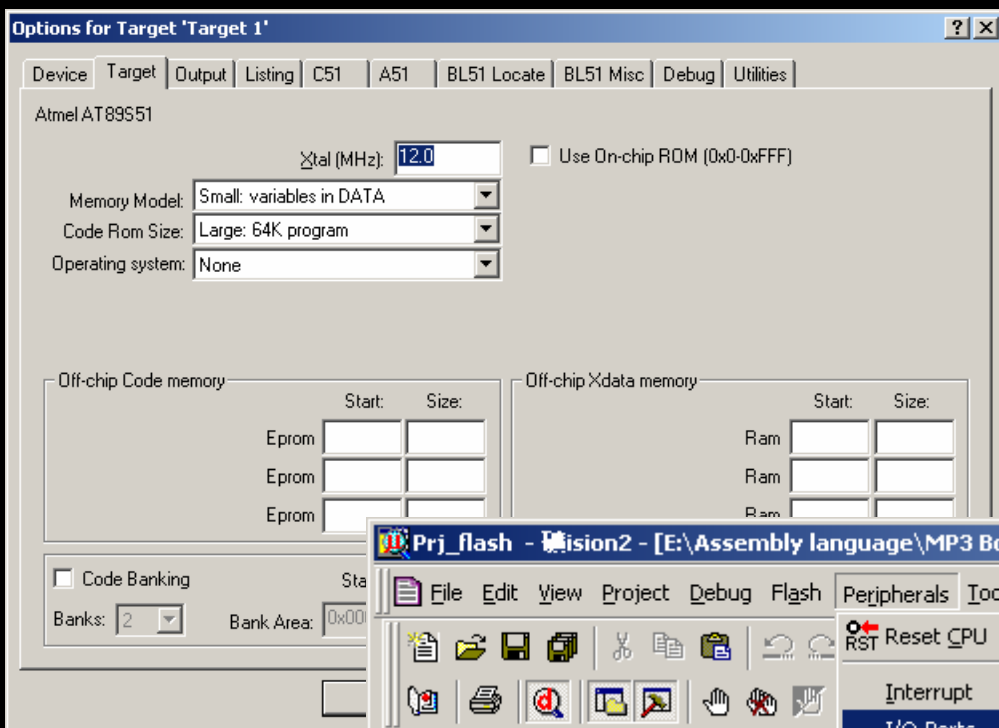
下载
操作
按钮



下载

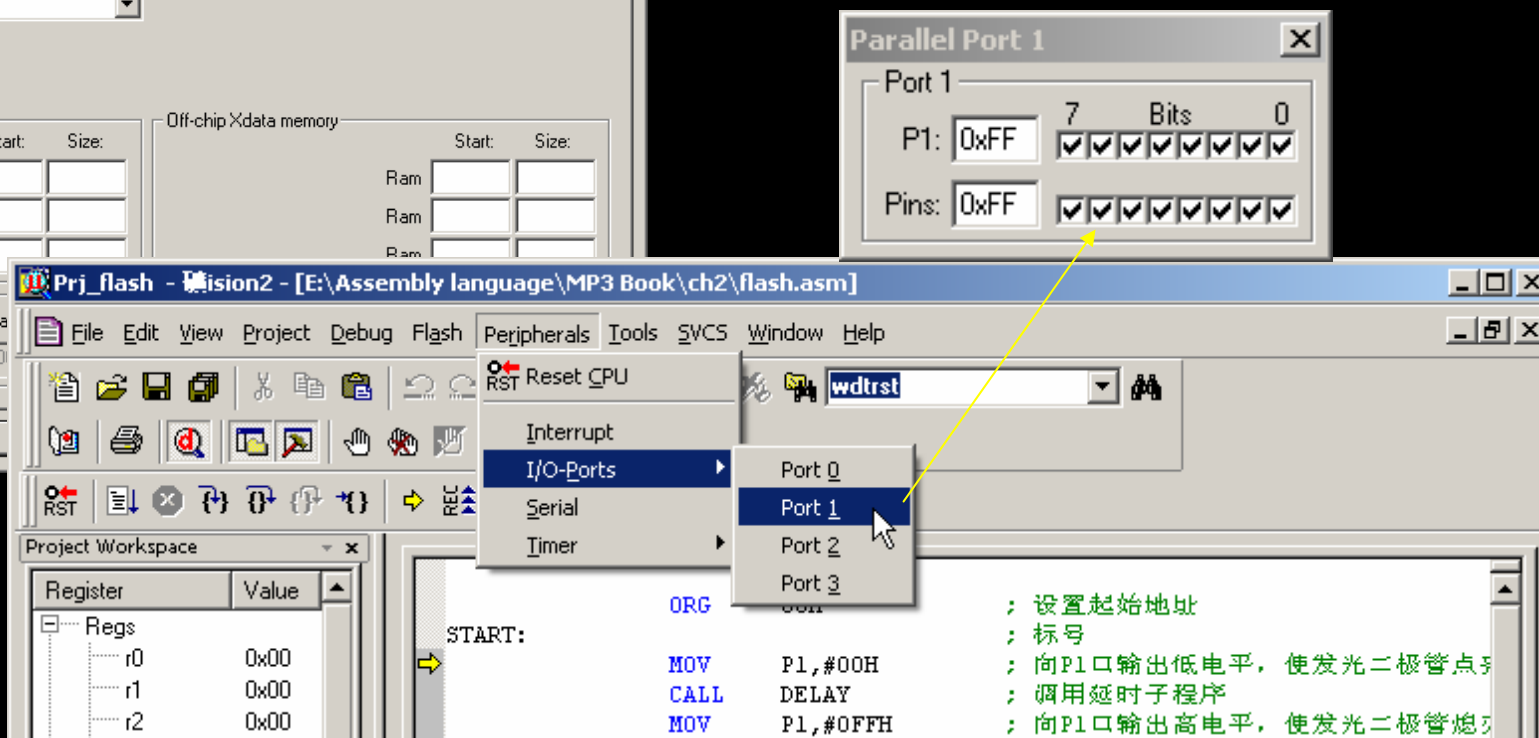
3.2 开发的线索小结与软件仿真方法

- μ Vision 的仿真功能



◁ 晶振频率设置

观察P1口的状态 ▷



3.2 开发的线索小结与软件仿真方法

完整的开发过程

明确系统功能: P1.0 口上的发光二极管以 500ms 为间隔闪烁



硬件设计: 单片机+外围器件（发光二极管等），完成电路图设计



搭建硬件平台: 在面包板上插入单片机和外围器件，并加入下载接



软件设计: 在 μ Vision 中书写汇编程序，进行汇编，修改语法错误



软件仿真: 在 μ Vision 设置系统选项，进入软件仿真功能，观察结果



下载到单片机: 如果软件仿真结果与设计相符，则下载到单片机中

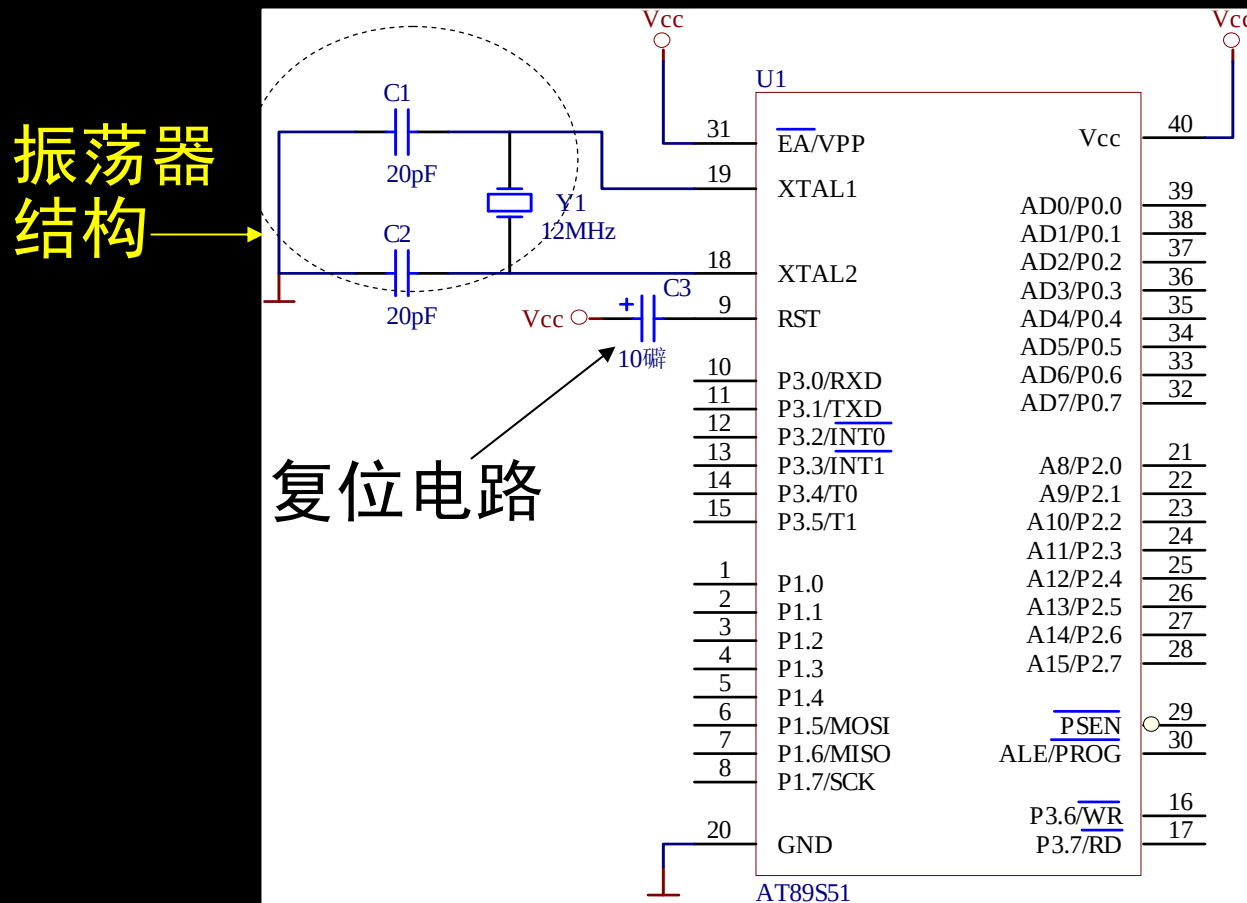
3.2 开发的线索小结与软件仿真方法

开发线索比较表

线 索	成 本	效 率	适 用
AMI+下载线	最低	较低	初学者
μ Vision+专用下载线	较高	较高	开发人员
编程器专用软件	较高	较高	开发人员

3.3 单片机最简（小）系统分析

- AT89S51单片机最简系统只包含单片机、振荡器结构、复位电路和电源的系统。

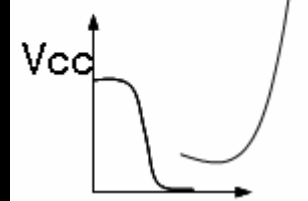
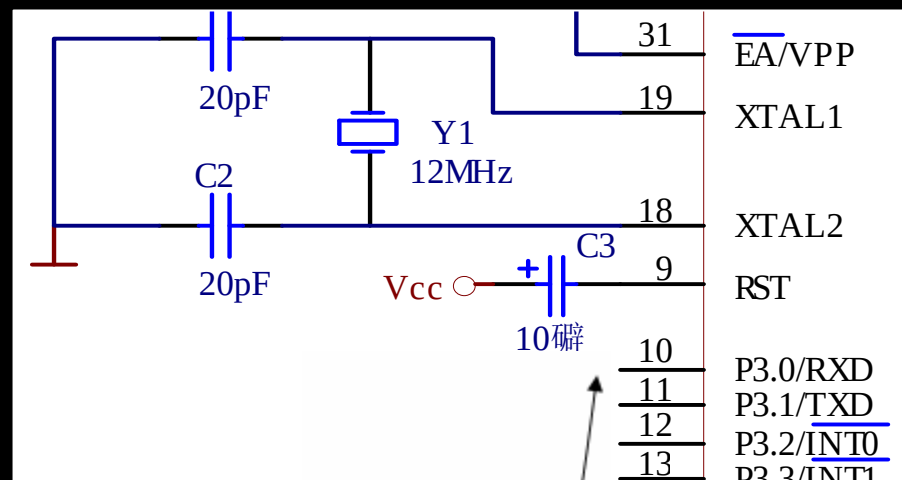
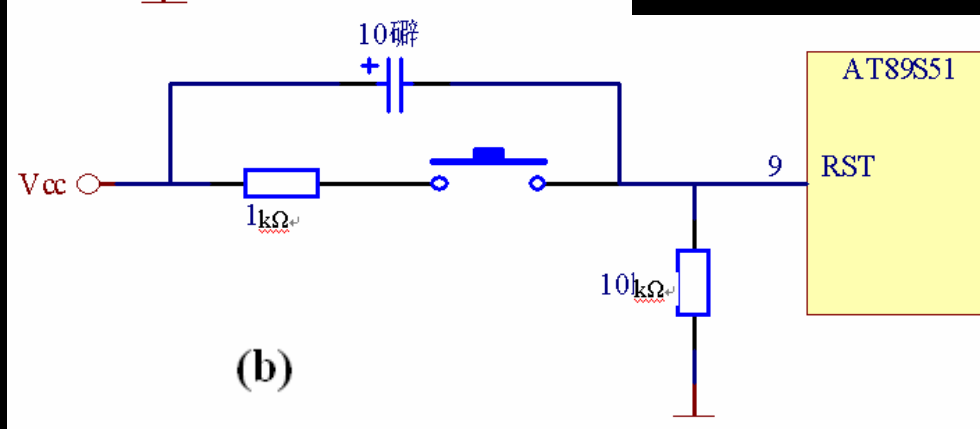
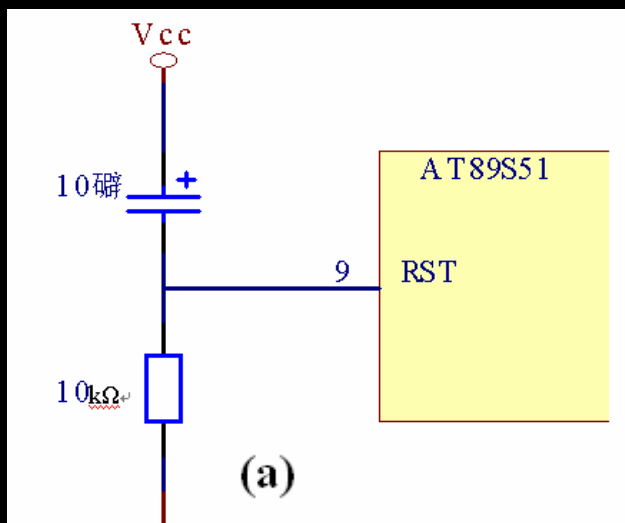


电源直流
+4.0~+5.5V

3.3 单片机最简（小）系统分析

• 复位电路

▷ 两种常用的复位电路



△ RST 管脚上的电平跳变

3.3 单片机最简（小）系统分析

- 外部程序存储器访问控制端（ $\overline{\text{EA}}/\text{VPP}$ ）
 - ✓ 如果 $\overline{\text{EA}}/\text{VPP}$ 接高电平Vcc，单片机则执行本单片机内部程序存储器中的程序，并在读取完AT89S51单片机内部程序存储器的0FFFH后自动转向读取外部的程序存储器；
 - ✓ 如果 $\overline{\text{EA}}/\text{VPP}$ 管脚接低电平，则单片机访问的全部是外部程序存储器；

3.4 实例点拨——流水灯

- 开发步骤:

明确系统功能 → 硬件设计 → 搭建硬件平台

→ 软件设计 → 下载程序到单片机并调试

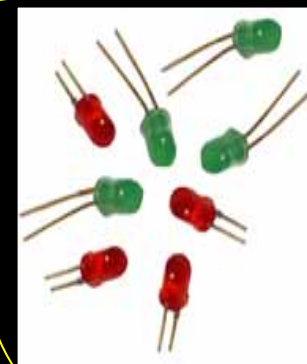
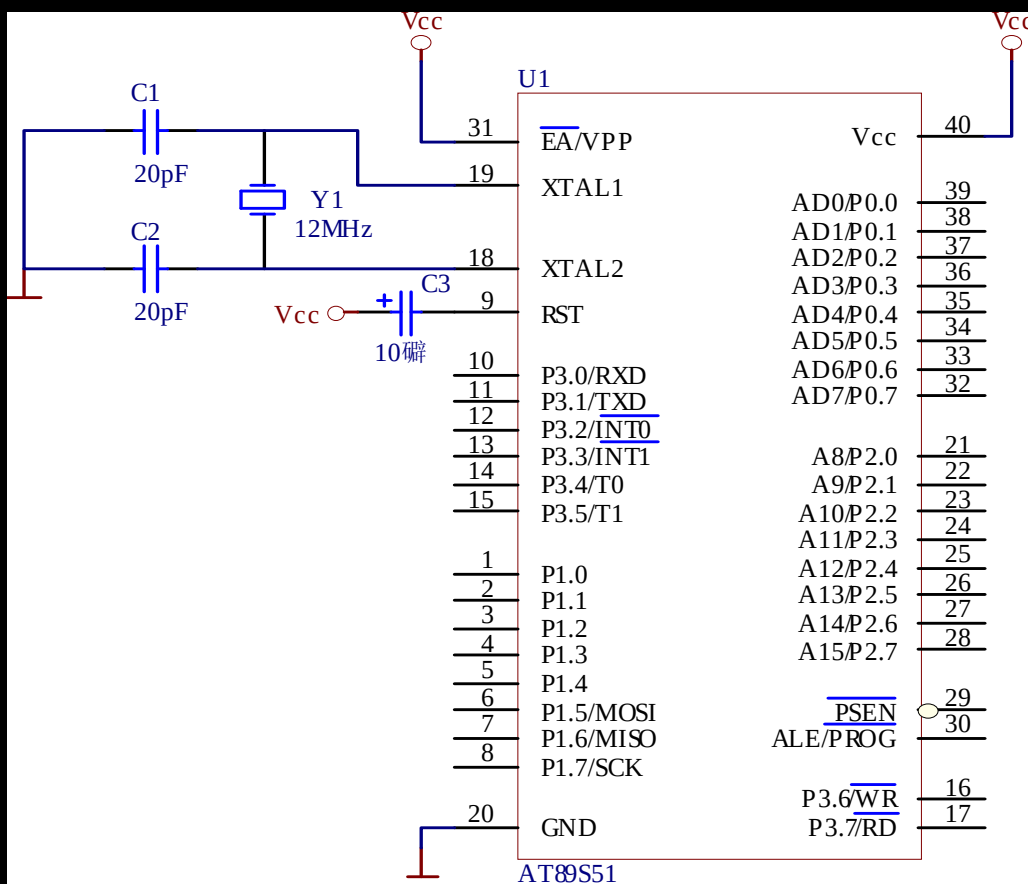
3.4 实例点拨——流水灯

① 明确系统功能

- ✓ 8支发光二极管排成一列；
- ✓ 从左至右再从右至左，做单一灯的流动，如此反复；
- ✓ 每一支发光二极管点亮时间为200ms；

3.4 实例点拨——流水灯

② 硬件设计



8支发光二极管

3.4 实例点拨——流水灯

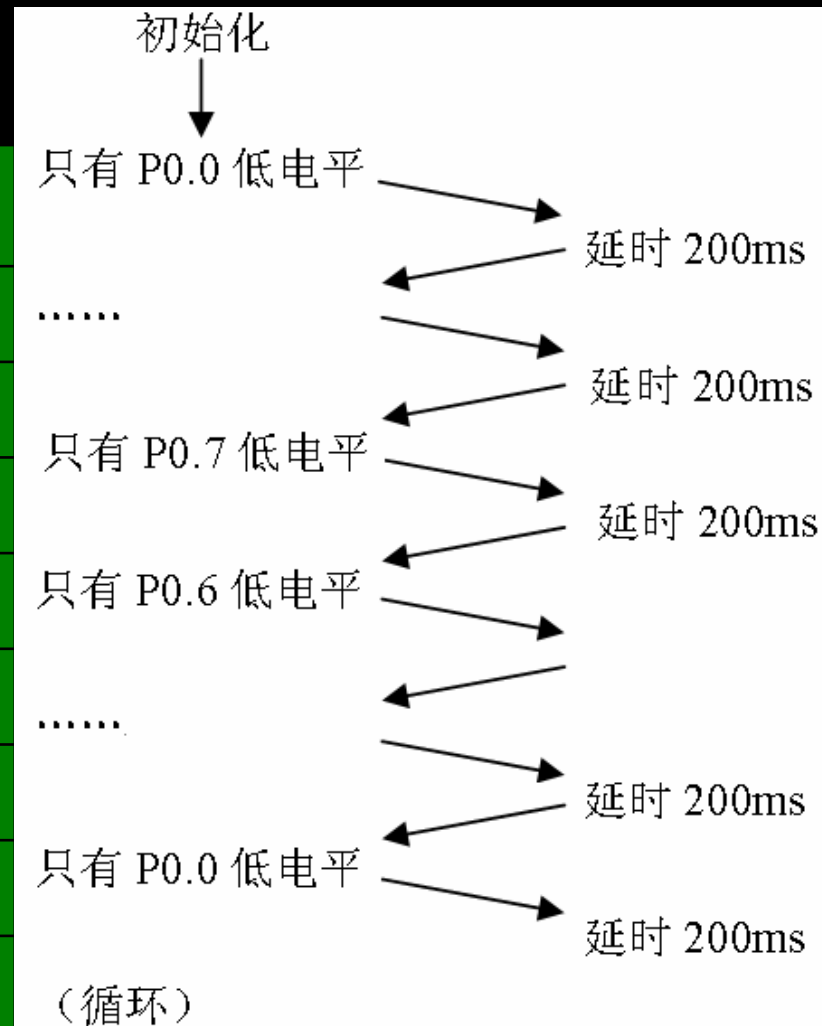
③ 搭建硬件平台

- ✓ 在面包板或万用板上插器件并使用导线完成特定电气连接的过程。
- ✓ 连接完成后，需要仔细对照电路图进行检查，最后连接电源，务必确保正负极没有弄错。

3.4 实例点拨——流水灯

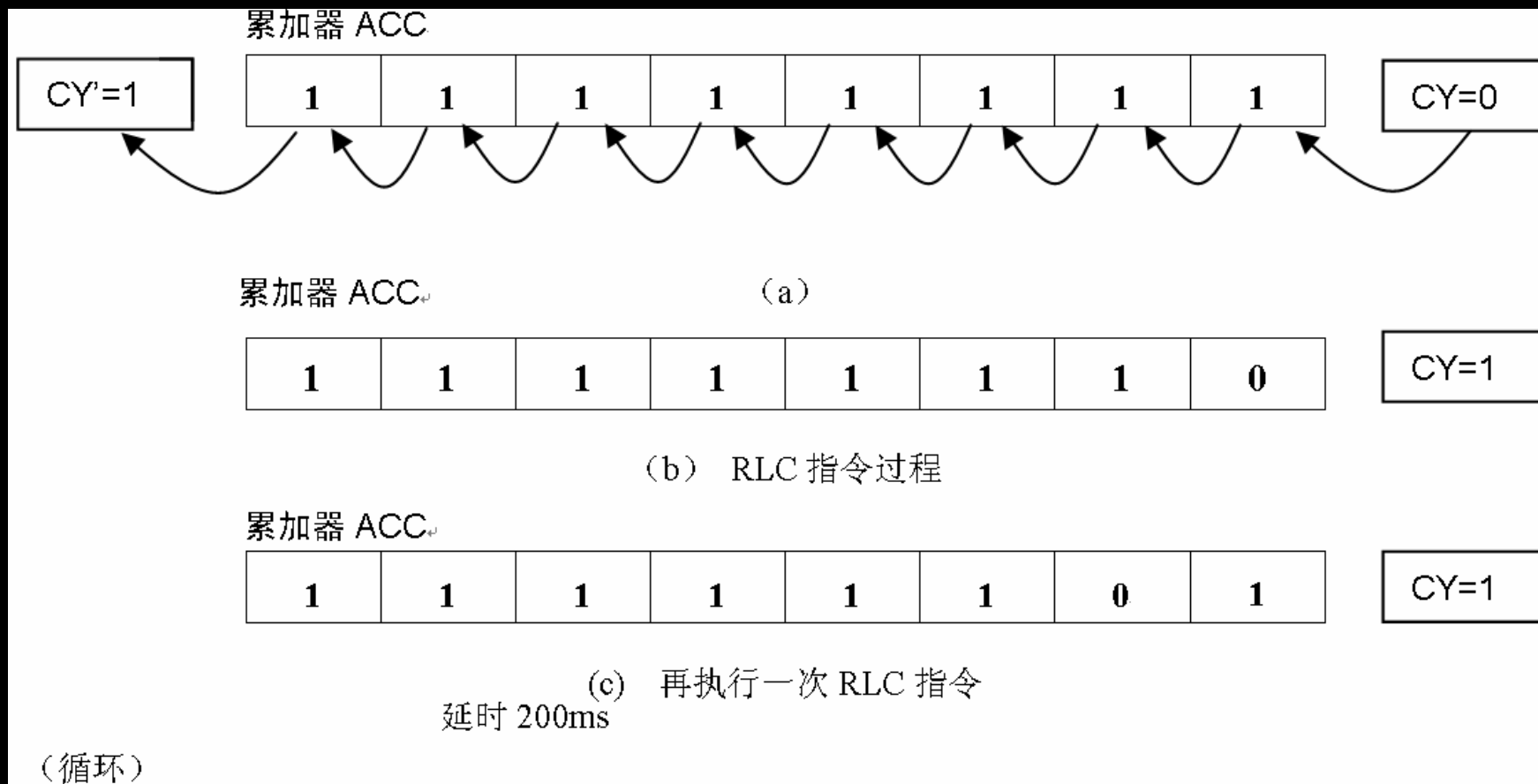
④ 软件设计（赋值命令）

p	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	十六进制
1	1	1	1	1	1	1	1	0	0FEH
2	1	1	1	1	1	1	0	1	0FDH
3	1	1	1	1	1	0	1	1	0FCH
4	1	1	1	1	0	1	1	1	0F7H
5	1	1	1	0	1	1	1	1	0EFH
6	1	1	0	1	1	1	1	1	0DFH
7	1	0	1	1	1	1	1	1	0BFH
8	0	1	1	1	1	1	1	1	07FH



3.4 实例点拨——流水灯

④ 软件设计（RLC和RRC命令）



PSW的8位

CY

AC

F0

RS1

RS0

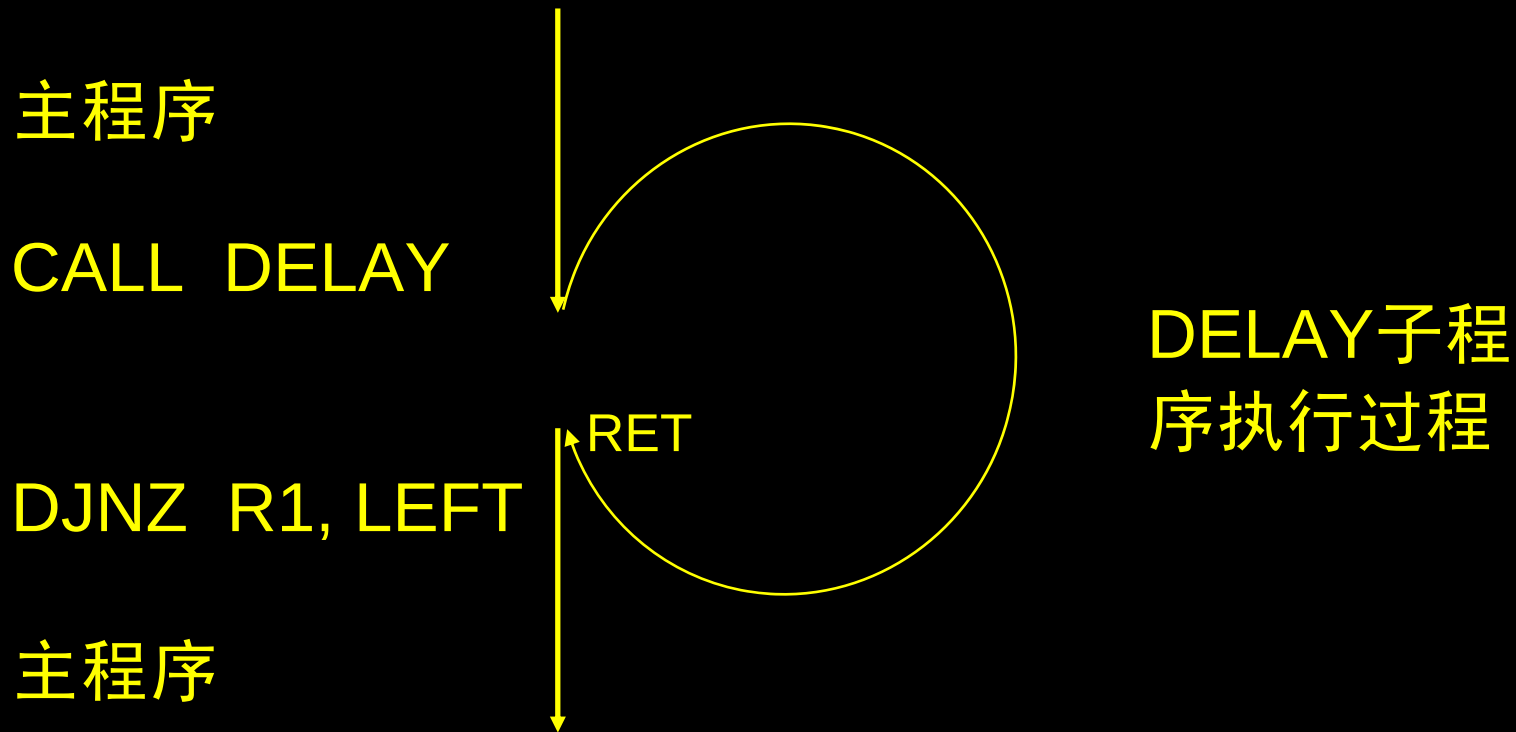
OV

无用位

P

3.4 实例点拨——流水灯

④ 软件设计（主程序和子程序）



3.4 实例点拨——流水灯

④ 软件设计（延时子程序）

D1:

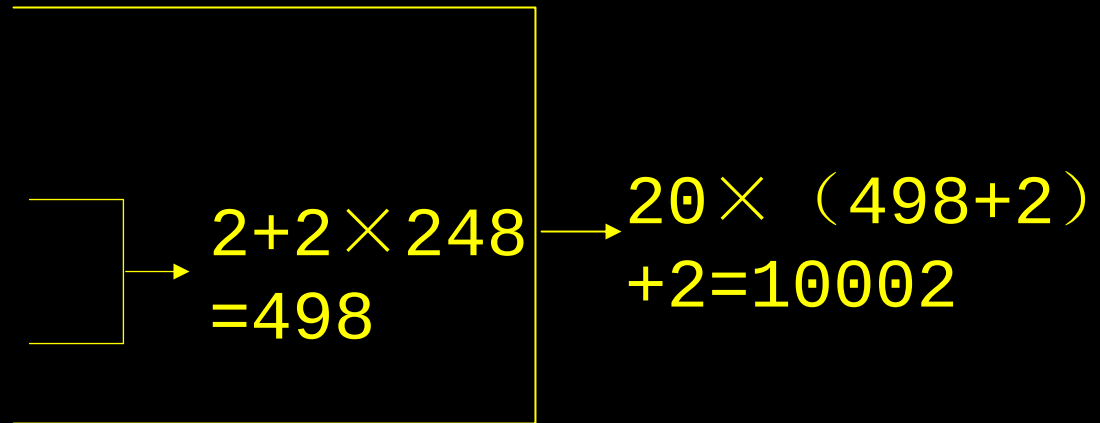
MOV R4, #20

D2:

MOV R5, #248

DJNZ R5, \$

DJNZ R4, D2



3.4 实例点拨——流水灯

⑤ 下载程序到单片机并调试

在 μ Vision中进行编辑和汇编，

再进行软件仿真观察结果是否与设计相符，

成功后就可以通过下载线下载到硬件平台中。

第4章 单片机的“触角”——I/O口

欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 4.1 解读AT89S51的I/O口
- 4.2 I/O口作输入端口使用——流水控制灯
- 4.3 七段数码管的控制——秒表
- 4.4 小键盘的控制
- 4.5 实例点拨——计时提醒器

4.1 解读AT89S51的I/O口

• 4.1.1 I/O口的整体印象

系统示意图	功 能	部 件	I/O功能
	电源	电源开关	输入
	启动/暂停	启动/暂停按钮	输入
	过程指示	指示灯	输出
	时间显示	七段数码管	输出
	程序控制	程序按钮	输入
	水位控制	水位按钮	输入
	注水控制	注水阀	输出
	称衣物重量	重量传感器	输入
	排水控制	排水阀	输出
	波轮旋转	电动机	输出

洗衣机与单片机的I/O控制

4.1 解读AT89S51的I/O口

• 4.1.1 I/O口的整体印象(89S51的I/O口)

P3口具有
双重功能

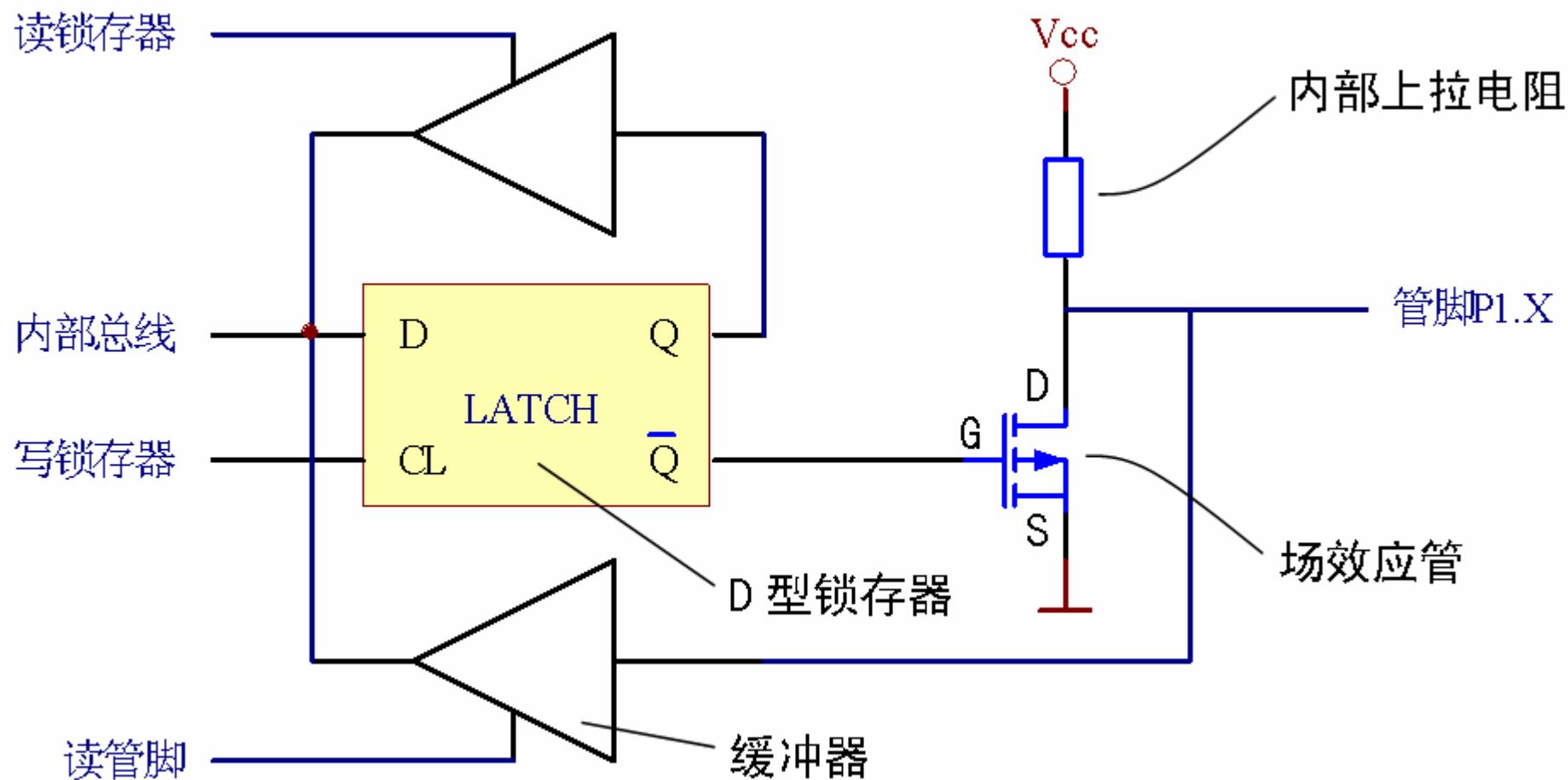
31	<u>EA/VPP</u>	Vcc	40
19	XTAL1		
		AD0/P0.0	39
		AD1/P0.1	38
18	XTAL2	AD2/P0.2	37
		AD3/P0.3	36
9	RST	AD4/P0.4	35
		AD5/P0.5	34
10	P3.0/RXD	AD6/P0.6	33
11	P3.1/TXD	AD7/P0.7	32
12	P3.2/INT0		
13	P3.3/INT1	A8/P2.0	21
14	P3.4/T0	A9/P2.1	22
15	P3.5/T1	A10/P2.2	23
		A11/P2.3	24
1	P1.0	A12/P2.4	25
2	P1.1	A13/P2.5	26
3	P1.2	A14/P2.6	27
4	P1.3	A15/P2.7	28
5	P1.4		
6	P1.5/MOSI	<u>PSEN</u>	29
7	P1.6/MISO	ALE/PROG	30
8	P1.7/SCK		
		P36/WR	16
20	GND	P37/RD	17

P0和P2口
访问外部存储器

P3口具有双重功能

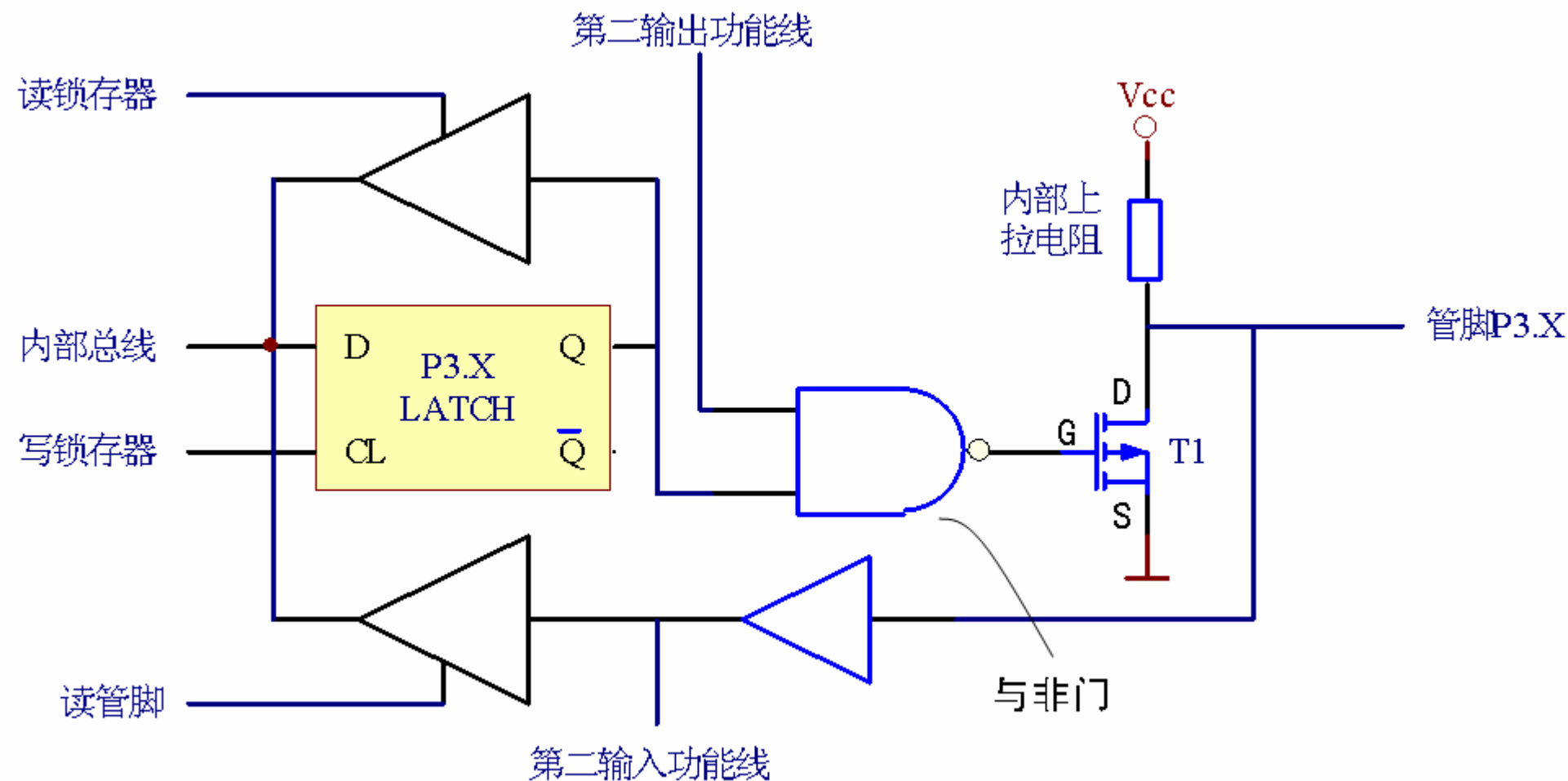
4.1 解读AT89S51的I/O口

• 4.1.2 深入观察I/O口的结构（P1口结构）



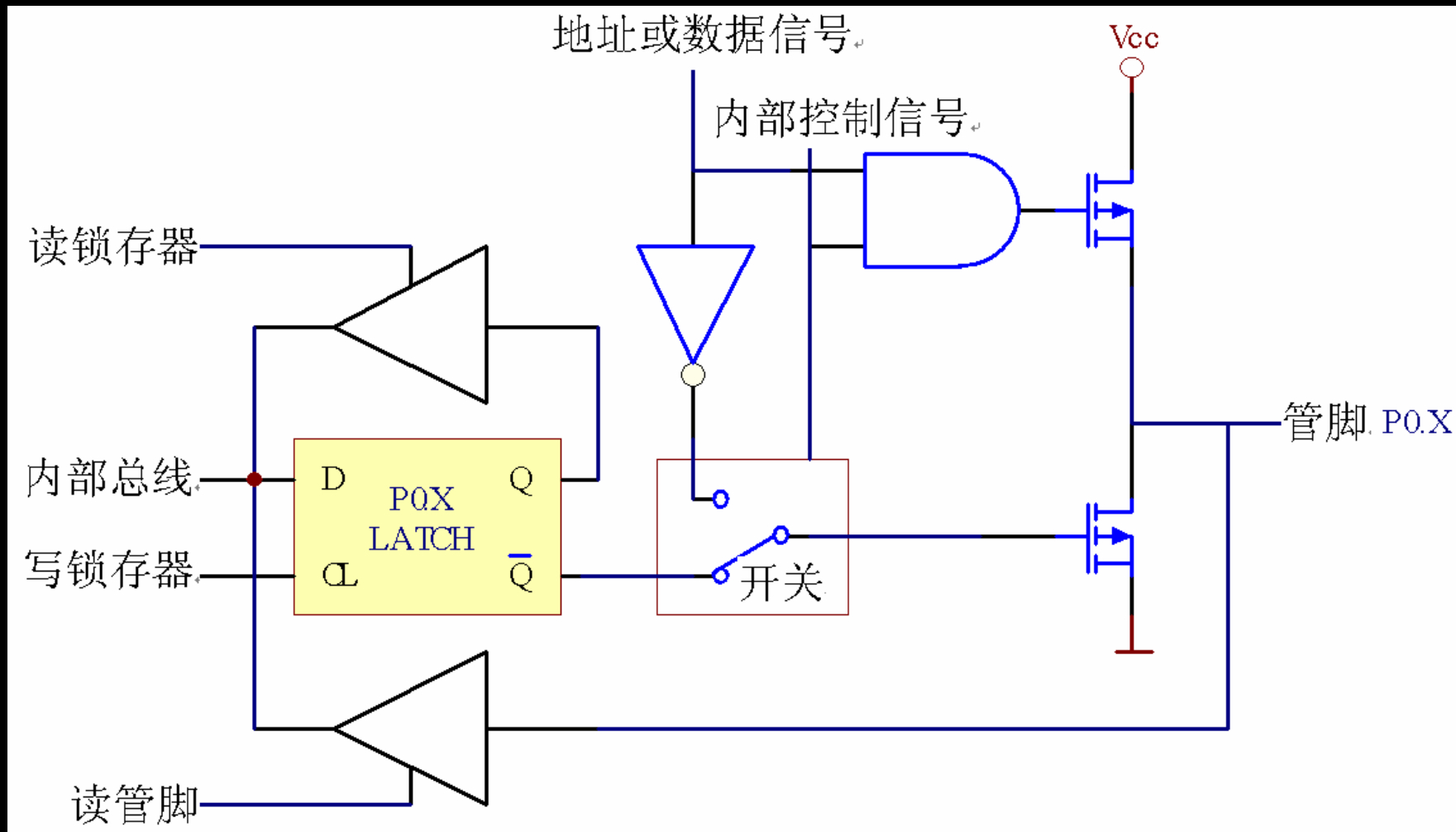
4.1 解读AT89S51的I/O口

• 4.1.2 深入观察I/O口的结构（P3口结构）



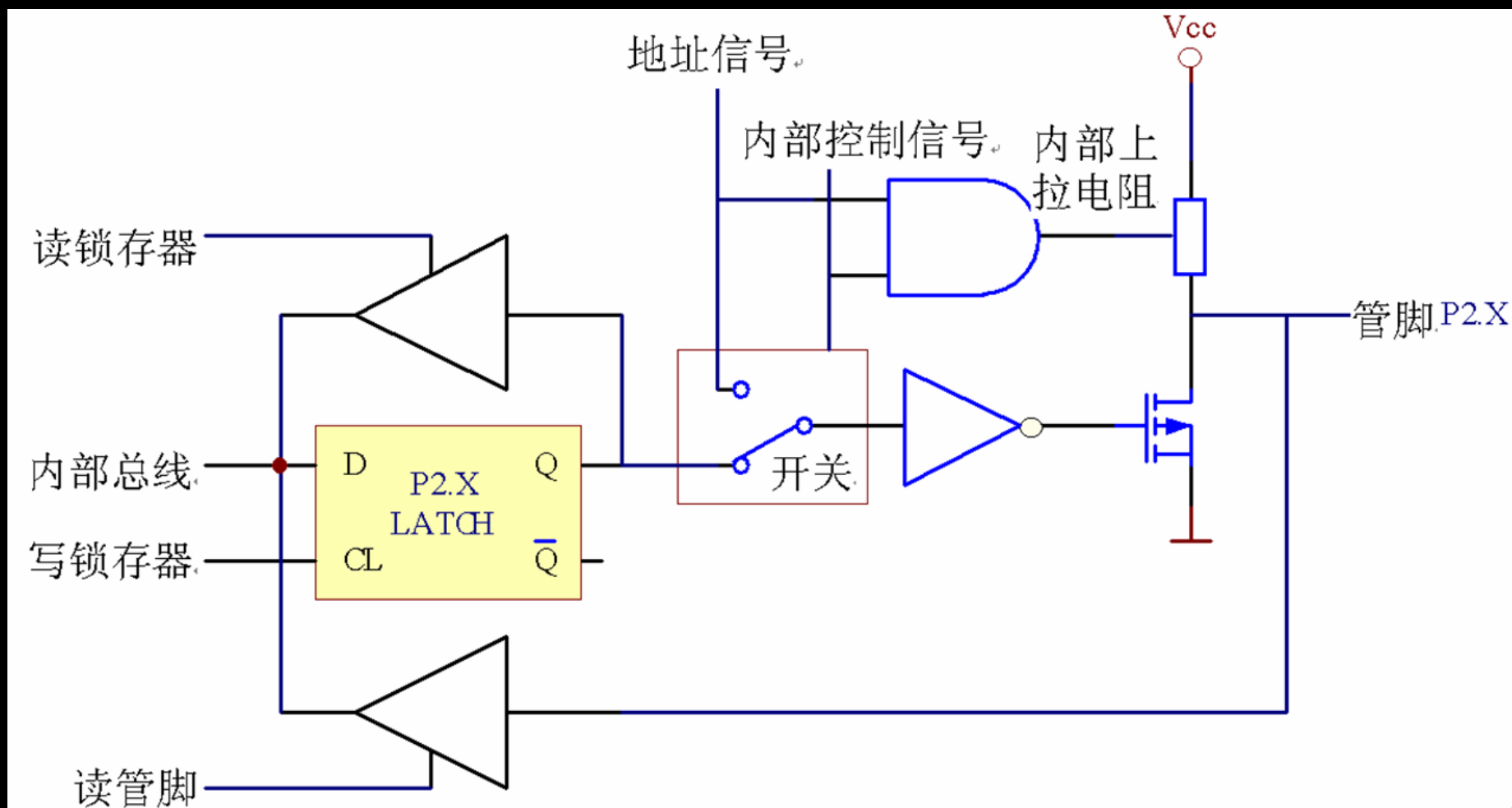
4.1 解读AT89S51的I/O口

• 4.1.2 深入观察I/O口的结构（P0口结构）



4.1 解读AT89S51的I/O口

• 4.1.2 深入观察I/O口的结构（P2口结构）



4.2 I/O口作输入端口使用

——流水控制灯

- 明确系统功能

8支发光二极管作为显示器件， 两个按钮开关A和B，

一开始， 8支发光二极管为全亮状态，

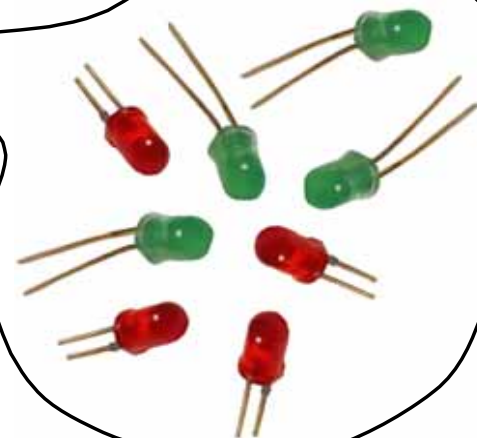
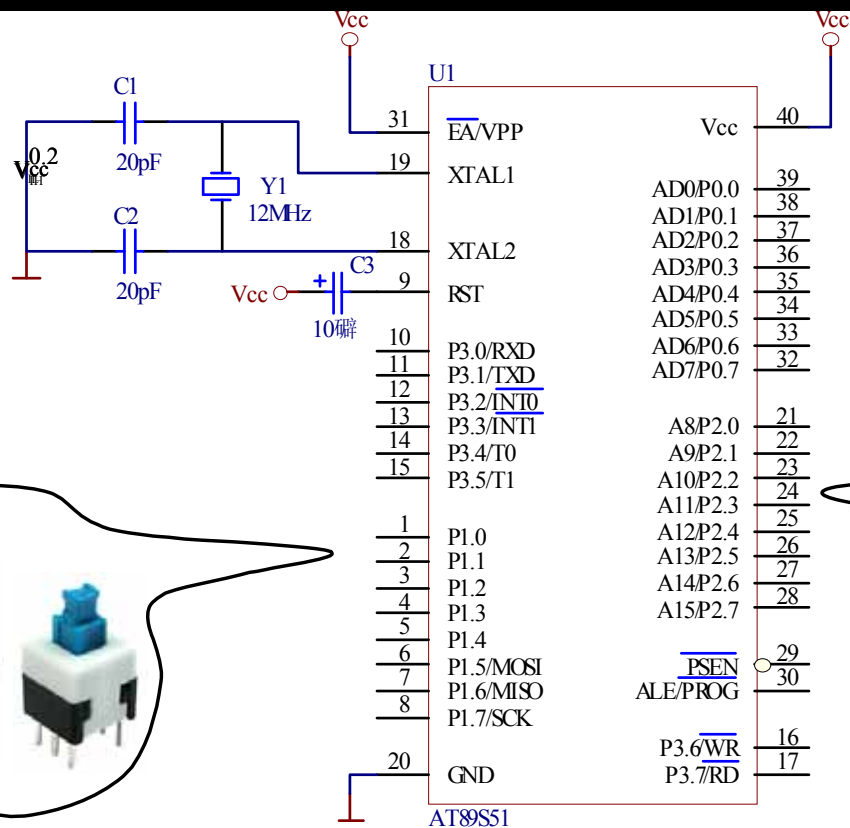
当按下按钮A时， 8支发光二极管做单一灯的从右向左流动，

当按下按钮B时， 8支发光二极管做单一灯的从左向右流动。

4.2 I/O口作输入端口使用

——流水控制灯

• 硬件设计

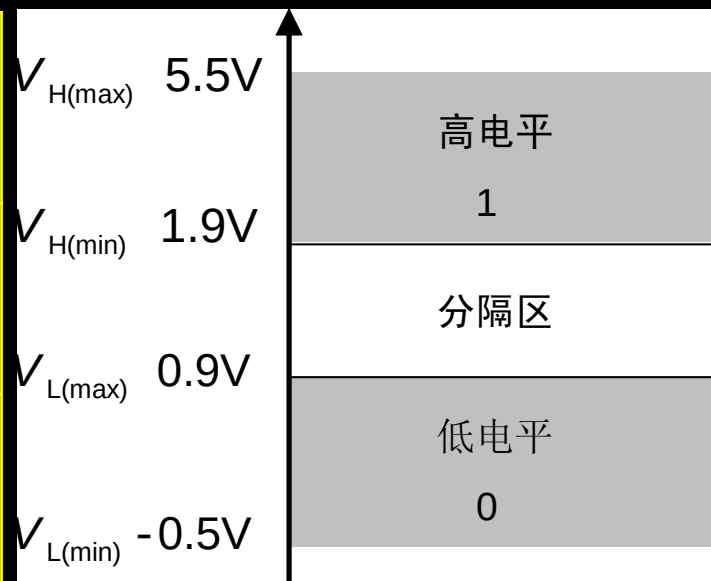


4.2 I/O口作输入端口使用

——流水控制灯

- 硬件设计 （高低电平）

表示符号	参数	最小值	最大值	单位
V_{IL}	输入低电平	-0.5	-0.1	V
V_{IH}	输入高电平	+0.9	+0.5	V

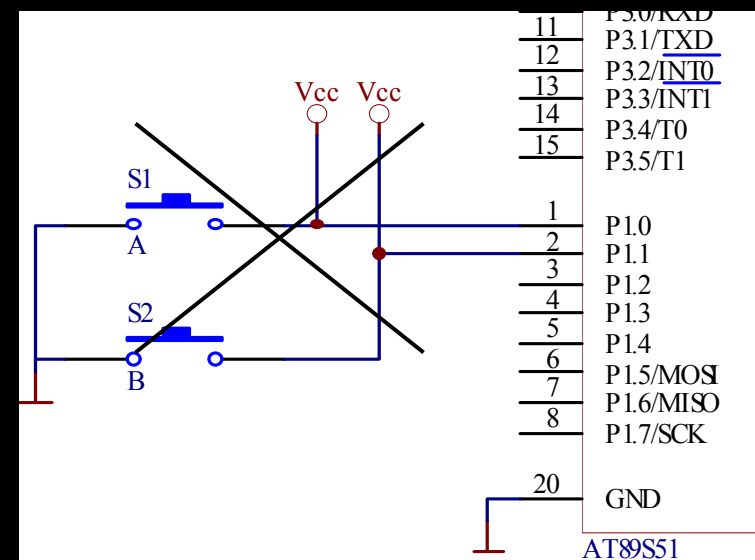
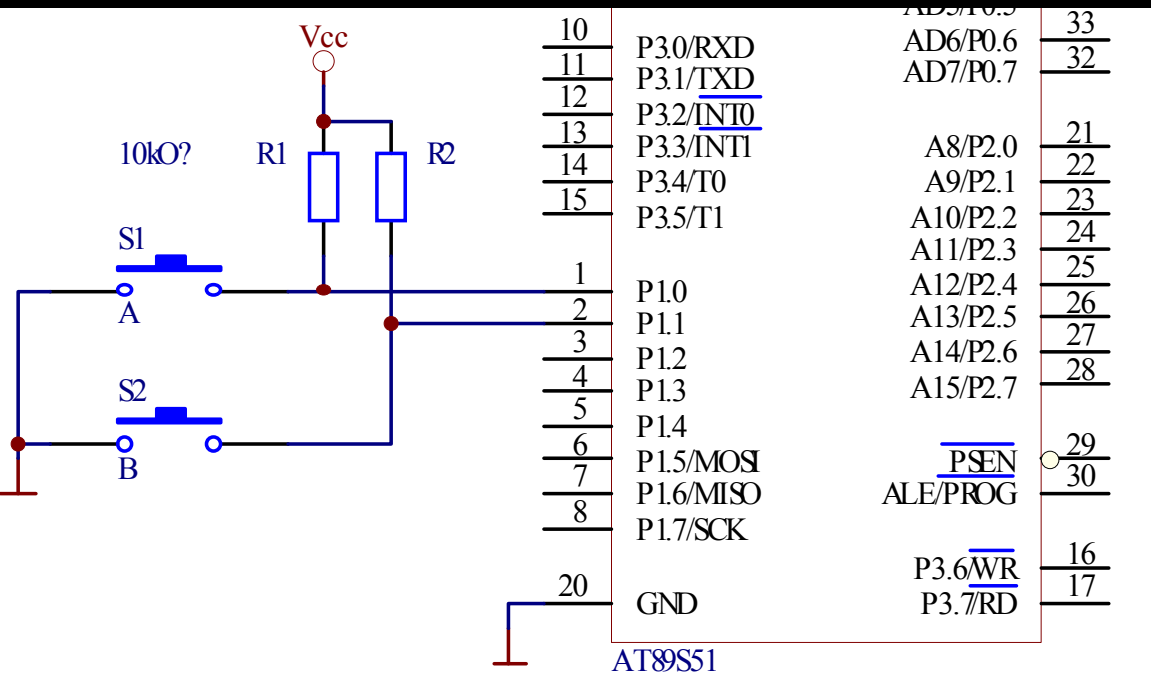


AT89S51单片机的逻辑电平描述

4.2 I/O口作输入端口使用

——流水控制灯

- 硬件设计（I/O口作为输入口）

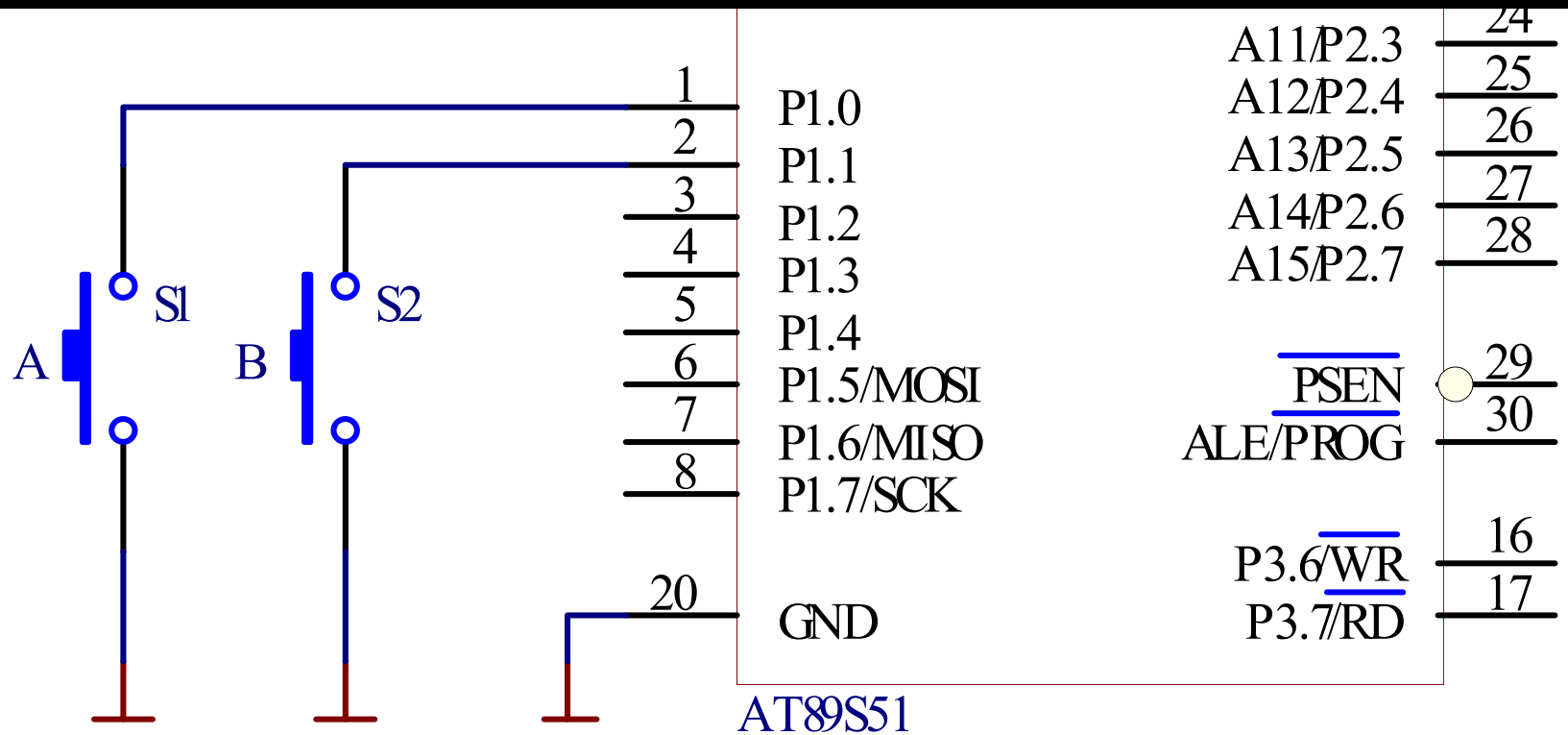


(a) 上拉电阻方案

4.2 I/O口作输入端口使用

——流水控制灯

- 硬件设计（I/O口作为输入口）

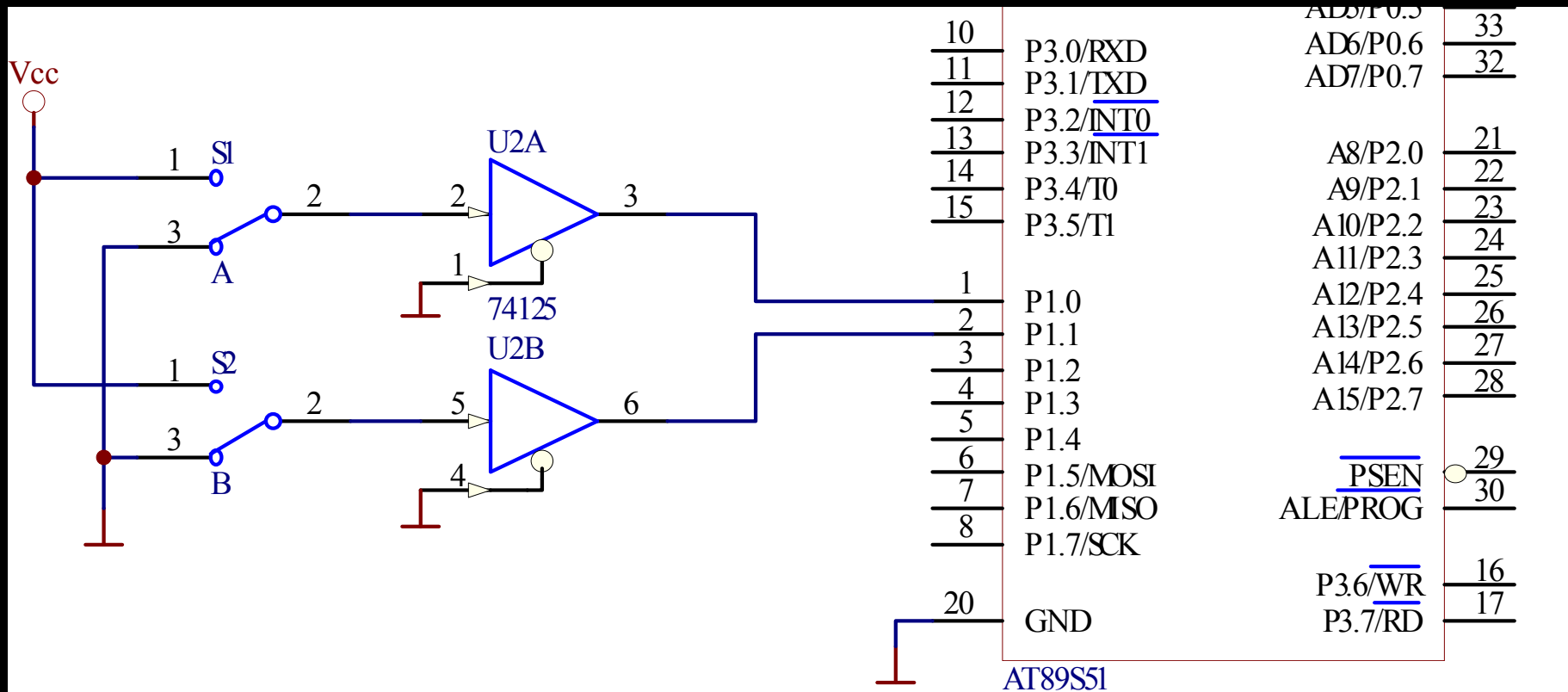


(b) 简易方案

4.2 I/O口作输入端口使用

——流水控制灯

- 硬件设计（I/O口作为输入口）

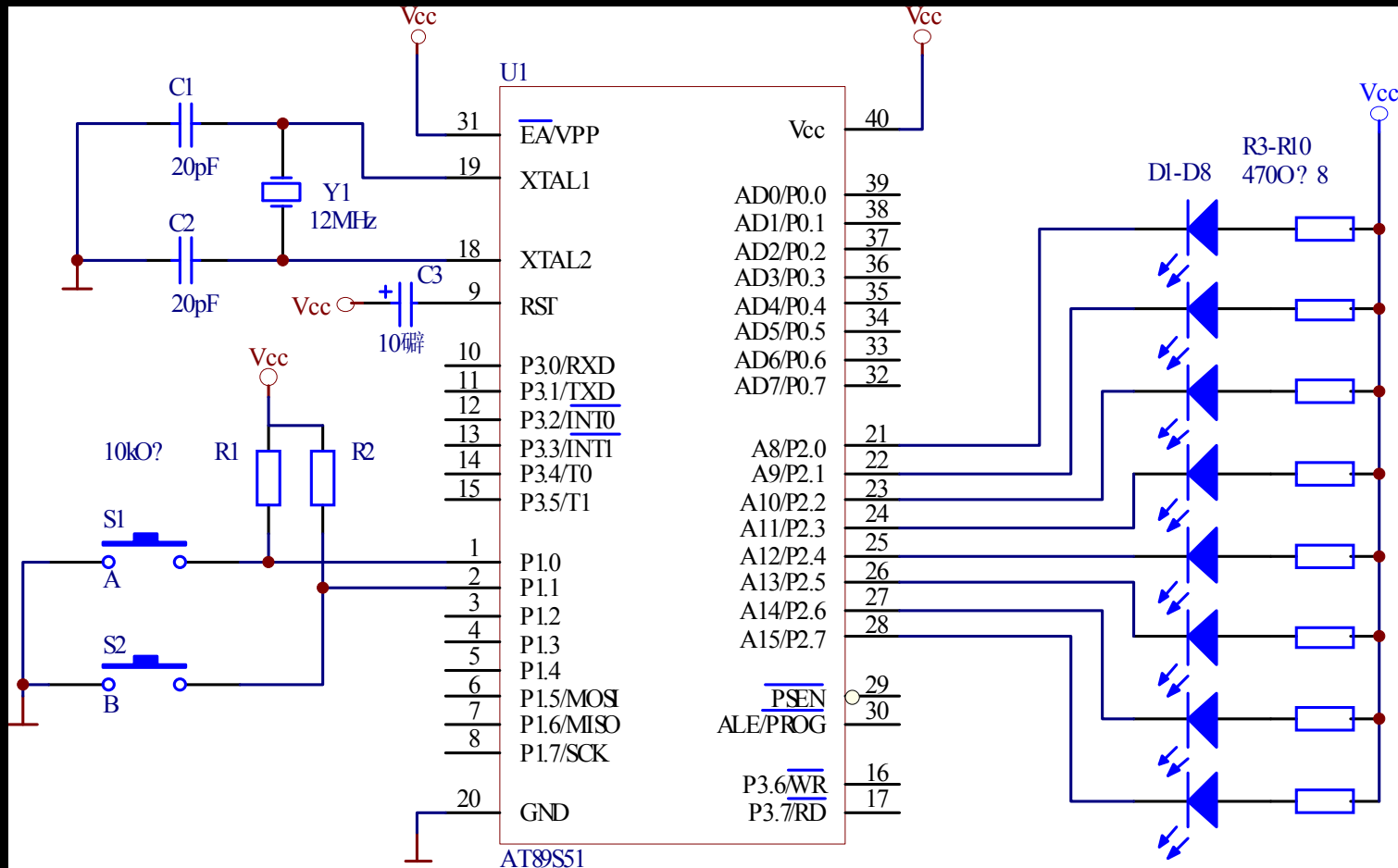


(c) 缓冲器方案

4.2 I/O口作输入端口使用

——流水控制灯

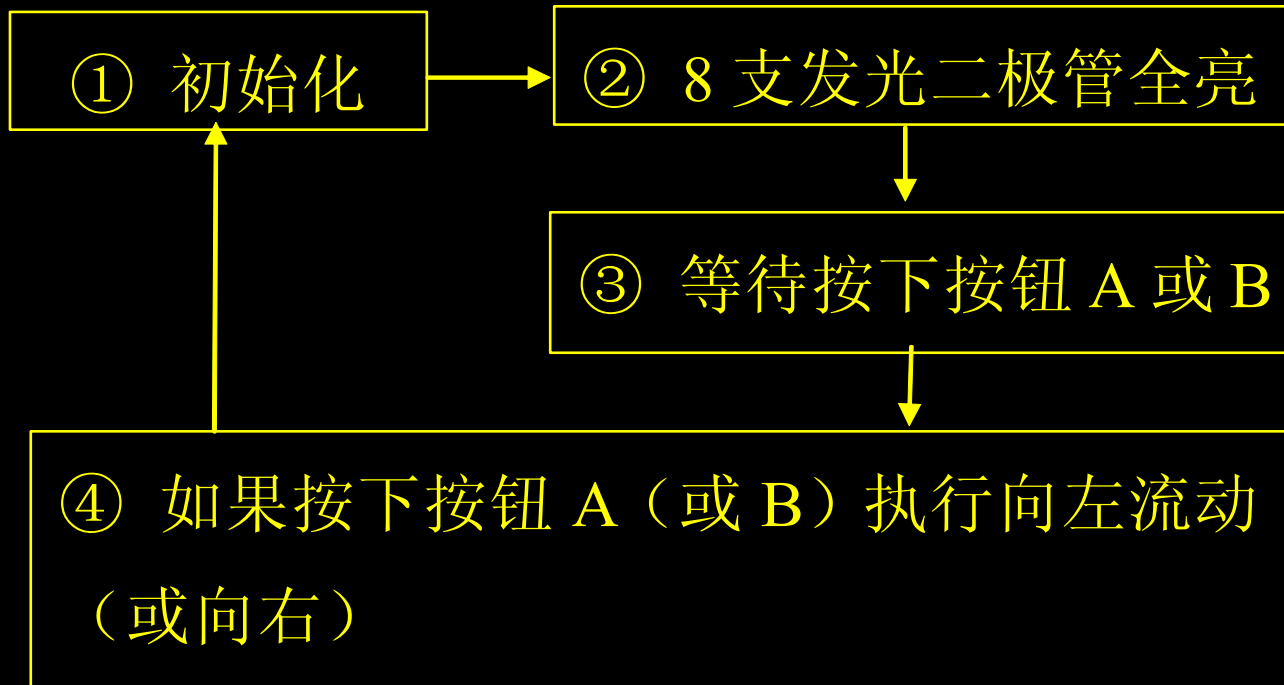
• 硬件设计（系统电路图）



4.2 I/O口作输入端口使用

——流水控制灯

- 搭建硬件平台
- 软件设计



程序设计思路

4.2 I/O口作输入端口使用

——流水控制灯

- 软件设计（指令简析“JB P1.0, WAIT_A”）

.....

WAIT_A:

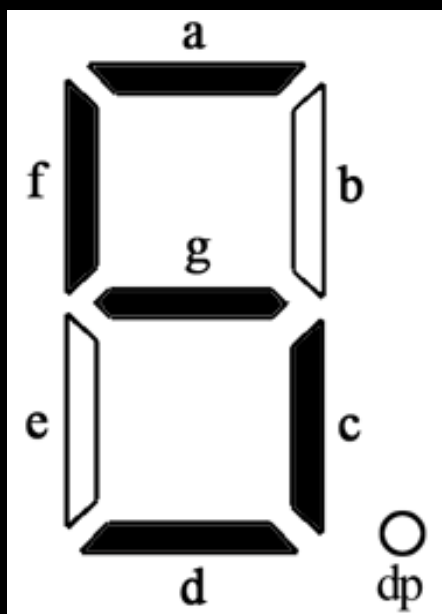
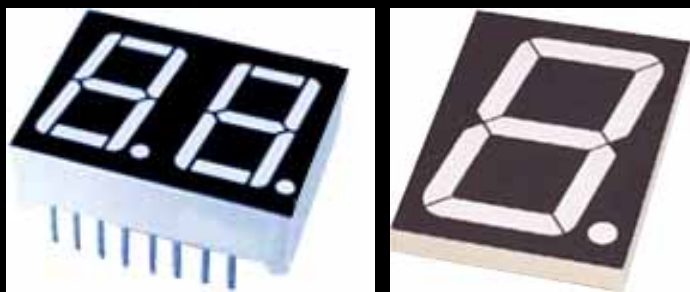
JB P1.0, WAIT_A ;如果按钮A没有按下,
;循环执行本行以继续判断

JMP LEFT

.....

4.3 七段数码管的控制——秒表

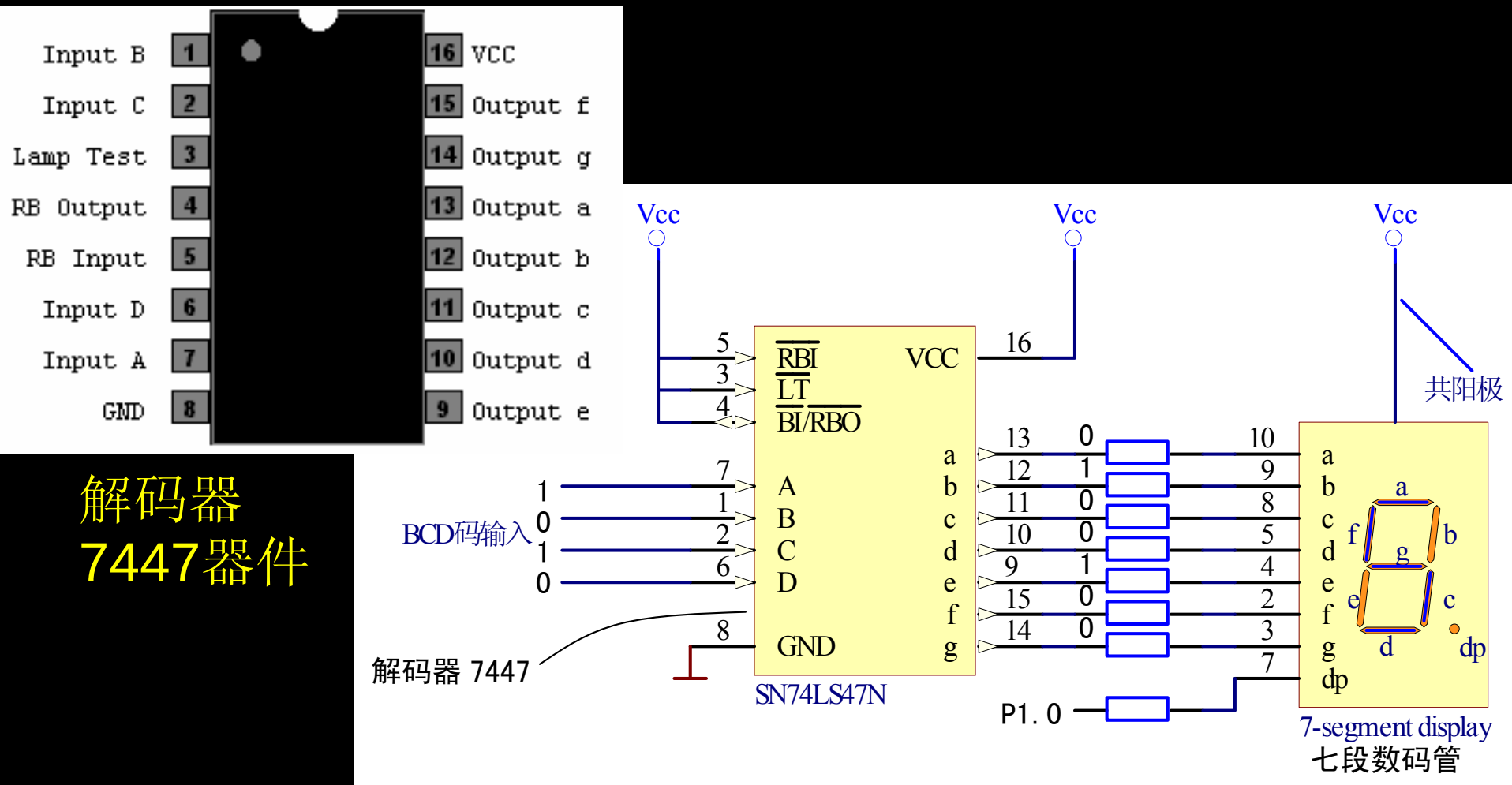
• 七段数码管介绍



数 字	亮 段
0	a,b,c,d,e,f
1	b,c
2	a,b,d,e,g
3	a,b,c,d,g
4	b,c,f,g
5	a,c,d,f,g
6	a,c,d,e,f,g
7	a,b,c
8	a,b,c,d,e,f,g
9	a,b,c,d,f,g

4.3 七段数码管的控制——秒表

• 七段数码管介绍 (解码器7447)



4.3 七段数码管的控制——秒表

• 七段数码管介绍 (解码器7447)

显 示	BCD码输入端				亮段控制输出端						
	D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	1	0	0	1	1	1	1
2	0	0	1	0	0	0	1	0	0	1	0
3	0	0	1	1	0	0	0	0	1	1	0
4	0	1	0	0	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0	1	0	0
6	0	1	1	0	0	1	0	0	0	0	0
7	0	1	1	1	0	0	0	1	1	1	1
8	1	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	0	0	1	0	0

4.3 七段数码管的控制——秒表

- 明确系统功能

两位七段数码管在开机时显示“00”，

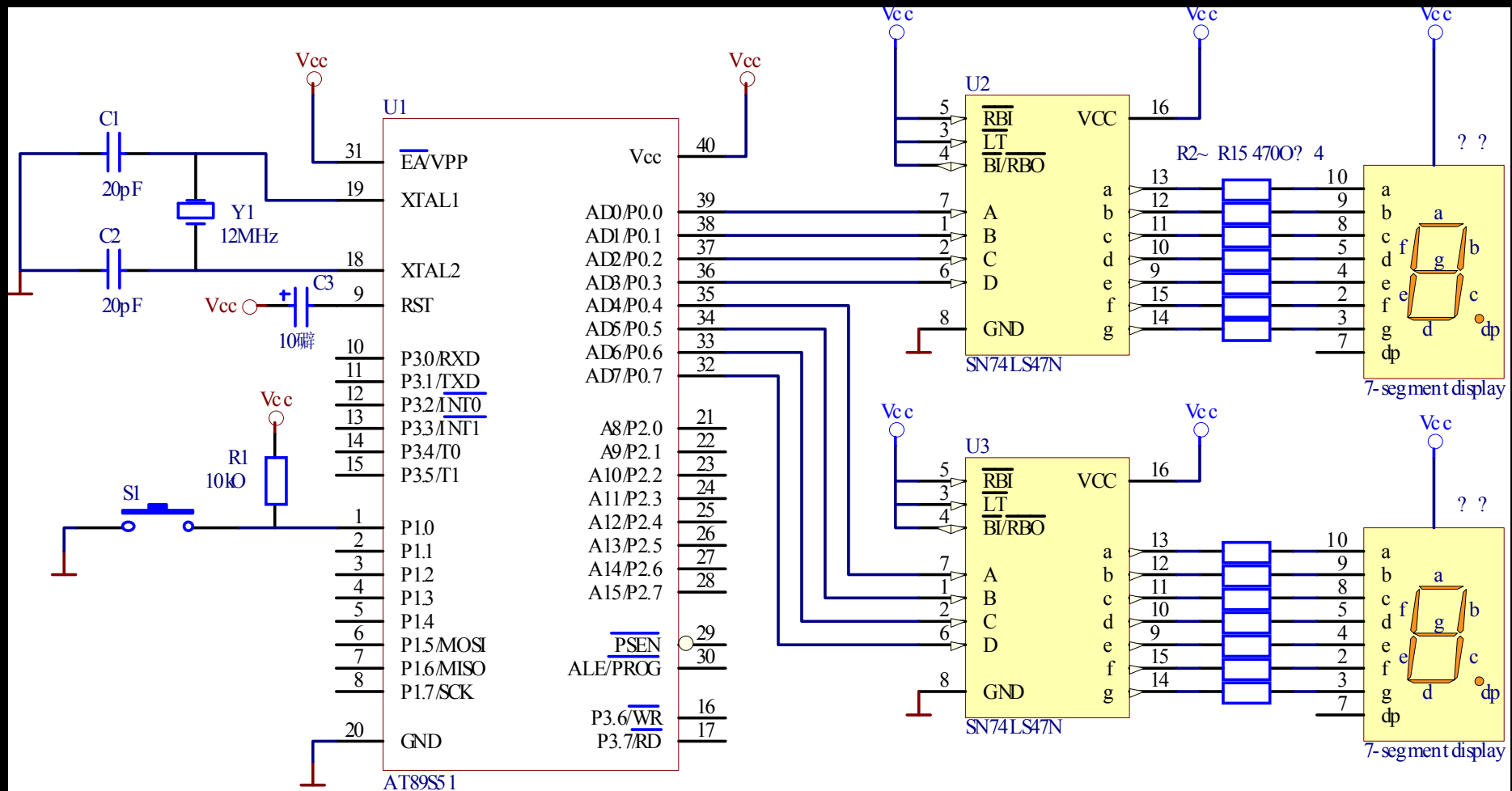
第1次按下按钮开关后秒表开始计时，

第2次按下后计时停止，

第3次按下后两个数码管清0，并回到一开始的计时状态。

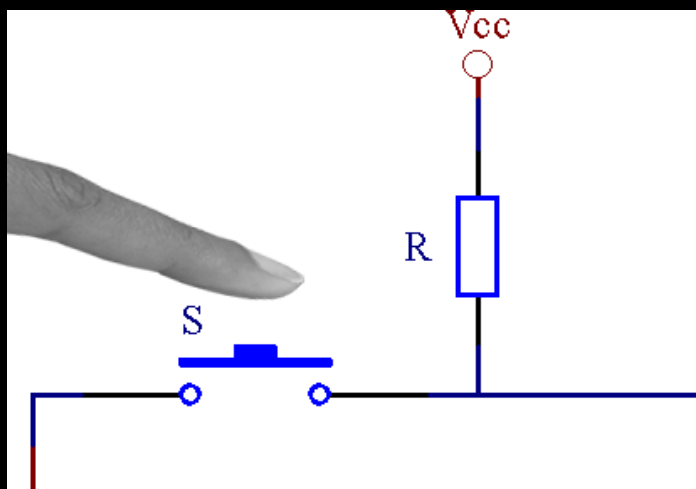
4.3 七段数码管的控制——秒表

• 硬件设计(系统电路图)

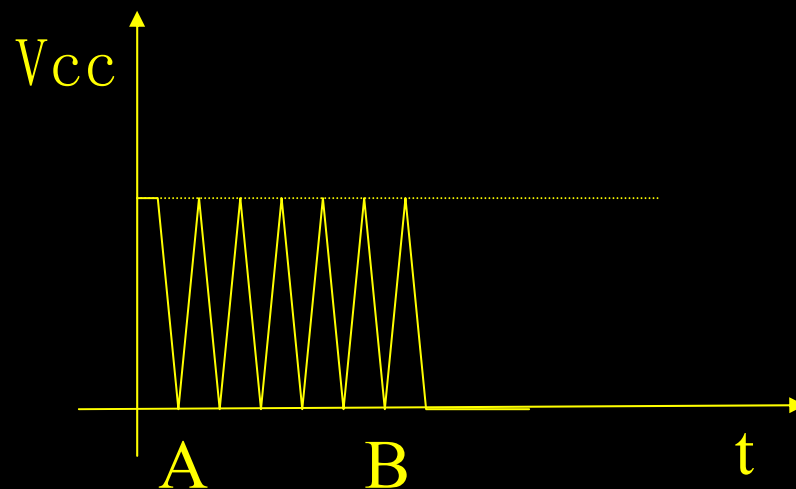


4.3 七段数码管的控制——秒表

- 软件设计前奏 (解决开关抖动)



△按下开关时手指的抖动

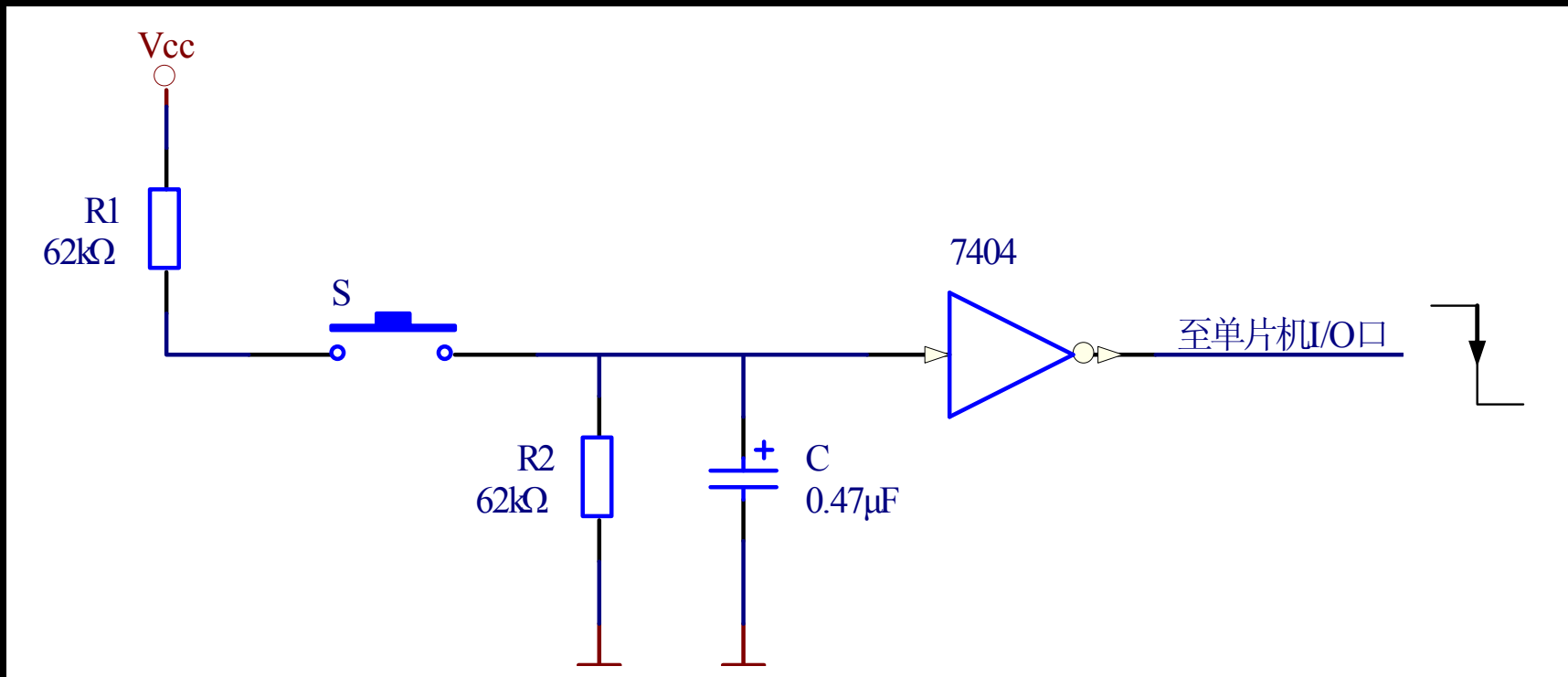


△输出端的电平变化

解决开关抖动的办法： 硬件的方法 软件的方法

4.3 七段数码管的控制——秒表

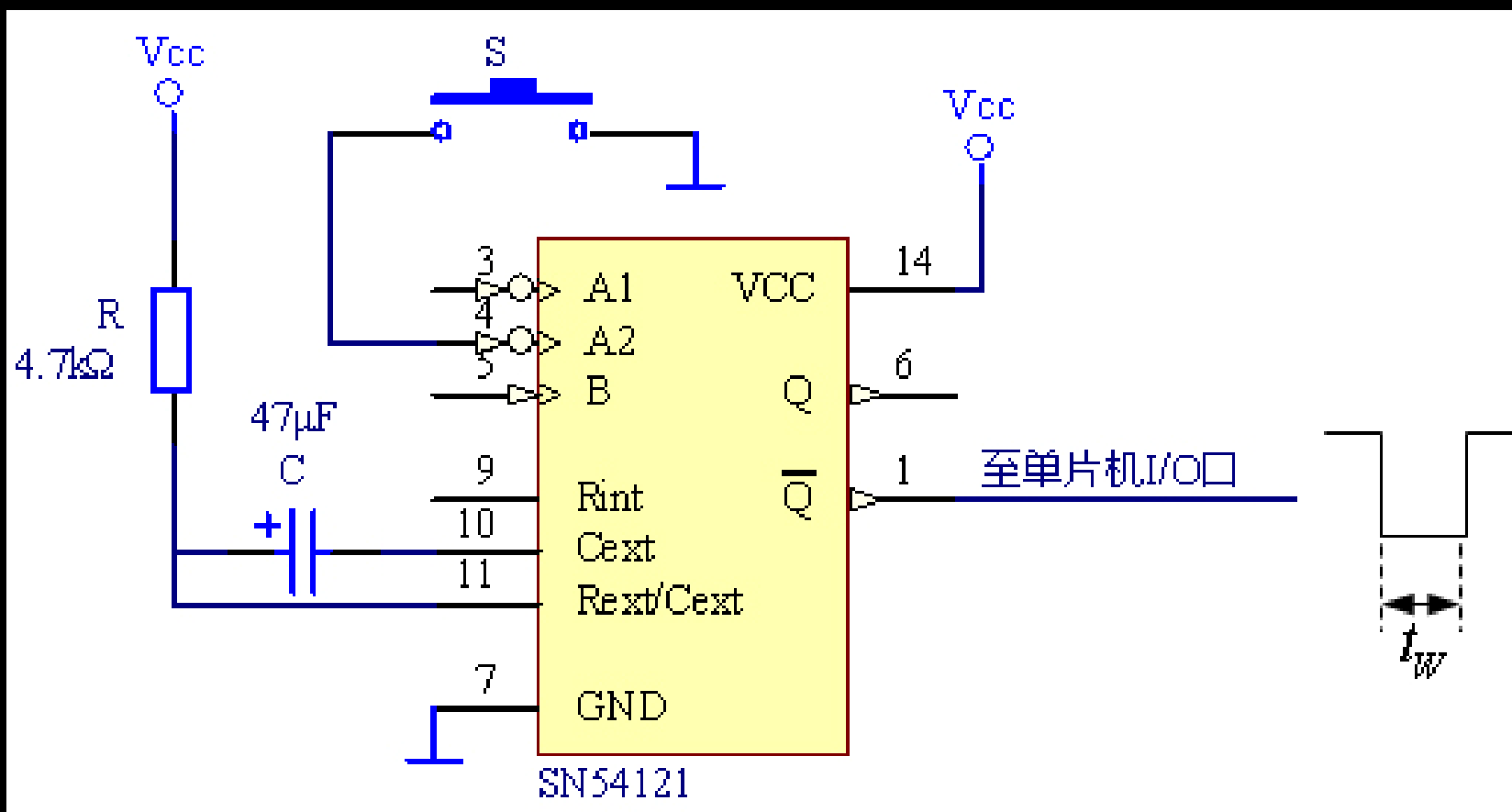
- 软件设计前奏 (硬件的方法解决开关抖动)



门控滤波电路

4.3 七段数码管的控制——秒表

- 软件设计前奏 (硬件的方法解决开关抖动)



振荡器滤波电路

4.3 七段数码管的控制——秒表

- 软件设计前奏 (软件的方法解决开关抖动)

JB P1.0, \$;如果P1.0为高电平, 就重复执行本行

CALL FILTER ;调消除开关抖动的子程序

JNB P1.0, \$;如果P1.0为低电平, 就重复执行本行

注释:

“CALL FILTER ”调用消除开关抖动的延时子程序

4.3 七段数码管的控制——秒表

- 软件设计

启动时，显示00和等待按钮按下

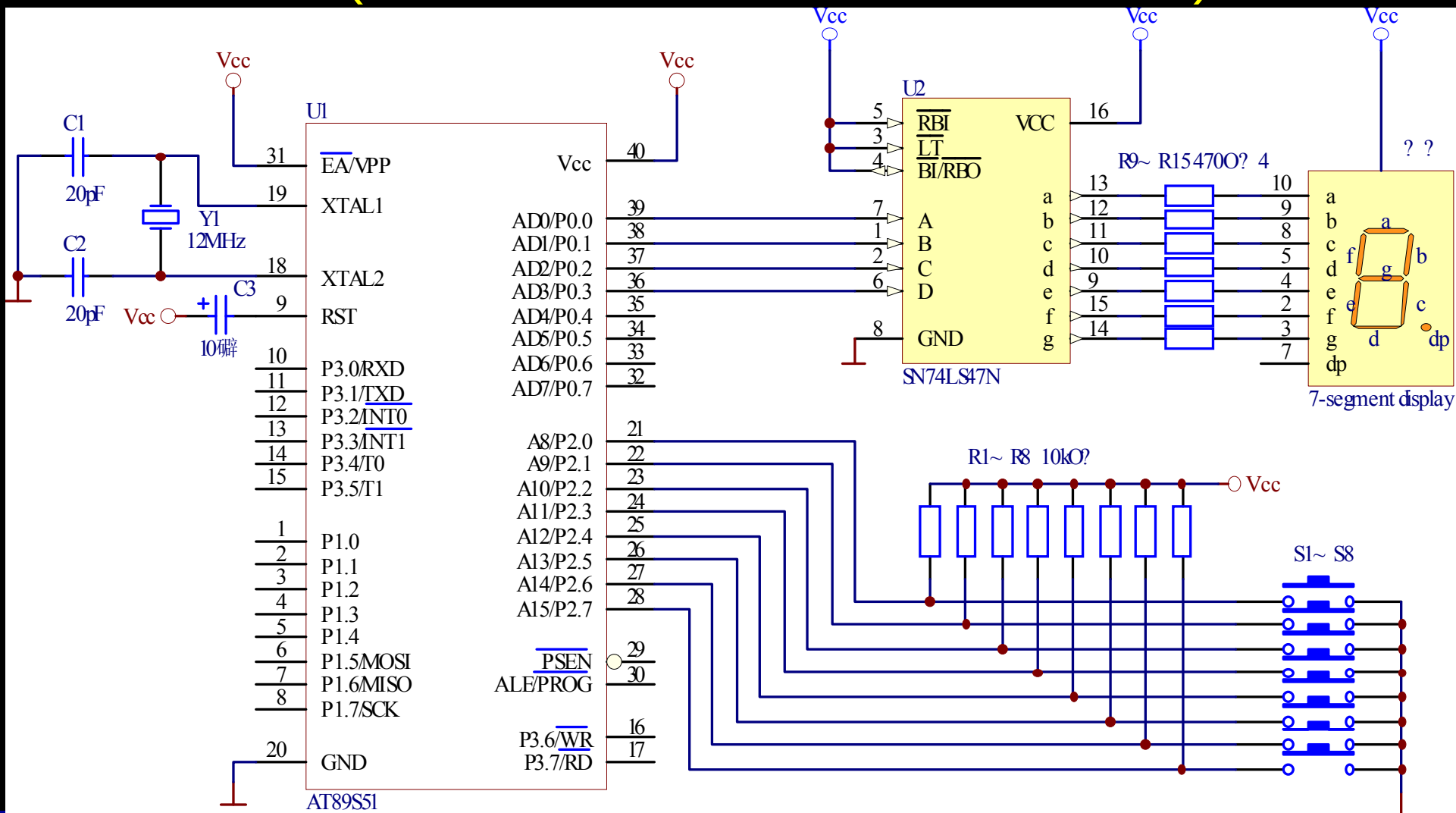
秒表计时显示和1秒延时

处理第2次、第3次按下按钮的处理

延时程序

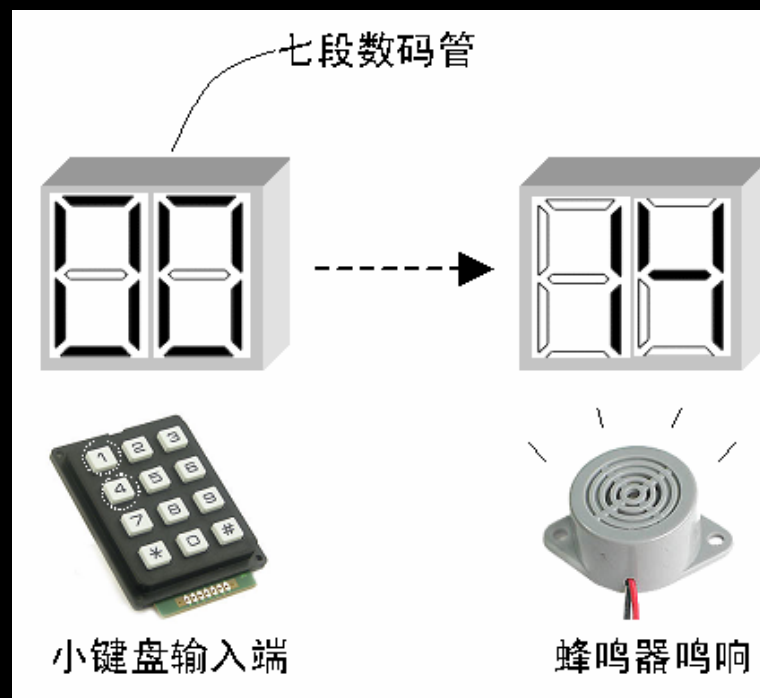
4.4 小键盘的控制

- 上拉电阻(避免与Vcc直连, 开关断路无电流)



4.5 实例点拨——计时提醒器

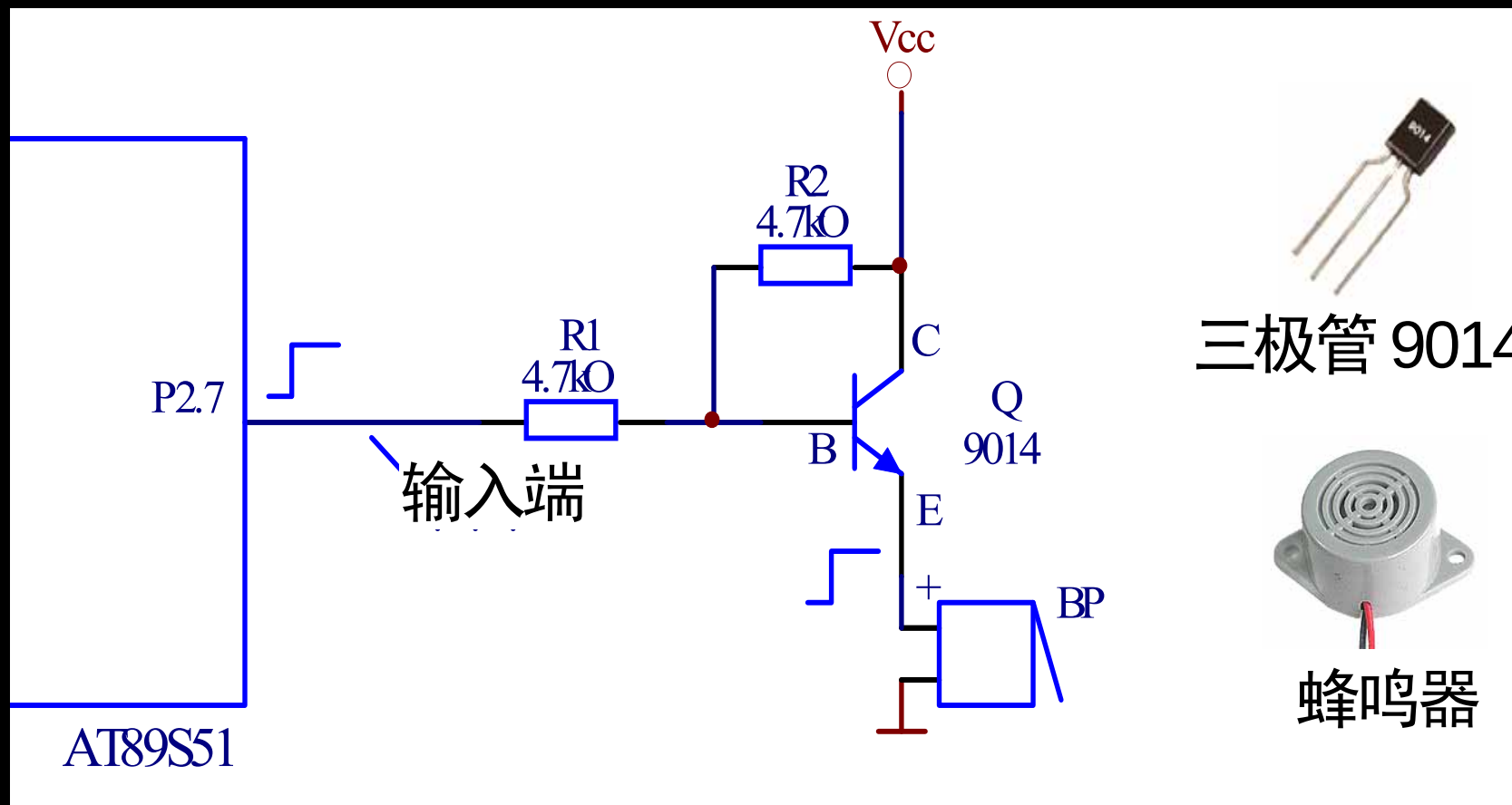
- 明确系统功能
 - ✓ 开机时，七段数码管显示“00”，等待输入计时时间，
 - ✓ 输入有误，按“#”键取消重新输入，
 - ✓ 按“*”键则确认，计时提醒器开始工作，
 - ✓ 计时完时，蜂鸣器发出“嘀、嘀……”的提示音，
 - ✓ 在计时过程中，如果按下“#”键则取消计时，系统回到开机时的状态。



4.5 实例点拨——计时提醒器

- 硬件设计

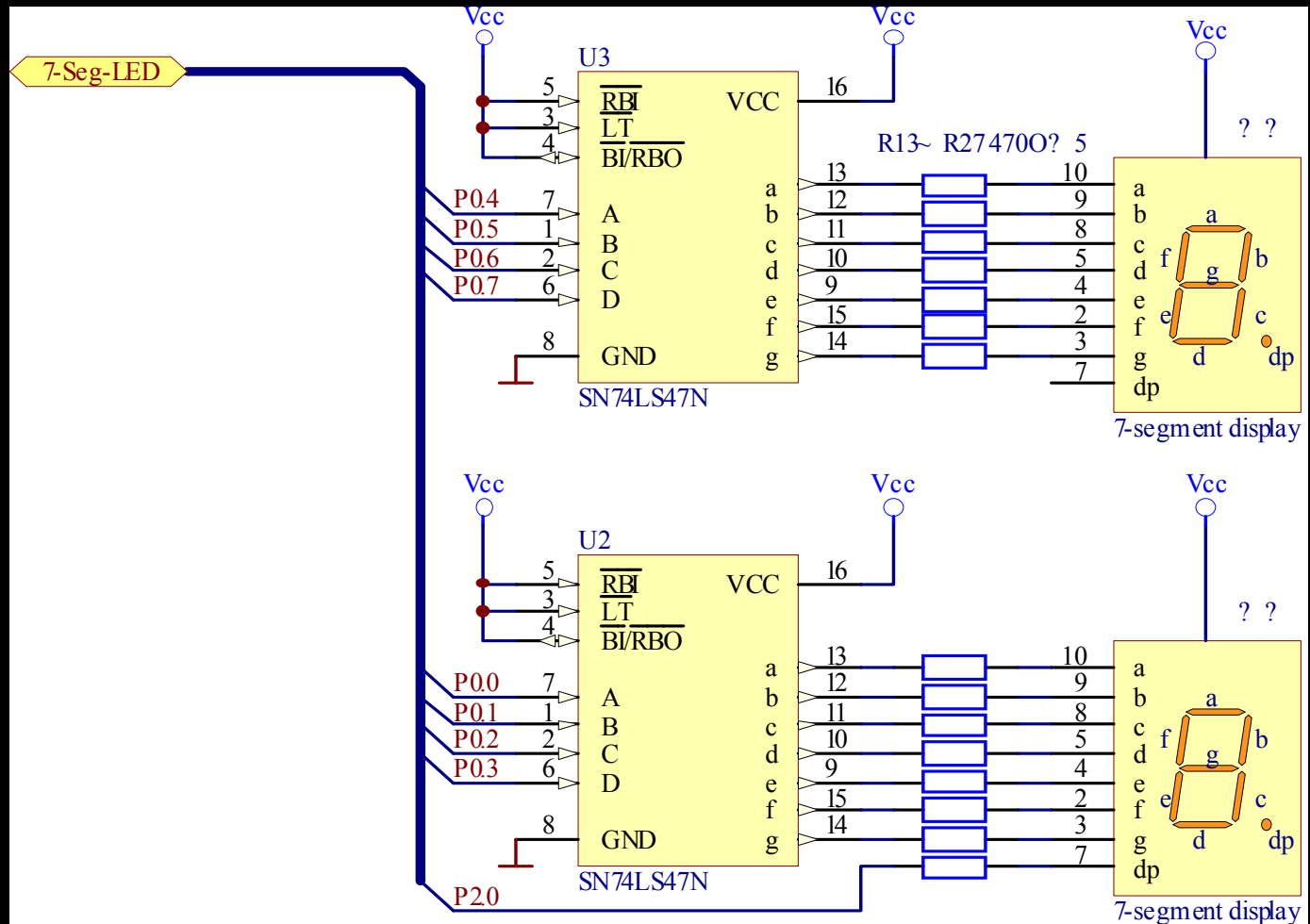
12个按键，两位七段数码管，蜂鸣器和单片机最简系统



4.5 实例点拨——计时提醒器

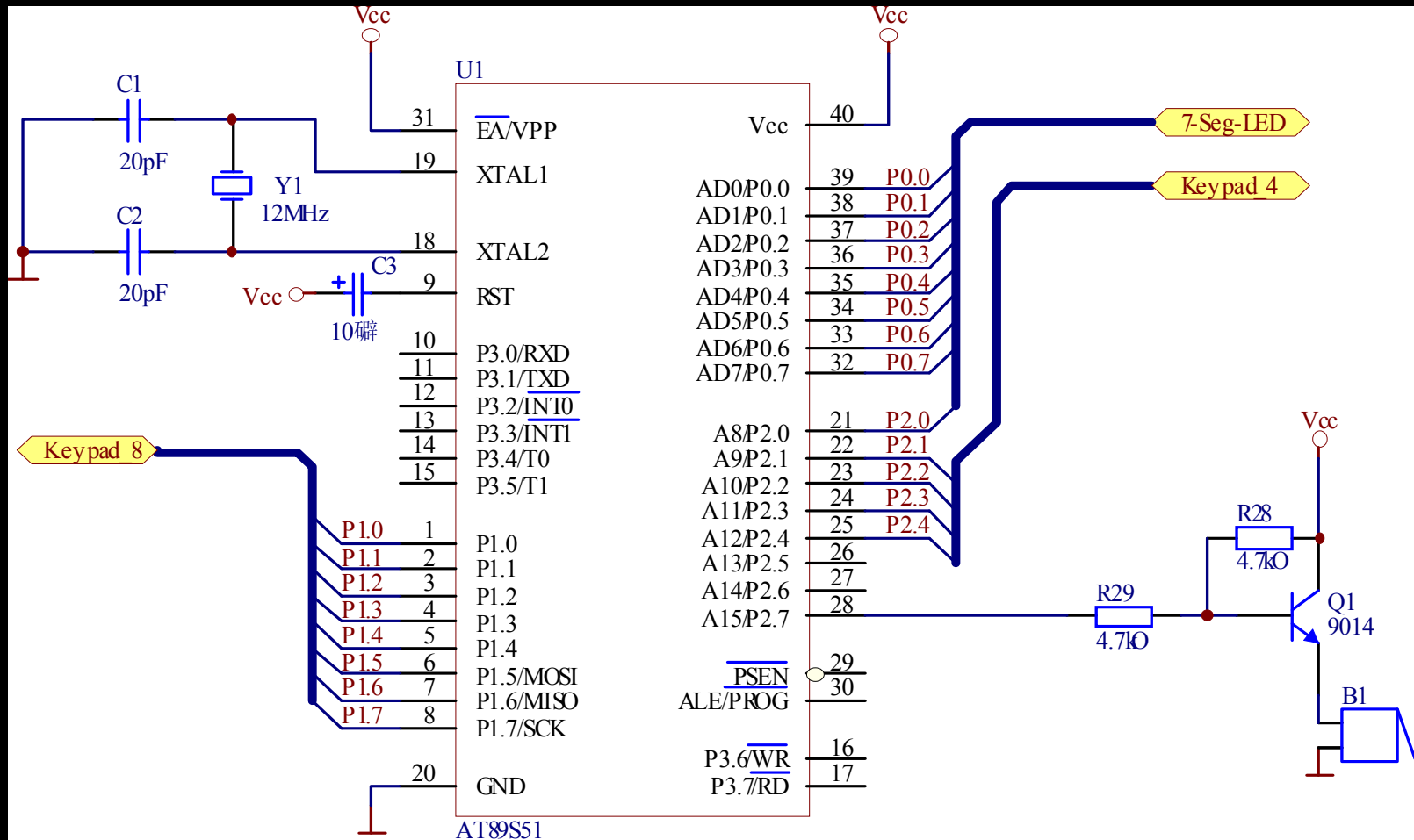
• 硬件设计

两位七
段数码
管和解
码器
7447



4.5 实例点拨——计时提醒器

• 硬件设计



最简系统和蜂鸣器

4.5 实例点拨——计时提醒器

- 软件设计（SWAP指令介绍）

MOV A, #11110000B

；向累加器ACC中载入二进制数11110000，后缀B
代表二进制数

SWAP A

；累加器ACC中的低4位与高4位对调，
(A) = 0000 1111

4.5 实例点拨——计时提醒器

• 软件设计（好习惯1/2）

✓ 阅读程序的好习惯

.....

MOV R1, #0F2H

MOV A, R1

SWAP A

ADD A, #01H

MOV P0, A

.....

.....

MOV R1, #0F2H → R1 = 0F2H

MOV A, R1 → A = R1 = 0F2H

SWAP A → A = 2FH

ADD A, #01H → A = 2F + 01 = 30H

MOV P0, A → P0 = A = 30H

.....

在纸上用笔记录每一行
指令产生的影响和结果

✓ 程序中的标号应该尽量起得有实际、具体、确切的含义，这样能提高程序的书写、编辑、调试的效率。

4.5 实例点拨——计时提醒器

- 软件设计（好习惯2/2）
- ✓ 一般来说，标号写在最左边，最好给标号单独占一行。在它的下面接着写程序。每一行指令的起始点，也就是汇编指令的助记符应该比标号偏右一些，以利于区别标号和指令。

```

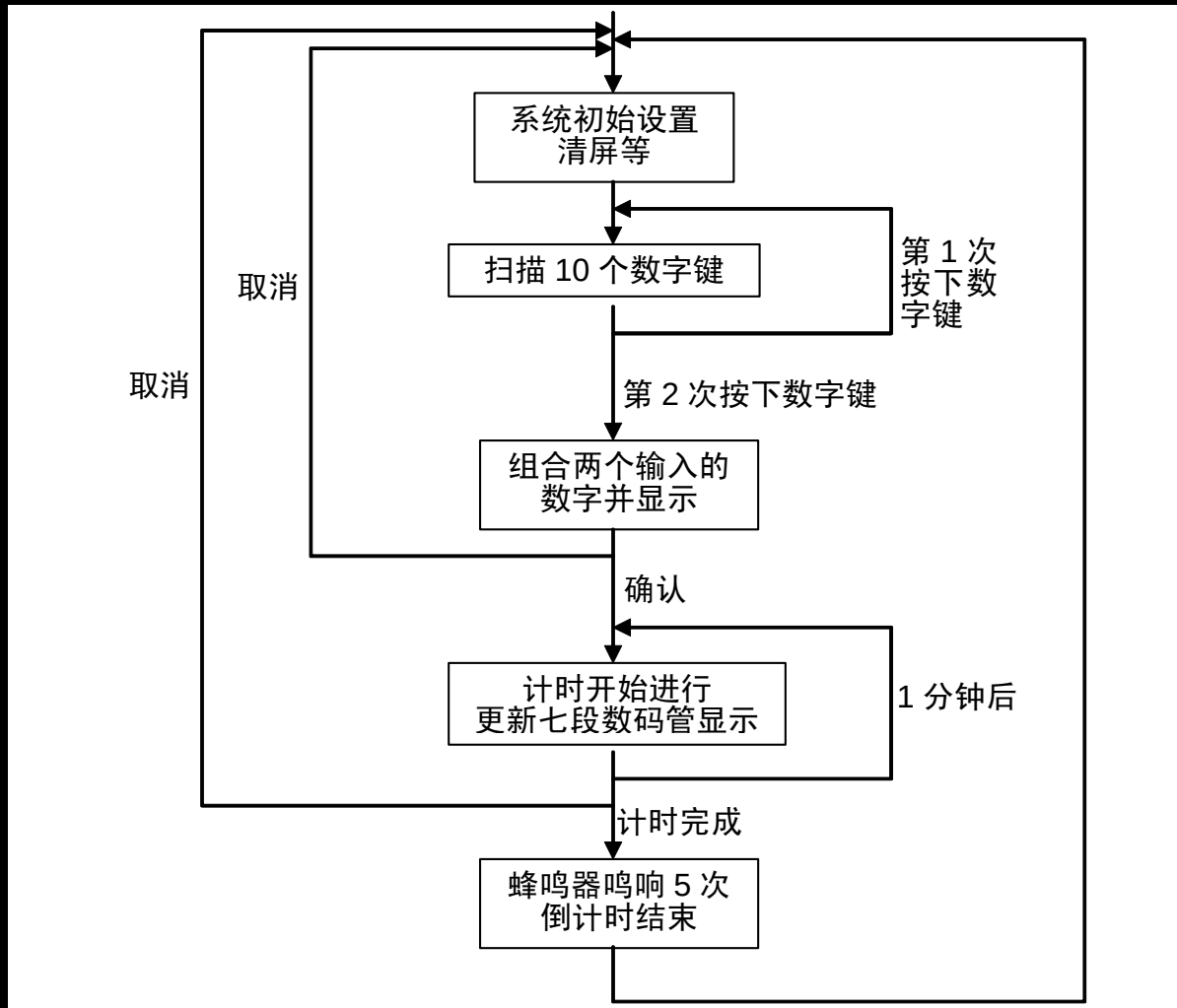
ORG      00H
START:   MOV      P0,#00H
          MOV      R0,#2
          CLR      C
          CLR      P2.7
SETTING:  CHK_0:
          JB       P1.0,CHK_1
          CALL     FILTER
          JNB      P1.0,$
          MOV      R1,#0
          JMP      TIMERSET
  
```

```

                                ORG      00H
START:
                                MOV      P0,#00H
                                MOV      R0,#2
                                CLR      C
                                CLR      P2.7
SETTING:
                                CHK_0:
                                JB       P1.0,CHK_1
                                CALL     FILTER
                                JNB      P1.0,$
                                MOV      R1,#0
                                JMP      TIMERSET
  
```

4.5 实例点拨——计时提醒器

- 软件设计（程序流程图）



4.5 实例点拨——计时提醒器

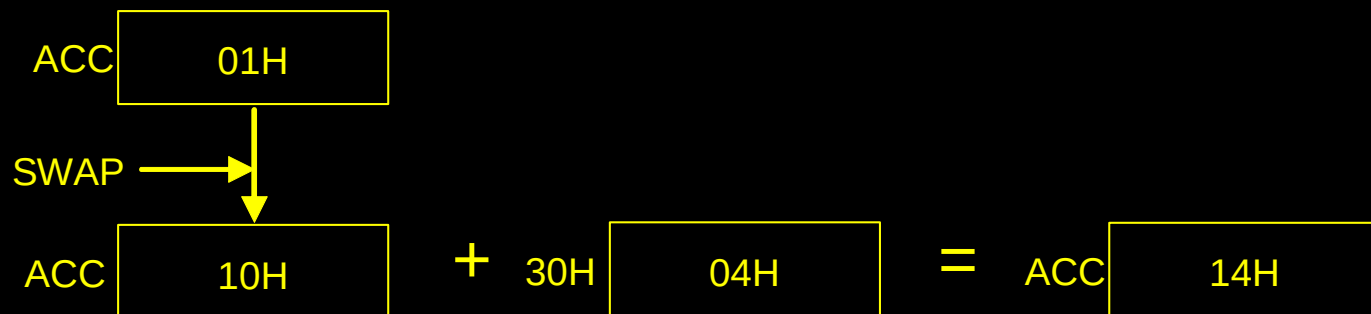
- 软件设计（程序解析）
 1. 计时提醒器接受键盘输入

CHK_0:

JB	P1.0,CHK_1	;判断按钮按下的JB
CALL	FILTER	;消除抖动子程序的CALL
JNB	P1.0, \$	;判断按钮松开的JNB
MOV	R1, #0	
JMP	TIMERSET	

4.5 实例点拨——计时提醒器

2. 显示和存储所设定的时间



```

TIMERSET:      ;这是显示和存储所设定的时间的程序段
               DJNZ  R0, LSB      ; 如果 R0 不等于 0, 说明按键设置的是时间的低位
               JMP    MSB
LSB:           MOV    30H, R1      ; 把低位时间值存储到地址 30H 中
               JMP    SETTING     ; 跳回扫描数字键程序段, 继续扫描高位时间的输入
MSB:           MOV    A, R1        ; 把高位时间载入累加器 ACC 中
               SWAP   A            ; 累加器 ACC 的高 4 位与低 4 位互换
               ADD    A, 30H       ; 累加器 ACC 与地址 30H 中的数据相加, 得到两位时间值
               MOV    31H, A       ; 把设定好的时间保存到 31H 中
               MOV    P0, A        ; 把设定好的时间输出到七段数码管显示
  
```

4.5 实例点拨——计时提醒器

3. *号键确认，或“#”号键取消

CONFIRM: ;这是确认设置时间的程序段

OK:

JB P2.3,CANCEL ; 判断是否按下确认键"*"

CALL FILTER

JNB P2.3,\$

MOV A, #00H ; 计时初始值载入 ACC 中

JMP TIMING ; 如果确认，就跳到 TIMING 程序段开始计时

CANCEL:

JB P2.4,OK

CALL FILTER

JNB P2.4,\$

RESTART:

JMP START ; 如果按取消键"#", 则跳回一开始重新扫描设置时间

4.5 实例点拨——计时提醒器

4. 正常的计时阶段

TIMING:

```
        MOV        P0, A            ; 更新显示时间
MIN:
        MOV        R0, #60          ; 60 个 1 秒为 1 分钟, R0 作为分钟计数
SEC:
        MOV        R1, #10          ; 延时 1 秒
SEC_10:
        MOV        R2, #200         ; 延时 1/10 秒
        CALL       DELAY
        DJNZ       R1, SEC_10
        DJNZ       R0, SEC
        ADD        A, #1             ; 计时值加 1
        DA         A                ; 十进制调整
        CJNE       A, 31H, TIMING    ; 如果计时值不等于 31H 保存的设置值,
                                     就继续计时
```

4.5 实例点拨——计时提醒器

5. 蜂鸣器的“嘀、嘀……”提示音

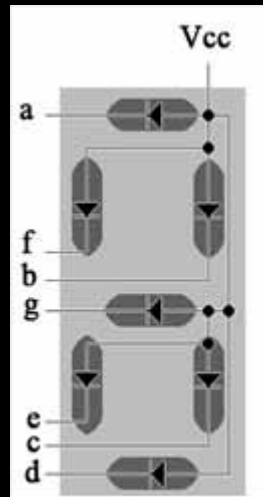
BEEP:

```
        MOV        R2, #5           ; 设置蜂鸣器鸣响 5 次
REPEAT:
        SETB       P2.7             ; 把 P2.7 脚置高, 蜂鸣器鸣响
        CALL        LONGDELAY
        CLR        P2.7             ; 把 P2.7 脚置低, 蜂鸣器关闭
        CALL        LONGDELAY
        DJNZ       R2, REPEAT
        JMP        START            ; 完成后跳回一开始扫描数字键
```

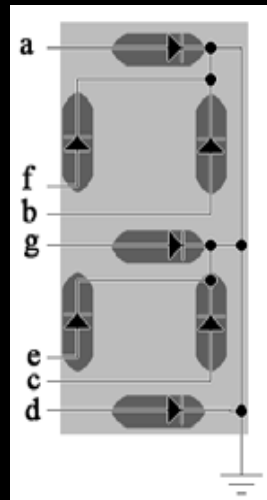

器件介绍

1. 七段数码管

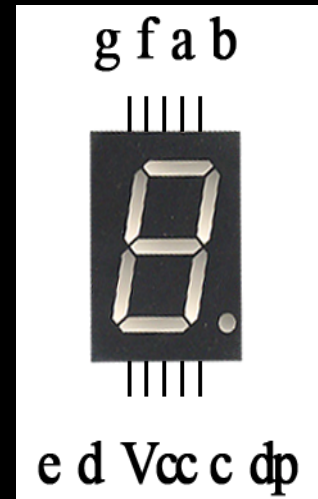
共阳



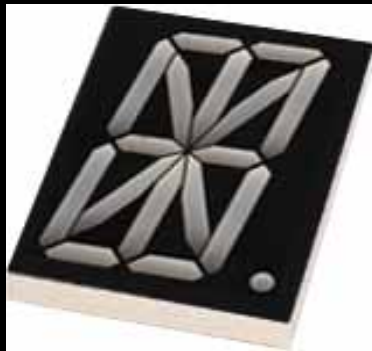
共阴



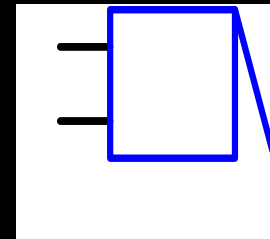
管脚排布



2. 米字数码管



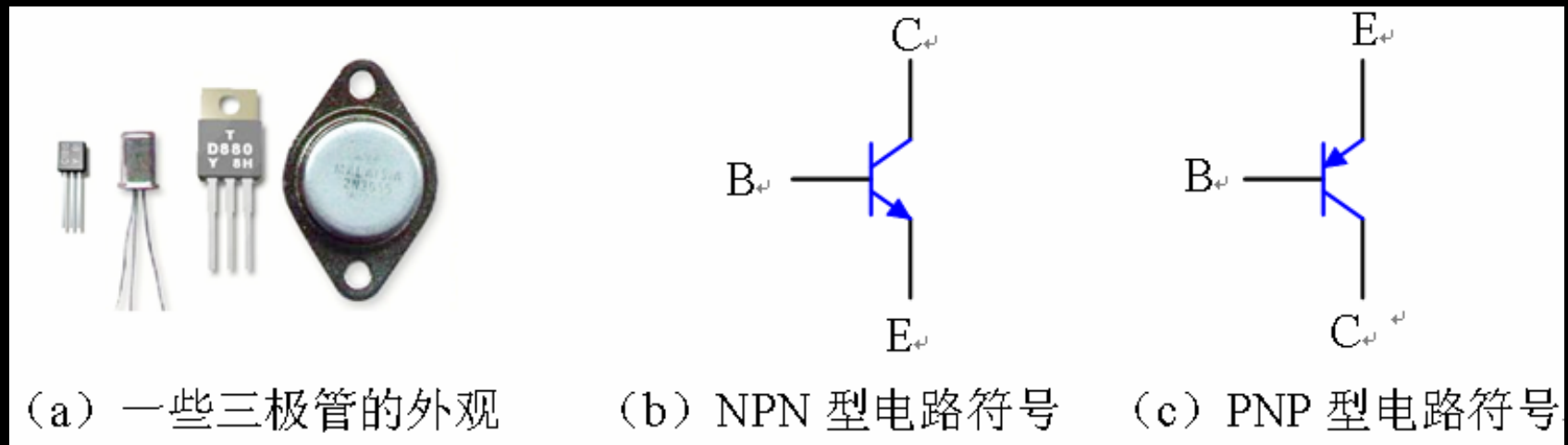
3. 蜂鸣器



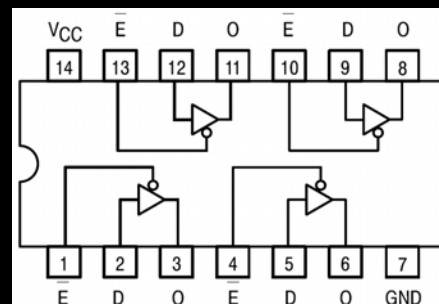
蜂鸣器及其电路符号

器件介绍

4. 三极管



5. 三态缓冲器74125



第5章解剖单片机——观察存储器

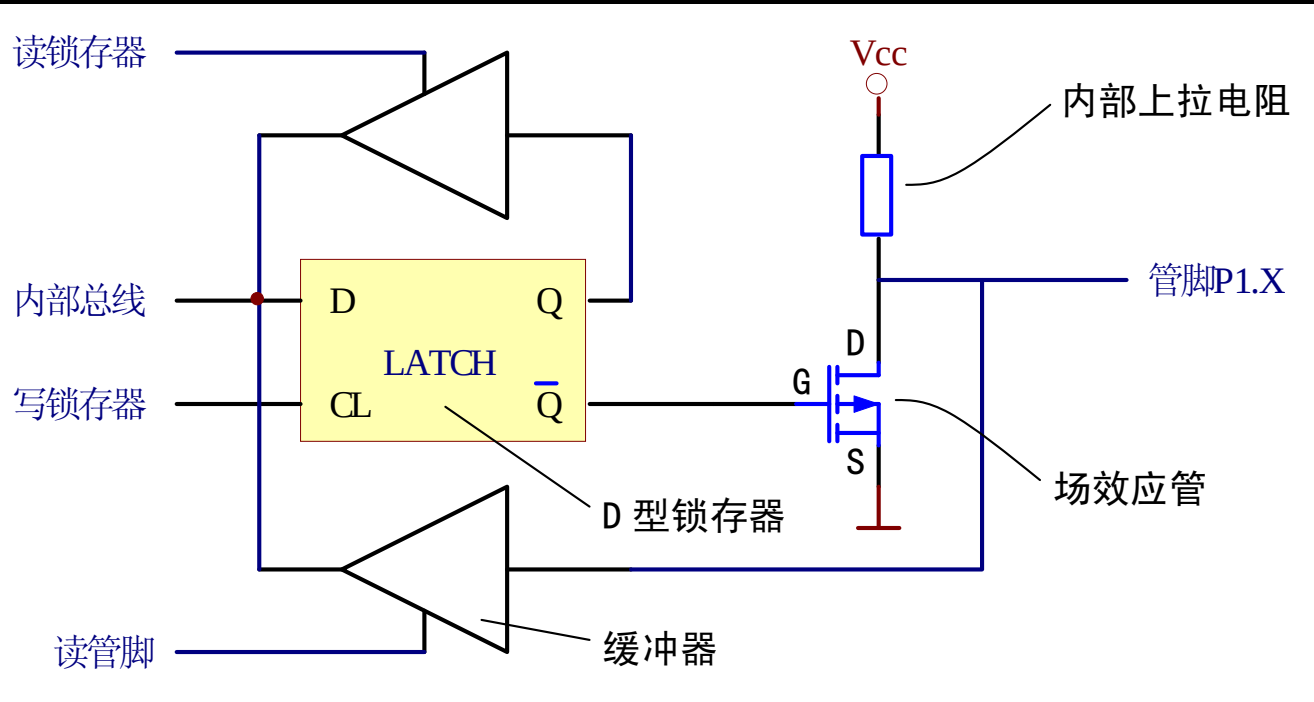
欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

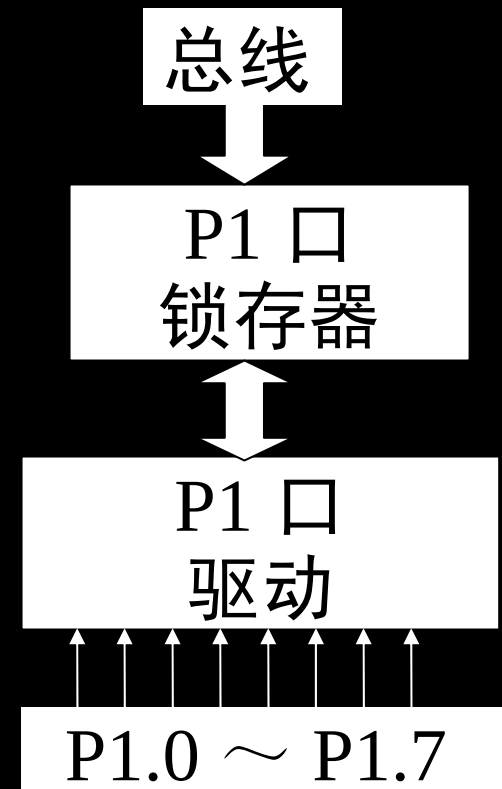
- 5.1 从I/O口到单片机的“解剖结构”
- 5.2 存储器家族
- 5.3 随机访问存储器——RAM
- 5.4 只读存储器——ROM
- 5.5 实例点拨——数据是如何保存在存储器中的

5.1从I/O口到单片机的“解剖结构”

• 5.1.1 I/O口的“解剖图”



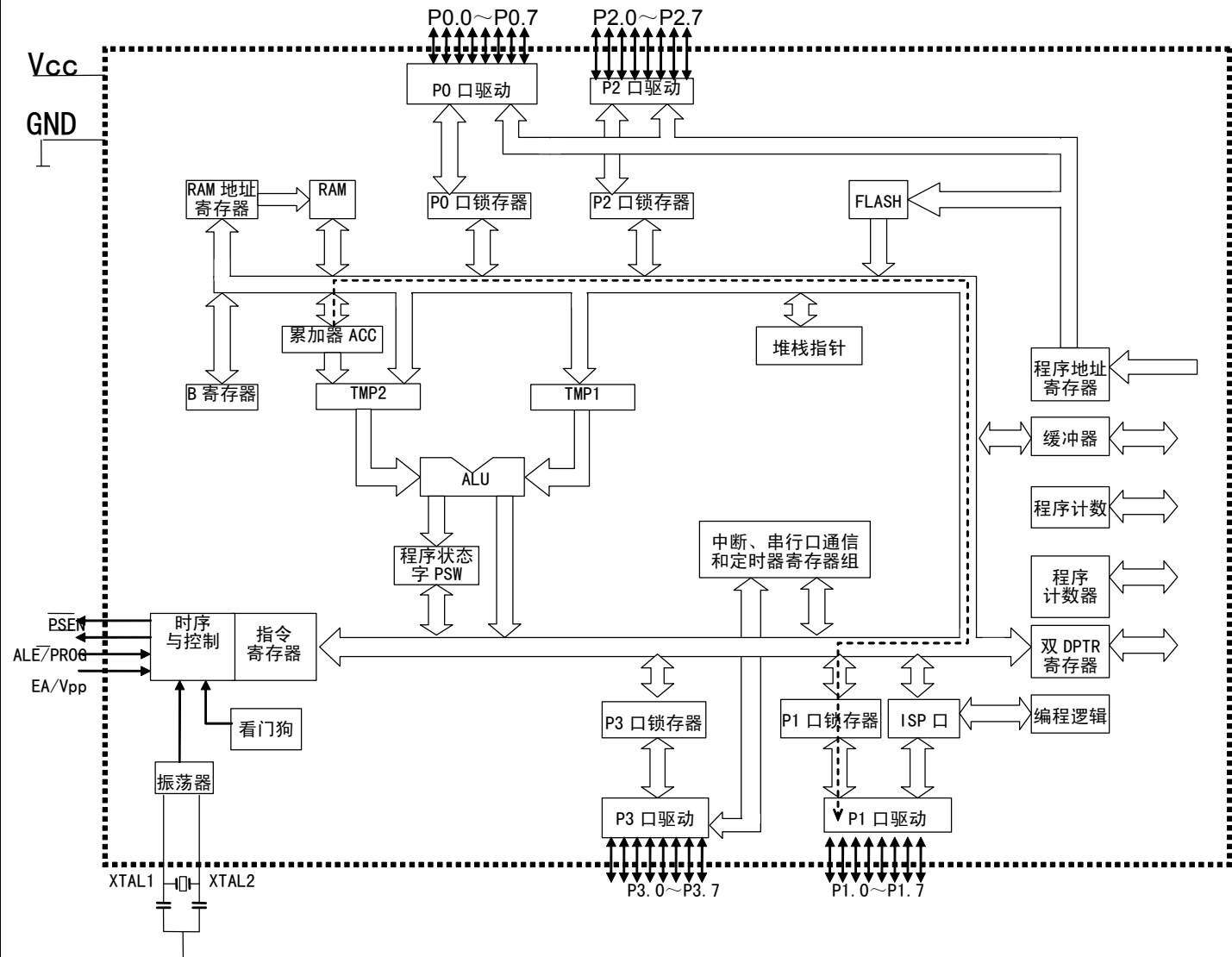
P1.X口结构



P1口的抽象

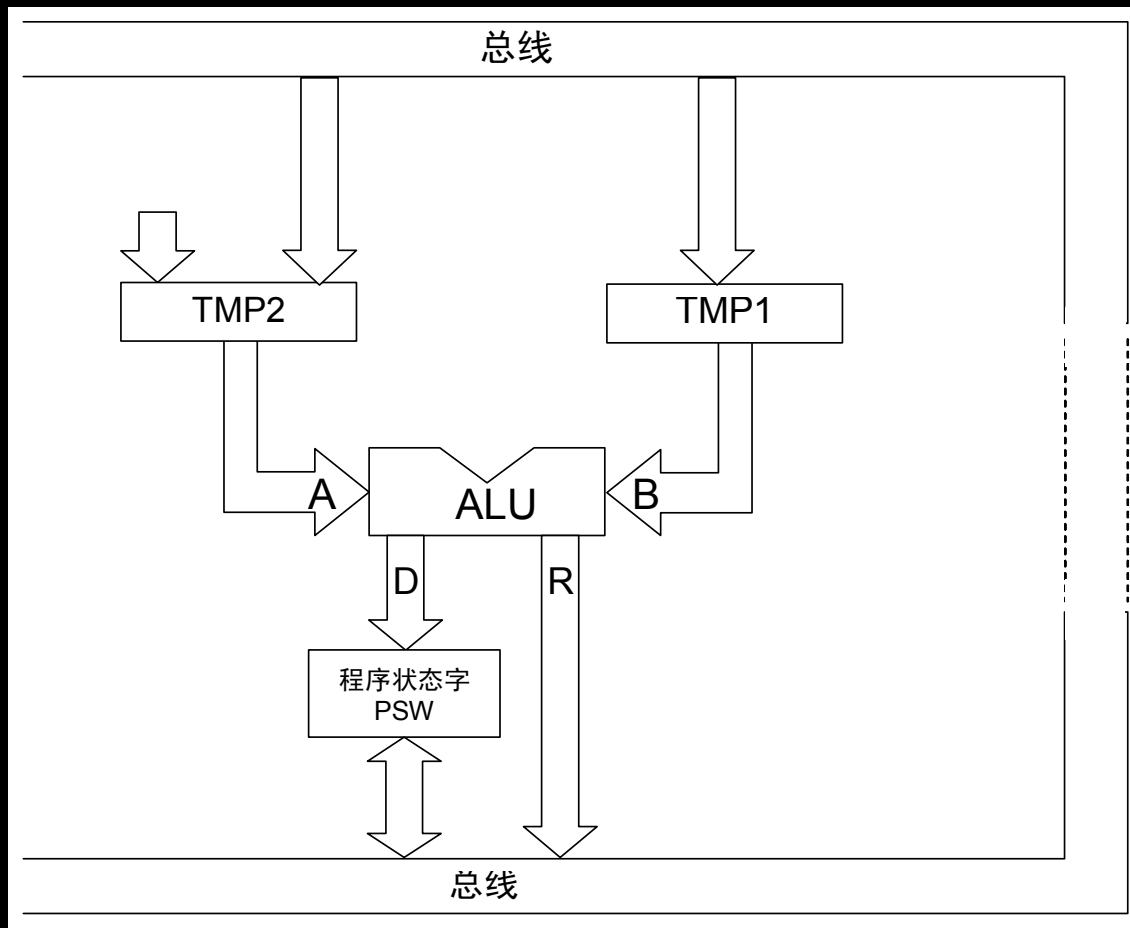
5.1从I/O口到单片机的“解剖结构”

5.1.2 单片机的“解剖结构”



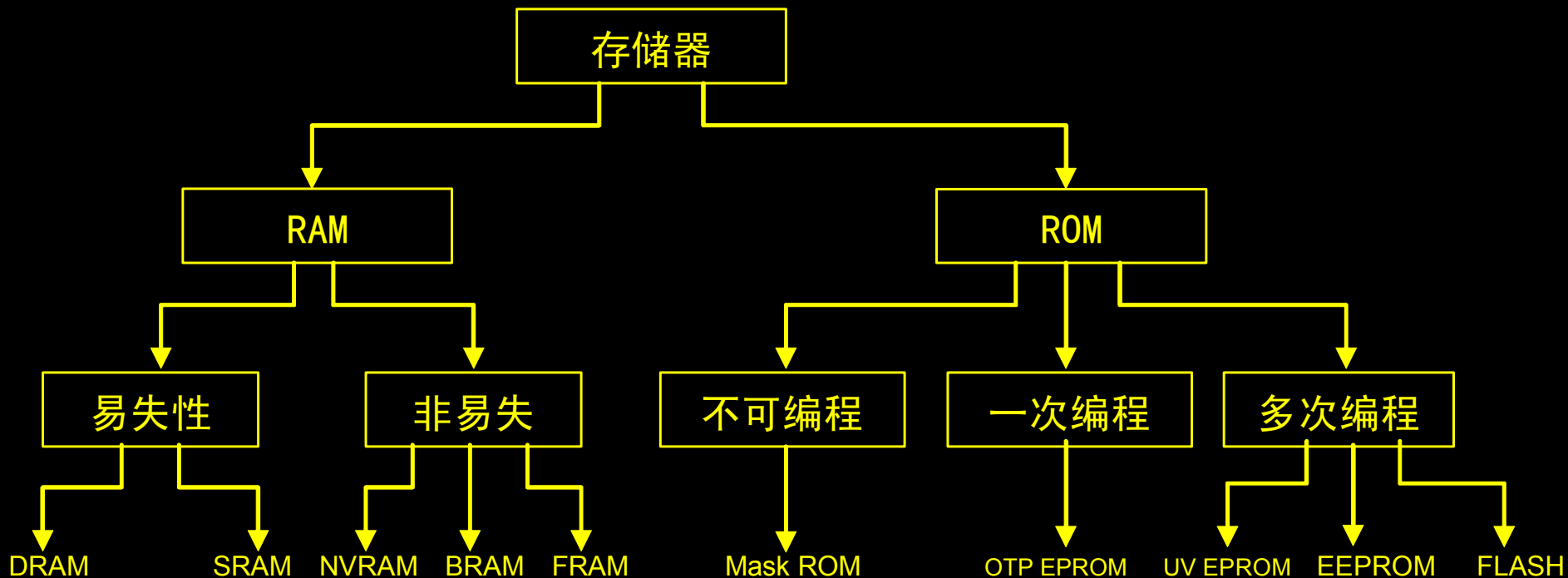
5.1从I/O口到单片机的“解剖结构”

• 5.1.3 深入“解剖”——算术逻辑单元ALU



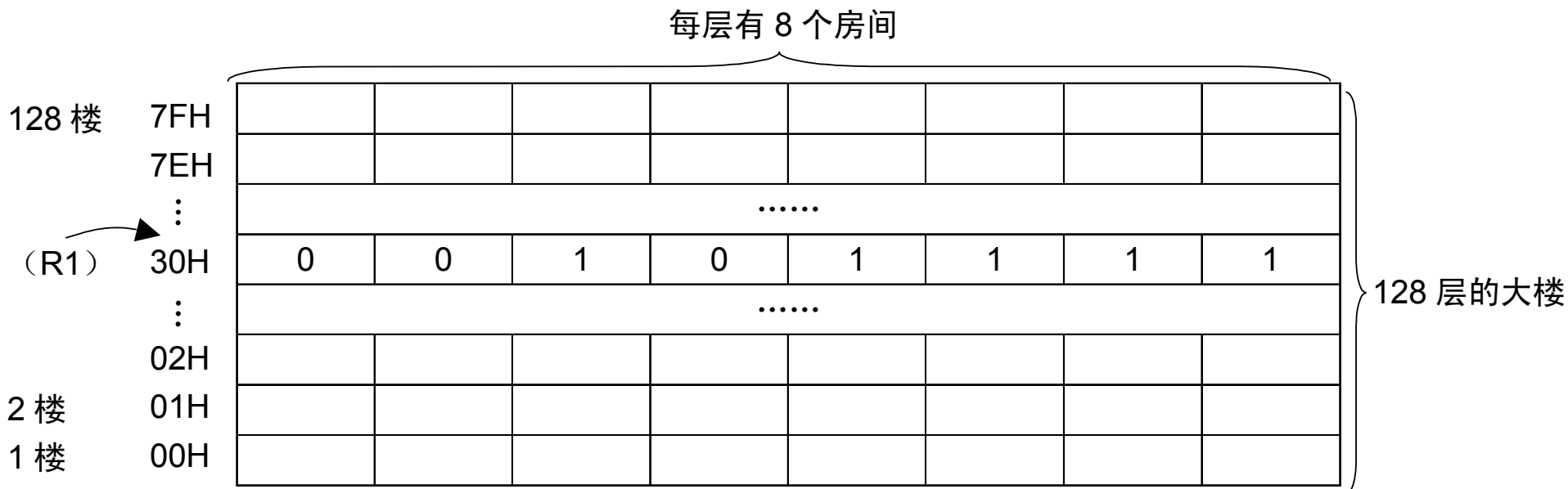
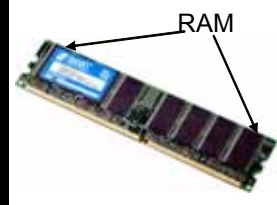
5.2 存储器家族

- 常见存储器的种类



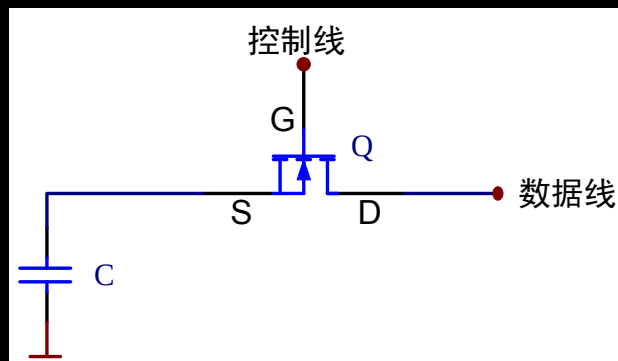
5.3 随机访问存储器——RAM

- RAM之所以称为随机访问存储器（random access memory），是因为它能在同样的时间内访问RAM中任意地址上的数据，而不需要从头到尾顺序地对地址上的数据进行访问。



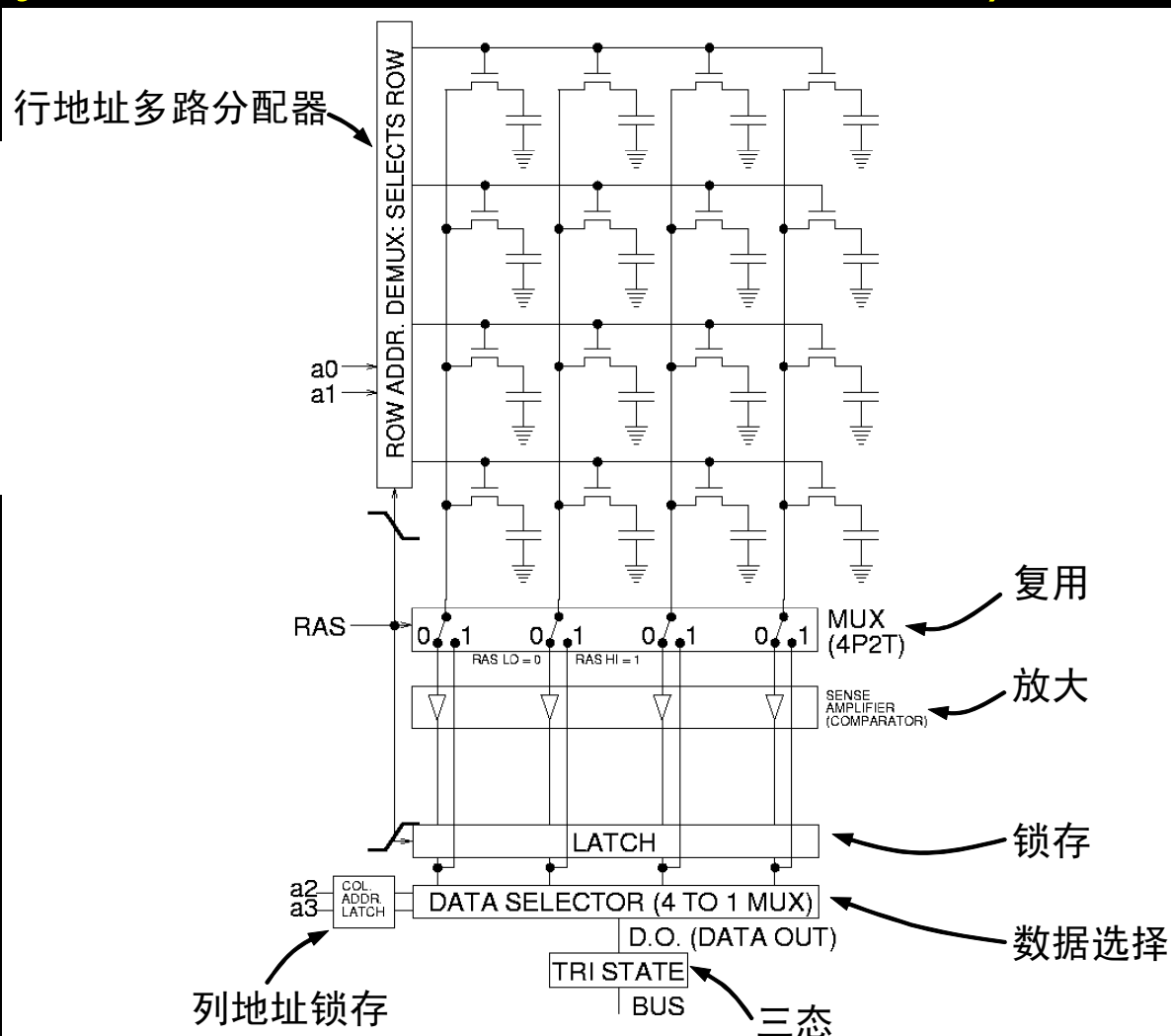
5.3 随机访问存储器——RAM

• 5.3.1 DRAM (Dynamic RAM, 动态RAM结构)



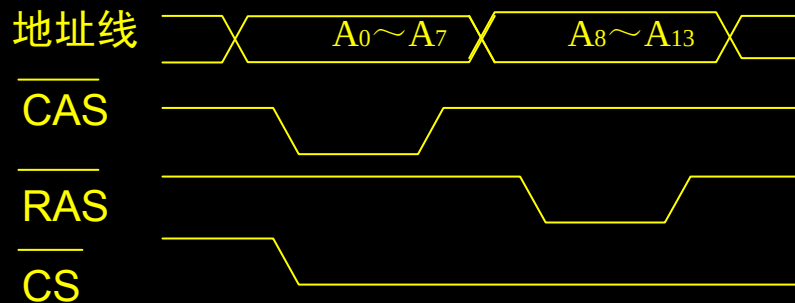
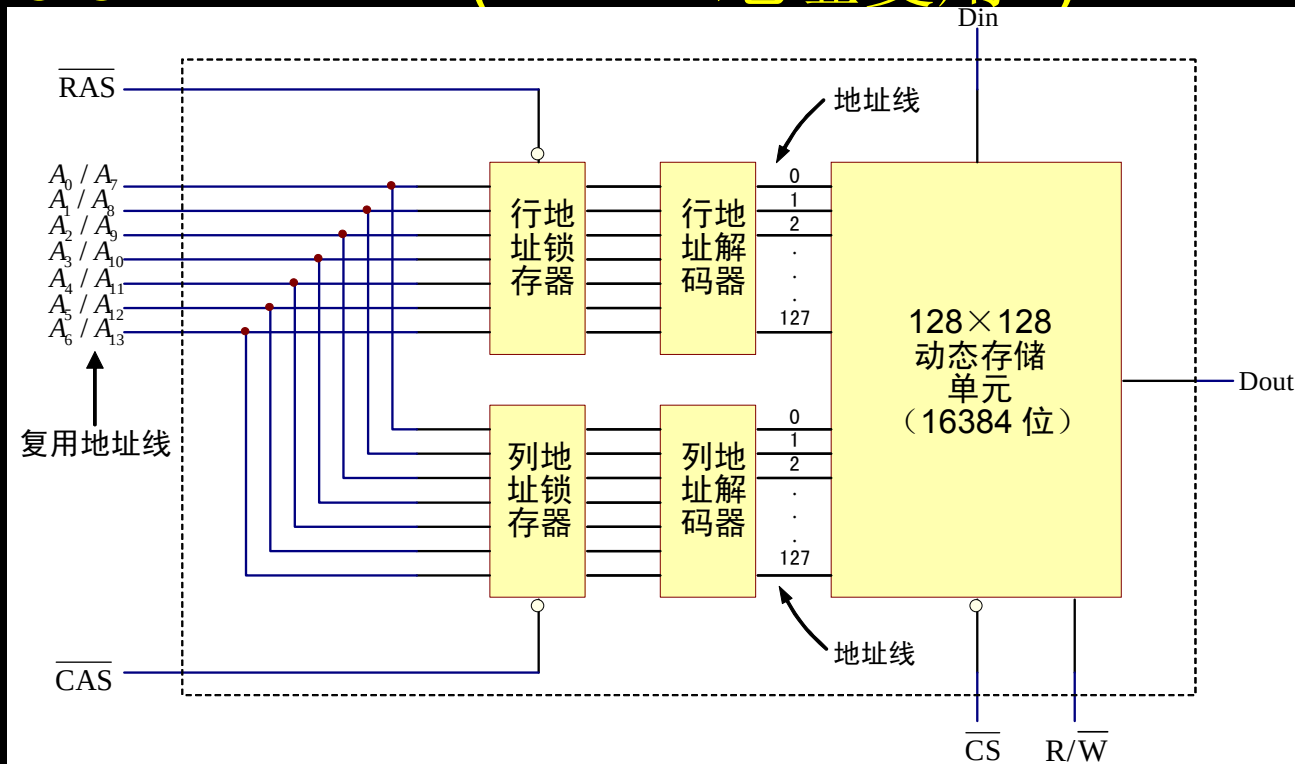
△DRAM的存储单元

DRAM较为真实的内部结构



5.3 随机访问存储器——RAM

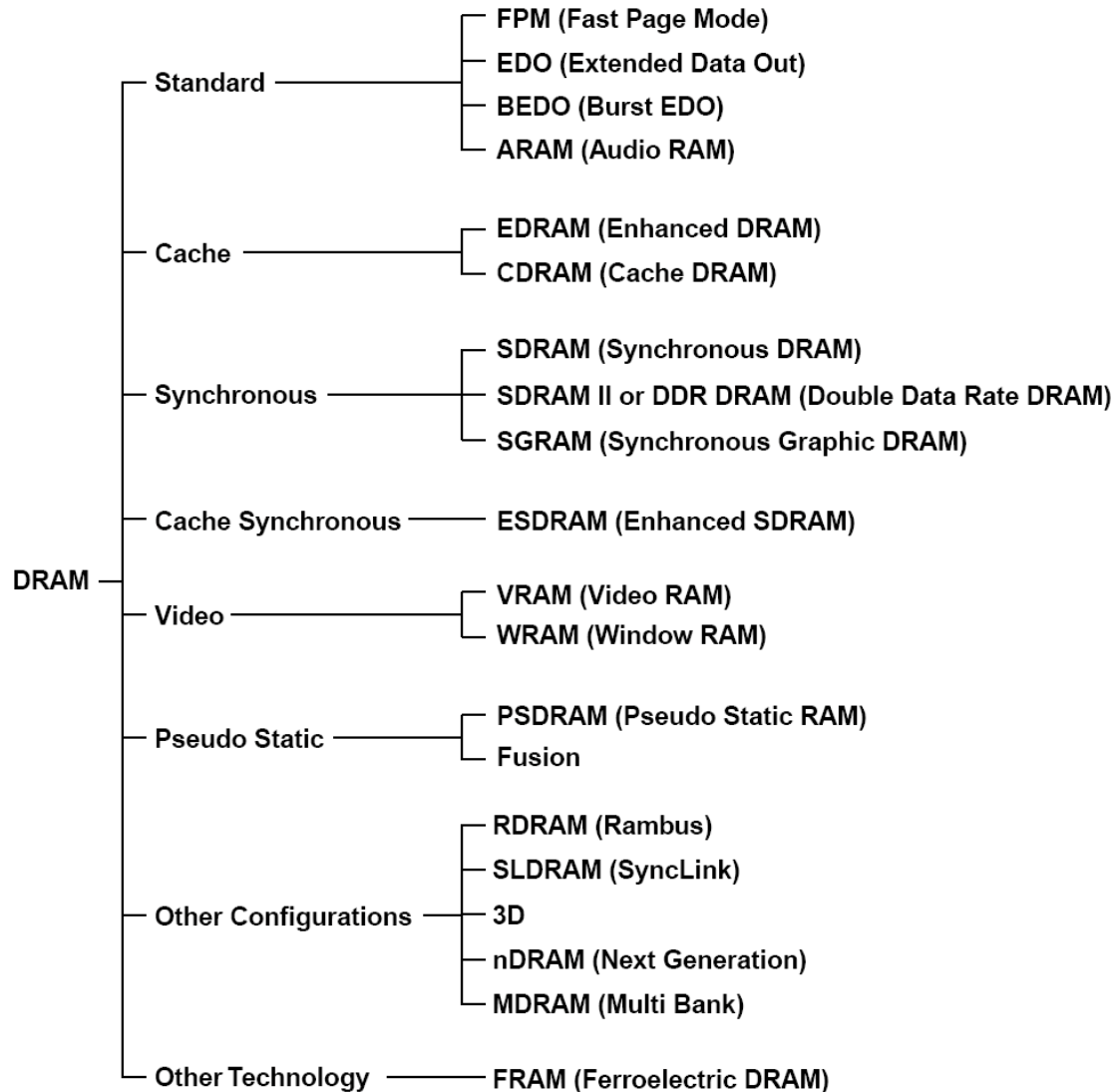
• 5.3.1 DRAM (DRAM地址复用)



地址复用的时序图

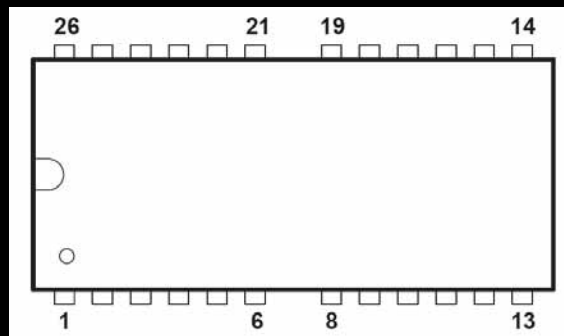
5.3 随机访问存储器——RAM

• 5.3.1 DRAM (应用及分类)

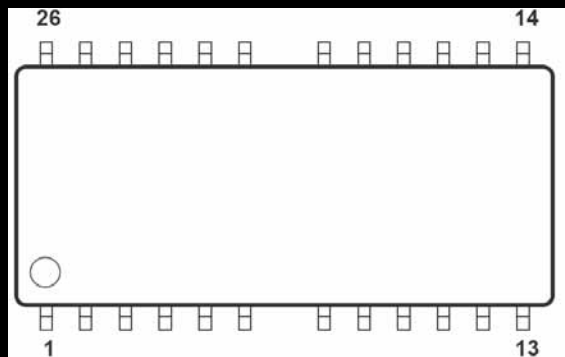


5.3 随机访问存储器——RAM

• 5.3.1 DRAM (相关知识学习, TMS416400为例)



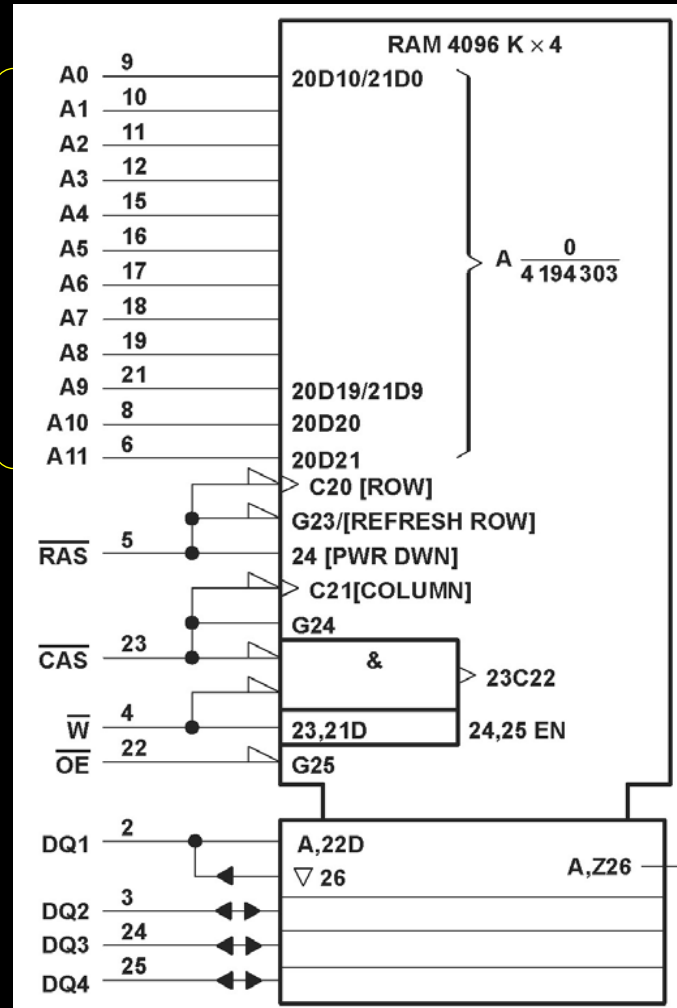
△ DJ封装



△ DGA封装

地址线

数据输入
输出端

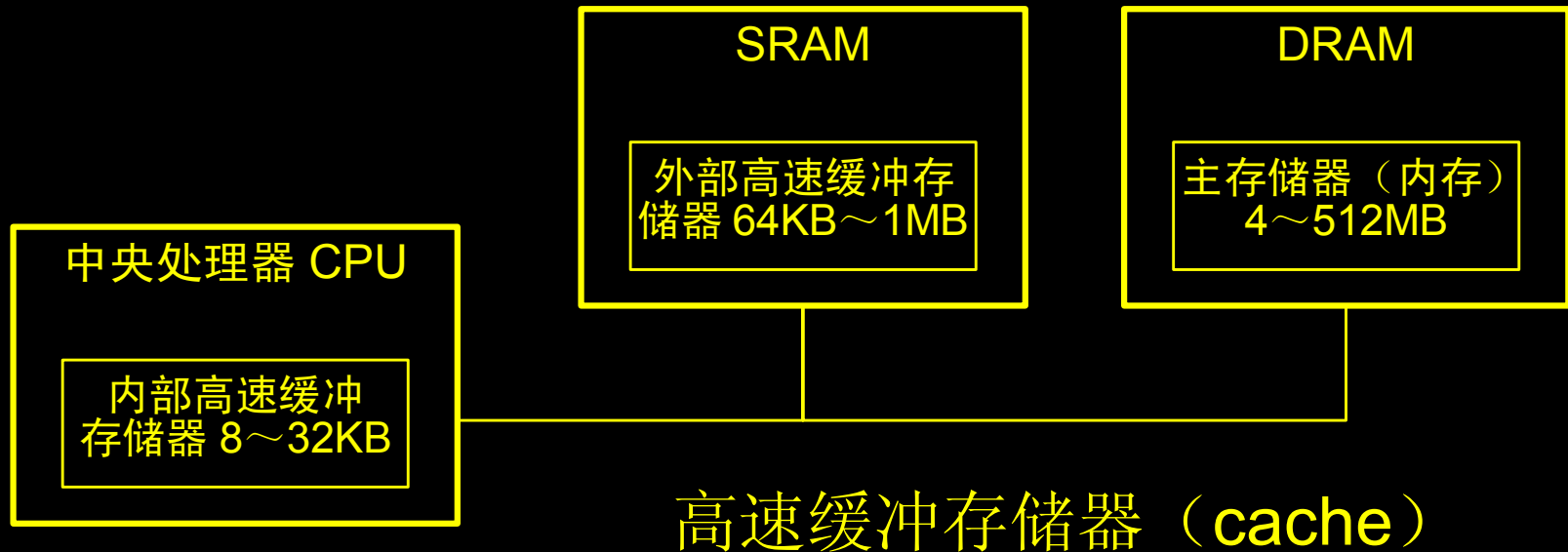


△
TMS41
6400A
DRAM
的逻辑
符号

5.3 随机访问存储器——RAM

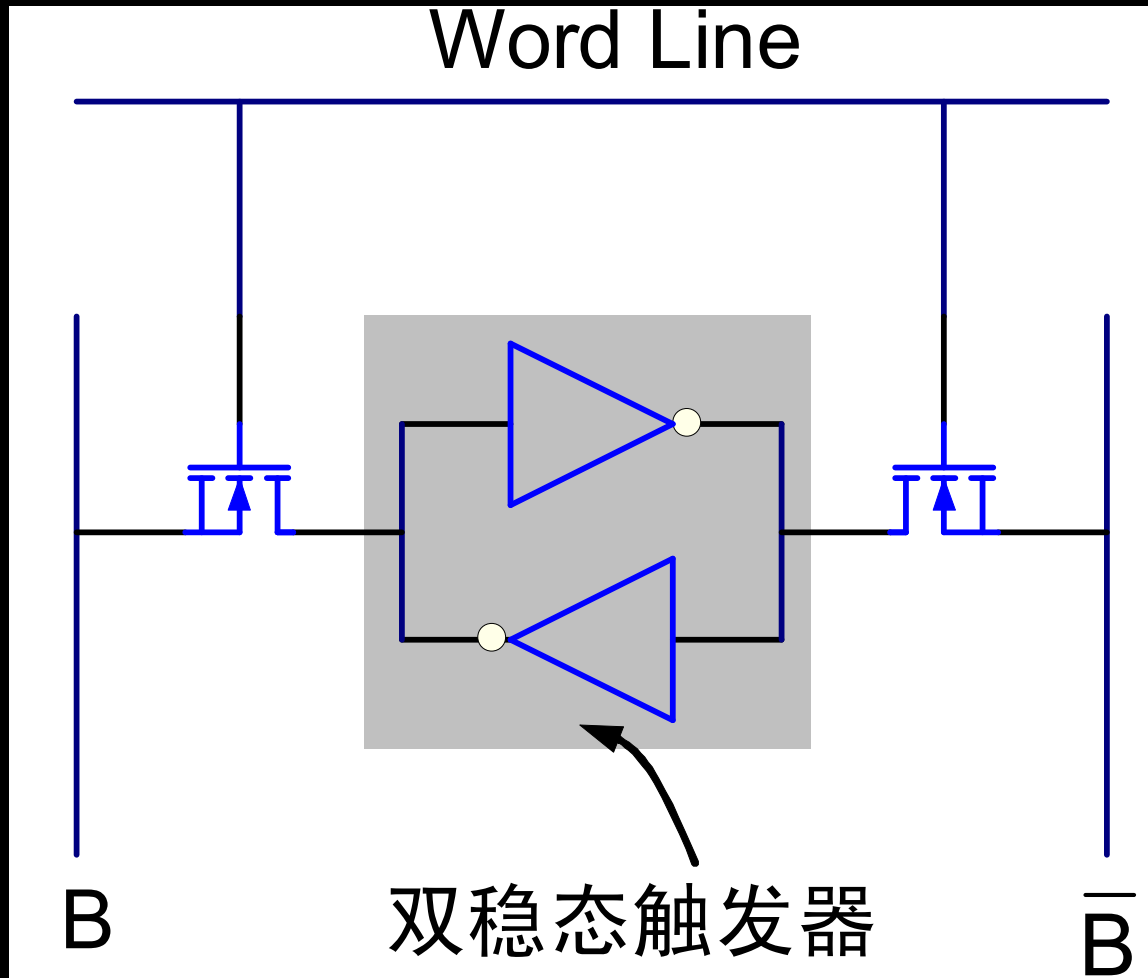
- 5.3.2 SRAM (静态随机访问存储器 static random access memory)

SRAM可以进行随机访问，不需要对数据进行重新装载，只要保持给它供电，它就能保持住原来存储的数据，完成高速而低功耗的存储任务。



5.3 随机访问存储器——RAM

• 5.3.2 SRAM (SRAM的存储单元)

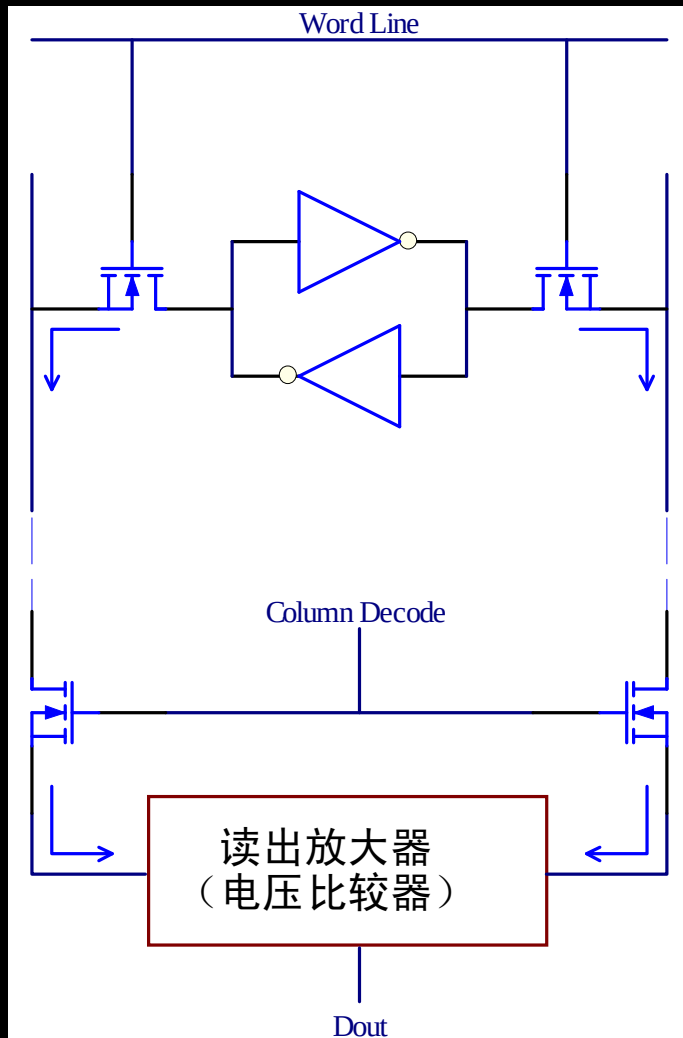


字线也称为行地址。
位线也称为列地址。

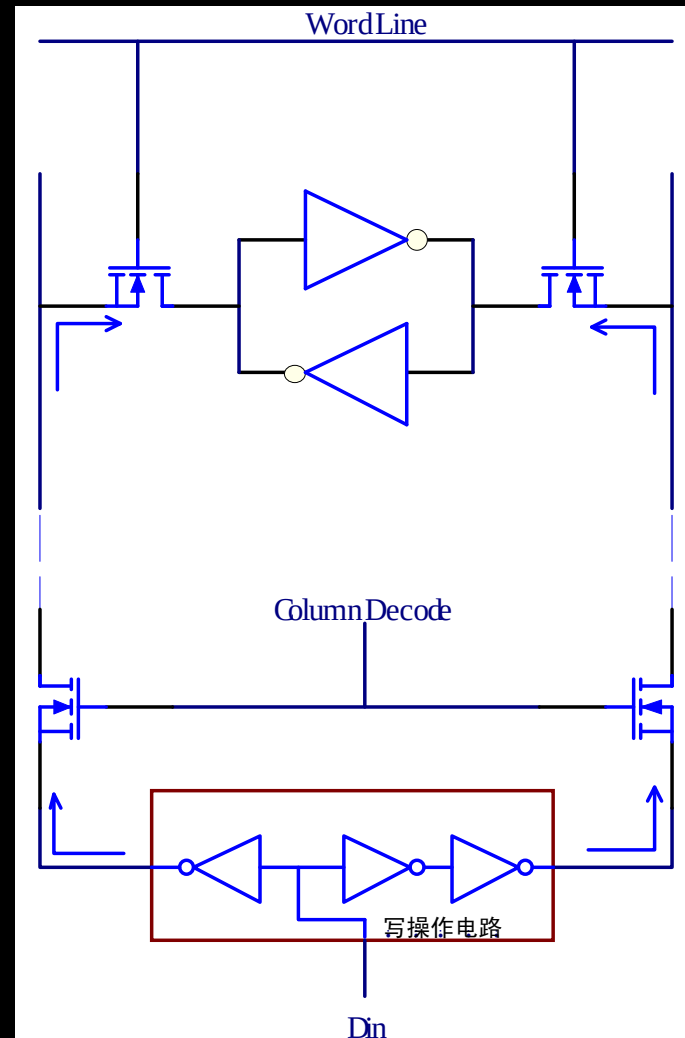
5.3 随机访问存储器——RAM

• 5.3.2 SRAM (SRAM的读写操作)

读操作

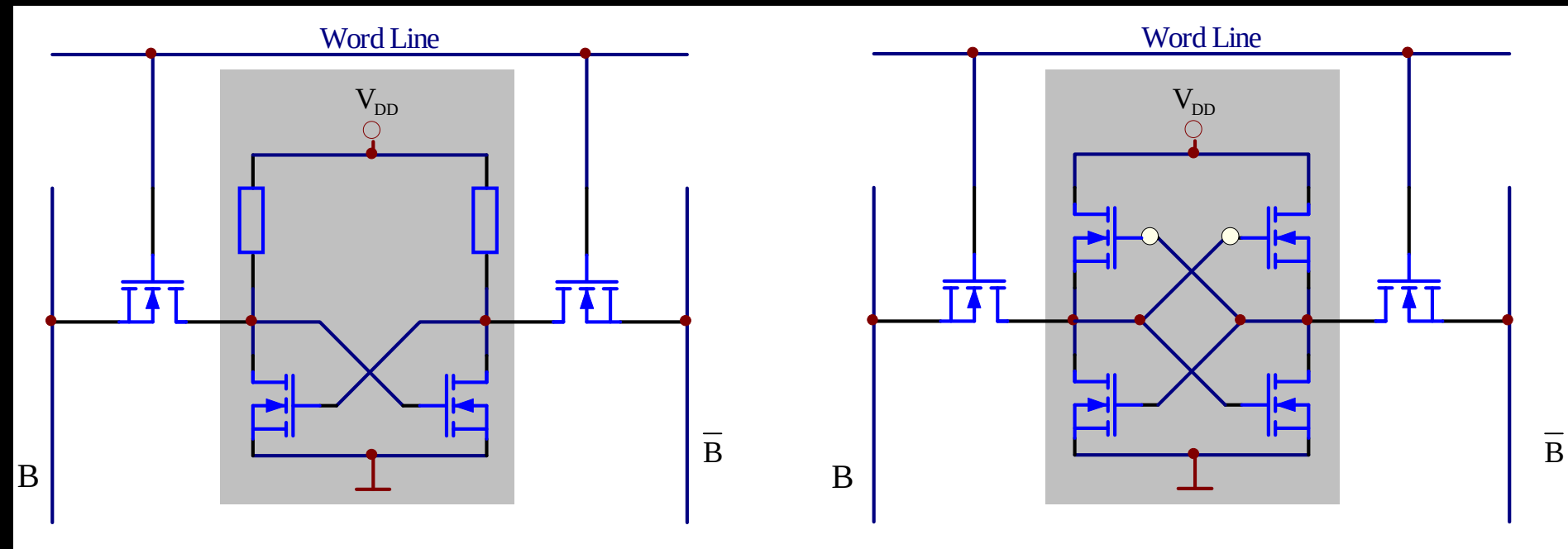


写操作



5.3 随机访问存储器——RAM

• 5.3.2 SRAM (存储单元中双稳态触发器的结构)

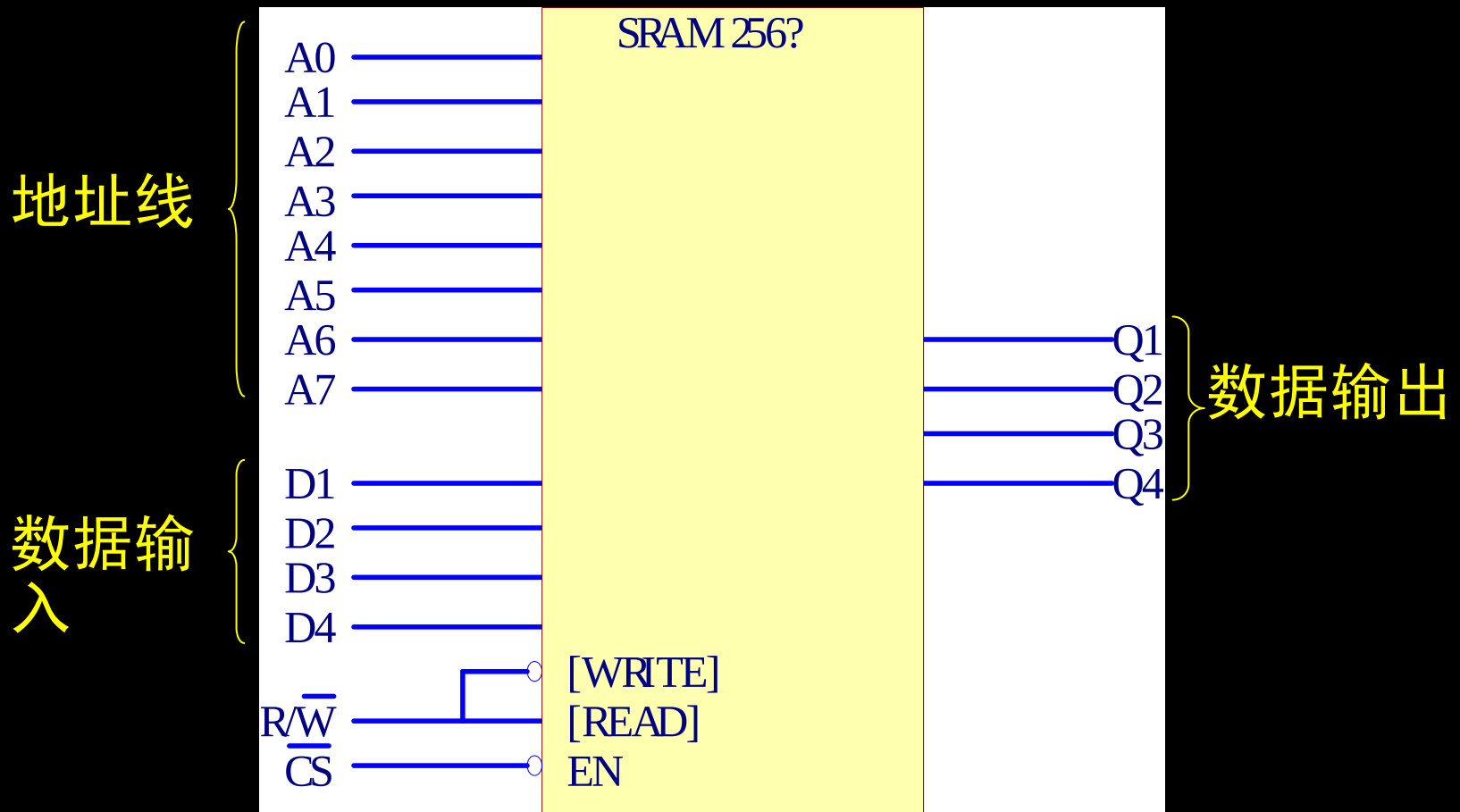


(a) 4个场效应管

(b) 6个场效应管

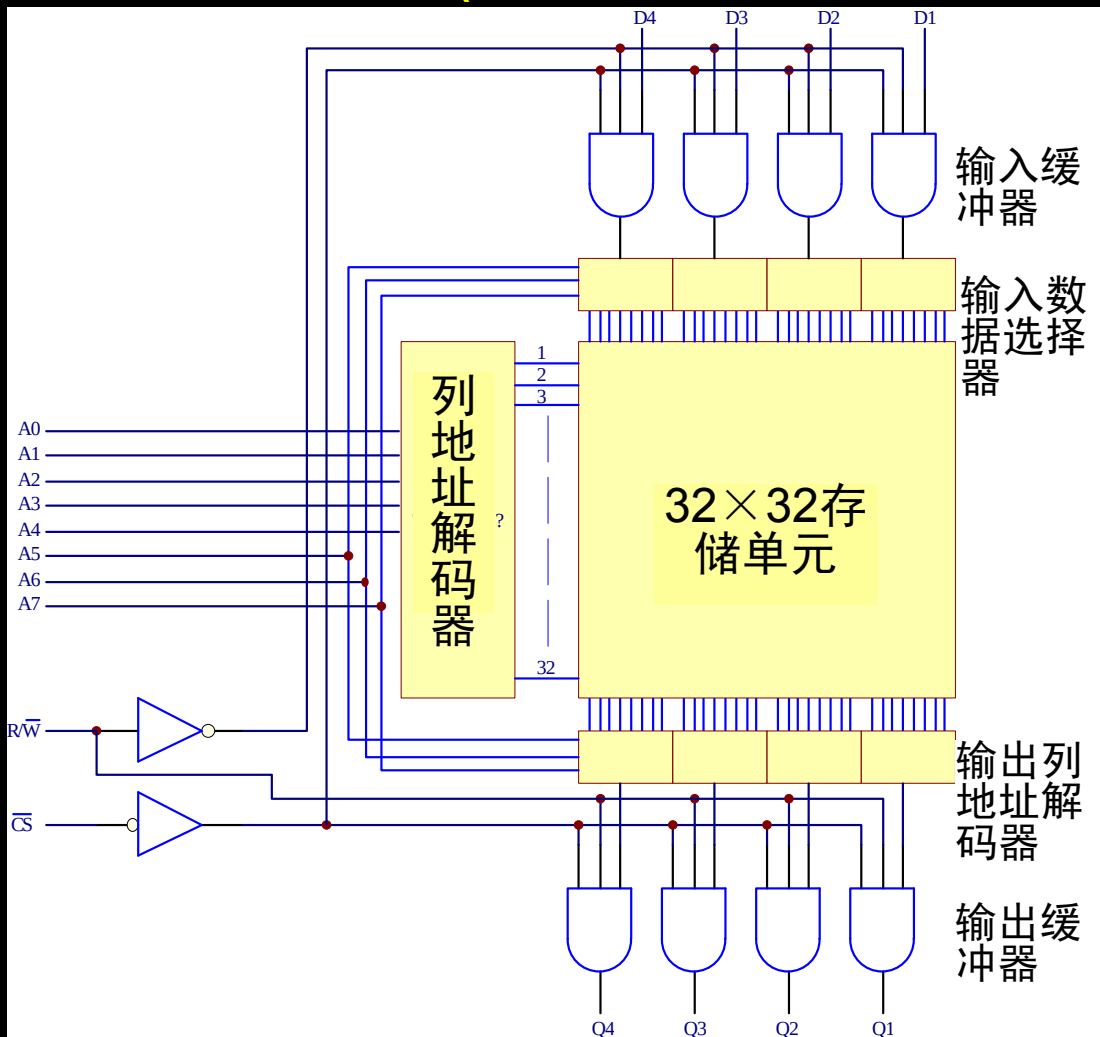
5.3 随机访问存储器——RAM

• 5.3.2 SRAM (实际器件展示)



5.3 随机访问存储器——RAM

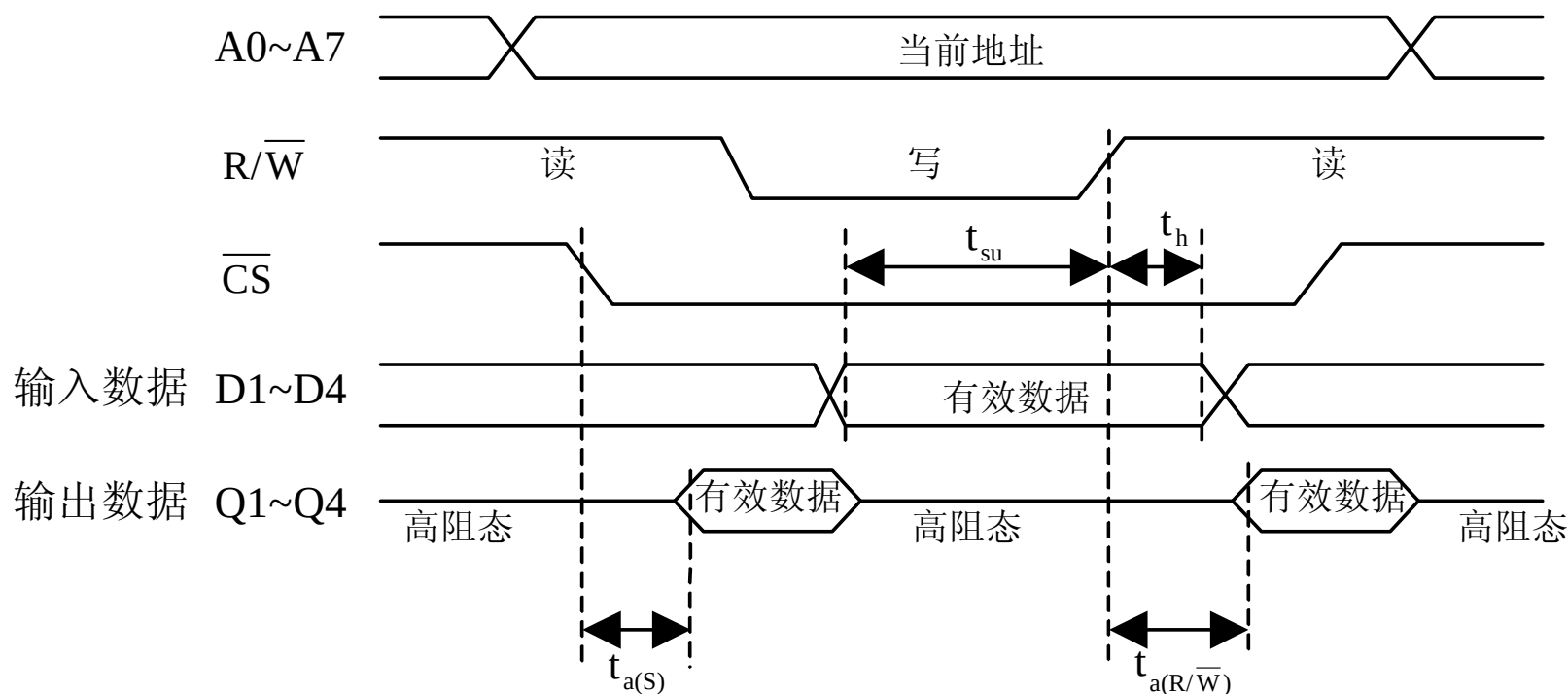
- 5.3.2 SRAM (实际器件的组织结构)



32 × 32 的存储单元矩阵

5.3 随机访问存储器——RAM

• 5.3.2 SRAM (SRAM的操作时序图)



$t_{a(S)}$: 访问时间, 从 $\overline{CS}$ 到数据输出

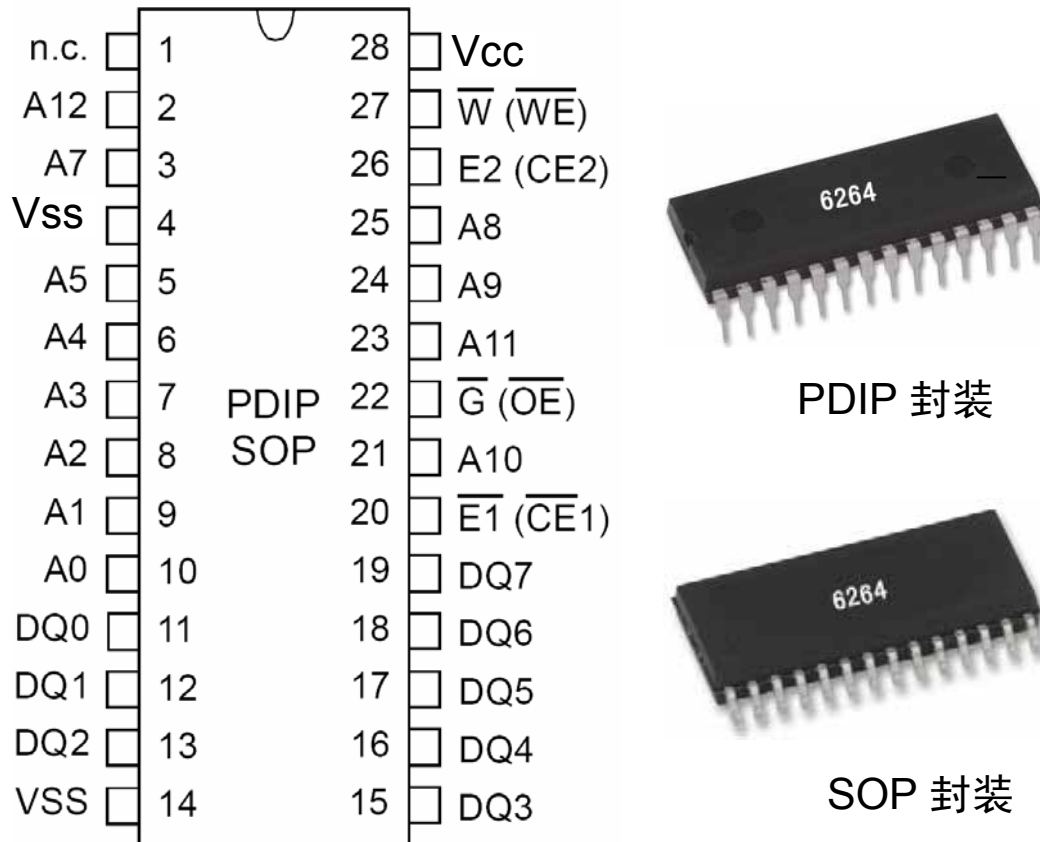
$t_{a(R/\overline{W})}$: 访问时间, 从 $R/\overline{W}$ 到数据输出

t_{su} : 写数据时间

t_h : 保持时间

5.3 随机访问存储器——RAM

• 5.3.2 SRAM (SRAM 6264 简介)

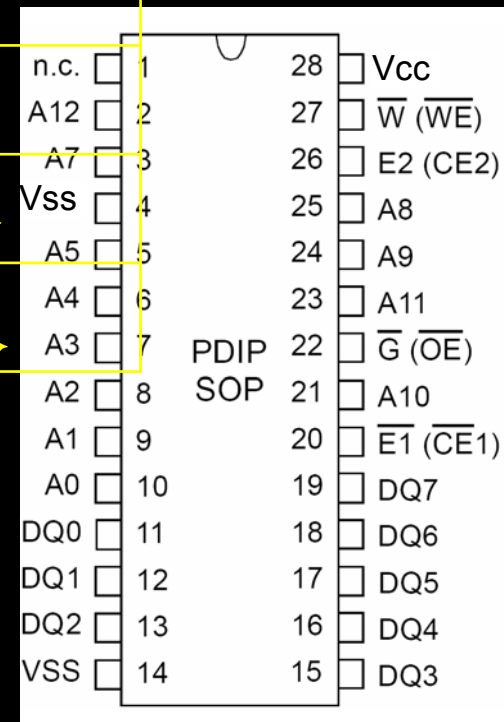


管脚术语	解释
A0~A12	地址线
DQ0~DQ7	数据输入/输出线
$\overline{E1}$ ($\overline{CE1}$)	使能 1
$\overline{E2}$ ($\overline{CE2}$)	使能 2
$\overline{G}$ ($\overline{OE}$)	输出使能
$\overline{W}$	写操作使能
Vcc	供电端
Vss	接地端
n.c.	空脚，不接

5.3 随机访问存储器——RAM

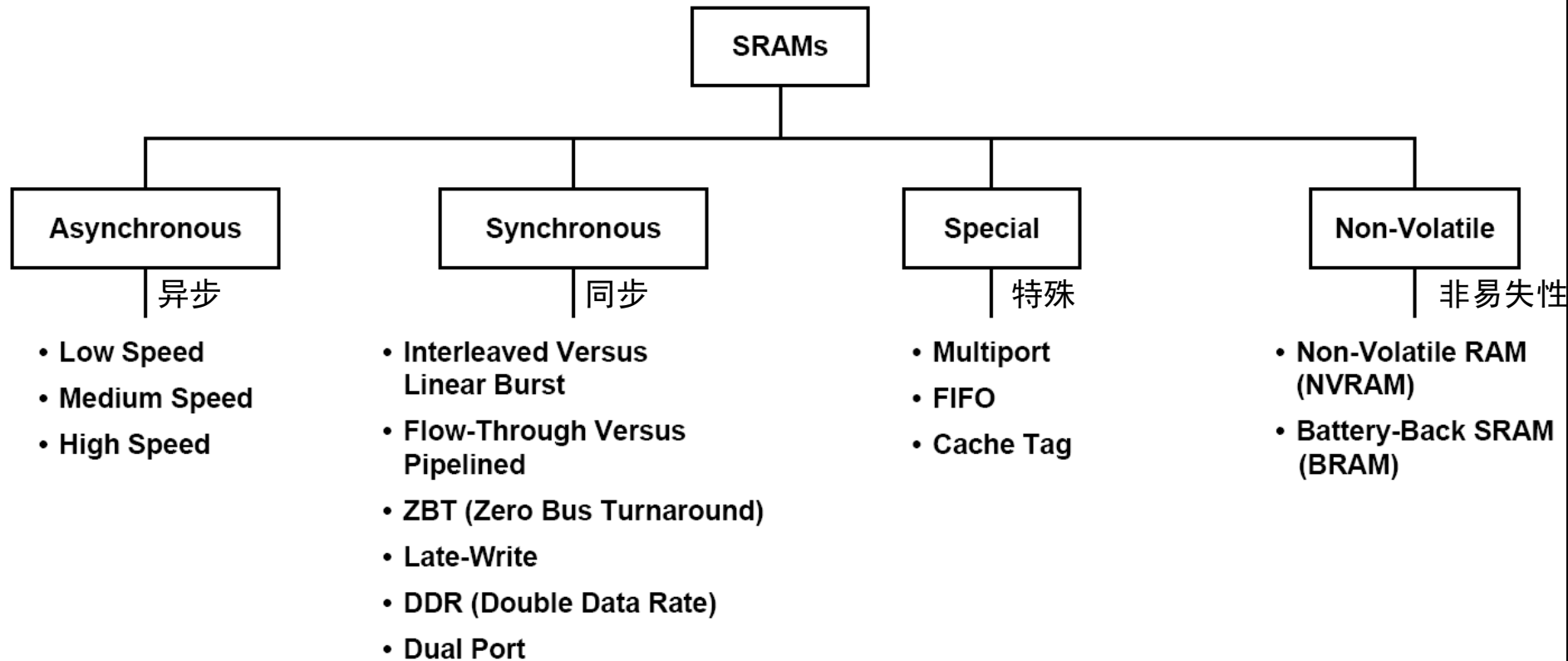
• 5.3.2 SRAM (SRAM 6264 简介)

操作模式	E1	E2	W	G	DQ0~DQ7
待机/非被选中	H/L	L	H/L	H/L	高阻态
	H	H/L	H/L	H/L	高阻态
内部读操作	L	H	H	H	高阻态
读操作	L	H	H	L	数据输出
写操作	L	H	L	H/L	数据输入



5.3 随机访问存储器——RAM

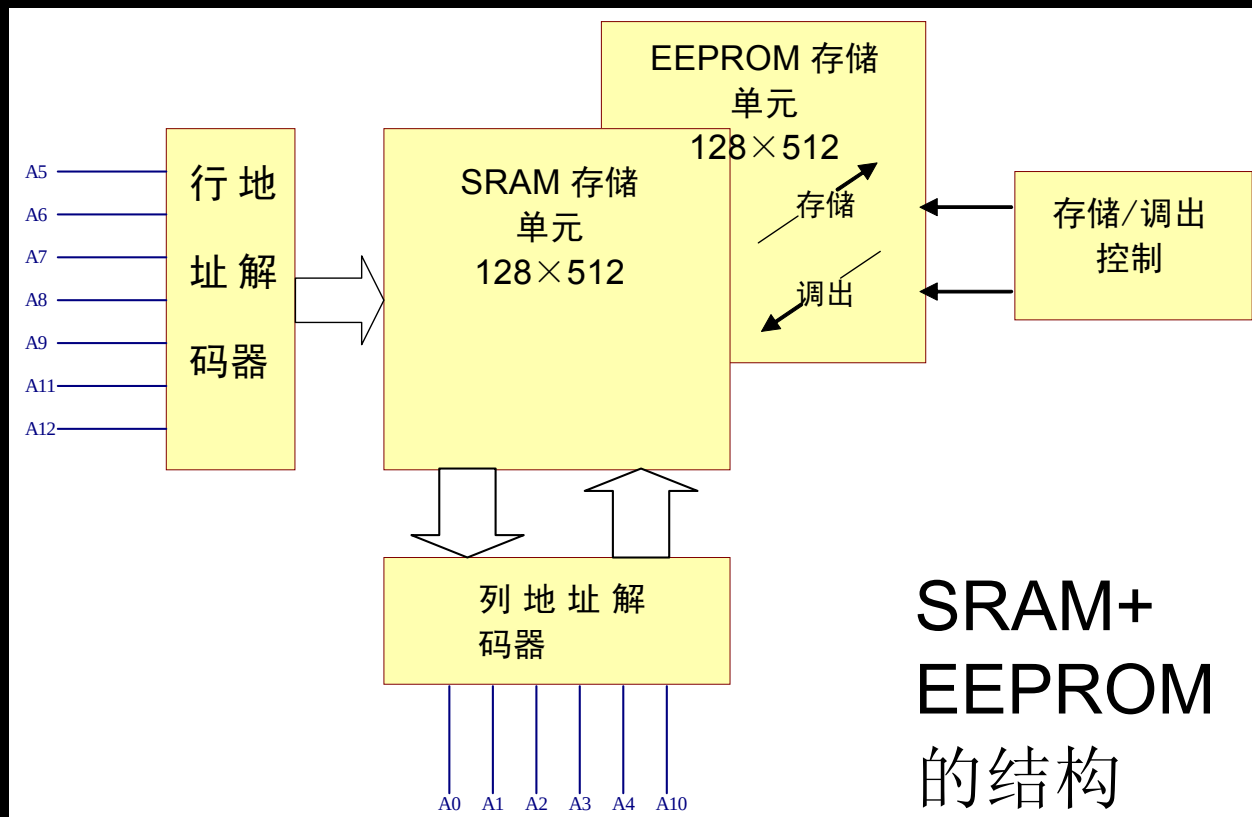
• 5.3.2 SRAM (SRAM 应用及分类)



5.3 随机访问存储器——RAM

- 5.3.3 NVRAM (非易失性随机访问存储器 non-volatile random access memory)

NVRAM可以进行随机访问，掉电后数据仍然保存着。



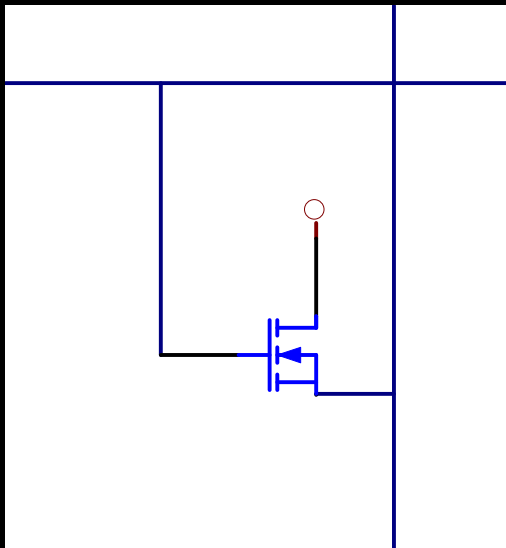
5.4 只读存储器——ROM

- ROM (read-only memory)
- ✓ ROM只能读数据，而不能往里面写数据。
- ✓ ROM是非易失性存储器，当存储器掉电后，存储器中的数据不会丢失。

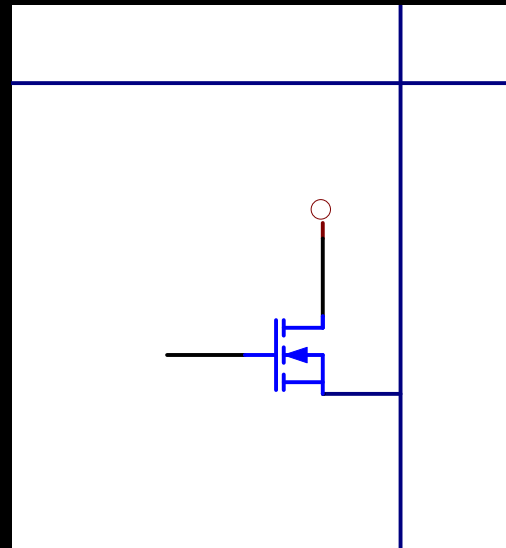
5.4 只读存储器——ROM

• 5.4.1 Mask ROM

Mask ROM存储的内容无法被用户修改，所以它通常存储商品的信息，如投影仪开机时显示的品牌名称和标志；计算机开机时显示的主板版本和厂商信息等。



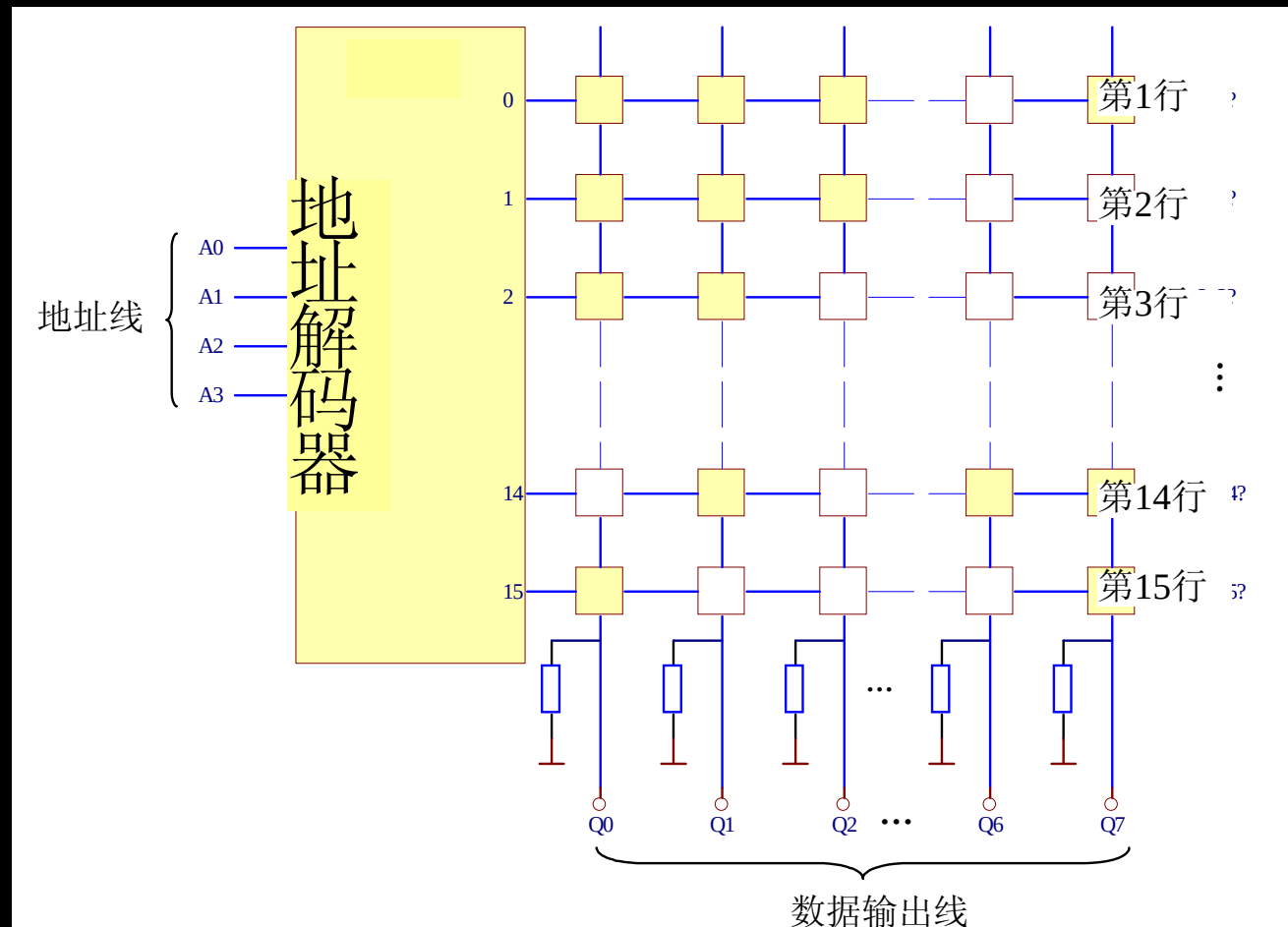
(a) 存储1



(b) 存储0

5.4 只读存储器——ROM

• 5.4.1 Mask ROM(小规模Mask ROM简介)



16×8位
ROM的
结构

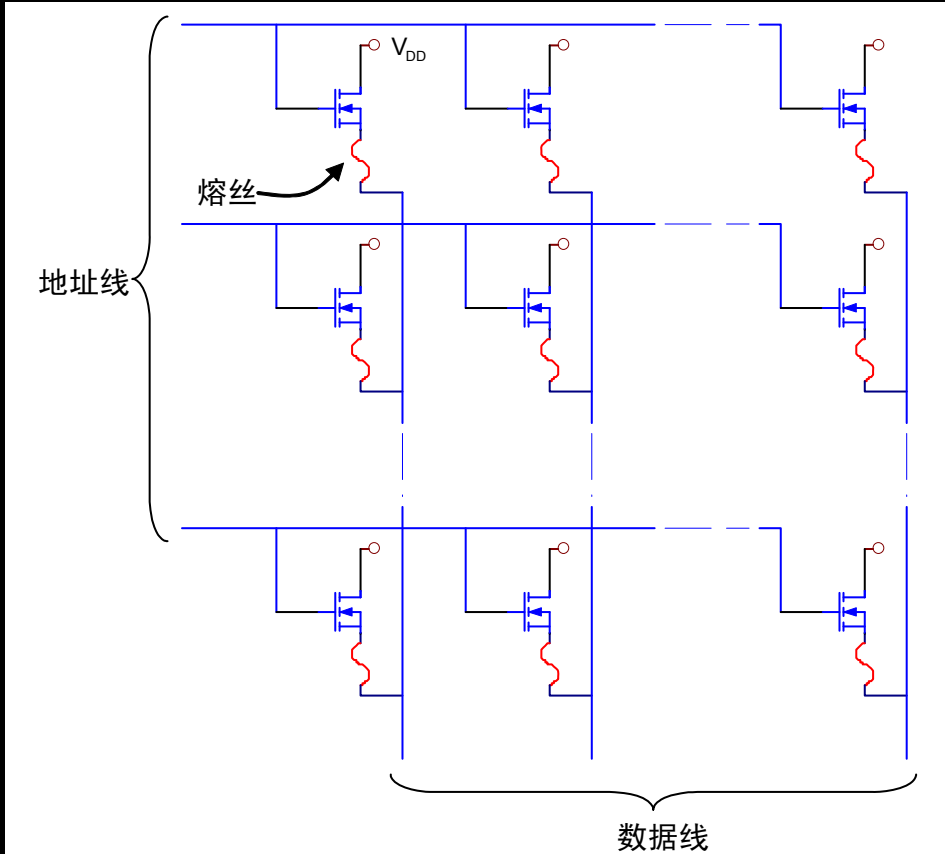
5.4 只读存储器——ROM

- 5.4.2 PROM (programmable ROM)

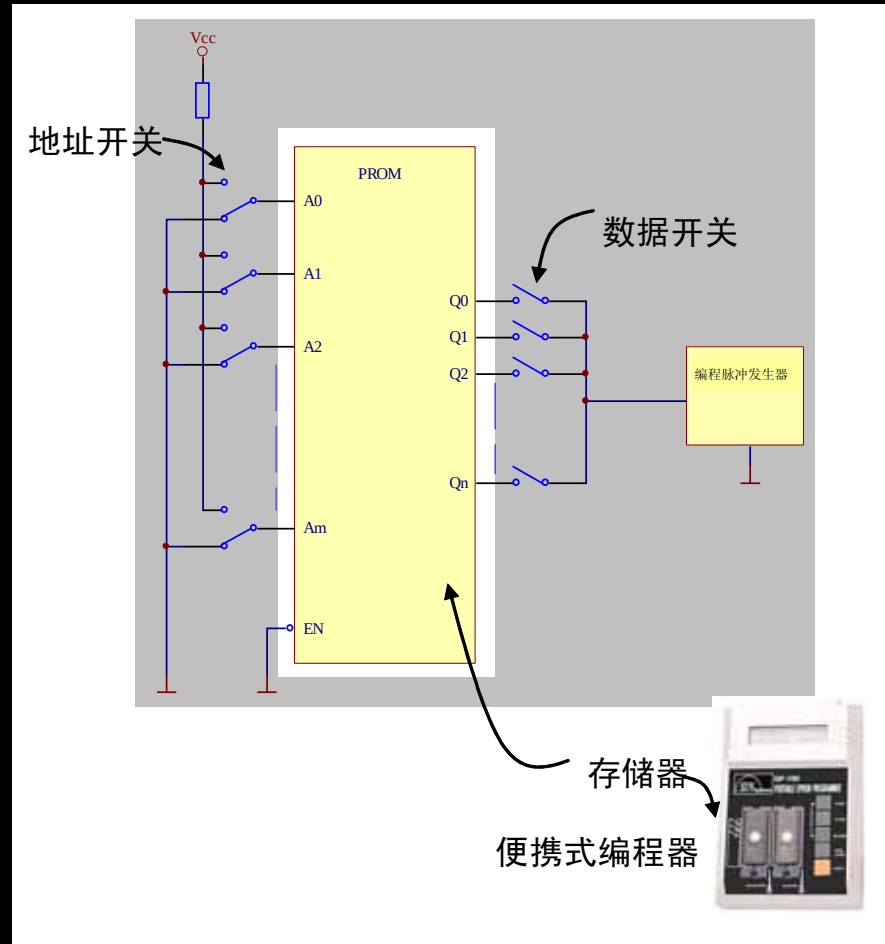
用户可以对新买回来的可编程**ROM**器件进行编程，也就是固化数据到**PROM**中，一旦固化完成后，**PROM**就像一个**Mask ROM**一样，只能读取数据而不能再写入或更改数据了。

5.4 只读存储器——ROM

• 5.4.2 PROM (PROM存储单元及烧写原理)



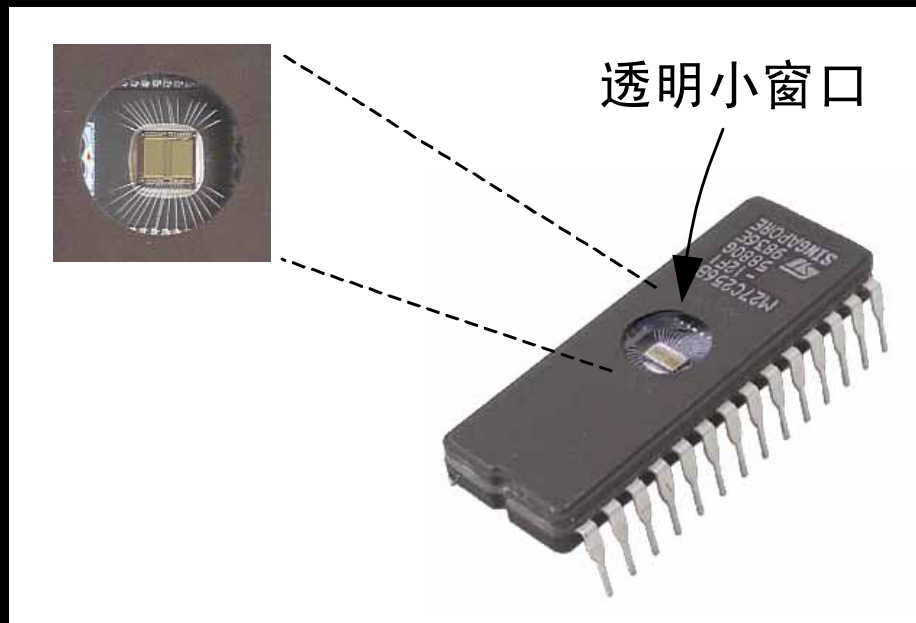
熔丝连接场效应管的
源极和PROM的数据线



5.4 只读存储器——ROM

• 5.4.3 EPROM (erasable PROM)

可擦写PROM可以在任何时候把上一次烧写的数
据擦除掉，再往其中写入新的数据，是一种可以重复
编程的ROM器件。

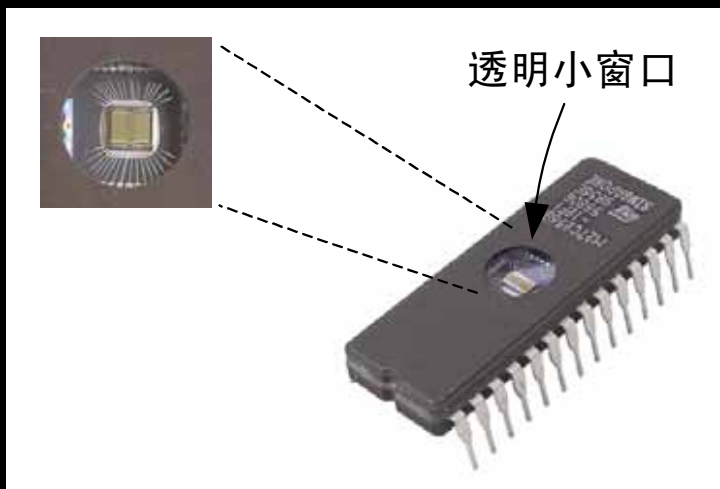


5.4 只读存储器——ROM

• 5.4.3 EPROM (erasable PROM)

高强度的紫外线照射到透明小窗口上大概20分钟，**EPROM**内的数据就会被擦除，

存储单元使用的是**NMOSFET**场效应管，照射的时间足够长，所有存储单元中场效应管栅极上的电子都被移除，数据都恢复到原来的1。



5.4 只读存储器——ROM

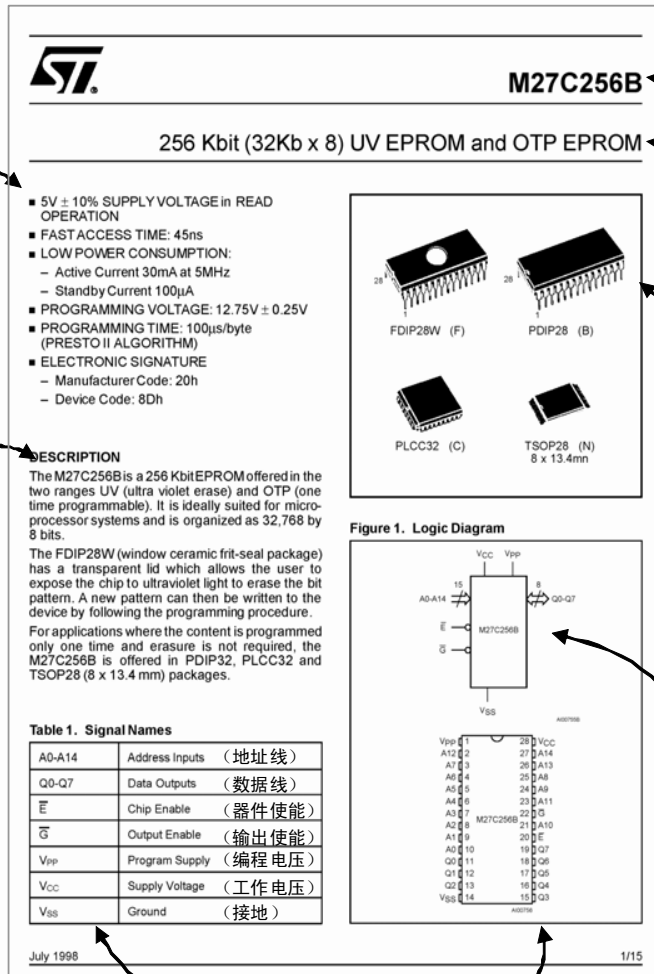
• 5.4.3 EPROM (实例介绍EPROM 27C256)

器件特色:

- 读数据时 $5V \pm 10\%$ 供电
- 快速访问时间: 45ns
- 低功耗
- 编程电压: $12.75V \pm 0.25V$
- 编程时间: $100\mu s/\text{字节}$
- 电子签名

器件描述:

M27C256B 是一款 256Kbit 的 EPROM, 这个型号的器件有两个等级的产品: 可多次通过紫外线擦写型 (UV) 和只供一次编程型 (OTP), 它特别适用于以 32768×8 组织的单片机系统中。该器件的 FDIP28W 封装提供一个透明的小窗口, 以便使用紫外线照射完成数据的擦除。新的数据的烧入方法可以从本文档中找到。如果在只需要一次编, 可以选用以 PDIP32、PLCC32 和 TSOP28 为封装的产品。



器件名称:
M27C256B

容量:
256Kbit, 以 $32K \times 8$ 组织, 有两种产品 UV EPROM 和 OTP EPROM

封装图:
有 4 种封装:
FDIP28W ,
PDIP28 ,
PLCC32 ,
TSOP28

逻辑符号:

术语表

管脚分布图

5.4 只读存储器——ROM

- 5.4.4 EEPROM(E²PROM/electrically erasable)

电可擦写ROM直接用电信号就能实现存储器中数据的擦除和写入。

内部使用了与UV ERPOM相似的结构——栅极隔离且电荷状态代表数据状态，这种隔离的结构可以通过电信号来控制电荷是否充斥。

5.4 只读存储器——ROM

• 5.4.4 EEPROM(实例介绍EEPROM 28C256)

A14	1	28	Vcc
A12	2	27	$\overline{WE}$
A7	3	26	A13
A6	4	25	A8
A5	5	24	A9
A4	6	23	A11
A3	7	22	$\overline{OE}$
A2	8	21	A10
A1	9	20	$\overline{CE}$
A0	10	19	I/O7
I/O0	11	18	I/O6
I/O1	12	17	I/O5
I/O2	13	16	I/O4
GND	14	15	I/O3



管脚术语	解 释
A0~A14	地址输入
I/O0~I/O7	数据输入/输出
$\overline{CE}$	片选
$\overline{OE}$	输出使能
$\overline{WE}$	写操作使能
Vcc	供电端
GND	接地端

5.4 只读存储器——ROM

• 5.4.5 FLASH存储器

FLASH存储器混合了EPROM和EEPROM的技术，对整块数据进行操作，可以进行电擦写。

FLASH存储器的存储单元，由一个隔离栅极（或称为浮栅极）的场效应管构成。

常常使用多层存储单元的结构，能在一个存储单元中存储多于1位的数据。



5.4 只读存储器——ROM

• 5.4.5 FLASH存储器(NOR FLASH)

存储单元与标准的MOSFET场效应管相似，但它有两个栅极——控制栅极（CG），浮栅极（FG）。浮栅极被绝缘的氧化物层隔离着，一旦有电子经过，浮栅极就捕获电子，电子数量决定栅极电压阈值，电压阈值决定是否产生电流，数据的状态由电流的存在与否决定，从而实现信息的存储。

多层存储单元结构中，每一个存储单元中存储多于1位的数据，电流的强度决定数据状态。

NOR FLASH的传输效率很高，但写入和擦除速度低。

5.4 只读存储器——ROM

• 5.4.5 FLASH存储器(NAND FLASH)

NAND FLASH写入和擦除的速度很快，所有的U盘都使用NAND FLASH作为存储介质。

选择FLASH的权衡因素：

- ✓ NOR的读速度比NAND稍快一些。
- ✓ NAND的写入速度比NOR快很多。
- ✓ NAND的2~4ms擦除速度远比NOR的0.6~5s快。
- ✓ NAND的擦除单元更小，相应的擦除电路更少。

5.4 只读存储器——ROM

- 5.4.5 FLASH存储器(NOR和NAND FLASH比较)

接口：NOR 带有SRAM接口

NAND 使用复杂的I/O口来串行地存取数据

成本和容量：

NAND单元尺寸几乎是NOR的一半

NAND 价格较低

可靠性和耐用性：

NAND最大擦写次数是一百万次

NOR 擦写次数是十万次

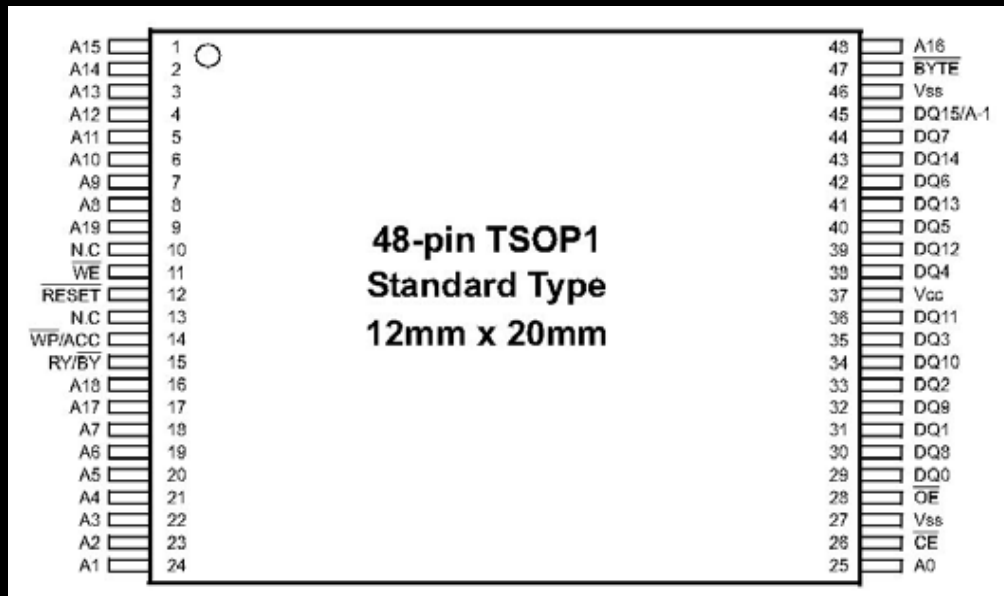
使用方便性：

NOR 可以在上面直接运行代码，不需软件支持

5.4 只读存储器——ROM

• 5.4.5 FLASH存储器(实例简介K8D1716UTC)

K8D1716UTC是SAMSUNG公司的一款容量为16MB的NOR FLASH存储器，它以2MB×8的结构来组织。



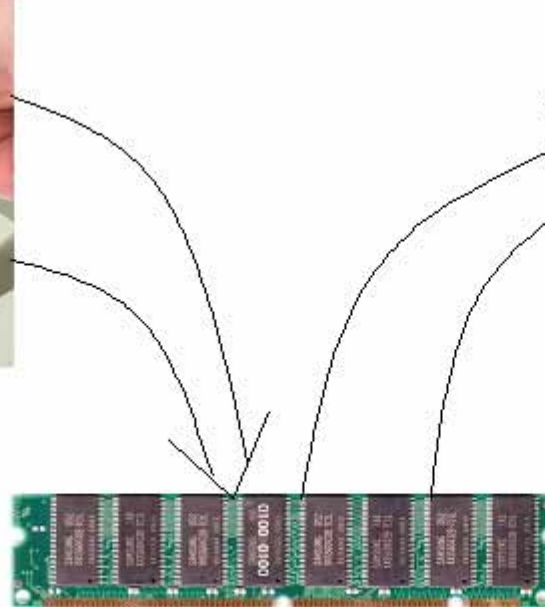
管脚术语	解 释
A0~A19	地址线
DQ0~DQ14	数据输入/输出
DQ15/A-1	DQ15 数据输入/输出 A-1 LSB 地址线
$\overline{\text{BYTE}}$	字/字节选择控制线
$\overline{\text{CE}}$	片选
$\overline{\text{OE}}$	输出使能
$\overline{\text{RESET}}$	硬件复位端
$\overline{\text{RY/BY}}$	准备/繁忙信号输出端
$\overline{\text{WE}}$	写操作使能
$\overline{\text{WP/ACC}}$	硬件写保护/编程加速
Vcc	电源正极
Vss	接地端
N.C.	空脚，不接

5.5 实例点拨

——数据是如何保存在存储器中的



输入字母 D



内存中产生 0100 0100



显示器显示 D



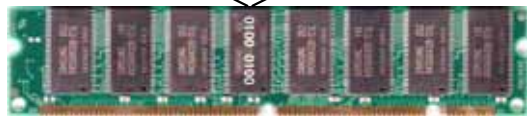
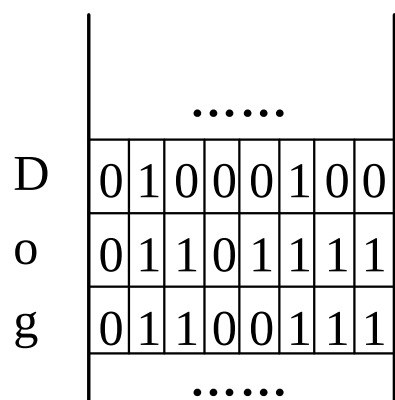
硬盘中存储 0100 0100

字母D的产生、显示、存储

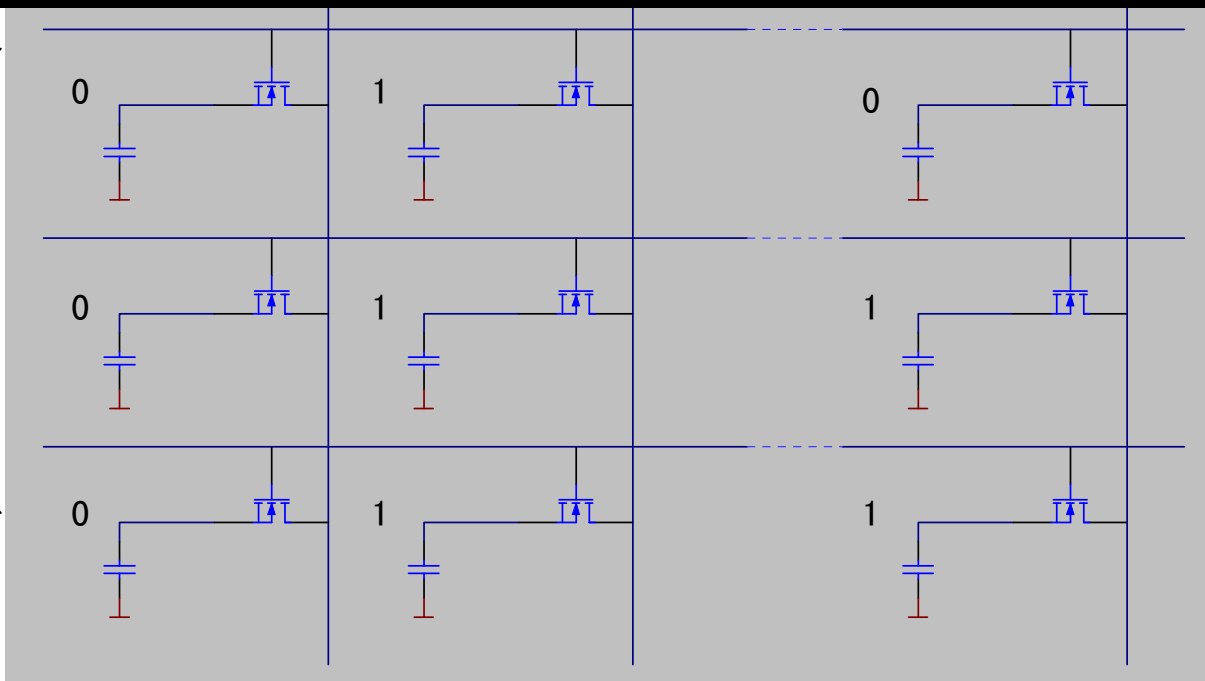
5.5 实例点拨

——数据是如何保存在存储器中的

• 5.5.1 文字数据与存储器



(a) 内存中的 Dog



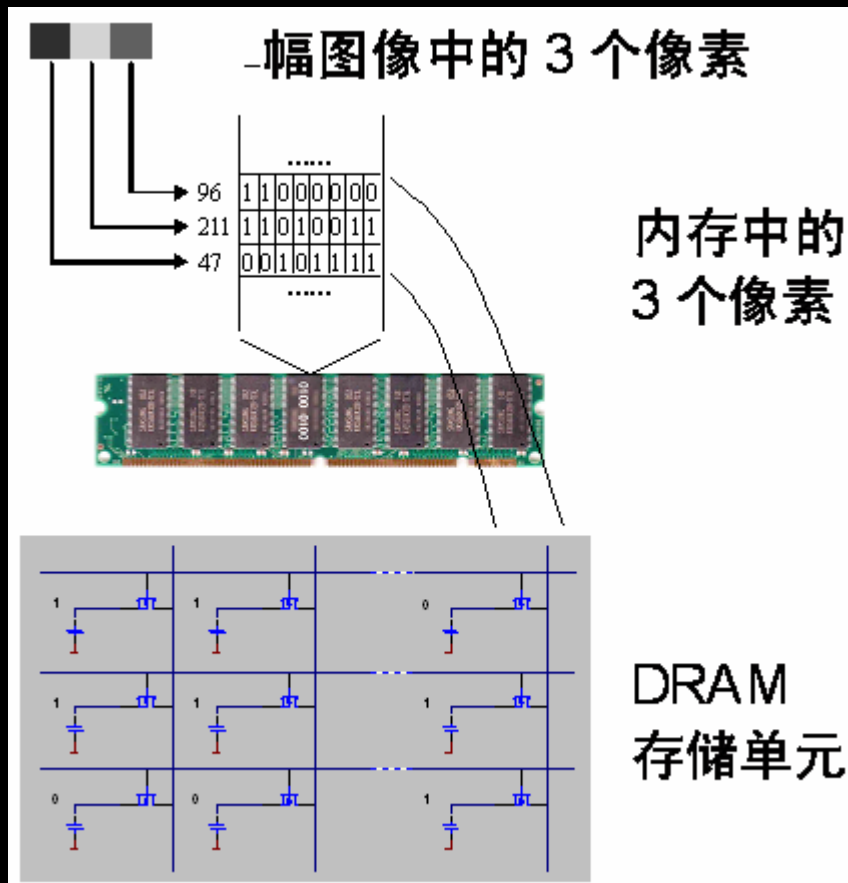
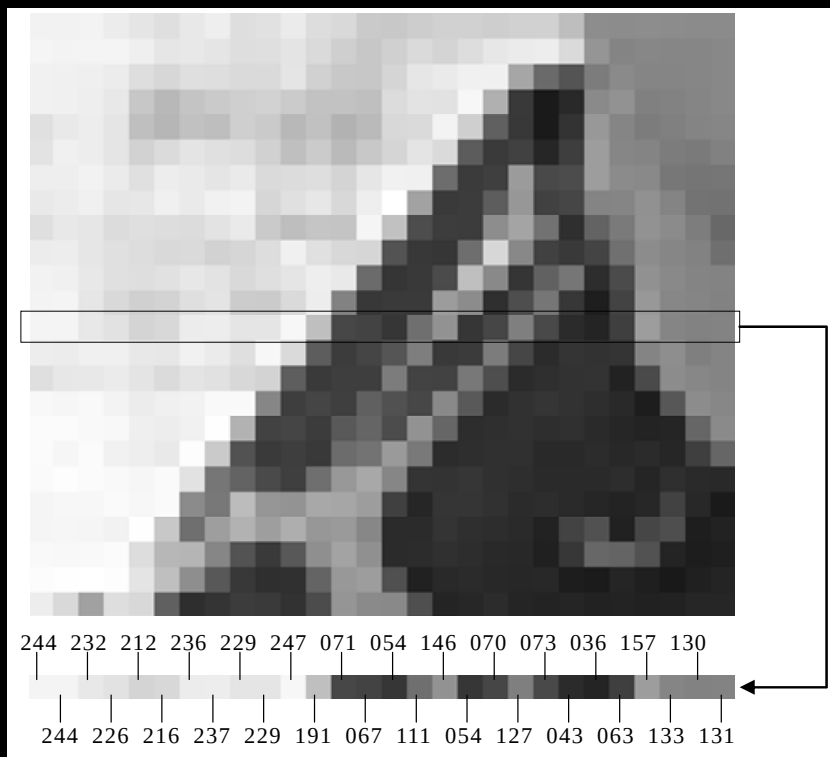
(b) DRAM 存储单元

单词Dog的存储

5.5 实例点拨

——数据是如何保存在存储器中的

• 5.5.2 图像数据与存储器



图像由不同灰度值的像素组成

存储3个像素的灰度值

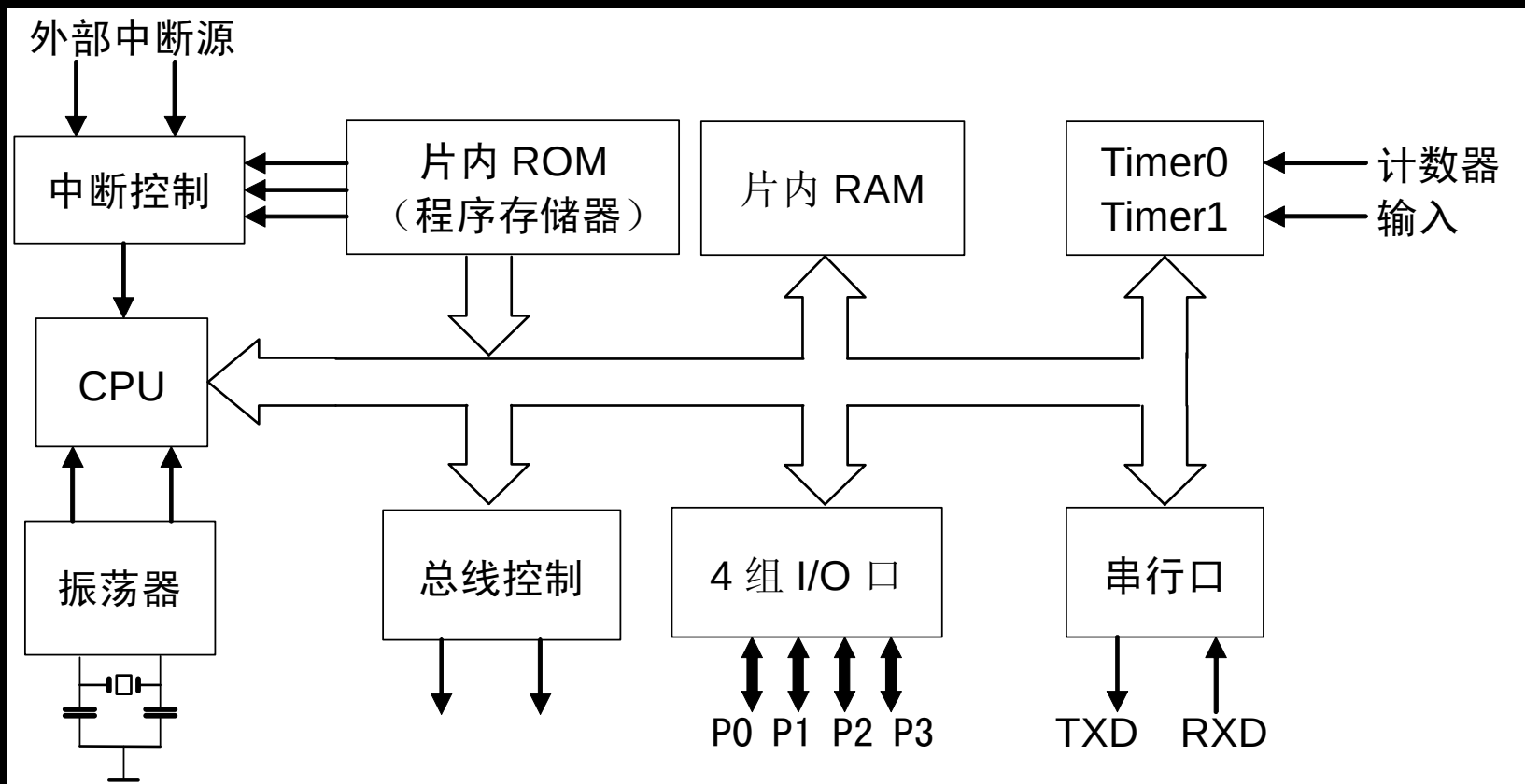
第6章 探寻单片机内部 ——存储器组织

欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 6.1 单片机的程序存储器
- 6.2 单片机的数据存储器
- 6.3 单片机的特殊功能寄存器SFR
- 6.4 实例点拨——用取表方式控制流水灯

- Intel公司首推的8051单片机有128 Bytes的片内RAM、4K Bytes的片内ROM、两个Timer、1个串行口和4组I/O口。



8051单片机的内部结构框图

6.1 单片机的程序存储器

• 6.1.1 单片机的程序下载到哪里

程序经过成功地汇编后，就会生成一个**.HEX**的文件（十六进制代码），十六进制代码很容易就转换成二进制代码，按一定的顺序（地址）下载到单片机的程序存储器（**program memory**）中。

AT89S51单片机中的程序存储器就是一个**FLASH**存储器。

除了使用**FLASH**存储器作为程序存储器外，**PROM**、**EPROM**等存储器也用在一些单片机中。

还有一些单片机甚至连程序存储器都没有，这种单片机称为**ROMless**单片机，必须通过读取外部的程序存储器中的程序才能正常使用。

6.1 单片机的程序存储器

• 6.1.2 AT89S51单片机的程序存储器

4K Bytes可在线编程的FLASH存储器，可用0000H~0FFFH来指向，如果把这1000H个字节的存储空间比喻成一幢大楼，大楼的楼层可表示为：0000H层~0FFFH层，共1000H层。

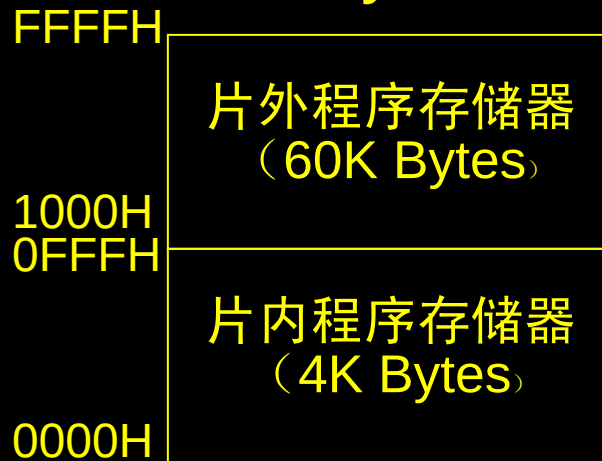
	0FFFH	1	1	0	0	1	0	1	1
	0FFEh	0	0	1	1	0	1	0	1
	...								
	...								
	0002H	1	1	1	0	0	0	1	1
PC →	0001H	1	0	0	0	0	0	0	1
	0000H	0	0	0	0	1	1	0	0

AT89S51单片机的片内程序存储器

6.1 单片机的程序存储器

• 6.1.2 AT89S51单片机的程序存储器(容量)

在单片机中，有一个程序计数器（**program counter, PC**）用于告诉单片机下一条将要执行的指令的地址。程序计数器**PC**是一个两个字节（**16位**）的寄存器，故**AT89S51**单片机最大的寻址范围是**0000H~FFFFH**，共**64K Bytes**。

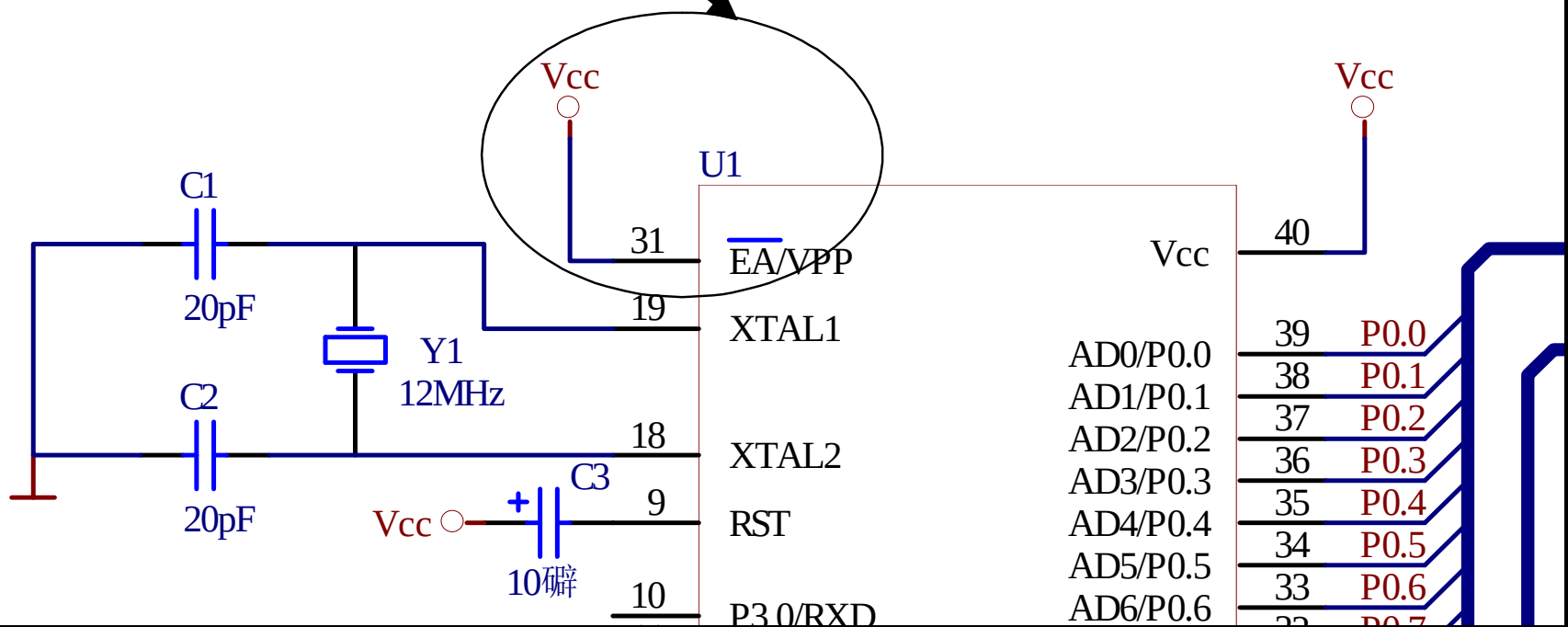


AT89S51单片机的片内片外程序存储器总容量

6.1 单片机的程序存储器

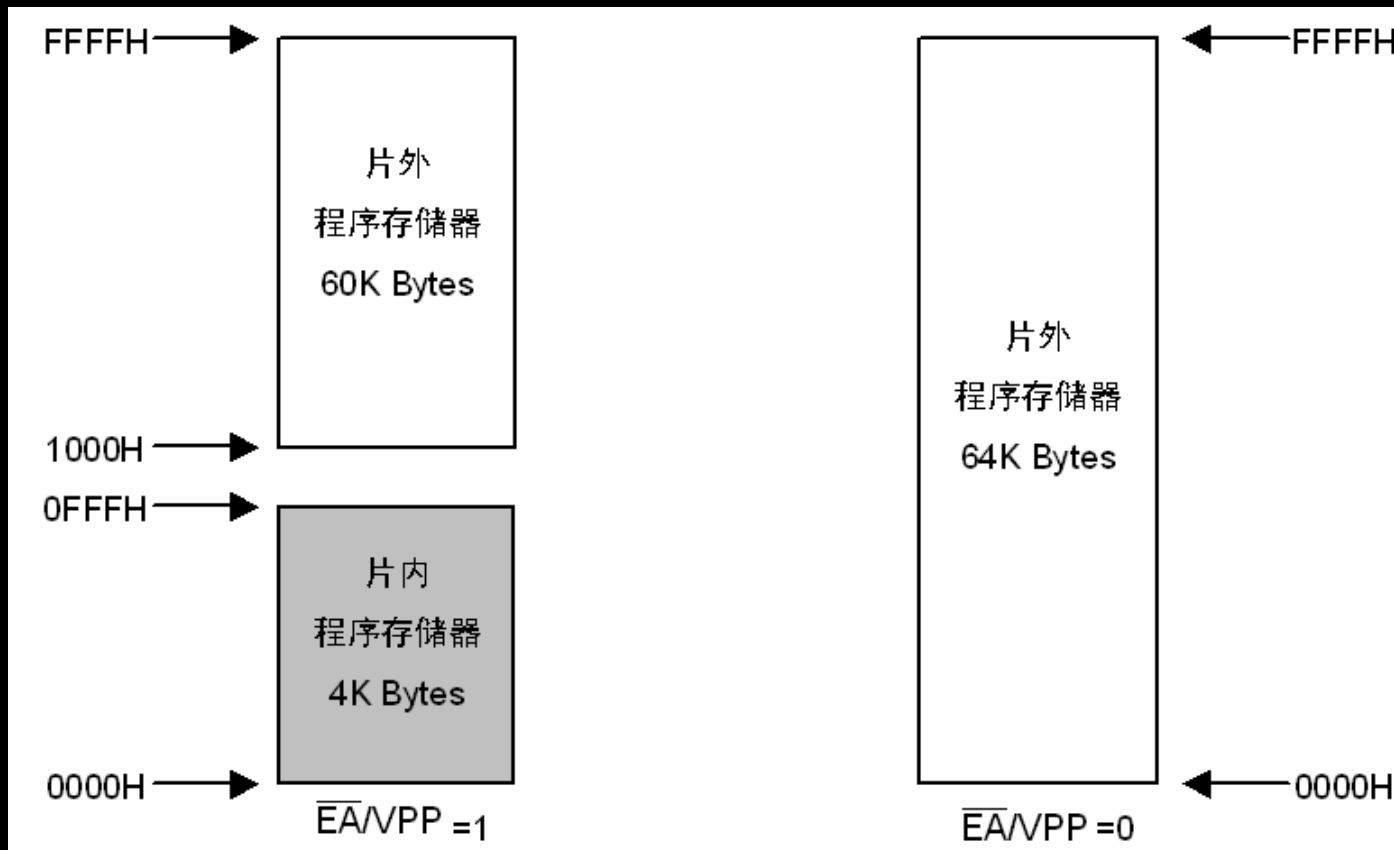
• 6.1.3 是片内还是片外程序存储器

$\overline{\text{EA/VPP}}$ 管脚，接高电平，单片机
执行片内程序存储器中的程序



6.1 单片机的程序存储器

• 6.1.3 是片内还是片外程序存储器



$\overline{EA}/VPP=0$ 时，全部自动执行片外程序存储器

$\overline{EA}/VPP=1$ 时，先执行片内程序存储器，执行完4K Bytes后，自动执行片外程序存储器。

6.2 单片机的数据存储器

- 6.2.1 数据存储器用来作什么

AT89S51单片机提供容量为 $128 \times 8\text{-bit}$ 的片内RAM，用于存储运行时产生的数据的地方，也称这个片内RAM为单片机的数据存储器。

例如指令“MOV 30H, R1”在执行时，工作寄存器R1中的数据载入地址30H中，这30H就是指单片机数据存储器的地址。

6.2 单片机的数据存储器

- 6.2.2 片内和片外数据存储器

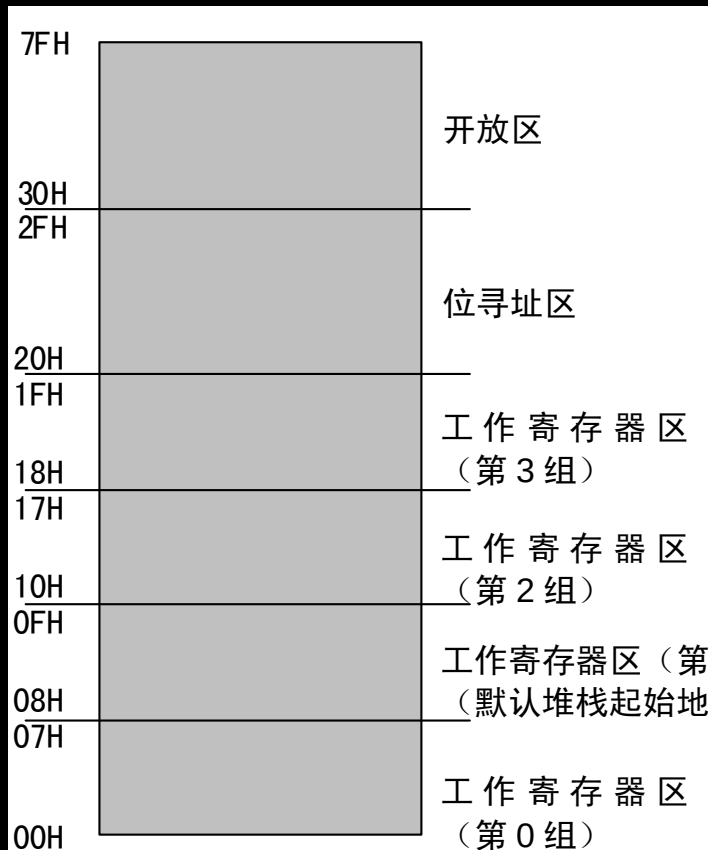
片内数据存储器就是单片机中原有的数据存储器，即片内**RAM**。

片外数据存储器是外部扩展的部分，它可以由**SRAM**来充当。

访问数据存储器时可通过数据指针**DPTR**进行。数据指针**DPTR**是一个长度为两个字节（**16位**）的寄存器。最大的数据存储器寻址范围也为**0000H~FFFFH**。

6.2 单片机的数据存储器

• 6.2.2 片内和片外数据存储器



AT89S51单片机数据存储器结构 (128×8-bit)

在单片机运行期间，**128×8-bit**的片内数据存储器并不是完全开放给用户使用的，只有其中的**20H~7FH**共**96**个字节（开放区+位寻址区）是给用户使用的，而**00H~1FH**一般先不去直接使用。

6.2 单片机的数据存储器

• 6.2.3 00H~1FH——工作寄存器区(组别映射)

第0组		第1组		第2组		第3组	
地址	工作寄存器	地址	工作寄存器	地址	工作寄存器	地址	工作寄存器
00H	R0	08H	R0	10H	R0	18H	R0
01H	R1	09H	R1	11H	R1	19H	R1
02H	R2	0AH	R2	12H	R2	1AH	R2
03H	R3	0BH	R3	13H	R3	1BH	R3
04H	R4	0CH	R4	14H	R4	1CH	R4
05H	R5	0DH	R5	15H	R5	1DH	R5
06H	R6	0EH	R6	16H	R6	1EH	R6
07H	R7	0FH	R7	17H	R7	1FH	R7

工作寄存器组别映射的片内数据存储器地址

6.2 单片机的数据存储器

- 6.2.3 00H~1FH——工作寄存器区(组别选择)
更改程序状态字PSW中的第3位(RS0)和第4位RS1)，改变当前程序使用的工作寄存器组别。

程序状态字PSW								当前工作寄存器组别
CY	AC	F0	RS1	RS0	OV	保留	P	
×	×	×	0	0	×	×	×	第0组 (00H~07H)
×	×	×	0	1	×	×	×	第1组 (08H~0FH)
×	×	×	1	0	×	×	×	第2组 (10H~17H)
×	×	×	1	1	×	×	×	第3组 (18H~1FH)

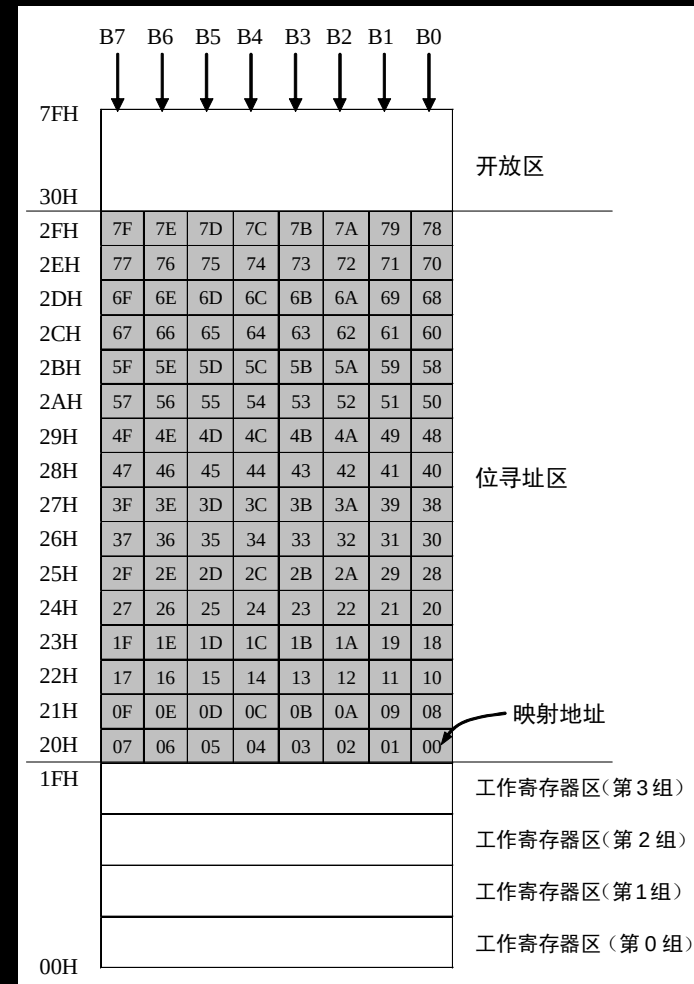
程序状态字PSW与工作寄存器组别

6.2 单片机的数据存储器

• 6.2.4 20H~2FH——位寻址区

片内数据存储器的20H~2FH（共16个字节）为位寻址区，这16个字节共有 $16 \times 8 = 128$ 位可进行位寻址，也就是可以以位为单位进行操作。所用到的指令是SETB——置1； CLR——清0； CPL——取反； ANL——“与”操作等。

20H~2FH位寻址区也可以当成是一般的开放区使用。



每个地址的映射地址

6.2 单片机的数据存储器

- 6.2.5 30H~7FH——开放区

30H~7FH是片内数据存储器中开放给用户使用的地址空间。用户可以在这个空间里存储系统运行时产生的数据，也可以读取存储的数据到工作寄存器、累加器等中。

一般来说，AT89S51单片机的开放区可以包括位寻址区20H~2FH，所以，AT89S51单片机开放给用户使用的空间为20H~7FH共96个字节。

6.3 单片机的特殊功能寄存器SFR

特殊功能寄存器SFR (special function registers)是用来对单片机内各功能模块进行管理、控制的控制寄存器，也是指示单片机运行状态的状态寄存器，它是一个具有特殊功能的RAM区。

累加器ACC，程序状态字PSW，I/O口P0~P3都在特殊功能寄存器中。

其地址映射空间紧接着片内数据存储器(00H~7FH)为80H~0FFH。

6.3 单片机的特殊功能寄存器SFR

• 6.3.1 特殊功能寄存器SFR分布图

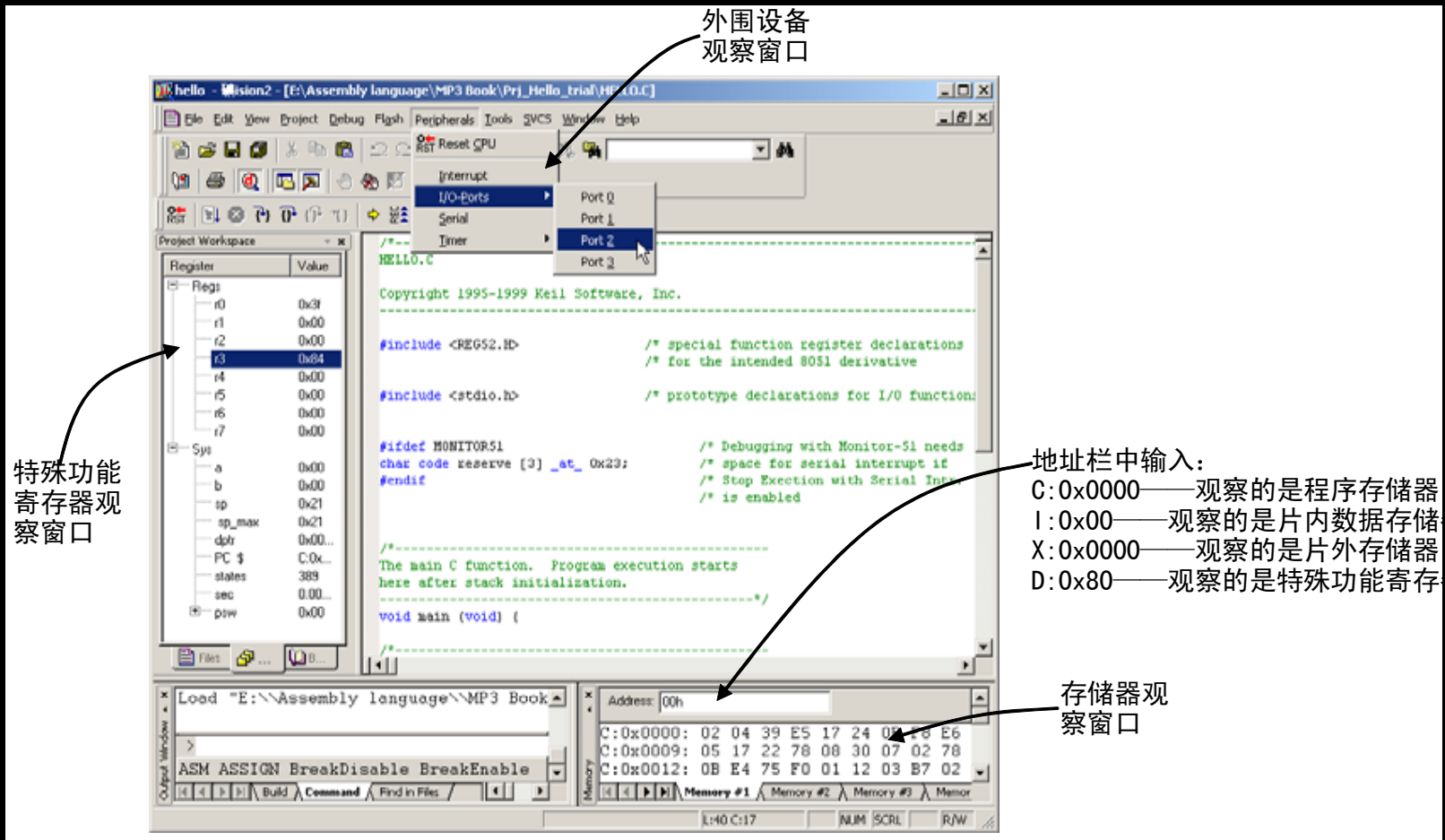
0F8H									0FFH
0F0H	B								0F7H
0E8H									0EFH
0E0H	ACC								0E7H
0D8H									0DFH
0D0H	PSW								0D7H
0C8H									0CFH
0C0H									0C7H
0B8H	IP								0BFH
0B0H	P3								0B7H
0A8H	IE								0AFH
0A0H	P2		AUXR1				WDTRST		0A7H
98H	SCON	SBUF							9FH
90H	P1								97H
88H	TCON	TMOD	TL0	TL1	TH0	TH1	AUXR		8FH
80H	P0	SP	DP0L	DP0H	DP1L	DP1H		PCON	87H

没有被占用的地址有可能在单片机中不存在。

AT89S51单片机特殊功能寄存器分布图

6.3 单片机的特殊功能寄存器SFR

• 6.3.1 特殊功能寄存器SFR分布图



μVision中的特殊功能寄存器及外围设备观察窗口

6.3 单片机的特殊功能寄存器SFR

• 6.3.2 特殊功能寄存器的操作方法

——字节操作与位操作

AT89S51单片机中支持位操作的特殊功能寄存器是：

- ✓P0（P0口锁存器）
- ✓TCON（定时/计数器控制寄存器）
- ✓TMOD（定时/计数器模式控制寄存器）
- ✓P1（P1口锁存器）
- ✓SCON（串行口控制寄存器）
- ✓P2（P2口锁存器）
- ✓IE（中断使能寄存器）
- ✓P3（P3口锁存器）
- ✓IP（中断优先控制寄存器）
- ✓PSW（程序状态字寄存器）
- ✓ACC（累加器）
- ✓B（B寄存器）

字节操作  MOV P1,#00H

位操作  CLR P0.0
CLR P0.1
CLR P0.2
CLR P0.3
CLR P0.4
CLR P0.5
CLR P0.6
CLR P0.7

6.3 单片机的特殊功能寄存器SFR

• 6.3.3 特殊功能寄存器简介

AT89S51单片机的特殊功能寄存器共有26个。

- ✓ P0 (80H) ——P0口锁存器
- ✓ SP (81H) ——堆栈指针
- ✓ DPxL和DPxH (82H~85H) ——数据指针 (x代表0或1)
- ✓ PCON (87H) ——电源控制寄存器
- ✓ TCON (88H) ——定时/计数器控制寄存器
- ✓ TOMD (89H) ——定时/计数器模式控制寄存器
- ✓ TL0、TL1、TH0、TH1 (8A~8DH) ——Timer寄存器

6.3 单片机的特殊功能寄存器SFR

• 6.3.3 特殊功能寄存器简介

AT89S51单片机的特殊功能寄存器共有26个。

- ✓ AUXR (8EH) ——辅助寄存器
- ✓ P1 (90H) ——P1口锁存器
- ✓ SCON (98H) ——串行口控制寄存器
- ✓ SBUF (99H) ——串口缓冲寄存器
- ✓ P2 (0A0H) ——P2口锁存器
- ✓ AUXR1 (0A2H) ——辅助寄存器1
- ✓ WDTRST (0A6H) ——看门狗复位控制端

6.3 单片机的特殊功能寄存器SFR

• 6.3.3 特殊功能寄存器简介

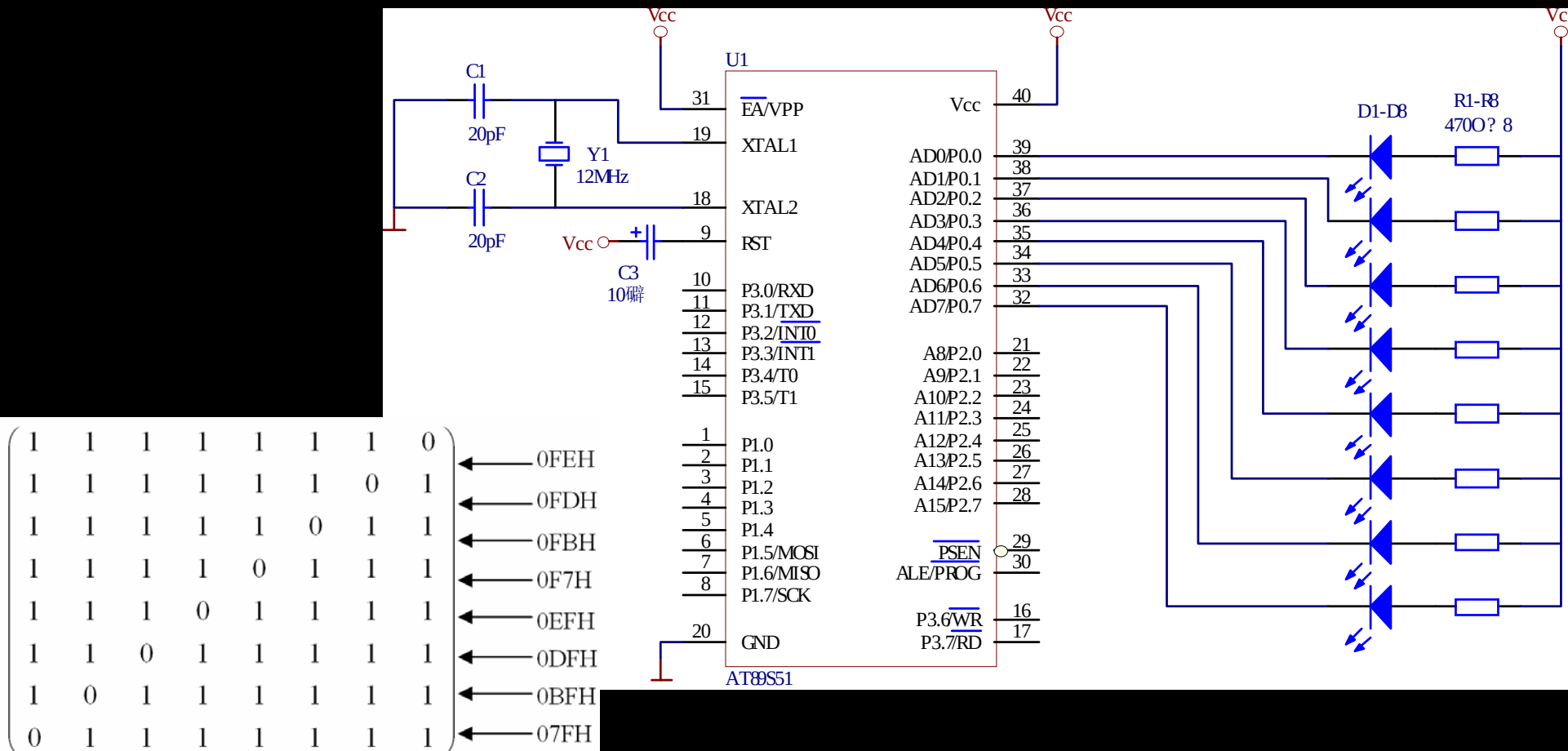
AT89S51单片机的特殊功能寄存器共有26个。

- ✓ IE (0A8H) ——中断使能寄存器
- ✓ P3 (0B0H) ——P3口锁存器
- ✓ IP (0B8H) ——中断优先级控制寄存器
- ✓ PSW (0D0H) ——程序状态字
- ✓ ACC (0E0H) ——累加器
- ✓ B (0F0H) ——B寄存器

6.4 实例点拨

——用取表方式控制流水灯

• 6.4.1 流水灯的显示数据转换



6.4 实例点拨

——用取表方式控制流水灯

• 6.4.2 取表法

数据存在依次取用的特点，一般用取表的方法。

在汇编程序末尾附加一个数据表，这个数据表可以是字节型的数据，也可是英文字母等数据。数据表与汇编程序一起被汇编，数据表就成为程序的一部分包含在生成的.HEX文件中。

```
TABLE:                                ;本程序的数据表
DB      0FEH, 0FDH, 0FBH, 0F7H      ; 左移
DB      0EFH, 0DFH, 0BFH, 07FH
DB      07FH, 0BFH, 0DFH, 0EFH      ; 右移
DB      0F7H, 0FBH, 0FDH, 0FEH
DB      88H                          ; 结束码
```


6.4 实例点拨

——用取表方式控制流水灯

• 6.4.2 取表法

当程序下载到单片机后，指令的执行代码和数据表中的数据一起保存在程序存储器中，存在存储器的某一段地址空间中。而数据表的名称TABLE就像一个指针，指向第一个数据0FEH所在的地址。

程序存储器地址	程序存储器中的指令代码																
0000H	90	00	1F	E4	93	B4	01	02	-	80	F6	F5	80	7B	14	12	00
0010H	14	A3	80	EF	7C	14	7D	F8	-	DD	FE	DC	FA	DB	F6	22	FE
0020H	FD	FB	F7	EF	DF	BF	7F	7F	-	BF	DF	EF	F7	FB	FD	FE	88

TABLE

数据表中的数据依次排列在程序存储器

6.4 实例点拨

——用取表方式控制流水灯

• 6.4.2 取表法(取表程序介绍)

```

START:                                ;装表, 即 DPTR 指向表 TABLE 的表头
MOV    DPTR, #TABLE                    ; 数据指针寄存器 DPTR 指向表 TABLE 头
LOADTABLE:                             ;取表程序段, 表中的数据载入累加器 ACC 中, 并判断是否已经完成
CLR    A                                ; 累加器 ACC 清 0
MOVC   A, @A+DPTR                      ; 基址加变址寻址, 将表中数据载入 ACC
CJNE   A, #88H, DISPLAY                ; 如果取到 88H 表明表已经取完, 否则显示
JMP     START                          ; 如果取完, 跳回 START 重新开始
DISPLAY:                               ; 显示程序段, 累加器 ACC 的数据输出到 P0
MOV    P0, A                           ; 将 ACC 中数据输出至 P0 口
CALL   DELAY                           ; 调延时子程序
INC    DPTR                            ; DPTR 加 1, 准备取表中的下一个数据
JMP    LOADTABLE                       ; 跳回取表程序段

```

6.4 实例点拨

——用取表方式控制流水灯

• 6.4.3 方法总结

取表法的实质是把存放在程序存储器中的数据，通过@A+DPTR指针来寻址，并载入ACC中，再经过指令“MOV P0,A”将数据从I/O口输出。



取表法设计思路

第7章 单片机“起舞” ——指令的执行

欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 7.1 汇编语言详谈
- 7.2 时钟与指令的执行
- 7.3 寻址方式
- 7.4 实例点拨——程序存储器和数据存储器的寻址

7.1 汇编语言详谈

- 汇编语言很原始，直接与计算机的机器码打交道。
- 高级语言让程序员减少出错率和提高工作效率。

（用汇编语言写成的）汇编程序

（汇编器）的汇编

（十六进制的）执行代码

（用高级语言写成的）高级语言程序

（编译器）的编译

（可执行的）文件

汇编与编译的过程

7.1 汇编语言详谈

• 7.1.1 汇编器

几个定义：

- 指令（**instruction**）：控制单片机的命令行
- 汇编程序（**assembly program**）：控制目标系统（如单片机）实现特定功能的指令集合。
- 汇编语言（**assembly language**）：它是一种低级的计算机语言。
- 源文件（或源代码，**source code file**）：以.asm为后缀的文件，其中包含一条一条的指令。
- 目标文件（或目标代码，**object code**）：编译器/汇编器从源代码文件中生成的文件。
- 执行代码（**opcode or operation code**）：指定了操作如何执行，如.HEX文件中的十六进制代码。

7.1 汇编语言详谈

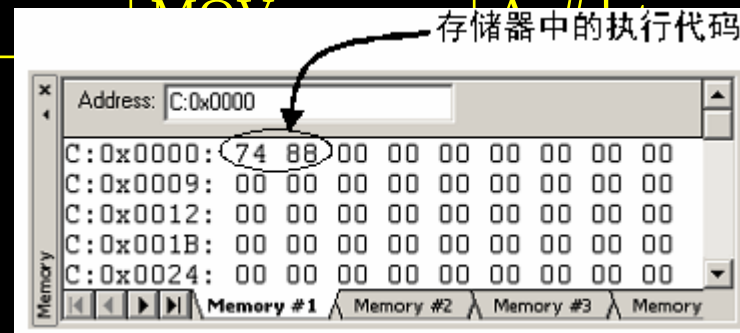
7.1.1 汇编器

汇编器的功能为：把（源文件中的）指令汇编成（目标文件中的）执行代码。汇编器还可以自动定义汇编程序中的标号，即自动在存储器中为标号分配地址。伪指令不会被汇编器汇编。

执行代码（十六进制）	字节数	助记符	操作数
.....			
73	1	JMP	@A+DPTR
74	2	MOV	A, #1

```
ORG    00H  
START:  
      MOV    A, #88H  
      END
```

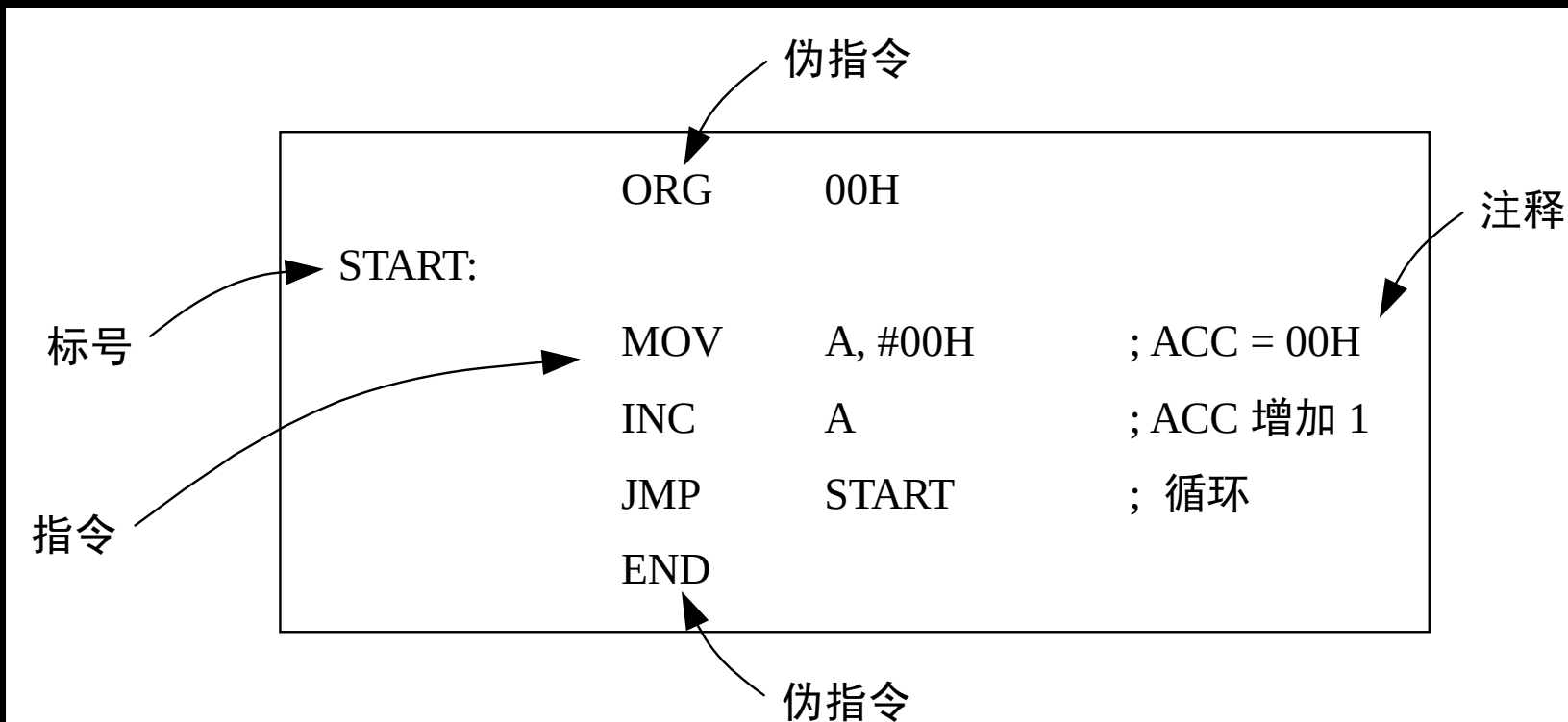
(a) 汇编程序



(b) 程序存储器中的执行代码

7.1 汇编语言详谈

• 7.1.2 汇编程序书写格式



汇编程序书写格式

7.1 汇编语言详谈

• 7.1.3 伪指令

伪指令（pseudo opcode）不被汇编器汇编，没有执行代码，但却告知汇编器一些汇编信息。

伪指令简介：

✓ BIT——用于汇编程序的一开始创建一个常量。

FLASH_COUNT BIT 3EH

； 创建一个名为FLASH_COUNT的常量，并把立即数3EH赋给这个常量，在程序中就可以直接把FLASH_COUNT等同于立即数3EH进行操作。

7.1 汇编语言详谈

• 7.1.3 伪指令

伪指令简介：

- ✓ **DATA**——定义一个指向特殊功能寄存器区地址的变量。

DPTRSW DATA 0A2H

； **DPTRSW**指向特殊功能寄存器**0A2H**地址上

- ✓ **DB**——用于汇编程序中定义若干个长度为**1**个字节的字，这若干个字使用逗号分隔开，如果逗号之间没有数据，汇编器默认为**00H**。

DB 10H, 11H, , 3FH, 20H

； 在目标文件中生成**10H, 11H, 00H, 3FH, 20H**

7.1 汇编语言详谈

• 7.1.3 伪指令

伪指令简介：

- ✓ **DS**——用于保留一块存储器空间给程序变量使用或别的用途。

STORAGE DS 10

；保留一块名叫“STORAGE”的10字节存储空间

- ✓ **DW**——定义若干个长度为两个字节的字，这若干个字使用逗号分隔开，如果逗号之间没有数据，汇编器默认为0000H。

DW 0FFFEH, , 0102H

；在目标文件中生成代码：FFH，FEH，00H，00H，01H，02H

7.1 汇编语言详谈

• 7.1.3 伪指令

伪指令简介：

- ✓ **END**——该伪指令告诉汇编器程序的结束点。
- ✓ **EQU**——定义某一个符号的值，一旦一个符号被定义后，就不能被另一个**EQU**或**SET**指令重复定义。

BEEP_COUNT EQU 1+1

；表达式把2定义给符号**BEEP_COUNT**

7.1 汇编语言详谈

• 7.1.3 伪指令

伪指令简介：

- ✓ IF、ELSE、ENDIF——这3个伪指令是条件选择语句，它们告诉汇编器根据表达式的值，是否汇编某一块程序，没有汇编的块在目标文件中是不存在对应的执行代码的。

IF P1.0; 如果P1.0=1，就汇编下一行

DB 01H, 02H, 03H; 在存储器中定义字01H、02H、03H

ENDIF; 条件选择结束，如果P1.0≠1，上一行不被汇编

7.1 汇编语言详谈

• 7.1.3 伪指令

伪指令简介：

- ✓ **INCL**——该伪指令用于在汇编时把其他文件与当前文件结合在一起汇编。

INCL **"const.def"**

； 即把文件“const.def”与当前文件结合在一起汇编。

- ✓ **ORG**——该伪指令用于设置程序计数器PC的初始值。

ORG **0000H**

； 指令的执行代码在单片机的程序存储器中从0000H开始存储（也可简单写成00H）。

7.1 汇编语言详谈

• 7.1.3 伪指令

伪指令简介：

- ✓ **SET**——该伪指令类似**EQU**，但不同的是**SET**可以通过另一个**SET**伪指令重复定义变量的值。

```
COUNT          SET      3
COUNT          SET      1 ; 最终COUNT=1
```

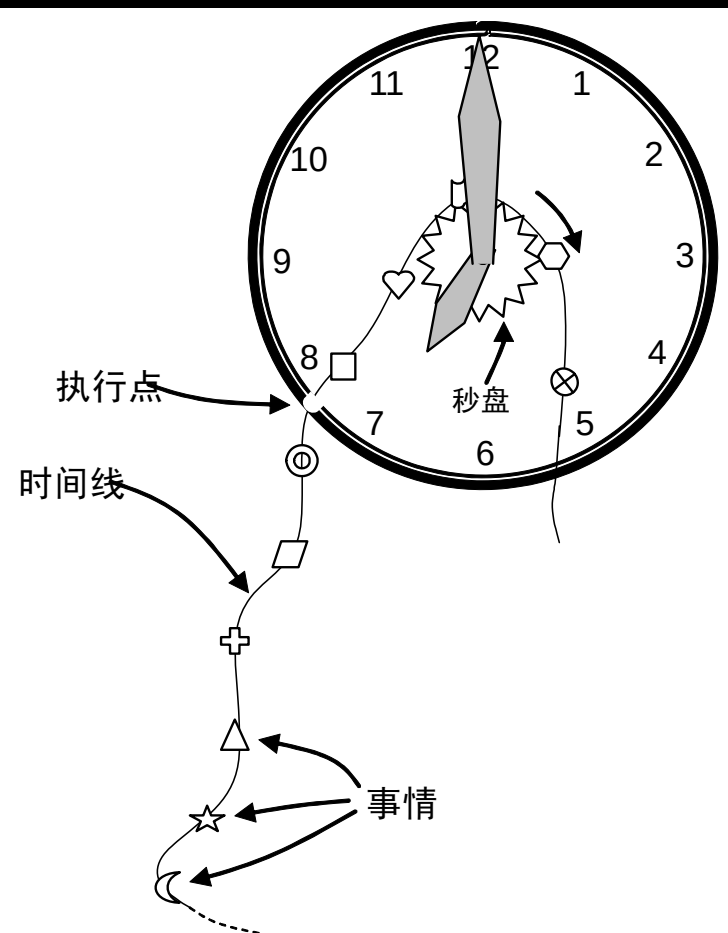
- ✓ **\$**——美元符号表示当前地址，意味着程序计数器**PC**的值不变，在程序中表示“本行程序”。

```
DJNZ            R5,     $
```

工作寄存器**R5**减1，如果不等于0就跳回本行——直到**R5**减至0为止，执行下一条指令。

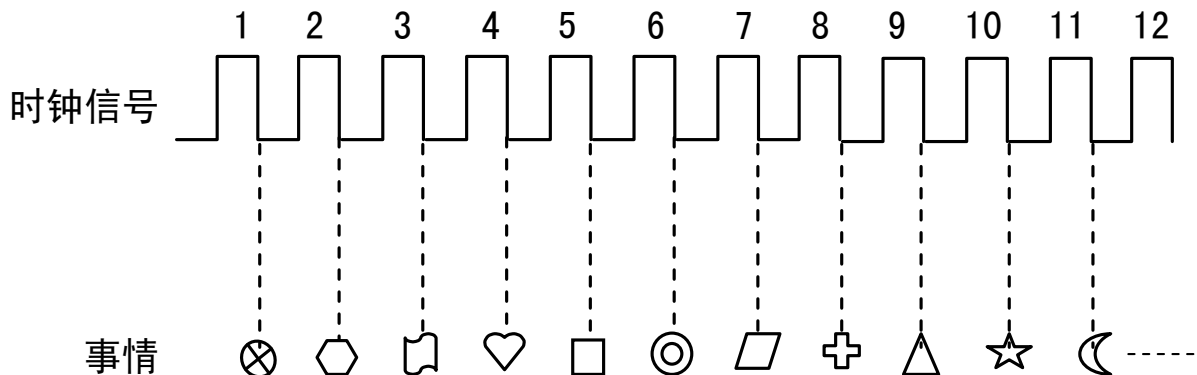
7.2 时钟与指令的执行

• 7.2.1 时钟究竟是什么



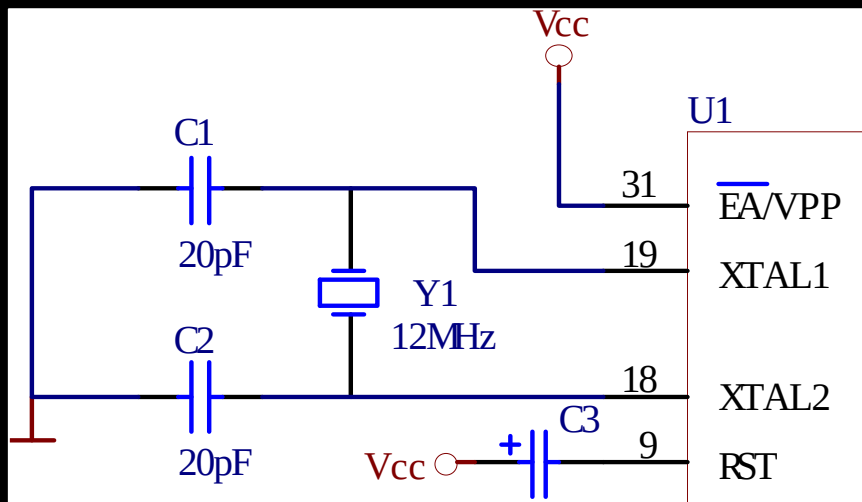
◁ 事情与时间线

▽ 时钟信号波形与事情

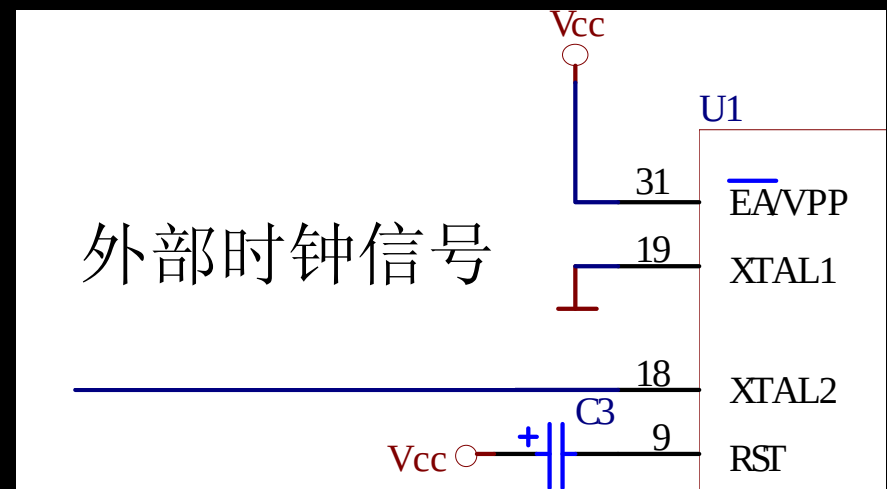


7.2 时钟与指令的执行

• 7.2.1 时钟究竟是什么(单片机的时钟信号源)



内部时钟



外部时钟

7.2 时钟与指令的执行

- 7.2.1 时钟究竟是什么(有关时钟信号周期的概念)
 - ✓ 振荡周期——振荡器产生的时钟信号的周期。
 - ✓ 机器周期——1个机器周期=12个振荡周期。
 - ✓ 指令周期——执行一条指令所占用的全部时间，一般为1~4个机器周期。

例：使用的是内部时钟，晶振频率为12MHz，则有：

振荡周期=1/晶振频率=1/12μs

机器周期=12×振荡周期=1μs

“DJNZ R4,DELAY”的指令周期为两个机器周期，执行时间为2×机器周期=2×1μs =2μs

7.2 时钟与指令的执行

• 7.2.2 程序计数器PC的角色

程序计数器PC，它是一个两个字节长度的寄存器，这个寄存器中的数值告诉单片机要到哪一个地址去找下一条指令来执行。

程序一开始定义“ORG 00H”，程序计数器PC从0000H开始，当第一条指令执行后，PC自增1，指示下一条指令从0001H中拿出执行。

程序计数器PC是不能被直接修改的，但可以使用跳转指令来改变PC的值，如指令“LJMP 1234H”执行后，PC= 1234H。

没有指令能够读PC的值。

7.2 时钟与指令的执行

• 7.2.3 指令的执行

时钟信号提供时序，程序计数器PC指示执行地址，单片机便可以顺利地执行指令。
单片机的指令执行大致分为以下5个步骤。

- ① 抓取指令——单片机通过总线向程序存储器读取执行代码和等待处理的过程。
 - ② 单片机内部数据处理——将抓取到的数据和某个寄存器做运算。
 - ③ 逻辑或标志处理——运算完成后，对逻辑或标志进行处理。
 - ④ 转移判断——依照判断的结果进行转移，条件为真时如何动作，条件为假时如何动作。
 - ⑤ 返回——将结果保存到其他寄存器或存储器中。
- 抓取数据 → 处理数据 → 存储数据

7.2 时钟与指令的执行

• 7.2.3 指令的执行(详解指令执行)

“MOV A, 32H”其执行代码为“E5 32”。

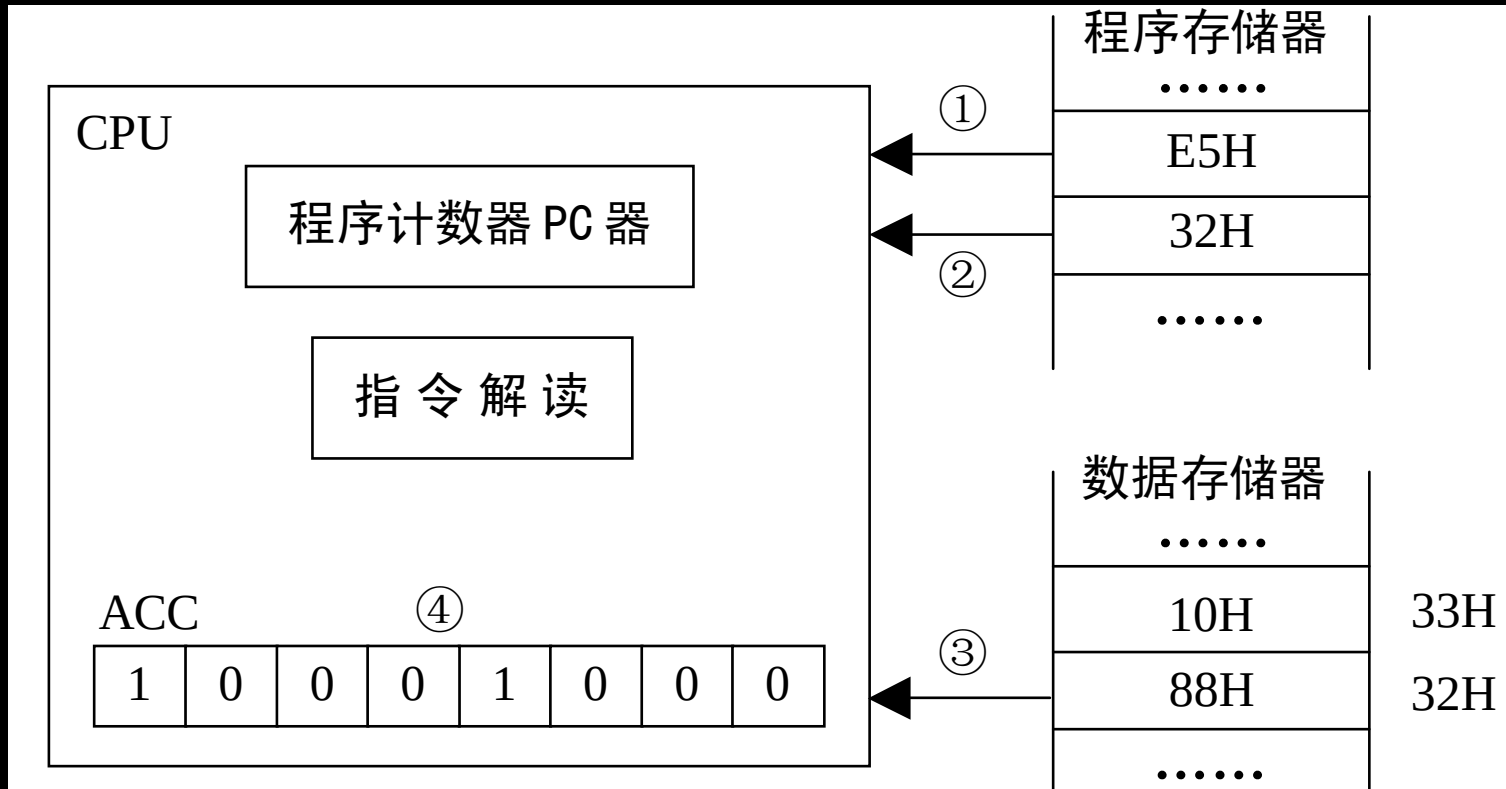
理解：

- ① 当单片机抓取“E5”时，就知道要执行的操作是把下一个执行代码所指向的数据存储器地址中的数据载入ACC。
- ② 于是单片机往下抓取下一个执行代码“32”。单片机根据这个值知道要去取数据存储器地址32H中的内容。
- ③ 抓取数据存储器32H中的内容“88H”后。
- ④ 再把88H载入到ACC中，于是最终（A）=88H。

抓取数据→处理数据→存储数据

7.2 时钟与指令的执行

• 7.2.3 指令的执行(详解指令执行)



- ① 读取执行代码E5；② 获知目标地址是32H；
③ 取目标地址中的内容；④ 数据载入累加器ACC中。

寻址示意图

7.3 寻址方式

6种寻址方式

在指令中的地址或立即数如果以十六进制中的A、B、C、D、E或F开头需要在地址或立即数前加“0”。

寻 址 方 式	例 子
直接寻址	MOV A, 30H
间接寻址	MOV A, @R0
寄存器寻址	MOV A, R0
寄存器特征寻址	INC A
立即寻址	MOV A, #8FH
变址寻址	MOVC A, @A+DPTR

7.3 寻址方式

• 7.3.1 直接寻址

直接寻址（direct addressing）是指一个直接地址的内容载入一个寄存器中或相反。单片机片内数据存储器的00H~7FH（但常用30H~7FH）以及特殊功能寄存器SFR能被直接寻址。

直接寻址的内容不包含在执行代码中，而是在单片机的数据存储器中，所以执行直接寻址指令的速度还是很快的。

```
MOV  A, 30H    ;把30H地址的内容载入累加器ACC
MOV  50H, B     ;把B寄存器的值载入50H地址内
ADD  A, 60H     ;ACC和60H地址的内容相加，结果存回ACC
```


7.3 寻址方式

• 7.3.2 间接寻址

- ✓ 间接寻址（indirect addressing）是用R0、R1、SP、DPTR中的某一寄存器来代替直接寻址中的直接地址来寻址。例如，直接寻址“MOV A, 32H”与间接寻址是相同的。

MOV R1, #32H ; R1=32H

MOV A, @R1; R1所指向的地址的内容载入ACC中

- ✓ 间接寻址可以访问片内和片外数据存储器。

MOV A, @R0 ; 取得R0所指的存储器地址的内容，并载入ACC

MOVX A, @DPTR ; 取得DPTR所指的片外数据存储器的内容，并载入ACC

7.3 寻址方式

• 7.3.3 寄存器寻址

寄存器寻址（register instructions）是指与工作寄存器R0～R7有关的寻址指令。程序状态字PSW中的RS1和RS0决定使用AT89S51单片机4组R0～R7寄存器中的哪一组。

MOV A, R0 ;将R0的值载入累加器ACC

ADD A, R5 ;将ACC与R5的值相加，并把结果存回ACC

MOV R7, A ;将ACC的值载入R7中

7.3 寻址方式

• 7.3.4 寄存器特征寻址

寄存器特征寻址（**register-specific instructions**）是一类与特定寄存器有关的寻址方式。指令不涉及用于指向的地址字节，不对其他地址和数据产生什么影响。例如：

INC A ;累加器ACC自增1

SWAP A ;ACC的高4位与低4位互换

INC DPTR ;数据指针寄存器DPTR自增1

7.3 寻址方式

- 7.3.5 立即寻址

立即寻址（immediate addressing）是一类与立即数相关的寻址方式。当程序中需要向寄存器或地址中载入某个立即数，就可采用立即寻址。立即数的特征就是在常数前加一个“#”号。这类寻址方式比较简单。

例如：

MOV A, #100 ;将立即数100（64H）载入累加器AC

MOV 33H, #10H ;将立即数10H载入数据存储器33H中

MOV R0, #0FFH ;R0=FFH

MOV DPTR, #2046H ;DPTR=2046H

7.3 寻址方式

• 7.3.6 变址寻址

变址寻址（indexed addressing）针对的是程序存储器，只能从程序存储器中读数据。通常我们对程序存储器访问得较多的是数据表中的数据。在变址寻址中，使用程序计数器PC或数据指针寄存器DPTR作为间接地址，有时还加上累加器ACC。

例如：

MOVC A, @A+DPTR ;查表操作：累加器ACC和数据指针寄存器DPTR的值相加得到一个程序存储器上的地址，将该地址的内容载入ACC中

MOVX @DPTR, A ;访问片外存储器：将ACC的值载入DPTR所指的片外存储器的地址上

7.4 实例点拨

——程序存储器和数据存储器的寻址

• 7.4.1 把程序存储器中的数据载入工作寄存器中

例：将程序存储器中的200H~202H上的“C”、“A”、“T”3个字节数据载入工作寄存器R0、R1、R2中。

```
MOV    DPTR, #200H           ; DPTR = 0200H
CLR     A                     ; ACC 清 0
MOVCA, @A+DPTR               ; 200H 上的数据载入 ACC
MOV     R0, A                 ; R0 = ACC
INC     DPTR                  ; DPTR + 1
```

7.4 实例点拨

——程序存储器和数据存储器的寻址

• 7.4.1 把程序存储器中的数据载入工作寄存器中

```
INC      DPTR                ; DPTR + 1
CLR      A                   ; ACC = 0
MOVCA,   @A+DPTR             ; 201H 上的数据载入 ACC
MOV      R1, A                ; R1 = ACC
INC      DPTR                ; DPTR + 1
CLR      A                   ; ACC = 0
MOVCA,   @A+DPTR             ; 202H 上的数据载入 ACC
MOV      R2, A                ; R2 = ACC
JMP      $                   ; 停机
```

7.4 实例点拨

——程序存储器和数据存储器的寻址

- 7.4.1 把程序存储器中的数据载入工作寄存器中

ORG 0200H; 数据表TABLE的起始地址定义为0200H

TABLE:

DB "CAT"; 数据表中的数据“CAT”

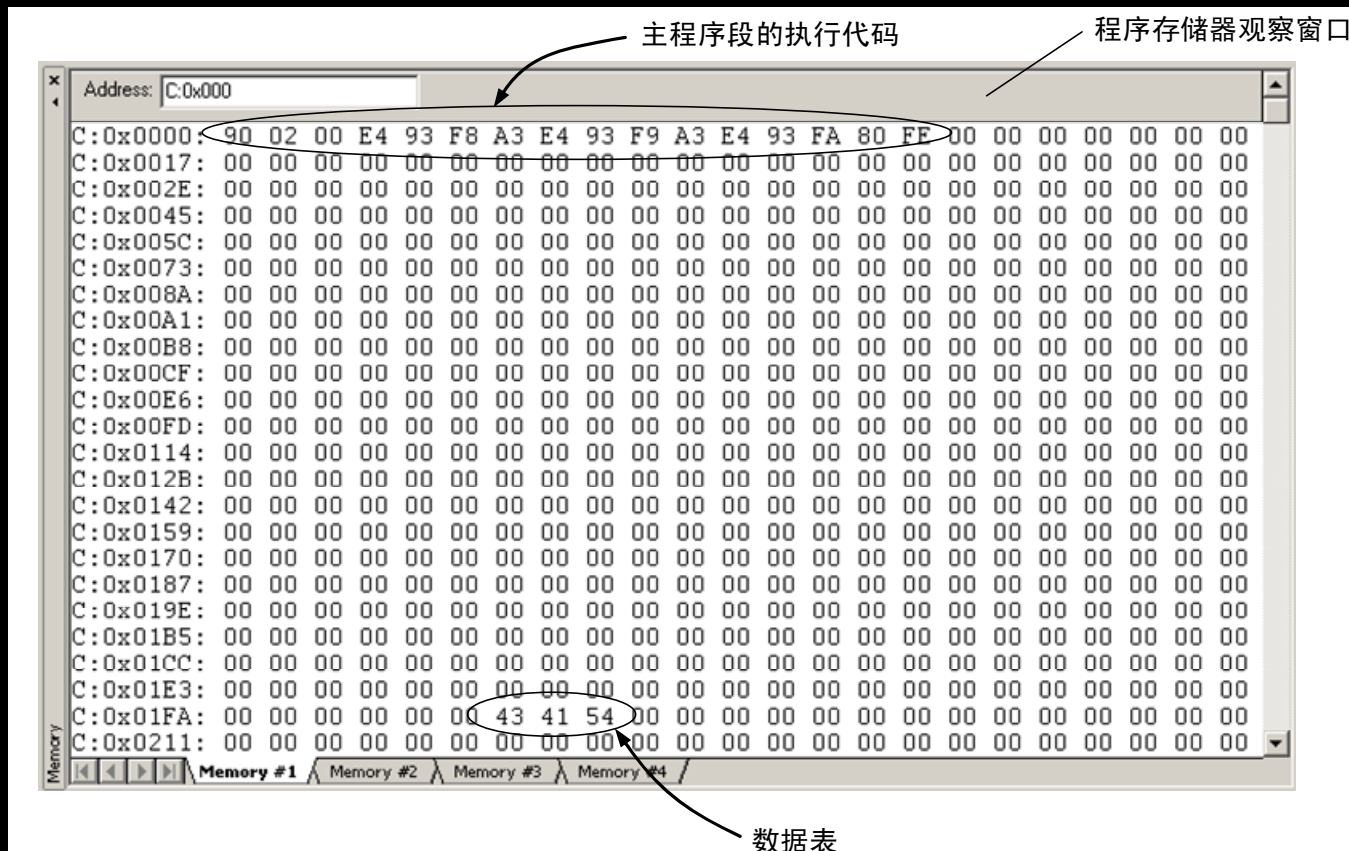
END

“MOV DPTR, #200H”用数据指针寄存器DPTR指向数据表TABLE的表头，也可以写成“MOV DPTR, #TABLE”

7.4 实例点拨

——程序存储器和数据存储器的寻址

• 7.4.1 把程序存储器中的数据载入工作寄存器中



伪指令定义数据表的起始地址

7.4 实例点拨

——程序存储器和数据存储器的寻址

• 7.4.2 把程序存储器中的数据载入数据存储器中

把程序存储器中数据表中的一个单词“CHINA”装载到数据存储器的30H~34H中。

```
ORG      000H                      ; 主程序的起始地址定义为 0000H

MOV      DPTR, #TABLE              ; DPTR 指向 TABLE
MOV      R0, #30H                  ; R0 = 30H

REPEAT:

CLR      A                        ; ACC 清 0
MOVCA, @A+DPTR                    ; 200H 上的数据载入 ACC
JZ       FINISH                   ; 如果 ACC = 0, 则跳到 FINISH
MOV      @R0, A                   ; ACC 的值载入 R0 所指的地址上
```

指令“MOV @R0, A”将ACC中的数据装到R0所指的数据存储器。

7.4 实例点拨

——程序存储器和数据存储器的寻址

• 7.4.2 把程序存储器中的数据载入数据存储器中

```
INC    DPTR                ; DPTR + 1
INC    R0                  ; R0 + 1, 即间接地址增加 1
JMP    REPEAT              ; 跳至 REPEAT

FINISH:
JMP    $                   ; 停机

ORG    250H                ; 定义数据表的起始地址为 250H

TABLE:
DB     "CHINA", 0          ; 数据表中的数据, "0" 为结束标志
```

第8章 给单片机下命令——指令畅谈

欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

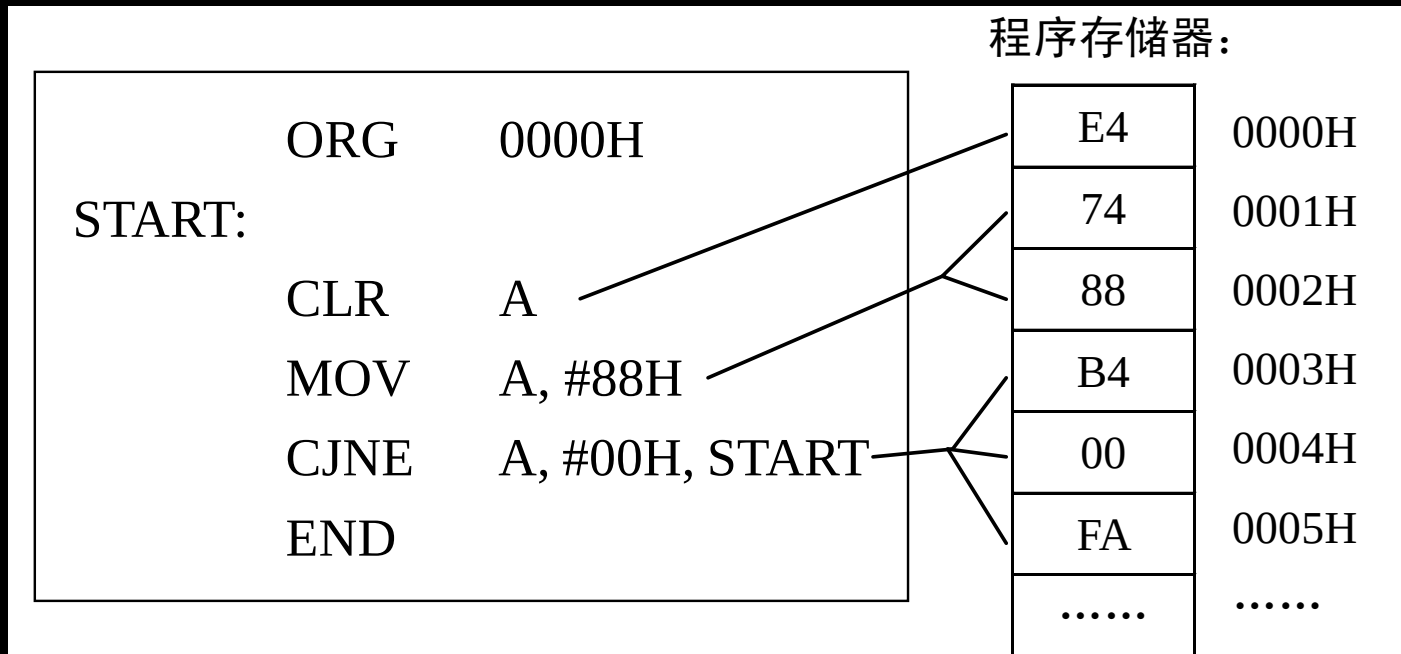
- 8.1 指令概述
- 8.2 算术指令
- 8.3 逻辑指令
- 8.4 片内数据装载指令
- 8.5 片外数据装载指令
- 8.6 查表指令
- 8.7 布尔指令
- 8.8 调用子程序指令
- 8.9 跳转与循环指令
- 8.10 单片机对于带符号数的处理和溢出问题
- 8.11 实例点拨——指令应用（程序）实例

单片机的命令

- 51单片机的指令总有**111**条，配合与不同的操作数共有**255**个指令的写法，即**255**个指令的执行代码。这个数量比起其他的处理器来说并不为多。但在实际应用中，常用的指令一般不过**50**个。
- 一条指令只要经过几次反复使用和理解就能掌握。

8.1 指令概述

• 8.1.1 指令的长度



指令的长度

多数指令的长度为单字节或双字节的。指令的长度与指令周期（指令执行所花费的时间）没有必然的联系。有的单字节指令周期比双字节或三字节指令要长，如指令“**MUL AB**”长度为**1**个字节，而指令周期却为**4**个机器周期。

8.1 指令概述

• 8.1.2 影响程序状态字PSW的指令

程序状态字PSW用来存放CPU运算和执行状态的寄存器，长度为1个字节，其8个位分别如下：

7	6	5	4	3	2	1	0
CY	AC	F0	RS1	RS0	OV	保留	P

其中有4个位在执行某些指令时可能会被改变，以指示CPU的运算和执行状态，如下：

CY——进位标志。

AC——辅助进位标志。

OV——溢出标志。

P——奇偶标志位。

8.1 指令概述

• 8.1.2 影响程序状态字PSW的指令

指 令	标 志 位			指 令	标 志 位		
	CY	OV	AC		CY	OV	AC
ADD	1/0	1/0	1/0	CLR C	0		
ADDC	1/0	1/0	1/0	CPL C	1/0		
SUBB	1/0	1/0	1/0	ANL C, bit	1/0		
MUL	0	1/0		ANL C, /bit	1/0		
DIV	0	1/0		ORL C, bit	1/0		
DA	1/0			ORL C, /bit	1/0		
RRC	1/0			MOV C, bit	1/0		
RLC	1/0			CJNE	1/0		
SETB C	1						

表中“1”表明执行该指令时会将标志位置1；“0”表明会清0标志位；“1/0”表明可能置1，也可能清0；没有标注的说明不会影响标志位。

8.2 算术指令

• 8.2.1 加法指令——ADD A, <src-byte>

A代表累加器ACC，<src-byte>代表“源操作数-以字节形式”

指 令	说 明	字 节	机器周期
ADD A,Rn	将Rn与ACC的值相加，结果存回ACC	1	1
ADD A,direct	将直接地址direct的内容与ACC相加，结果存回ACC	2	1
ADD A,@Ri	将间接地址@Ri的内容与ACC相加，结果存回ACC	1	1
ADD A,#data	将立即数#data与ACC的值相加，结果存回ACC	2	1

Ri 表示R0或R1；Rn 表示R0~R7的其中之一

关于ADD指令有以下几点需要说明。

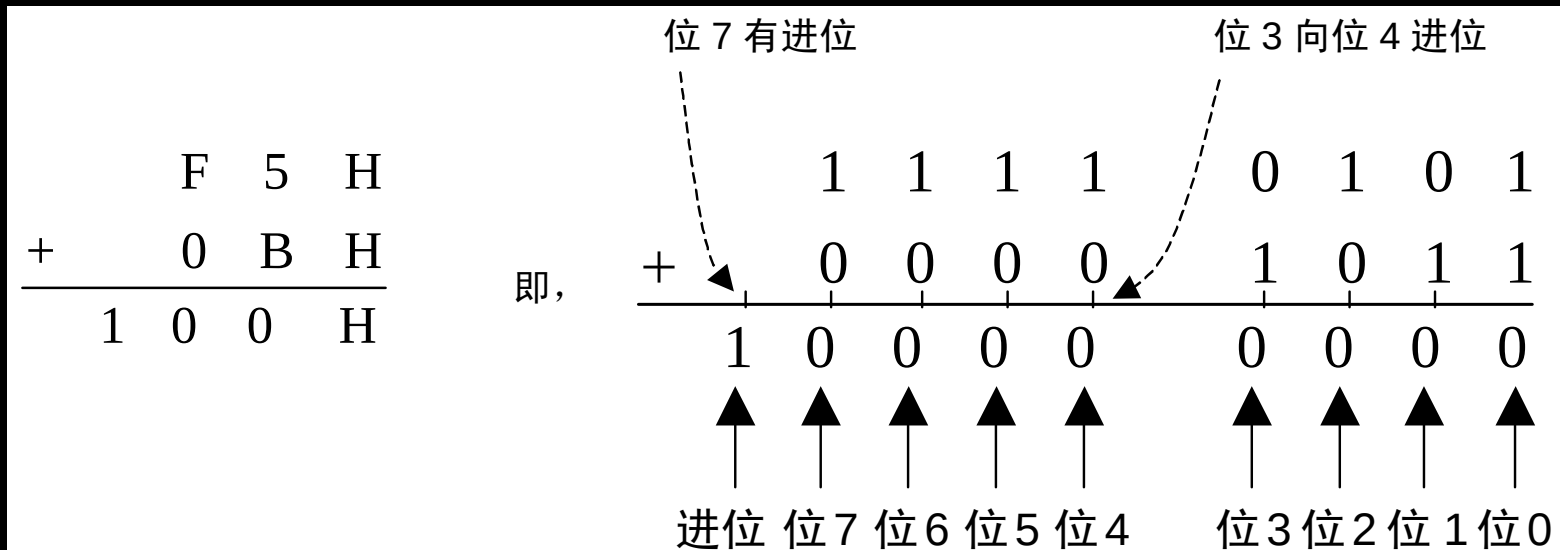
- ① 相加的操作总是在累加器ACC中发生，源操作数可以是一个工作寄存器的值、直接地址的内容、间接地址的内容或立即数。
- ② ADD指令可能影响PSW中的标志位CY、OV、AC和P。如果相加过程中，位3有进位则辅助进位标志位AC=1；位7有进位则进位标志位CY=1。

8.2 算术指令

• 8.2.1 加法指令——ADD A, <src-byte>

例: MOV A, #0F5H ; ACC=0F5H

ADD A, #0BH ; ACC=0F5+0BH=100H



运算结果: ACC=00, 标志位情况如下:

CY=1——由于位7有进位, AC=1——由于位3向位4进位,

P=1——由于ACC=0000 0000, 1的个数是0 (偶数)。

8.2 算术指令

• 8.2.1 加法指令——ADD A, <src-byte>

例: MOV A, #0F5H ; ACC=0F5H
ADD A, #0BH ; ACC=0F5+0BH=100H

溢出标志位OV的变化是：如果位6有进位而位7没有进位，或者位7有进位而位6没有，则溢出标志OV=1，否则OV=0。注意，OV的状态只有在带符号数加法运算时才有意义。当两个带符号数相加时，OV=1表示加法运算超出了累加器ACC所能表示的带符号数的有效范围（-128~+127），即产生了溢出，因此运算结果是错误的；否则OV=0说明无溢出产生，运算结果是正确的。

8.2 算术指令

• 8.2.2 带进位的加法指令——ADDC A, <src-byte>

带进位的加法指令ADDC相当于在ADD的运算基础上再加上进位CY。

指 令	说 明	字 节	机 器 周 期
ADDC A,Rn	将Rn与ACC的值及进位CY相加，结果存回ACC	1	1
ADDC A,direct	将直接地址direct的内容与ACC及进位CY相加，结果存回ACC	2	1
ADDC A,@Ri	将间接地址@Ri的内容与ACC及进位CY相加，结果存回ACC	1	1
ADDC A,#data	将立即数#data与ACC的值及进位CY相加，结果存回ACC	2	1

8.2 算术指令

• 8.2.2 带进位的加法指令——ADDC A, <src-byte>

ADDC和ADD指令一起使用时一般用于处理2个字节（16位）的加法运算。由于累加器ACC的长度为1个字节，所以在处理2个字节的加法时先用ADD指令进行低位字节的运算，把结果保存在某一地址空间或寄存器中。如果低位字节相加有进位将会影响进位标志CY，于是再使用ADDC指令进行高位字节的加法运算，把结果保存在另一个地址空间或寄存器中，这两个加法运算的结果合在一起就是2个字节数据的和。

8.2 算术指令

• 8.2.2 带进位的加法指令——ADDC A, <src-byte>

```
.....  
CLR      C                ; 清进位 CY  
MOV      A, #0E7H         ; ACC=0E7H, 载入低位字节  
ADD      A, #08DH         ; 低位字节加法, ACC=E7+8D=74H, 且 CY=1  
MOV      R0, A            ; 在 R0 中保存和的低位字节  
MOV      A, #3CH          ; ACC=3CH, 载入高位字节  
ADDC     A, #3BH          ; 带进位的加法运算, 3C + 3B + 1 = 78H  
MOV      R1, A            ; 和的高位字节保存在 R1 中  
.....
```

运算结果：R1=78H，R0=74。

如果在进行高位字节加法时也使用ADD指令将会漏掉低位字节相加的进位CY，而导致最终运算的错误。

8.2 算术指令

• 8.2.3 带借位的减法指令——SUBB A,<src-byte>

减法指令SUBB是“subtract with borrow”的缩写，当进行减法运算时，PSW中的CY位就变成了借位标志位。

指 令	说 明	字 节	机器周期
SUBB A,Rn	ACC减去Rn的值及借位CY，结果存回ACC	1	1
SUBB A,direct	ACC减去直接地址direct的内容及借位CY，结果存回ACC	2	1
SUBB A,@Ri	ACC减去间接地址@Ri的内容及借位CY，结果存回ACC	1	1
SUBB A,#data	ACC减去立即数#data及借位CY，结果存回ACC	2	1

8.2 算术指令

• 8.2.3 带借位的减法指令——SUBB A,<src-byte>

关于SUBB指令对标志位的影响如下：

- ① 当位7有借位时，标志CY=1；否则CY=0。也就是说，如果无符号数做减法时，减数比被减数大，CY=1。
- ② 当位3有借位时，标志AC=1；否则AC=0。
- ③ 溢出标志位OV=1表示溢出；OV=0表示未溢出。OV的值可由差的位7的借位与位6的借位做XOR（异或）的逻辑判断得到。
- ④ 由于SUBB指令连CY一起减，若不想减CY，可先将CY清0。

8.2 算术指令

- 8.2.4 自增/自减指令——INC <byte> / DEC <byte>
进位标志CY不受影响，可对DPL和DPH进行INC/DEC操

指 令	说 明	字 节	机器周期
INC A	ACC的值自增1	1	1
INC Rn	工作寄存器Rn的值自增1	1	1
INC direct	直接地址direct的内容自增1	2	1
INC @Ri	间接地址@Ri的内容自增1	1	1
INC DPTR	DPTR值自增1	1	1
DEC A	ACC的值自减1	1	1
DEC Rn	工作寄存器Rn的值自减1	2	1
DEC direct	直接地址direct的内容自减1	1	1
DEC @Ri	间接地址@Ri的内容自减1	1	1

8.2 算术指令

• 8.2.5 乘法指令——MUL AB

指 令	说 明	字节	机器周期
MUL AB	ACC与B寄存器的值相乘 ($A \times B$)，积的低位字节存回ACC，高位字节存回B寄存器	1	4

说明：

① 做无符号的1个字节运算时，累加器ACC与B寄存器的值相乘所得的积（2个字节），其低位字节存回ACC中，高位字节存入B寄存器。

② 如果积大于00FFH，则OV=1，否则OV=0。

③ 执行时，进位标志CY会被清除为0。

例： MOV A, #66H ; ACC=66H
 MOV B, #77H ; B=77H
 MUL AB ; 积=66×77=2F6AH

运算结果：

A=6AH,
 B=2FH,
 CY=0, 由于
 2F6AH>FFH
 , 所以
 OV=1。

8.2 算术指令

• 8.2.6 除法指令——DIV AB

指 令	说 明	字节	周期
DIV AB	ACC除以B寄存器的值 ($A \div B$)， 商存回ACC，余数存回B寄存器	1	4

说明：

① DIV指令做无符号的除法运算。如果B寄存器=0，执行时OV被置1，表示运算是错误的，因为除数不应该为0。

② DIV指令正确执行后，商存回ACC，余数存回B寄存器，进位标志CY及溢出标志OV都等于0。例：

MOV A, #66H ; ACC=66H

MOV B, #04H ; B=04H

DIV AB ; $66 \div 04 = 19H \dots 02H$ ，即商=19H，
；余数=02H

运算结果：A=19H，B=02H，CY=0，OV=0。

8.2 算术指令

• 8.2.7 十进制调整指令——DA A

指令DA只适用于加法指令ADD或ADDC后且只对累加器ACC产生作用，而不能使用在如INC等指令之后。

指 令	说 明	字 节	机器周期
DA A	累加器ACC作十进制调整	1	1

DA指令进行十进制调整的方法为：

- ① 在ADD或ADDC运算后，若ACC低位>9或AC=1，则ACC+06H。
- ② 在ADD或ADDC运算后，若ACC高位>9或CY=1，则ACC+60H。

例：MOV A, #47H ; ACC=47H
 MOV B, #25H ; B=25H
 ADD A, B ; ACC=6CH, ACC的低位C>9, ACC+06H
 DA A ; 十进制调整后ACC=6C+06=72H

8.3 逻辑指令

• 8.3.1 AND操作——ANL <dest-byte>,<src-byte>

指 令	说 明	字 节	机器周期
ANL A,Rn	将Rn与ACC的值做AND运算，结果存回ACC	1	1
ANL A,direct	将直接地址direct的内容与ACC做AND运算，结果存回ACC	2	1
ANL A,@Ri	将间接地址@Ri的内容与ACC做AND运算，结果存回ACC	1	1
ANL A,#data	立即数#data与ACC做AND运算，结果存回ACC	2	1
ANL direct,A	ACC与直接地址direct的内容做AND运算，结果存回该直接地址中	2	1
ANL direct,#data	立即数#data与直接地址direct的内容做AND运算，结果存回该直接地址中	3	2

8.3 逻辑指令

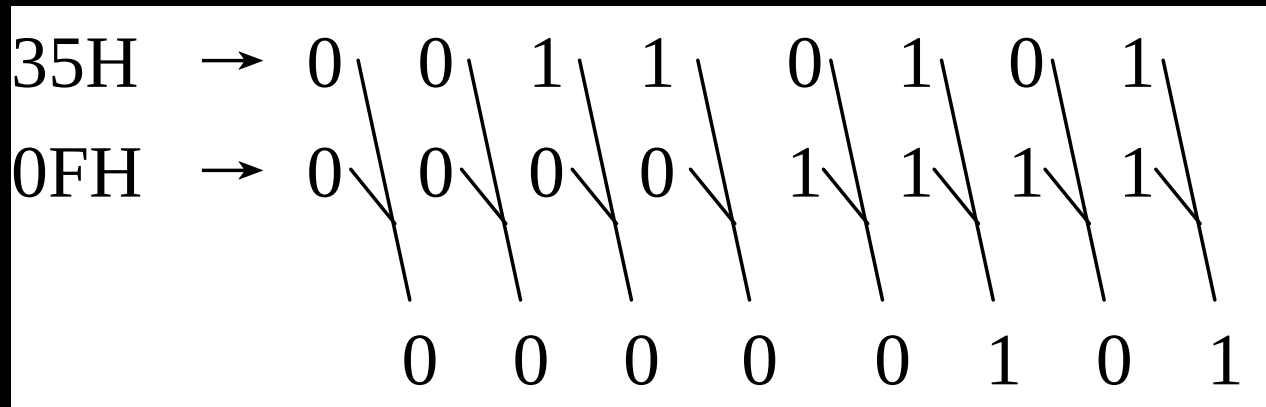
• 8.3.1 AND操作——ANL <dest-byte>, <src-byte>

例:

MOV A, #35H ; ACC=35H

ANL A, #0FH ; ACC=35H AND 0FH

运算过程:



X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

运算结果: 0000 0101 → ACC=05H

8.3 逻辑指令

• 8.3.2 OR操作——ORL <dest-byte>,<src-byte>

指 令	说 明	字节	机器周期
ORL A,Rn	将Rn与ACC的值做OR运算，结果存回ACC	1	1
ORL A,direct	将直接地址direct的内容与ACC做OR运算，结果存回ACC	2	1
ORL A,@Ri	将间接地址@Ri的内容与ACC做OR运算，结果存回ACC	1	1
ORL A,#data	立即数#data与ACC做OR运算，结果存回ACC	2	1
ORL direct,A	ACC与直接地址direct的内容做OR运算，结果存回该直接地址中	2	1
ORL direct,#data	立即数#data与直接地址direct的内容做OR运算，结果存回该直接地址中	3	2

8.3 逻辑指令

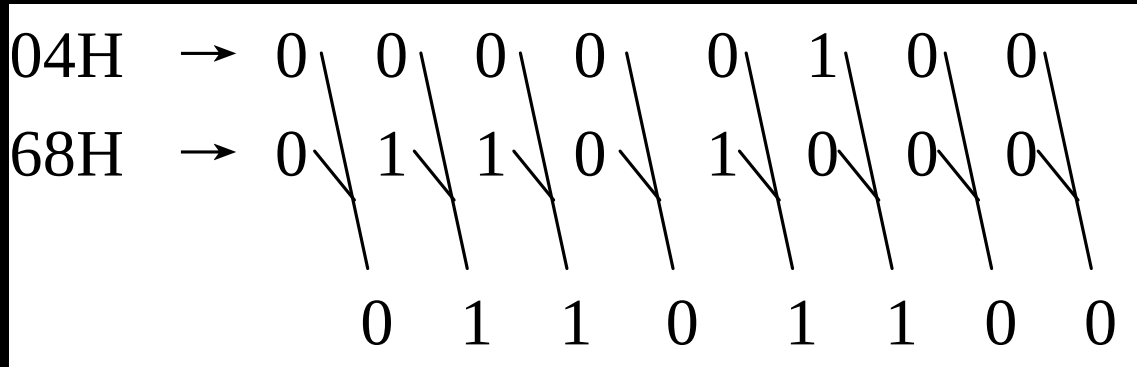
8.3.2 OR操作——ORL <dest-byte>, <src-byte>

例：

MOV A, #04H ; ACC=04H

ORL A, #68H ; ACC=04H OR 68H

运算过程：



X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	1

运算结果： **0110 1100 → ACC=6CH**

8.3 逻辑指令

• 8.3.3 XOR操作——XRL <dest-byte>, <src-byte>

指 令	说 明	字节	机器周期
XRL A,Rn	将Rn与ACC的值做XOR运算，结果存回ACC	1	1
XRL A,direct	将直接地址direct的内容与ACC做XOR运算，结果存回ACC	2	1
XRL A,@Ri	将间接地址@Ri的内容与ACC做XOR运算，结果存回ACC	1	1
XRL A,#data	立即数#data与ACC做XOR运算，结果存回ACC	2	1
XRL direct,A	ACC与直接地址direct的内容做XOR运算，结果存回该直接地址中	2	1
XRL direct,#data	立即数#data与直接地址direct的内容做XOR运算，结果存回该直接地址中	3	2

8.3 逻辑指令

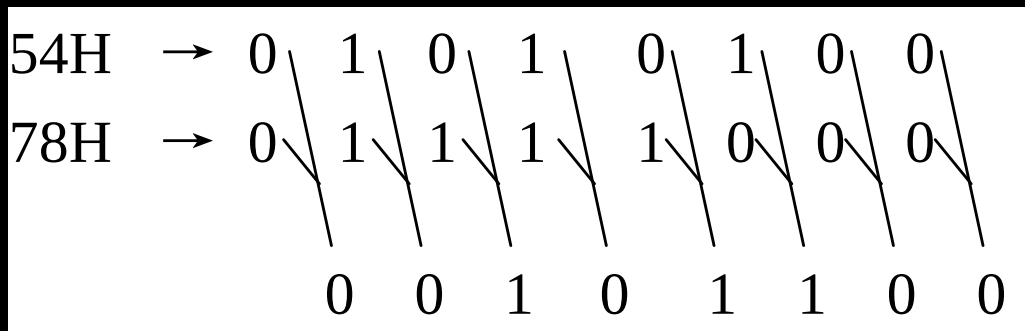
• 8.3.3 XOR操作——XRL <dest-byte>, <src-byte>

例：

MOV A, #54H ; ACC=54H

XRL A, #78H ; ACC=54H XOR 78H

运算过程：



运算结果： **0010 1100 → ACC=2CH**

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

8.3 逻辑指令

• 8.3.4 清0操作——CLR A

指 令	说 明	字 节	机器周期
CLR A	累加器ACC清0	1	1

说明：将累加器ACC清0。

执行指令“CLR A”后，ACC=00H。

8.3 逻辑指令

• 8.3.5 取反操作——CPL A

指 令	说 明	字 节	机器周期
CPL A	累加器ACC每一位的值反相	1	1

例：

MOV A, #47H ; ACC=47H

CPL A ; ACC=B8H

运算过程：

X	Y
0	1
1	0

取反每一位，
运算结果：

47H → 0 1 0 0 0 1 1 1
 ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
 1 0 1 1 1 0 0 0 → ACC = B8H

8.3 逻辑指令

• 8.3.6 位移动操作——RL、RLC、RR、RRC

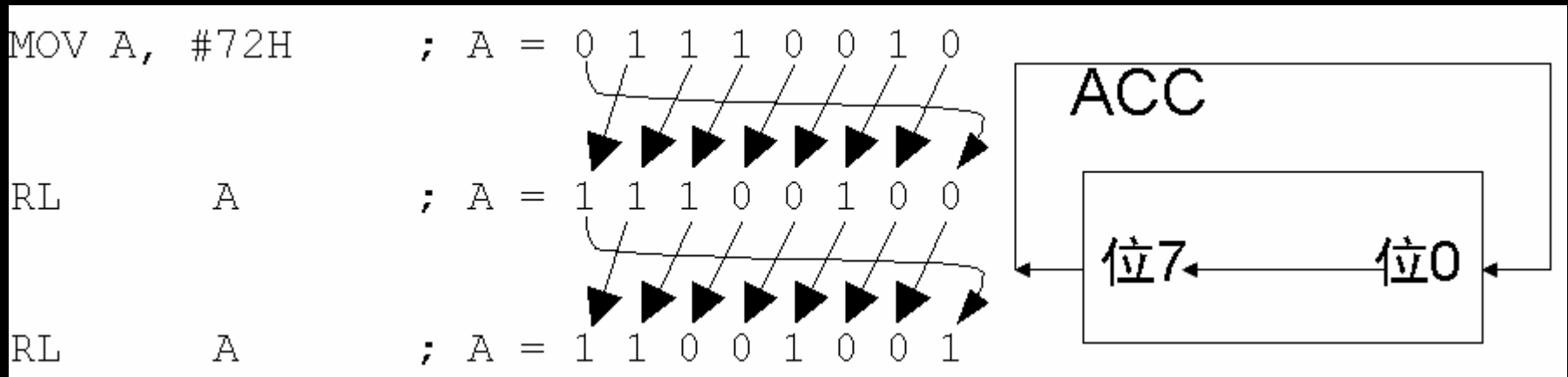
指 令	说 明	字 节	机器周期
RL A	累加器ACC左移一位	1	1
RLC A	累加器ACC含进位CY左移一位	1	1
RR A	累加器ACC右移一位	1	1
RRC A	累加器ACC含进位CY右移一位	1	1

- 这4条指令用于累加器ACC内部位的移动，这4条指令只适用于ACC。

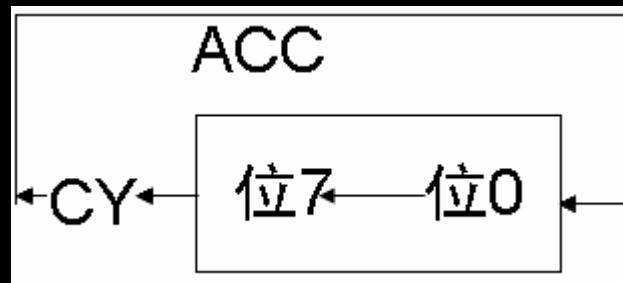
8.3 逻辑指令

• 8.3.6 位移动操作——RL、RLC、RR、RRC

① **RL A**——ACC左移一位。每次移出ACC的位7进入位0。



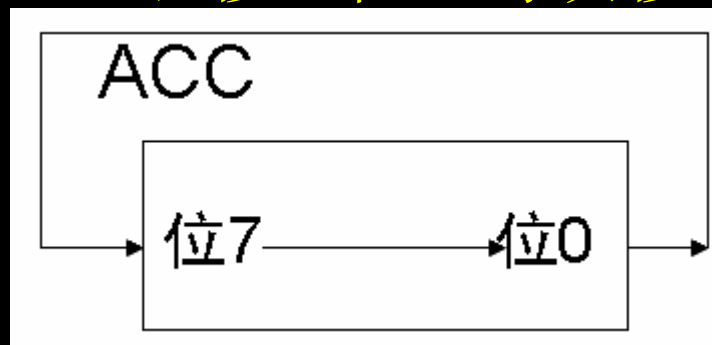
② **RLC A**——ACC含进位CY左移一位。每次移出ACC的位7进入进位CY中，而进位CY则进入位0中。



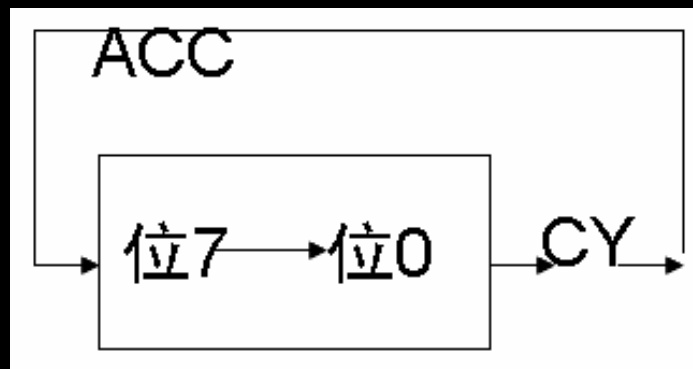
8.3 逻辑指令

• 8.3.6 位移动操作——RL、RLC、RR、RRC

③ RR A——ACC右移一位。每次移出ACC的位0进入位7。



④ RRC A——ACC含进位CY右移一位。每次移出ACC的位0进入进位CY中，而进位CY则进入位7中。



8.3 逻辑指令

• 8.3.7 累加器高低位交换操作——SWAP A

指 令	说 明	字 节	机器周期
SWAP A	累加器ACC的高4位与低4位互换	1	1

例：

MOV A, #28H ; ACC=28H

SWAP A ; ACC=82H

运行结果：

累加器ACC中的高4位与低4位互换，得
ACC=82H。

8.4 片内数据装载指令

- 片内数据装载指令是一类在片内存储器内“搬运”数据的指令，在单片机程序中使用频率最高。
- 这类指令中一多半都以MOV为助记符，指令形式为“MOV <dest>, <src>”，其中<dest>为目的操作数，<src>为源操作数。
- <dest>和<src>代表片内数据存储器空间地址或特殊功能寄存器SFR，指令在进行不同地址空间或寄存器之间装载时不需要通过累加器ACC的参与。

8.4 片内数据装载指令

• 8.4.1 数据装载指令——MOV <dest>, <src>

指 令	说 明	字 节	机 器 周 期
MOV A,Rn	将Rn的内容载入ACC	1	1
MOV A,direct	将直接地址direct的内容载入ACC	2	1
MOV A,@Ri	将间接地址@Ri的内容载入ACC	1	1
MOV A,#data	将立即数#data载入ACC	2	1
MOV Rn,A	将ACC的值载入Rn	1	1
MOV Rn,direct	将直接地址的内容载入Rn	2	2
MOV Rn,#data	将立即数#data载入Rn	2	1
MOV direct,A	将ACC的值载入直接地址direct中	2	1
MOV direct,Rn	将Rn的值载入直接地址direct中	2	2

8.4 片内数据装载指令

• 8.4.1 数据装载指令——MOV <dest>, <src>

指 令	说 明	字节	机器周期
MOV direct1,direct2	将直接地址direct2中的内容载入直接地址direct1中	3	2
MOV direct,@Ri	将间接地址@Ri的内容载入直接地址direct中	2	2
MOV direct,#data	将立即数#data载入直接地址direct中	3	2
MOV @Ri,A	将ACC的值载入间接地址@Ri中	1	1
MOV @Ri,direct	将直接地址direct的内容载入间接地址@Ri中	2	2
MOV @Ri,#data	将立即数#data载入间接地址@Ri中	2	1

8.4 片内数据装载指令

• 8.4.1 数据装载指令——MOV <dest>, <src>

根据目的操作数的不同，MOV指令分成以累加器ACC、工作寄存器Rn、直接地址direct、间接地址@Ri为目的操作数的4种不同形式。

MOV指令都不会改变源操作数，也不会影响标志位。

注意：

- ① 立即数前必须要有“#”号，否则装载的将是地址空间上的数据（直接地址的内容）。
- ② 如果立即数小于10H，即0~F，则高位会被系统自动补上0。
- ③ 累加器ACC或工作寄存器等一些寄存器加载大于FFH的立即数将会引发错误，因为这些寄存器的长度只有1个字节。

8.4 片内数据装载指令

- 8.4.2 数据指针寄存器装载指令——MOV DPTR, #data16

指 令	说 明	字 节	机器周期
MOV DPTR, #data16	将16位的立即数#data载入 DPTR	3	2

- ✓ 说明：该指令用于向数据指针寄存器DPTR装载16位的地址数据，地址空间的范围为0000H~FFFFH。
- ✓ AT89S51单片机为了方便同时访问片内、片外数据存储器，提供了两个16位的DPTR寄存器——DPTR0和DPTR1。在特殊功能寄存器中，每一个DPTR分成两个8位的寄存器DPH和DPL。
- ✓ 例：当AUXR1中的DPS位=0时，指令影响DPTR0，“MOV DPTR, #33FFH”，向DP0H装载33H，向DP0L装载FFH。

8.4 片内数据装载指令

• 8.4.3 堆栈指令——PUSH和POP

堆栈（**stack**）是单片机在片内数据存储器中开辟用于存储临时数据的区域。

程序运行过程中需要把某寄存器或地址空间中的数据暂时保存起来，以释放寄存器或地址空间暂作它用，等使用完毕后，再将刚才暂时保存的数据恢复到寄存器或地址空间中。

堆栈指针**SP**在特殊功能寄存器中的**81H**上，是一个8位的寄存器。

8.4 片内数据装载指令

• 8.4.3 堆栈指令——PUSH和POP

- ✓ 单片机上电复位时SP=07H，SP指向的是片内数据存储器中的07H，正好是第0组工作寄存器R7的地址。此时如果执行压栈指令“PUSH direct”，执行的步骤是SP先自增1，于是SP=08H，然后direct的数据就压入08H中，此时（08H）=direct，数据就被保存到了08H上。

指 令	说 明	字 节	机器周期
PUSH direct	将直接地址direct的内容压入堆栈中，执行前SP+1	2	2
POP direct	从堆栈中弹出数据到直接地址direct中，执行后SP-1	2	2

8.4 片内数据装载指令

• 8.4.3 堆栈指令——PUSH和POP（例）

```
ORG          00H
MOV          A, #89H          ; 向累加器ACC中载入立即数89H
PUSH        ACC              ; 压ACC的数值入堆栈, (08H) =89H
MOV          A, #3FH          ; ACC=3FH
MOV          R0, #55H         ; R0=55H
ADD          A, R0            ; ACC=94H
MOV          30H, A           ; (30H) =94H, ACC完成运算任务
POP  ACC ; 弹栈指令将刚才压入堆栈的立即数89H弹回到ACC中,
      ACC=89H
MOV          SP, #40H         ; 堆栈指针SP=40H
PUSH  30H    ; 把30H中的数据94H压入堆栈中, (41H) =94H
JMP          $               ; 停机
END
```


8.4 片内数据装载指令

• 8.4.4 数据交换指令——XCH和XCHD

指 令	说 明	字节	周期
XCH A,Rn	将累加器ACC与Rn的值交换	1	1
XCH A,direct	将累加器ACC与直接地址direct的内容交换	1	2
XCH A,@Ri	将累加器ACC与间接地址@Ri的内容交换	1	1
XCHD A,@Ri	将累加器ACC的低4位与间接地址@Ri的低4位内容交换	1	1

XCH指令交换累加器ACC和工作寄存器、直接或间接地址的内容。例如，ACC=65H，R2=99H，执行“XCH A, R2”之后，ACC=99H，R2=65H。

指令XCHD交换的只是累加器ACC与间接地址的低4位内容，高4位内容不变。例如，地址空间（40H）=2FH，ACC=3DH，R1=40H，执行“XCHD A, @R1”之后，ACC=3FH，（40H）=2DH。

8.5 片外数据装载指令

- 访问单片机的片外存储器，寻址方式只有一种——间接寻址。用到的操作数除了累加器ACC外，还有1个字节长度的间接地址——@Ri（R0或R1）或2个字节长度的间接地址——@DPTR。至于程序中选择使用@Ri还是@DPTR要根据寻址的空间大小来确定。

指 令	说 明	字节	机器周期
MOVX A,@Ri	将间接地址@Ri所指的片外存储器的内容载入ACC中（8位）	1	2
MOVX A,@DPTR	将DPTR所指定的片外存储器的内容载入ACC中（16位）	1	2
MOVX @Ri,A	将ACC的值载入间接地址@Ri所指的片外存储器中（8位）	1	2
MOVX @DPTR,A	将ACC的值载入DPTR所指的片外存储器中（16位）	1	2

8.5 片外数据装载指令

- 例：片外程序存储器的1000H开始存储有8个数据，循环将这8个数据送到P1口用于显示。

```
MYXDATA EQU 1000H ; 开始地址1000H赋给变量MYXDATA
COUNT EQU 8 ; 计数器COUNT=8
ORG 00H ; 起始地址
START:
MOVDPTR, #MYXDATA ; DPTR指向地址1000H, DPTR = 1000H
MOVR1, #COUNT ; 计数器值载入R1, R1 = 8
LOOP:
MOVX A, @DPTR ; 间接寻址, @DPTR所指片外数据载入ACC
MOV P1, A ; 送到P1口输出
INC DPTR ; DPTR自增1, 准备取下一个数据
DJNZ R1, LOOP ; 判断是否取完8个数据
JMP START ; 跳回一开始, 循环
END
```

8.6 查表指令

指 令	说 明	字 节	机 器 周 期
MOVC A,@A+DP TR	ACC加DPTR的值作为间接地址，将该地址的内容载入ACC中	1	2
MOVC A,@A+PC	ACC加PC的值作为间接地址，将该地址的内容载入ACC中	1	2

✓使用“MOVC A, @A+DPTR”前，数据指针寄存器DPTR一般载入数据表的表头地址，这样指令就能实现将数据表中的数据逐一读入累加器ACC中。

8.6 查表指令

- 指令“MOVC A, @A+PC”讲解：

数据表TABLE要紧跟在USETABLE子程序段RET后。PC指向RET的地址，如果设置访问序号ENTRY_NUMBER=1，那么指令“MOVC A, @A+PC”就指向RET的下一个地址，即数据表TABLE的表头。

MOVA, ENTRY_NUMBER ; 将访问序号ENTRY_NUMBER载入ACC
CALL USETABLE ; 调子程序USETABLE

.....

USETABLE: ; USETABLE子程序段
MOVC A, @A+PC ; 此时PC已能指向RET随后的表TABLE
RET

TABLE: ; 数据表TABLE
DB 0FEH, 0FDH, 0FBH, 0F7H
DB 0EFH, 0DFH, 0BFH, 07FH

.....

END

8.7 布尔指令

- 51单片机提供了一套完整的布尔指令，布尔指令是一类针对位操作的指令。
- 除了片内数据存储器提供位寻址区外（20H～2FH），特殊功能寄存器SFR也提供了128个可寻址位，I/O口也可以进行位操作。
- 单片机提供位寻址的机制，是为了增强系统的灵活性。

8.7 布尔指令

• 8.7.1 清0、置1与取反操作——CLR、SETB、CPL

指 令	说 明	字 节	机器周期
CLR C	进位CY清0	1	1
CLR bit	将可位寻址的位清0	2	1
SETB C	将进位CY置1	1	1
SETB bit	将可位寻址的位置1	2	1
CPL C	将进位CY反向	1	1
CPL bit	将可位寻址的位反向	2	1

“bit”代表可位寻址的地址或寄存器，包括：
片内数据存储器的位寻址区20H~2FH（映射地址），
累加器ACC，B寄存器，程序状态字PSW，
中断优先级控制寄存器IP，中断使能寄存器IE，
串行控制寄存器SCON，定时/计数器控制寄存器TCON，
I/O口：P0、P1、P2、P3。

另外：对进位CY操作的指令会直接影响标志位。

8.7 布尔指令

• 8.7.2 布尔逻辑指令——ANL、ORL

指 令	说 明	字 节	机器周 期
ANL C,bit	将进位CY与可位寻址的位做AND运算，结果存回进位CY	2	2
ANL C,/bit	将进位CY与可位寻址的位的反相做AND运算，结果存回进位CY	2	2
ORL C,bit	将进位CY与可位寻址的位做OR运算，结果存回进位CY	2	2
ORL C,/bit	将进位CY与可位寻址的位的反相做OR运算，结果存回进位CY	2	2

说明：这类指令是对位的逻辑运算，执行结果存回进位CY，影响标志位。

8.7 布尔指令

• 8.7.3 位数据装载指令——MOV <dest-bit>,<src-bit>

指 令	说 明	字节	机器周期
MOV C,bit	将可位寻址的位载入进位CY	2	1
MOV bit,C	将进位CY载入可位寻址的位	2	2

说明：指令完成的是位之间的装载，

例：

SETB C ; CY=1

ORL C, P2.0 ; P2.0与CY做OR运算，由于CY=1，
;所以无论P2.0是1或0，OR运算结果都为1，于是CY=1

8.7 布尔指令

• 8.7.4 布尔跳转指令——JC、JNC、JB、JNB、

指 令	说 明	字节	机器周期
JC rel	若进位CY=1则跳到rel	2	2
JNC rel	若进位CY=0则跳到rel	2	2
JB bit,rel	若可位寻址的位=1则跳到rel	3	2
JNB bit,rel	若可位寻址的位=0则跳到rel	3	2
JBC bit,rel	若可位寻址的位=1则跳到rel，并将该位清0	3	2

说明：以上指令检测进位CY或可位寻址的位，如果条件成立就跳转到rel所指的地址，否则执行下一条指令。

8.8 调用子程序指令

- 调用子程序指令用于调用子程序，子程序一般是一些被频繁使用的实现特定功能的程序段，如延时子程序等。使用子程序可以节省存储空间和使程序结构科学化。在单片机的指令系统中，有两条指令用于调用子程序——**LCALL**（长调用）和**ACALL**（绝对调用），它们的主要区别在于调用子程序的目标地址范围不同。对于**AT89S51**单片机来说，可以笼统地使用**CALL**指令来自动配置子程序目标地址的范围。

8.8 调用子程序指令

• 8.8.1 长调用指令——LCALL

指 令	说 明	字节	机器周期
LCALL addr16	调用整个64K Bytes程序存储器范围内的子程序	3	2

LCALL拥有两个字节来指定子程序的目标地址，调用地址范围达64K Bytes，即0000H~FFFFH。为了确保子程序执行完毕之后单片机能正确回到原来主程序的地址继续执行，LCALL执行时把LCALL指令的下一条指令的地址保存在堆栈中。

8.8 调用子程序指令

• 8.8.2 绝对调用指令——ACALL

指 令	说 明	字 节	机器周期
ACALL addr11	调用2K Bytes程序存储器范围内的子程序	2	2

ACALL指令长度为2个字节，2个字节中的11位用来指向子程序的地址，所以说子程序的目标地址不能超过2K Bytes的范围。

8.8 调用子程序指令

• 8.8.3 返回指令——RET和RETI

指 令	说 明	字 节	机器周期
RET	子程序执行完毕后返回主程序	1	2
RETI	中断子程序执行完毕后返回主程序	1	2

- **RET**指令用于子程序的末尾，提示子程序结束，以返回主程序，**RETI**指令用于结束中断服务子程序。

8.9 跳转与循环指令

- 跳转与循环指令在程序中使用相当频繁，它们控制程序的走向和实现循环执行特定程序段。
- 用好跳转与循环指令，可以使系统程序得到优化和节省程序存储空间。

8.9 跳转与循环指令

• 8.9.1无条件跳转指令——LJMP、AJMP、SJMP、JMP

指 令	说 明	字节	机器周期
LJMP addr16	长跳转	3	2
AJMP addr11	绝对跳转	2	2
SJM P rel	短跳转（相对地址）	2	2
JMP @A+DPTR	间接跳转至@A+DPTR 所指的地址	1	2

LJMP是长跳转指令，跳转地址范围达64K Bytes。

AJMP是绝对跳转指令，目标地址不能超过2K Bytes。

SJMP是一个相对跳转指令，跳转范围为向前128个字节，向后127个字节。

JMP一个笼统的跳转指令，自动判断跳转的范围。

8.9 跳转与循环指令

• 8.9.2 条件跳转指令——JZ和JNZ

指 令	说 明	字 节	机器周期
JZ rel	ACC=0则跳转到rel	2	2
JNZ rel	ACC≠0则跳转到rel	2	2

这是两条以累加器ACC的值为条件的跳转指令。

JZ指令，当ACC=0时程序跳转到所指定的地址开始执行；

JNZ指令，当ACC≠0时才跳转到指定地址开始执行。

8.9 跳转与循环指令

• 8.9.2 条件跳转指令——JZ和JNZ

行号	PC	执行代	指令	
01	0000		ORG	00H
02	0000	7800	MOV	R0, #00H
03	0002	7455	MOV	A, #55H
04	0004	6003	JZ	NEXT
05	0006	08	INC	R0
06	0007	04	LOOP: INC	A
07	0008	04	INC	A
08	0009	2477	NEXT: ADD	A, #77H

分析：第4行的指令“JZ NEXT”，执行代码由60和03组成，分别存放在0004和0005地址空间中，执行代码03就是一个相对地址。“JZ NEXT”的下一条指令（第5行）的PC值为0006，所以跳转到 $0006+03=0009$ 的目标地址，即第8行的NEXT标号处。

8.9 跳转与循环指令

- 8.9.3 比较跳转指令——CJNE <dest-byte>, <src-byte>, rel

指 令	说 明	字 节	机 器 周 期
CJNE A,direct,rel	将直接地址direct的内容与ACC作比较，不相等则跳转到rel	3	2
CJNE A,#data,rel	将ACC的值与立即数#data作比较，不相等则跳转到rel	3	2
CJNE Rn,#data,rel	将立即数#data与Rn作比较，不相等则跳转到rel	3	2
CJNE @Ri,#data,rel	将立即数#data与间接地址@Ri的内容作比较，不相等则跳转到rel	3	2

8.9 跳转与循环指令

• 8.9.3 比较跳转指令——CJNE <dest-byte>, <src-byte>, rel

- ✓ CJNE指令是“compare and jump if not equal”的缩写，意思是将源操作数<src-byte>与目的操作数<dest-byte>的值相比，如果不相等就跳转到rel所指的地址。
- ✓ 它还改变进位标志CY的值以显示目的操作数较大还是较小。
- ✓ CJNE指令不会改变源操作数或目的操作数的值。

比较条件	影响进位CY
目的操作数>源操作数	CY=0
目的操作数<源操作数	CY=1

8.9 跳转与循环指令

• 8.9.4 循环指令——DJNZ <byte>,<rel-addr>

单片机中将一段程序重复执行一定次数的过程称为循环，通过指令DJNZ来实现的循环是一种最广泛应用的方法。

DJNZ执行时，工作寄存器或地址内容减1，如果不等于0，则程序跳转到rel指示地址。在执行DJNZ指令前，需要向相关工作寄存器或地址中载入计数值，该计数值就是循环的次数。

指 令	说 明	字 节	机器周期
DJNZ Rn,rel	将Rn的值减1，若不等于0则跳转到rel	3	2
DJNZ direct,rel	将直接地址direct的内容减1，若不等于0则跳转到rel	3	2

8.9 跳转与循环指令

• 8.9.5 无操作指令——NOP

NOP指令不进行任何操作，只是空耗费时间，并且更新程序计数器**PC**的计数值。每执行一次花去**1**个机器周期，执行完**NOP**后，就执行下一条指令。如果程序中需要等待一个很短的时间（若干个机器周期时间），可以结合使用**NOP**和循环指令**DJNZ**来实现。

指 令	说 明	字 节	机器周期
NOP	不执行任何动作	1	1

8.10 单片机对于带符号数的处理和溢出问题

• 8.10.1 无符号数与带符号数

8位时存储无符号数的数值范围是0~255,

7位存储带符号数的数值范围变成了-128~+127。



8.10 单片机对于带符号数的处理和溢出问题

• 8.10.2 二进制的二补数

在数字系统中，使用二进制表示负数时，最高有效位放置的是符号，而数值的大小以二进制的二补数（2's complement）形式存放在次高有效位到最低有效位。

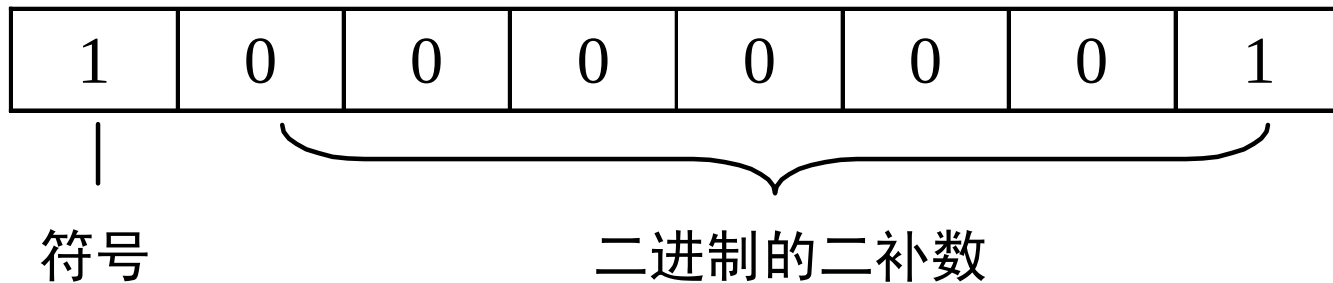
二进制的二补数是这样得到的：

- ① 把负数的绝对值用二进制表示。
- ② 将每一位取反。
- ③ 结果加1。

8.10 单片机对于带符号数的处理和溢出问题

• 8.10.2 二进制的二补数

例：-127这个数，其绝对值127表示成二进制是111 1111，将每一位取反得000 0000，最后加1就得到了如图8-6所示的结果：最高有效位1表示是负数，剩下的是二进制的二补数。



8.10 单片机对于带符号数的处理和溢出问题

• 8.10.3 带符号数操作中的溢出问题（例）

MOV A, #+96 ; ACC=0110 0000
 (60H)

MOV R1, #+70 ; R1=0100 0110 (46H)

ADD A, R1 ; ACC=1010 0110
 (A6H)

运算结果:

ACC=A6H, OV=1。

分析:

+96和+70是正数（带符号数），它们的和应该是
 $(+96) + (+70) = +166$ ，而运算的结果ACC=A6H，
 即-90，溢出标志位OV=1。为什么会出现-90这个错误的
 结果呢？这是因为+96与+70的和+166超出了8位空间
 存储带符号数的范围-128~+127，于是溢出标志

8.10 单片机对于带符号数的处理和溢出问题

• 8.10.4 什么时候会溢出

✓ 位6有进位而位7没有进位（CY=0），OV=1。

✓ 位7有进位（CY=1）而位6没有进位，OV=1。

例：MOV A, # -128 ; ACC=1000 0000 (80H)
 MOV R1, # -2 ; R1=1111 1110 (FEH)
 ADD A, R1 ; ACC=0111 1110 (7EH)

运算过程：

	1	0	0	0	0	0	0	0
+	1	1	1	1	1	1	1	0
	1	0	1	1	1	1	1	0

运算结果：

由于位7有进位，而位6没有——溢出。ACC=7EH，OV=1

8.10 单片机对于带符号数的处理和溢出问题

- 8.10.5 如何防止进位或溢出产生的错误
 - ✓ 在带符号数运算中，溢出标志OV暗示了运算结果是否正确——OV=1时结果错误，OV=0时结果正确。
 - ✓ 在无符号数操作中我们需要留意进位标志CY，而在带符号数操作中需要留意溢出标志OV。
 - ✓ OV是程序状态字PSW中的一位，且PSW可以被位寻址，所以我们可以用以下的办法来对溢出标志OV进行监视，从而控制程序的走向。

JB PSW.2, rel ; OV=1（溢出）则跳到rel

JNB PSW.2, rel ; OV=0（无溢出）则跳到rel

8.11 实例点拨

——指令应用（程序）实例

• 8.11.1 数据求和

- ✓ 5个数据以BCD码形式连续存储在片内数据存储器中，起始地址为40H。求这5个数据的和。

(40H) = 78, (41H) = 12, (42H) = 44, (43H) = 81, (44H) = 99,

```
ORG      00H

START:

MOV      R0, #40H      ; R0 作为数据的地址指针
MOV      R2, #5         ; 计数器
CLR      A              ; ACC 清 0
MOV      R7, A          ; R7 清 0, R7 用于存储和的高位字节
```

8.11 实例点拨

——指令应用（程序）实例

• 8.11.1 数据求和

```
LOOP:
    ADD    A, @R0        ; 将 R0 所指地址中的内容与 ACC 相加
    DA     A             ; 十进制调整
    JNC    NEXT          ; 如果 CY=0, 说明没有进位, 跳到 NEXT
    INC    R7            ; 如果 CY=1, 说明有进位, 和的高位字节加
NEXT:
    INC    R0            ; 地址指针增加 1
    DJNZ   R2, LOOP      ; 循环, 直到 5 个数据相加完成为止
    JMP    $             ; 停机
END
```

• 运行结果：R7=03，ACC=14。

8.11 实例点拨

——指令应用（程序）实例

- 8.11.2 减法与二进制的二补数
 - ✓ 带借位的减法指令SUBB，在单片机执行该指令时，如果借位CY=0，说明被减数大于减数，差是正数；如果借位CY=1，说明减数大于被减数，差是负数，且以二进制的二补数的形式存储在累加器ACC中。
 - ✓ 在SUBB执行时，单片机并不是直接将两个数相减，而是按照下面的方法进行：
相减操作 = 被减数 + 减数的二进制的二补数

8.11 实例点拨

——指令应用（程序）实例

• 8.11.2 减法与二进制的二补数

例：25H-1EH，减数1EH转换成其二进制的二补数得1110 0010，被减数25H与该二进制的二补数相加。

	0	0	1	0		0	1	0	1
+	1	1	1	0		0	0	1	0
	1	0	0	0	0		0	1	1

差是0000 0111，即07H。位7有进位，进位标志CY应该是1，但我们使用二进制的二补数形式分析减法时，进位标志CY、辅助进位标志AC、奇偶标志P的值要取反，也就是说CY=0，没有借位，表明差是正数。

8.11 实例点拨

——指令应用（程序）实例

• 8.11.2 减法与二进制的二补数

例：4CH-6EH，将减数6E转换成其二进制的二补数得1001 0010，被减数4CH与该二进制的二补数相加。

		0	1	0	0		1	1	0	0
+		1	0	0	1		0	0	1	0
	0	1	1	0	1		1	1	1	0

- 差是1101 1110，即DEH。这不是一个有效的结果，4CH-6EH应该得到一个负数的结果：-(6E-4C)=-22H。所以需要差DE再进行一次二进制的二补数转换，得0010 0010，即22H。从运算过程来看，位7没有进位，进位标志CY应该是0，但和①情况相同，标志位需要取反，于是得CY=1，有借位，表明差是负数。

8.11 实例点拨

——指令应用（程序）实例

- 8.11.3 XRL指令应用于比较寄存器数值

XRL指令可用来比较两个寄存器的数值是否相等。例如“XRL A, R1”将比较累加器ACC和工作寄存器R1的数值。如果ACC=R1，XRL执行结果是0，并存回ACC中，否则执行的结果为1，并存回ACC中。然后就可以使用指令JZ或JNZ来判断ACC的值从而控制程序的走向。

8.11 实例点拨

——指令应用（程序）实例

• 8.11.3 XRL指令应用于比较寄存器数值

P1口作为数据输入使用，程序判断P1口的输入数据是否等于45H，如果是则执行处理程序PROCESSING，否则继续作判断

断例：

```

ORG          00H
START:
    MOV  A, #45H          ; 将45H载入ACC
LOOP:
    MOV  R1, P1           ; 将P1口的输入数据载入R1
    XRL  A, R1            ; 比较ACC和R1的值
    JZ   PROCESSING      ; 如果相等，跳转到 PROCESSING
    JMP  LOOP            ; 如果不相等，跳转到LOOP继续判断
PROCESSING:
    .....
  
```

XRL指令还可以将位取反。如指令“XRL A, #0000 0100B”能把累加器ACC的位2取反，而保持其他位不变。

8.11 实例点拨

——指令应用（程序）实例

• 8.11.4 位数据的串行输出

有些场合，需要把并行的数据转换成串行输出，这时可以利用**RRC**指令，将累加器**ACC**中的位数据输出到进位**CY**中，然后将**CY**中的数据从**I/O**口输出，

RRC	A	； 将ACC的位0数据右移到CY中
MOV	P1. 3, C	； 从P1. 3口输出CY中的数据
RRC	A	； 将第2个位数据移到CY
MOV	P1. 3, C	； 从P1. 3口输出
RRC	A	； 将第3个位数据移到CY
MOV	P1. 3, C	； 从P1. 3口输出

8.11 实例点拨

——指令应用（程序）实例

• 8.11.5 利用布尔指令产生矩形波信号

矩形波信号简介：

矩形波中的主要参数：

脉冲幅度：脉冲信号所能达到的最大幅度值。

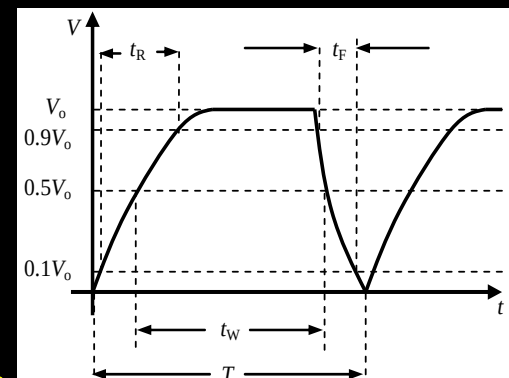
上升时间：脉冲从0.1上升到0.9所经历的时间。

下降时间：脉冲从0.9下降到0.1所经历的时间。

脉冲宽度：脉冲前0.5到后0.5的时间间隔。

脉冲周期 T ：对一个周期性的脉冲波，周期信号指两相邻脉冲出现的时间间隔。

占空比 D ：脉冲宽度与脉冲周期 T 之比称为占空比 D ，



$$D = \frac{t_w}{T} \times 100\%$$

8.11 实例点拨

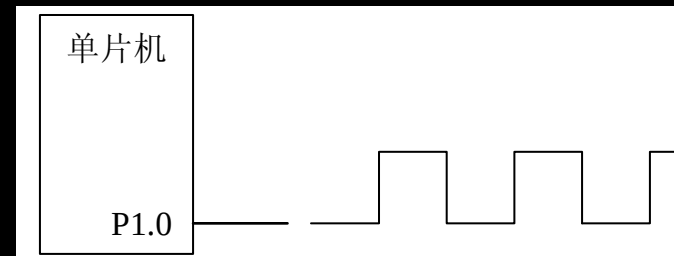
——指令应用（程序）实例

• 8.11.5 利用布尔指令产生矩形波信号

单片机的执行速度很快，指令SETB和CLR在晶振频率为12MHz时只需要1 μ s，也就是说如果使用某一I/O口作为波形的输出，那上升时间（SETB置1）或下降时间（CLR清0）过程只需1 μ s，可以应付一般场合的需要。

WAVEOUTPUT:

```
SETB  P1.0      ; P1.0口置1
LCALL DELAY     ; 延时子程序保持1
CLR    P1.0     ; P1.0口清0
LCALL DELAY     ; 延时子程序保持0
SJMP  WAVEOUTPUT ; 循环
```



8.11 实例点拨

——指令应用（程序）实例

• 8.11.6 布尔指令应用于控制

- ✓ 在一台电话机中，有一个单片机用于系统的控制，假设数据存储器22H上的B2位，即映射地址12H上的数据指示是否有电话打进来。如果映射地址12H上的数据是1，表明有电话打进来。写一段程序，控制液晶屏在映射地址12H=1时，显示“New Incoming Call”，否则显示“Standby”。

8.11 实例点拨

——指令应用（程序）实例

.....

CHECKBIT:

MOV	C, 12H	; 将 12H 载入进位 CY 中
JNC	STANDBY	; 如果 CY=0, 跳转到 STANDBY
MOV	DPTR, #NEWCALL	; DPTR 指向显示数据表 NEWCALL
LCALL	DISPLAY	; 调显示子程序
SJMP	CHECKBIT	; 循环

STANDBY:

MOV	DPTR, #NOCALL	; 如果没有电话打进, DPTR 指向显示数据表
LCALL	DISPLAY	; 调显示子程序
SJMP	CHECKBIT	; 循环

.....

8.11 实例点拨

——指令应用（程序）实例

• 8.11.6 布尔指令应用于控制

； 以下是液晶屏的显示数据表

```
NEWCALL : DB 'New Incoming Call'
NOCALL: DB 'Standby'
END
```

第9章 对第四维的测量

——定时/计数器

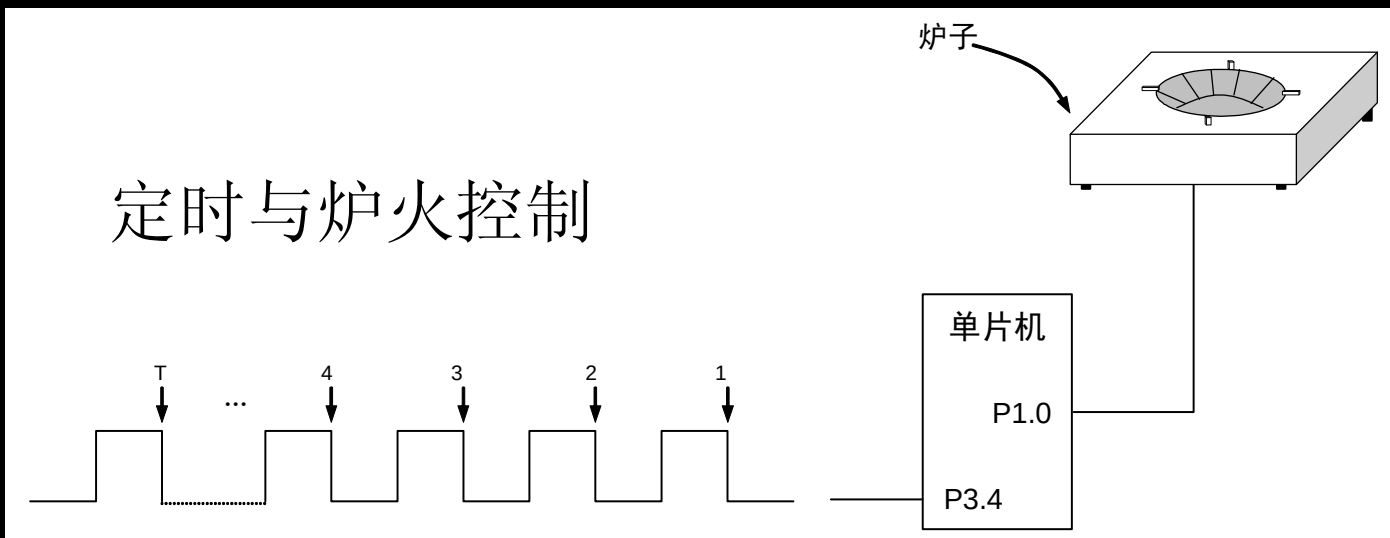
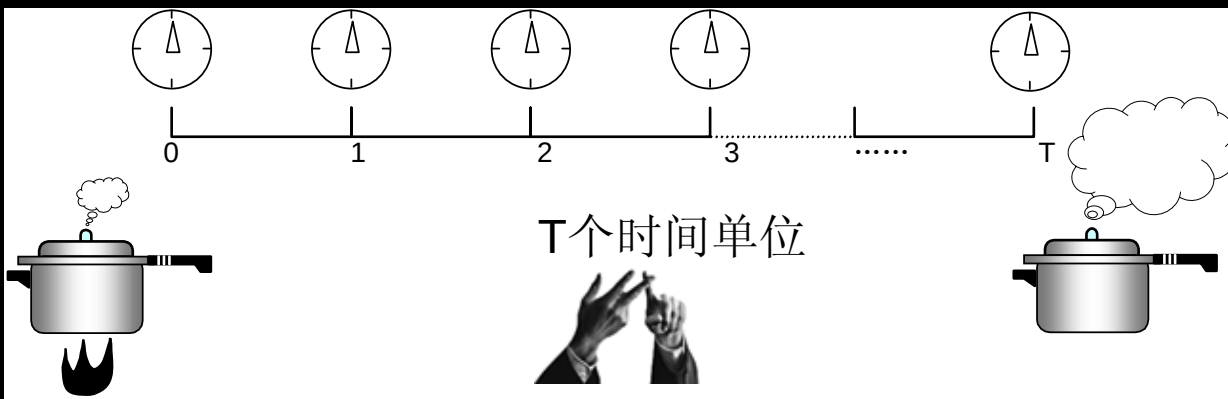
欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 9.1 定时与计数
- 9.2 单片机的定时/计数器
- 9.3 定时/计数器的控制者
- 9.4 模式1
- 9.5 模式0
- 9.6 模式2
- 9.7 模式3
- 9.8 计数器
- 9.9 看门狗
- 9.10 实例点拨——超声测距仪

9.1 定时与计数

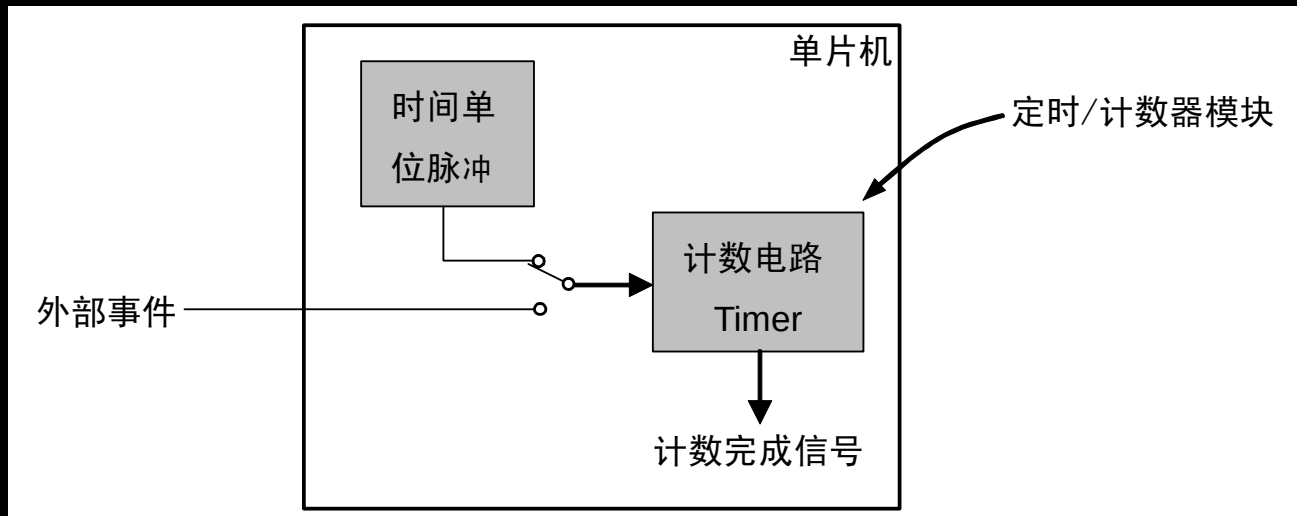
“计算个数”——定时计算的是时间单位的个数；
计数计算的是外部方波信号的个数。



9.2 单片机的定时/计数器

• 9.2.1 定时/计数器的实质

- ✓ 单片机的定时/计数功能模块使用同一个电路来实现，只是定时/计数功能模块“计算个数”的对象不一样——一个是时间单位的个数，另一个是外部事件的个数。
- ✓ 1个定时/计数器模块一次只能工作在一种功能下——要么是定时器，要么是计数器。当计数完成后，模块向CPU输出一个计数完成信号来中断计数操作。



9.2 单片机的定时/计数器

• 9.2.2 时间单位脉冲

- ✓ 单片机中的时间单位脉冲的周期是：1个机器周期，即12个振荡周期。
- ✓ 如果晶振的频率是12MHz，则1个机器周期=1μs，每1秒钟Timer所记录的时间单位脉冲的个数是：

$$\frac{1\text{s}}{1\mu\text{s}} = 10^6 \text{ 个}$$

- ✓ 如果Timer从0数到50000需要的时间可以这样计算：

$$\frac{50000 \text{ 个单位时间脉冲}}{10^6 \text{ 个/s}} = 0.05\text{s}$$

9.2 单片机的定时/计数器

• 9.2.3 Timer应用一例——延时子程序

程序解析:

```
SETB      TR0          ; 启动Timer0, 开始计时
```

```
SETUP:
```

```
    MOV    TL0, #0B0H ; 计数初始值
```

```
    MOV    TH0, #3CH
```

```
LOOP1:
```

```
    JBC    TF0, LOOP2; 如TF0=1, 10000数完, 清TF0
```

```
    JMP    LOOP1    ; 如果10000没数完, 循环检查TF0
```

```
LOOP2:
```

```
    CLR    TR0          ; 关闭Timer0
```

```
    RET                ; 返回主程序
```

9.3 定时/计数器的控制者

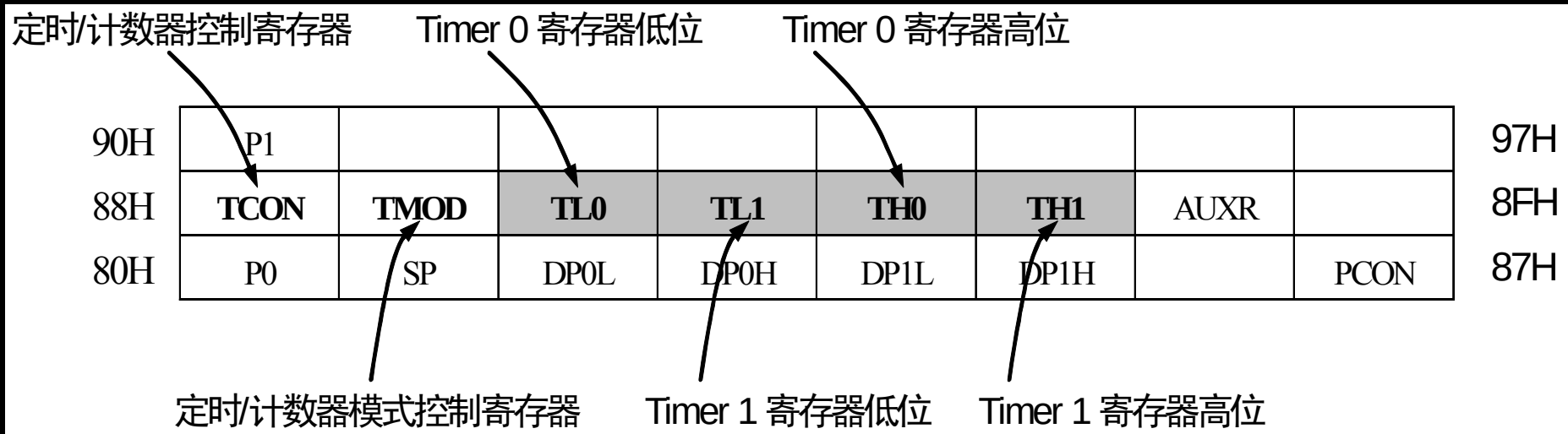
- Timer的启动、设置和关闭与特殊功能寄存器中的TCON、TL0、TH0等寄存器有关。
- T89S51单片机中有两个Timer，分别为Timer 0和Timer 1。有些单片机具有3个Timer，如AT89S52等。
- 两个Timer可以工作在定时器模式下，也可以工作在计数器模式下。

9.3 定时/计数器的控制者

• 9.3.1 Timer寄存器

Timer 0和Timer 1各有1个长度为2个字节的Timer寄存器，每个Timer寄存器由低位字节TL0或TL1和高位字节TH0或TH1两个特殊功能寄存器组成。

这4个寄存器可以像累加器ACC等寄存器一样进行数据的装载和读取。如指令“MOV TL0, #4FH”。



9.3 定时/计数器的控制者

- 9.3.2 定时/计数器模式控制寄存器TMOD
 - ✓ TMOD是“timer mode”的缩写，意思是“定时/计数器模式”，它在特殊功能寄存器区的89H上。
 - ✓ TMOD寄存器由高4位和低4位组成，分别控制Timer 1和Timer 0。
 - ✓ 在高4位或低4位中，M1和M0设置Timer 1或Timer 0的工作模式。

9.3 定时/计数器的控制者

• 9.3.2 定时/计数器模式控制寄存器TMOD

7	6	5	4	3	2	1	0
GATE	C/T#	M1	M0	GATE	C/T#	M1	M0
Timer 1				Timer 0			

GATE 门控位。 **C/T#** Timer定时/计数器选择位。

M1 Timer模式选择位1。 **M0** Timer 模式选择位0。设置方法为：

M1 M0 模式 工作模式

0 0 模式0 13位定时/计数器 (THx 8位, TLx 5位)

0 1 模式1 16位定时/计数器

1 0 模式2 8位自动重载入定时/计数器 (TLx), 溢出时从THx重载

1 1 模式3 Timer 0有此模式, Timer 1没有

*注: x表示0或1, 即情况适用于Timer 1和Timer 0 后同。

9.3 定时/计数器的控制者

• 9.3.3 定时/计数器控制寄存器TCON

- ✓ TCON是“timer control”的缩写，即“定时/计数器控制”的意思。它在特殊功能寄存器区的88H上。
- ✓ TCON的功能有：显示Timer溢出与否、启动/关闭Timer、外部中断方式控制、外部中断标志位。
TCON的8位中与Timer有直接关系的只有高4位。

7	6	5	4	3	2	1	0
TF1	TR1	TF0	TR0				
Timer 1		Timer 0					

TFx 溢出标志位。当Timer溢出时，该位由硬件置1；当执行中断子程序时，该位由硬件清0。

TRx Timer启动/停止位。TRx=1时Timer启动，TRx=0时Timer停止。

9.3 定时/计数器的控制者

• 9.3.4 Timer的时钟源

TMOD寄存器中的C/T#位=0，即Timer用作定时器，单片机的时钟信号就成了Timer的时钟源，时钟信号由晶振决定：

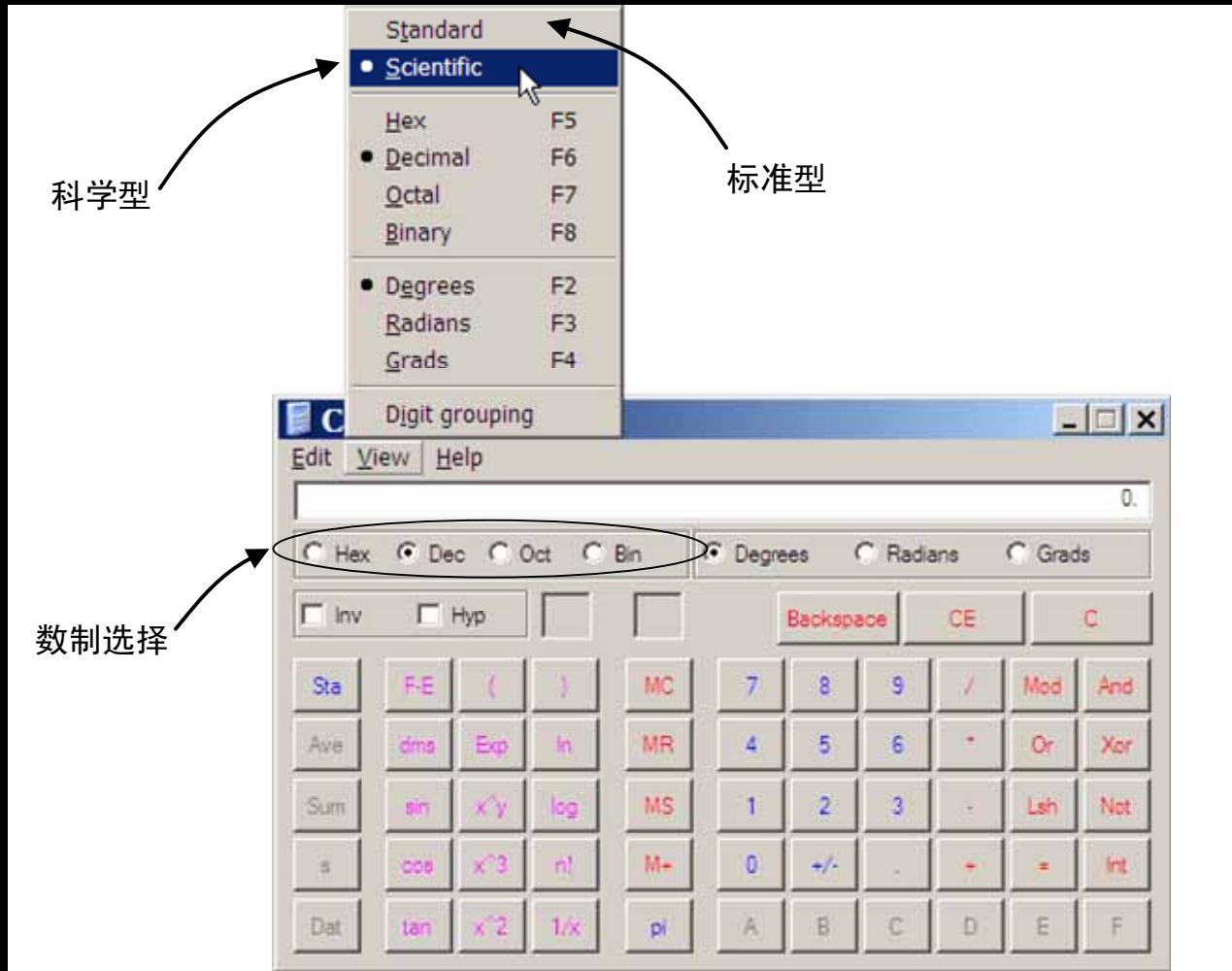
$$T = \frac{1}{f_c(\text{MHz}) \times \frac{1}{12}} \quad (\mu\text{s})$$

如晶振为12MHz,则单位时间 T 为：

$$T = \frac{1}{12\text{MHz} \times \frac{1}{12}} = 1 \quad (\mu\text{s})$$

9.3 定时/计数器的控制者

• 9.3.5 如何使用Windows计算器



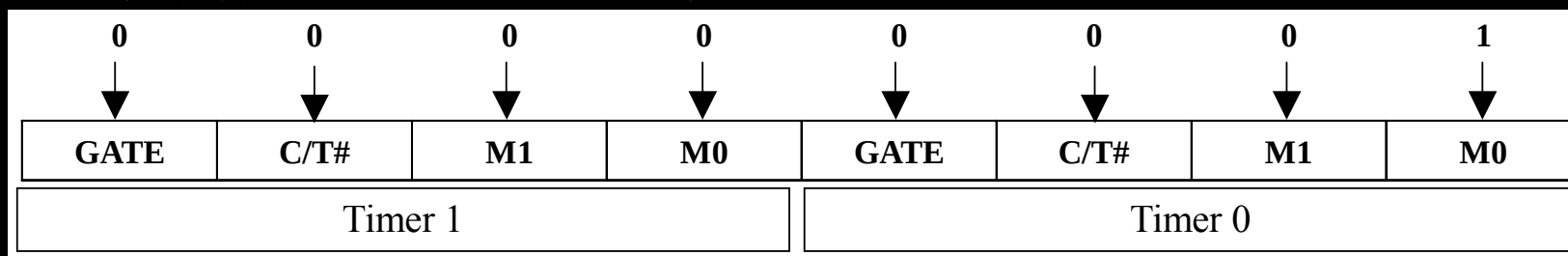
9.4 模式1

- 9.4.1 模式1下的Timer特点
 - ✓ Timer工作在模式1时是一个16位定时/计数器，计数初始值的低位和高位分别装载到Timer寄存器TLx和THx中。
 - ✓ Timer通过指令SETB将TRx置1启动。
 - ✓ 当TFx=1，表明计数完成，使用指令“CLR TRx”和“CLR TFx”来关闭Timer。
 - ✓ 想要重复Timer的计数过程，Timer寄存器中必须重新装载原来的计数初始值，并将TFx位清0。

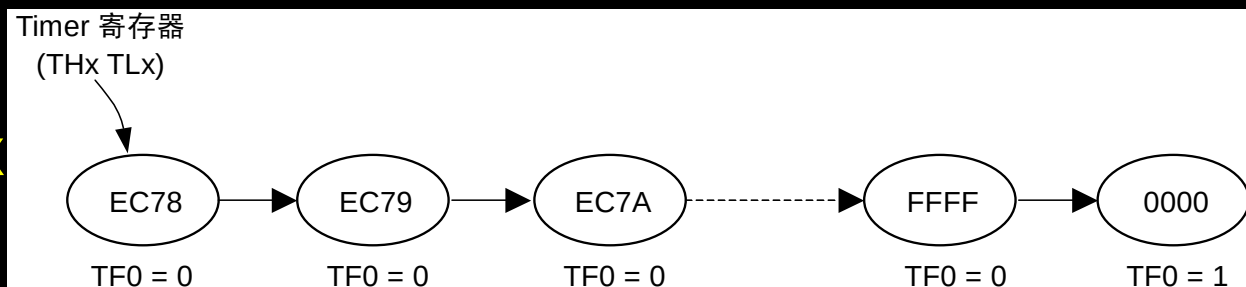
9.4 模式1

• 9.4.2 模式1的设置步骤

- ① 设置Timer的工作模式。程序中向定时/计数器模式控制寄存器TMOD中装载01H。



- ② 往TLx和THx中载入计数初始值。
- ③ 启动Timer。将TRx置1以启动Timer。
- ④ 监测Timer溢出标志TFx。
- ⑤ 关闭Timer。
- ⑥ 清溢出标志TFx
- ⑦ 重新装载。



9.4 模式1

• 9.4.3 Timer的计数时间与计数初始值

Timer寄存器的低位字节TLx和高位字节THx中装载的是Timer的计数初始值，当Timer启动后，从这个初始值开始每过1个机器周期计数值增加1，直到计数到16位的Timer寄存器的最大值——FFFFH后溢出为止。

$$t = Val(FFFF - MMLL + 1) \times \frac{1}{f_c(\text{MHz}) \times \frac{1}{12}} \mu\text{s}$$

其中MM是THx中的数值，LL是TLx中的数值。

9.4 模式1

• 9.4.3 Timer的计数时间与计数初始值

例：使用的晶振是12MHz，TH0和TL0中装载的计数初始值是EC78H，代入公式中得：

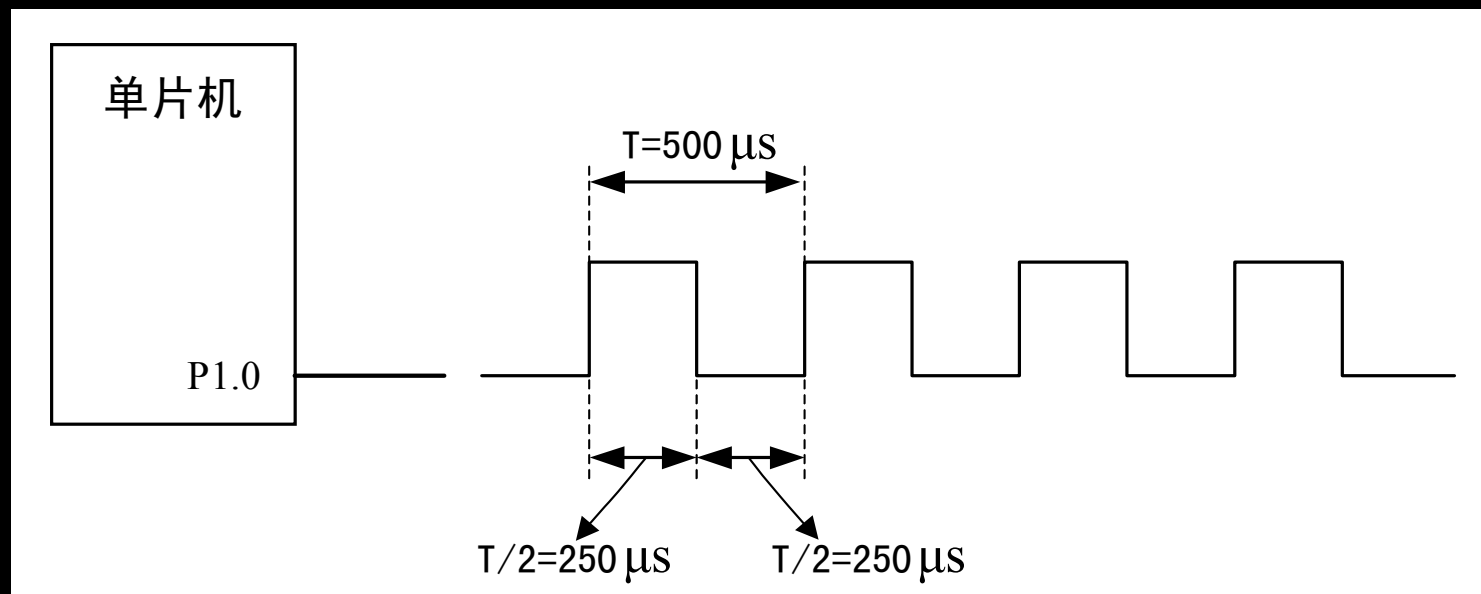
$$t = Val(FFFF - EC78 + 1) \times \frac{1}{12 \times \frac{1}{12}} = Val(1388) \times 1 = 5000 \times 1 = 5000(\mu s)$$

计算过程中，（FFFF-EC78+1）计算结果为1388（十六进制），使用科学计算器可以转换成十进制5000，然后再与单位时间相乘，最后结果为5000，即5ms。

9.4 模式1

• 9.4.4 如何选取Timer的计数初始值

例：设计一个方波信号发生器，要求从P1.0口输出频率 $f=2\text{kHz}$ 的方波，周期 $T=1/f=1/2\times 10^3=500\text{ }\mu\text{s}$ 。所以P1.0口输出高、低电平持续时间分别为 $250\text{ }\mu\text{s}$ 。



9.4 模式1

• 9.4.4 如何选取Timer的计数初始值

例：250即Timer计数时间，称之为设计时间。计数初始值步骤为：

- ① 将设计时间（以 μs 为单位，且小于65536）除以 $\frac{12}{f_c(\text{MHz})}$ 。
- ② 用65536减去步骤①得到的数。
- ③ 用科学计算器把步骤②中的得数转换成十六进制形式MMLL。如果转换结果 $<100\text{H}$ ，则MM=00。
- ④ 最后得计数初始值：THx=MM，TLx=LL。

根据这个步骤，假设晶振频率为12MHz，设计时间250：① $250\mu\text{s} / 1\mu\text{s} = 250$ ；② $65536 - 250 = 65286$ ；③ $65286 = \text{FF06H}$ ；④ THx = FF，TLx = 06。所以Timer寄存器的计数初始值为 FF06H。

9.4 模式1

- 9.4.5 如何利用Timer进行长时间定时
- ✓ Timer的设计时间 t 取决于两个因素：晶振频率，THx和TLx中装载的计数初始值。
- ✓ 硬件一旦完成，意味着晶振频率确定，Timer的设计时间就取决于THx和TLx中装载的计数初始值。
- ✓ 当MMLL=0000H时，设计时间 t 具有最大值，如果晶振频率为12MHz，设计时间的最大值=65536，即65.536ms。这个设计时间连0.1秒都不到，无法满足需要长时间定时的场合。

解决办法是多次重新装载计数初始值

9.4 模式1

• 9.4.5 如何利用Timer进行长时间定时

```

例:      ORG          00H
          START:
              MOV  TMOD, #10H      ; 设置Timer 1工作在模式1下
              MOV  R1, #50          ; R1为重复定时计数器, 定时50次
          LOAD:
              MOV  TL1, #0E0H      ; 往TL0和TH0中载入计数的初始值B1E0H
              MOV  TH1, #0B1H
              SETB TR1              ; 启动Timer 1
          CHECK:
              JNB  TF1, CHECK      ; 检测Timer溢出标志, 当TF1=1, 跳出
              CLR  TR1              ; 关闭Timer 1
              CLR  TF1              ; 将TF1清0
              DJNZ R1, LOAD         ; 如果R1≠0, 跳回LOAD重新装载计数初始值
              .....
              ; 这里是所要处理的程序
              .....
              JMP  START            ; 循环
          END
  
```

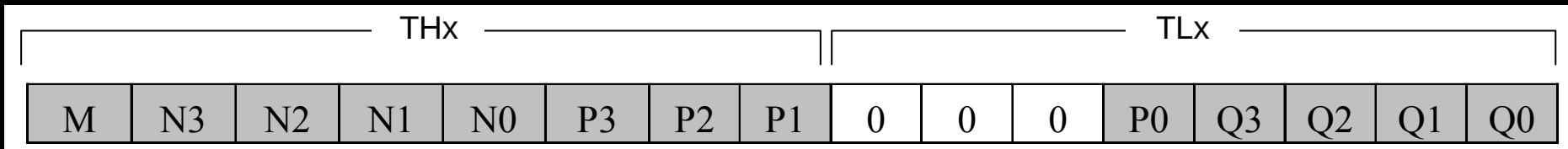
总延时 1s

9.5 模式0

- 模式0与模式1基本相同，不同的是模式1下Timer寄存器是16位的，而模式0下Timer寄存器是13位的。在载入计数初始值时，向THx载入高8位，向TLx的低5位上载入5位。
- 模式0的计数初值的计算方法如下：
 - ① 将设计时间（以为单位，且小于8192）除以 $\frac{12}{f_c(\text{MHZ})}$
 - ② 用8192减去步骤①得到的数。
 - ③ 用科学计算器把步骤②中的得数转换成13位二进制数，高位如果是空的用0补上，依次填入THx的8位和TLx的低5位中，TLx的高3位用0代替。
 - ④ 最后得计数初始值。

9.5 模式0

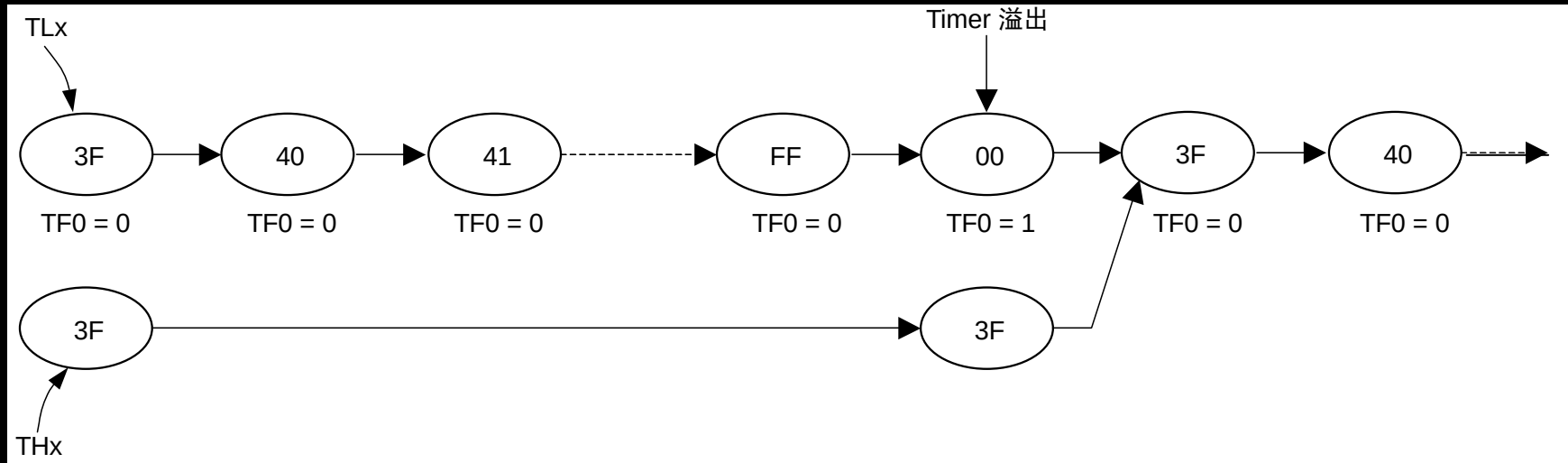
- Timer工作在模式0下产生一个500的延时，使用的晶振频率为12MHz。则计数初始值可以这样计算：
① $500\mu\text{s} / 1\mu\text{s} = 500$ ； ② $8192 - 500 = 7692$ ； ③ $7692 = 11110000\ 01100\text{B}$ ； ④ $\text{THx} = 1111\ 0000\text{B}$ ， $\text{TLx} = 0000\ 1100\text{B}$ 。转换成十六进制数： $\text{THx} = 0\text{F}0\text{H}$ ， $\text{TLx} = 0\text{C}\text{H}$ 。所以Timer寄存器计数初始值为F00CH。



9.6 模式2

• 9.6.1 模式2下的Timer特点

- ✓ Timer工作在模式2下是1个8位的定时/计数器，当计数至FFH之后就溢出，TFx位就会被硬件置1。
- ✓ 自动重新装载：当Timer溢出时，TLx自动从THx中复制原来保存的计数初始值。这时只需要把TFx位清0，就可以自动重复计数的过程。



模式2下的自动重新装载计数初始值

9.6 模式2

• 9.6.2 模式2的设置步骤

- ① 设置Timer的工作模式。向TMOD装载20H，以设置Timer 1工作在模式2。
- ② 往THx和TLx中载入相同的计数初始值。
- ③ 启动Timer。将TRX置1以启动Timer X。
- ④ 检测Timer溢出标志TFx。
- ⑤ 关闭Timer，将TRX清0。
- ⑥ 清溢出标志TFx，将溢出标志TF1清0。

9.6 模式2

• 9.6.3 如何设置计数初始值

✓ 模式2下Timer设计时间与计数初始值的计算式为：

$$t = Val(FF - TT + 1) \times \frac{1}{f_c(\text{MHz}) \times \frac{1}{12}} (\mu\text{s})$$

✓ 如果给定一个设计时间，计算THx和TLx中的计数初始值：

- ① 将设计时间（以为单位，且小于256）除以 $\frac{12}{f_c(\text{MHz})}$
- ② 用256减去步骤①得到的数。
- ③ 用科学计算器把步骤②中的得数转换成十六进制形式TT。
- ④ 最后得计数初始值：THx=TLx=TT。

9.7 模式3

- 对于AT89S51单片机来说，前面3种工作模式均适用于Timer 0和Timer 1，但本节将要谈论的模式3只适用于Timer 0，Timer 1不具有模式3。
- 在模式3下，Timer 0的TL0和TH0变成两个独立的8位的Timer寄存器。但不具备自动重新装载计数初始值的特性。
- 以TL0为Timer寄存器的Timer使用TMOD和TCON中原来Timer 0的控制位，设置的方法与前面相同。而以TH0为Timer寄存器的Timer使用原来Timer 1的溢出标志位TF1和启动/关闭位TR1，并且不能用作计数器使用。

9.7 模式3

- 例： AT89S51单片机的P1.0口输出周期为300、占空比为1/3的矩形波信号。

ORG 00H

START:

MOV TMOD, #03H; 设置Timer 0工作在模式3下

LOAD:

MOV TL0, #9CH ; 往TL0中载入计数初始值

MOV TH0, #38H ; 往TH0中载入另一个计数初始值

SETB P1.0 ; P1.0置1

CALL TL_DELAY ; 调以TL0为基础的延时子程序

CLR P1.0 ; P1.0清0

CALL TH_DELAY ; 调以TH0为基础的延时子程序

JMP LOAD ; 循环，重新装载计数初始值

9.7 模式3

TL_DELAY: ; 以TL0为Timer寄存器的Timer的延时子程序段

SETB TR0 ; 启动以TL0为寄存器的Timer

CHECK0:

JNB TF0, CHECK0 ; 检测溢出标志

CLR TR0 ; 关闭Timer

CLR TF0 ; 将TF0清0

RET ; 返回主程序

TH_DELAY: ; 以TH0为Timer寄存器的Timer的延时子程序段

SETB TR1 ; 启动以TH0为寄存器的Timer

CHECK1:

JNB TF1, CHECK1 ; 检测溢出标志

CLR TR1 ; 关闭Timer

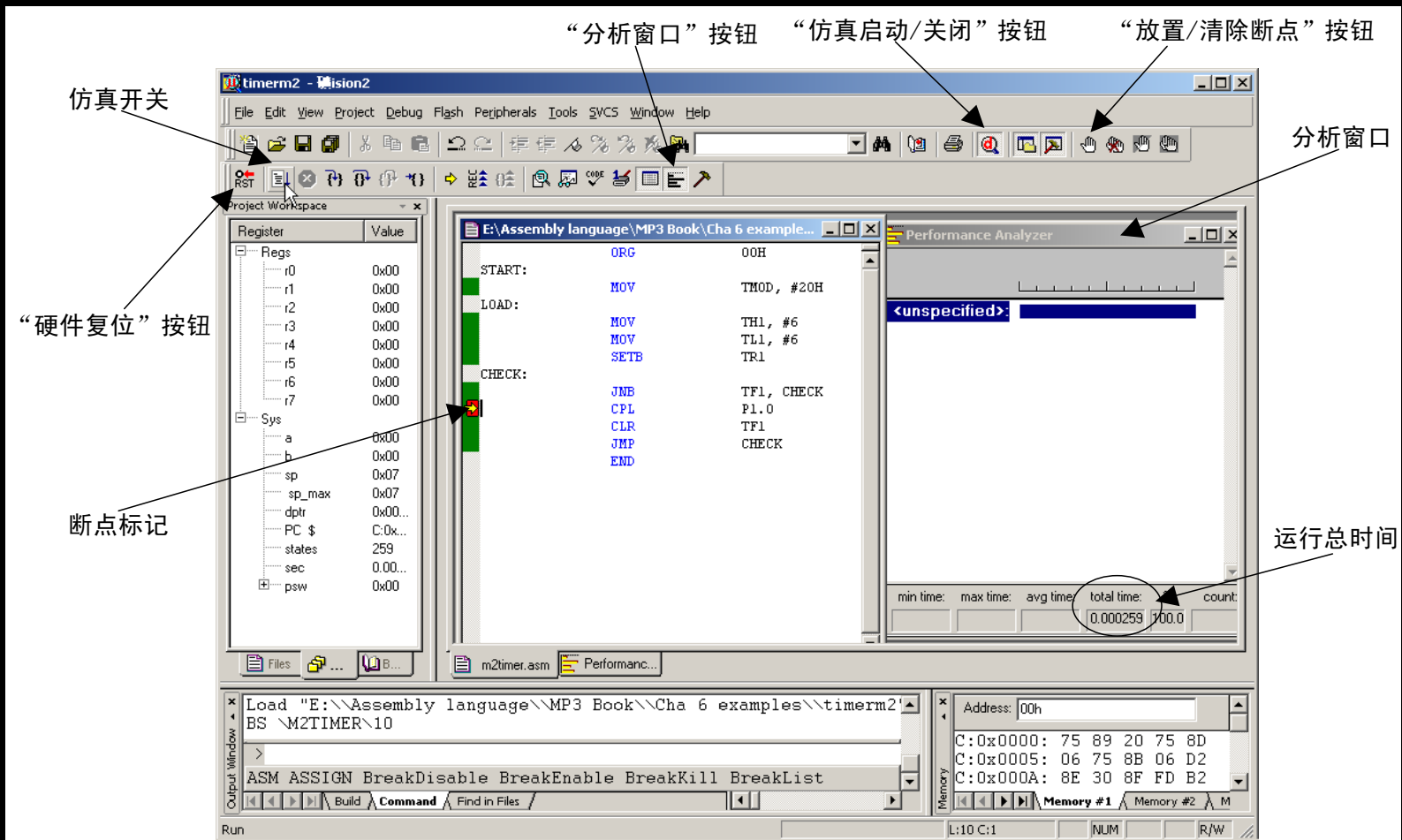
CLR TF1 ; 将TF1清0

RET ; 返回主程序

END

9.7 模式3

- 在μVision中仿真有关Timer的程序，结合断点来观察Timer延时的时长。



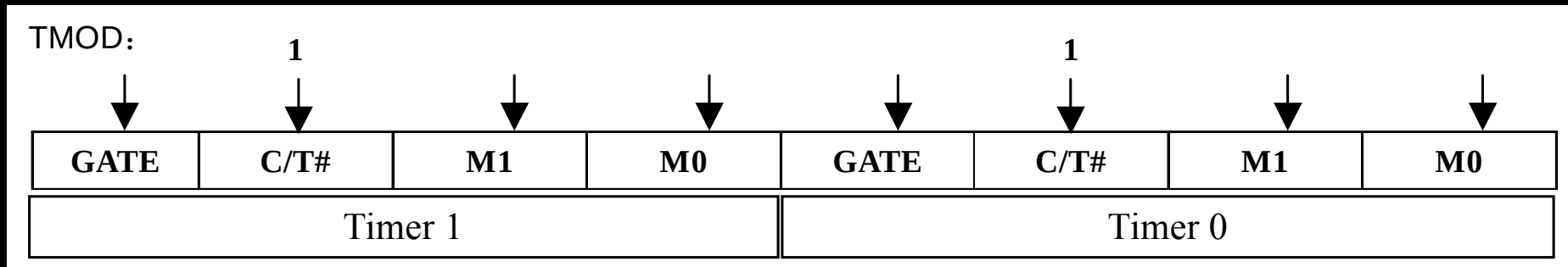
9.8 计数器

- Timer作计数器使用时，Timer寄存器THx和TLx的计数值记录的是外部事件的个数，例如输入的脉冲个数等。
- Timer作计数器使用时，定时/计数器模式控制寄存器TMOD和定时/计数器控制寄存器TCON的设置方法与前面相似。

9.8 计数器

• 9.8.1 Timer作计数器

如果TMOD中的C/T#位=0，Timer用作定时器；
如果C/T#位=1，Timer用作计数器，AT89S51单片机有Timer 0和Timer 1两个Timer，可以构成两个独立的计数器。



Timer用作计数器

9.8 计数器

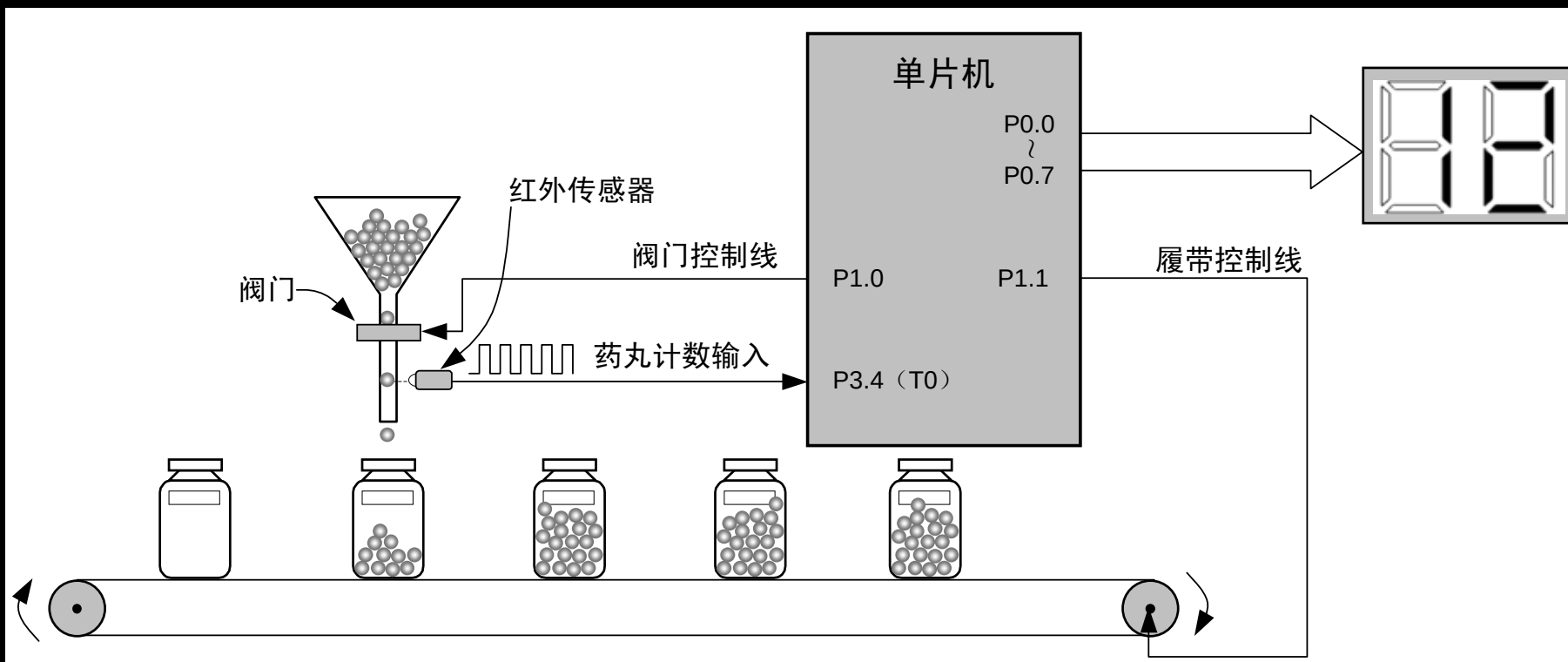
- 9.8.1 Timer作计数器

AT89S51单片机的P3.4口和P3.5口既能作为一般的I/O口使用，也可分别作为Timer 0和Timer 1的外部事件输入口。

管脚序号	I/O口	功能名称	描 述
14	P3.4	T0	Timer 0的外部事件输入端口
15	P3.5	T1	Timer 1的外部事件输入端口

9.8 计数器

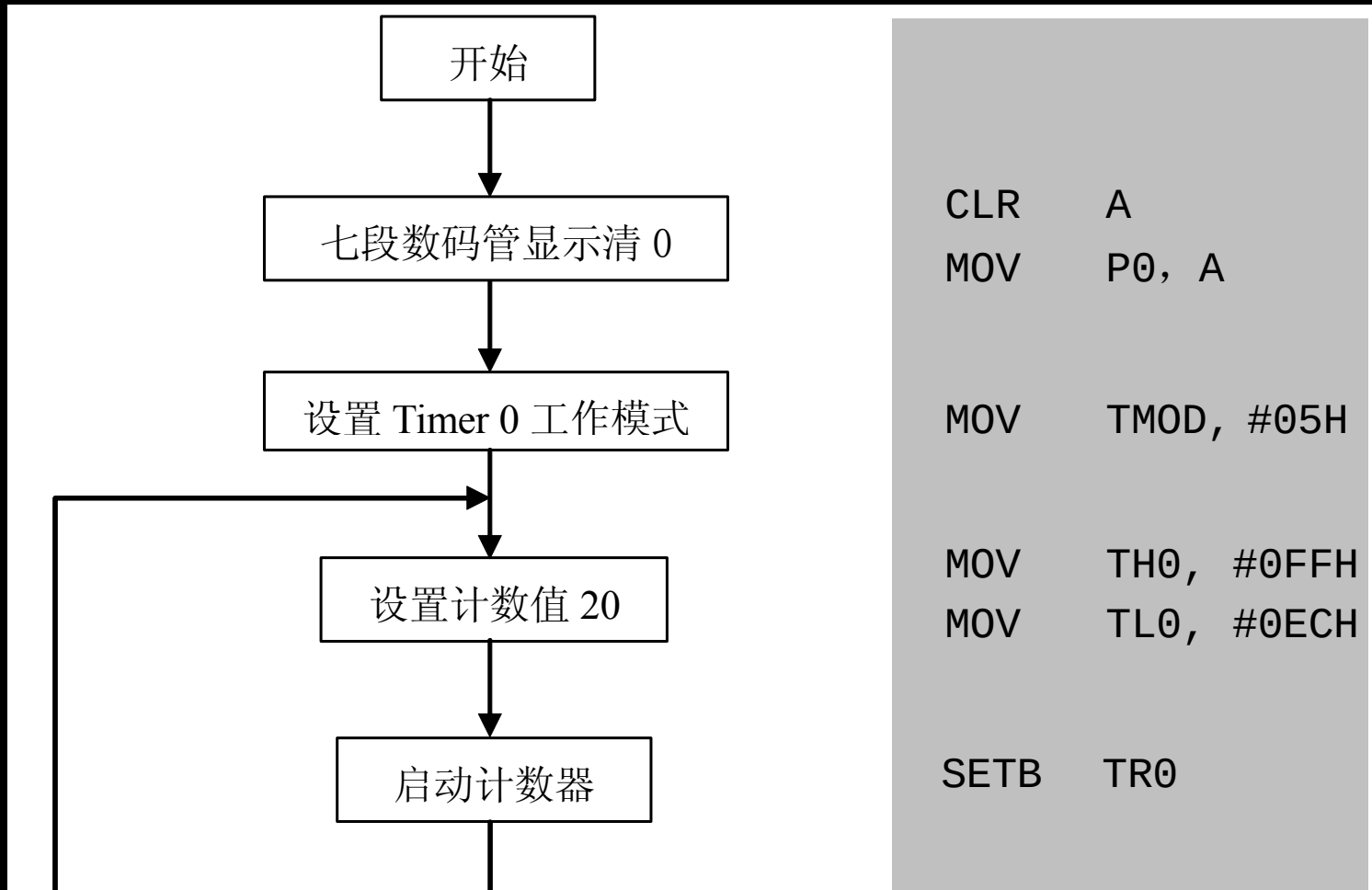
• 9.8.2 计数器的使用



装药丸生产线

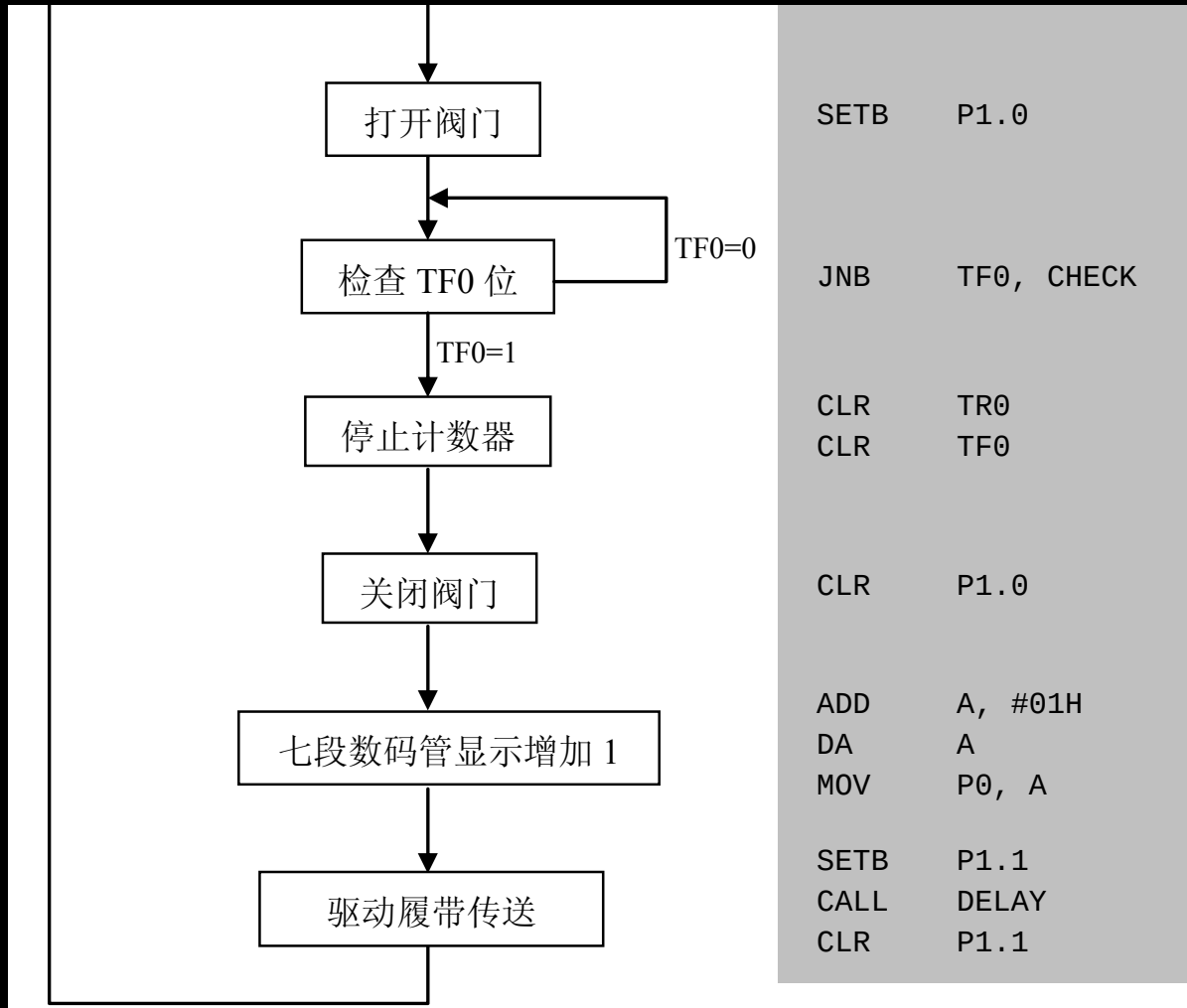
9.8 计数器

• 9.8.2 计数器的使用（装药生产线程序设计思路）



9.8 计数器

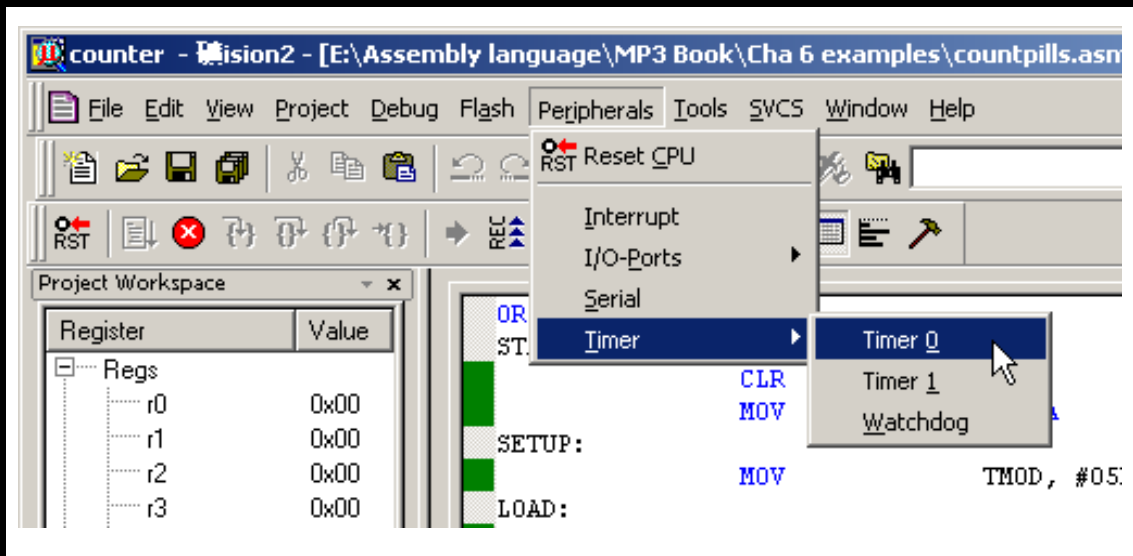
• 9.8.2 计数器的使用（装药生产线程序设计思路）



9.8 计数器

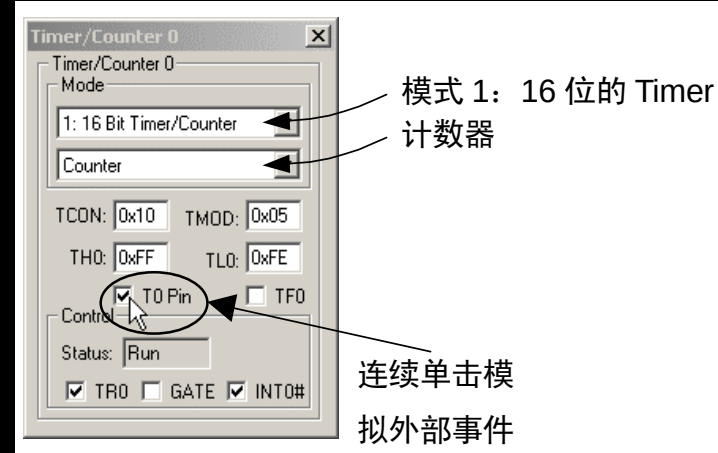
• 9.8.2 计数器的使用（ μ Vision仿真）

使用 μ Vision提供的一个功能来模拟从T0口（P3.4管脚）输入的外部事件。



△ 打开Timer属性对话框

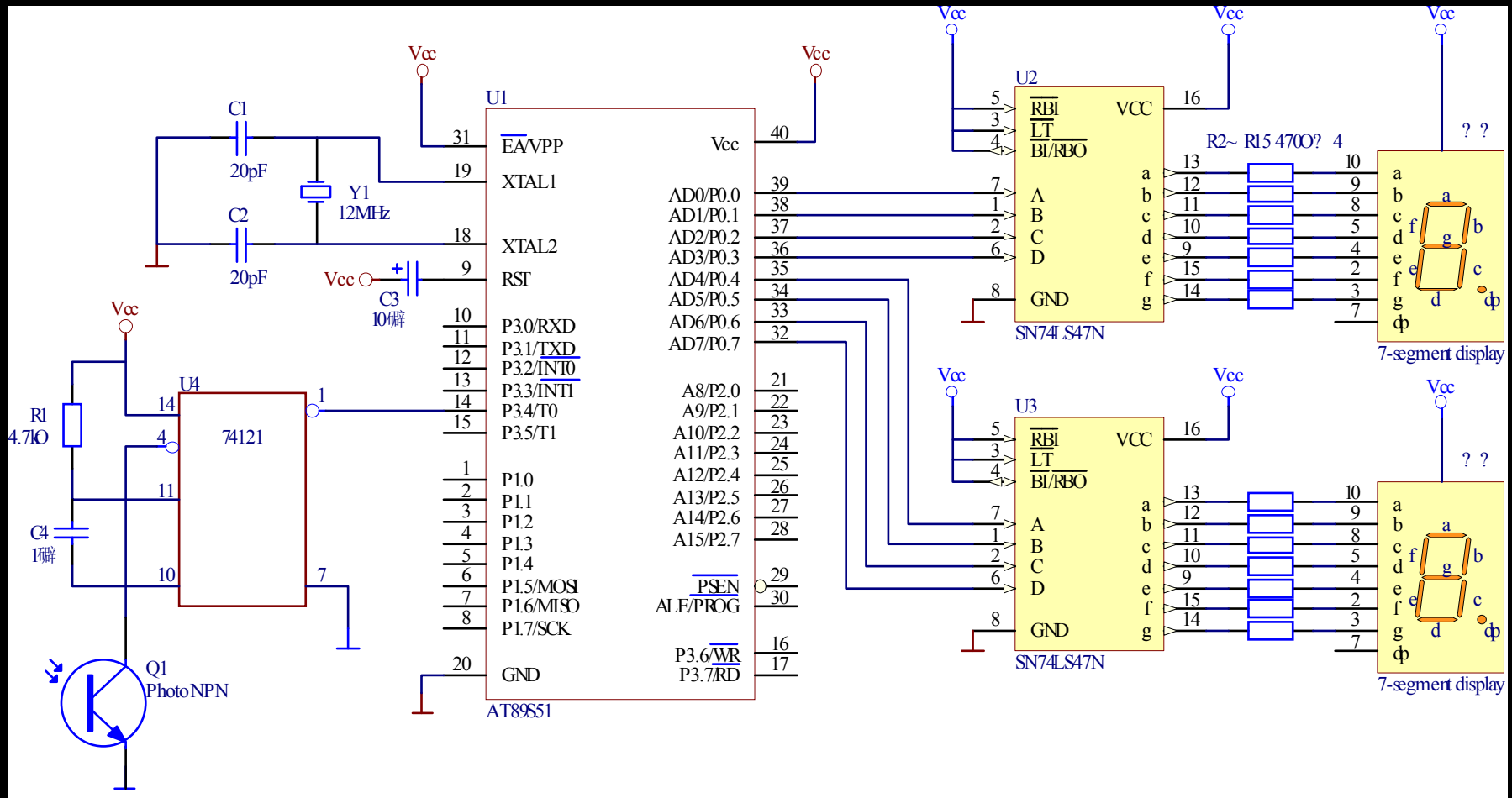
▽ Timer属性对话框



9.8 计数器

- 9.8.3 计数器的计数初始值设定
 - ✓ Timer在计数器下计算的是外部事件的发生次数。Timer工作在模式1下，是一个16位的计数器，其计数初始值通过指令“MOV TH0, #0FFH”和“MOV TL0, #0ECH”载入。这两条指令载入的计数初始值为FFECH。即（65536-20）的十六进制形式。
 - ✓ 计数初始值的载入也可以写成：
MOV TH0, # HIGH (65536 – 20)
MOV TL0, # LOW (65536 – 20)

• 9.8.4 计数器的4种工作模式（例）



制药厂装药丸生产线控制电路的一部分

9.8 计数器

- 9.8.4 计数器的4种工作模式（例）
使用4种模式下计数器来实现计数的任务。

① Timer 0作计数器工作在模式0下（13位计数器）

.....

SETUP:

MOV TMOD, #04H ; 设置Timer 0工作在计数器模式0下

LOAD:

MOV TH0, #(8192-20) / 32

; TH0中载入计数初始值的高位字节

MOV TL0, #(8192-20) MOD 32

; TL0中载入计数初始值的低位字节

SETB TR0 ; 启动Timer 0

.....

9.8 计数器

- 9.8.4 计数器的4种工作模式（例）

- ② Timer 0作计数器工作在模式1下（16位计数器）

.....

SETUP:

MOV TMOD, #05H; 设置Timer 0工作在计数器模式1下

LOAD:

MOV TH0, #HIGH(65536-20)

; TH0中载入计数初始值的高位字节

MOV TL0, #LOW(65536-20)

; TL0中载入计数初始值的低位字节

SETB TR0 ; 启动Timer 0

.....

9.8 计数器

- 9.8.4 计数器的4种工作模式（例）

- ③ Timer 0作计数器工作在模式2下（8位自动重新装载计数器）

.....

SETUP:

MOV TMOD, #06H; 设置Timer 0工作在计数器模式2下

LOAD:

MOV TH0, #(256-20) ; TH0中载入计数初始值的高位字节

MOV TL0, #(256-20) ; TL0中载入计数初始值的低位字节

SETB TR0 ; 启动Timer 0

.....

9.8 计数器

- 9.8.4 计数器的4种工作模式（例）

④ Timer 0作计数器工作在模式3下（8位计数器）

SETUP:

```
MOV    TMOD, #07H
```

；设置Timer 0工作在计数器模式3下

LOAD:

```
MOV    TL0, #(259-20)
```

；TH0中载入计数初始值的高位字节

```
SETB   TR0          ；启动Timer 0
```

.....

9.8 计数器

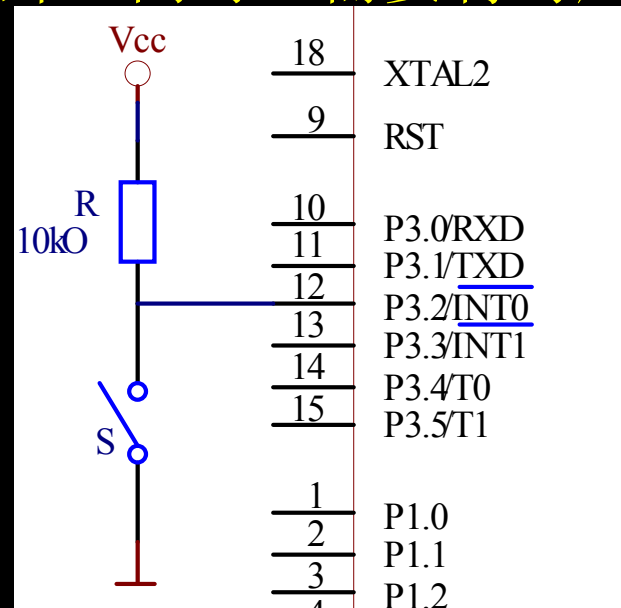
• 9.8.5 外部事件的信号要求

- ✓ 防止红外传感器可能输出的抖动信号使T0口误动作，红外传感器通过一个单稳态多谐振荡器74121芯片向单片机的T0口输入脉冲信号。由于T0口是Timer 0的外部事件输入端口，一般采用硬件的方法来消除抖动。
- ✓ 在计数器的应用中，当T0或T1口的电平由高向低跳变时，Timer计数值增加1。由于单片机“感知”这个电平的跳变需要一定的时间，所以外部事件发生的最高频率应小于晶振频率的1/24。
- ✓ Timer对外部事件信号的占空比没有什么限制，但为了安全起见，确保某一外部事件的电平信号在跳变之前至少被采样一次，则该电平至少要保持一个机器周期。

9.8 计数器

- 9.8.6 TMOD中的门控位GATE=1时
 - ✓ 在设置定时/计数器模式控制寄存器TOMD时，门控位GATE=0，Timer的启动或关闭由软件来控制，用“SETB TRx”启动Timer，用“CLR TRx”来关闭Timer。
 - ✓ 门控位GATE=1，则Timer的启动或关闭由硬件来控制。此时，尽管使用指令“SETB TRx”外，同时还需要将对应的管脚置1才能启动相应的Timer。

当Timer 0的门控位GATE=1时，
只有当开关S断开时=1，指令“SETB TR0”才能启动Timer 0。



9.9 看门狗

• 9.9.1 什么是看门狗

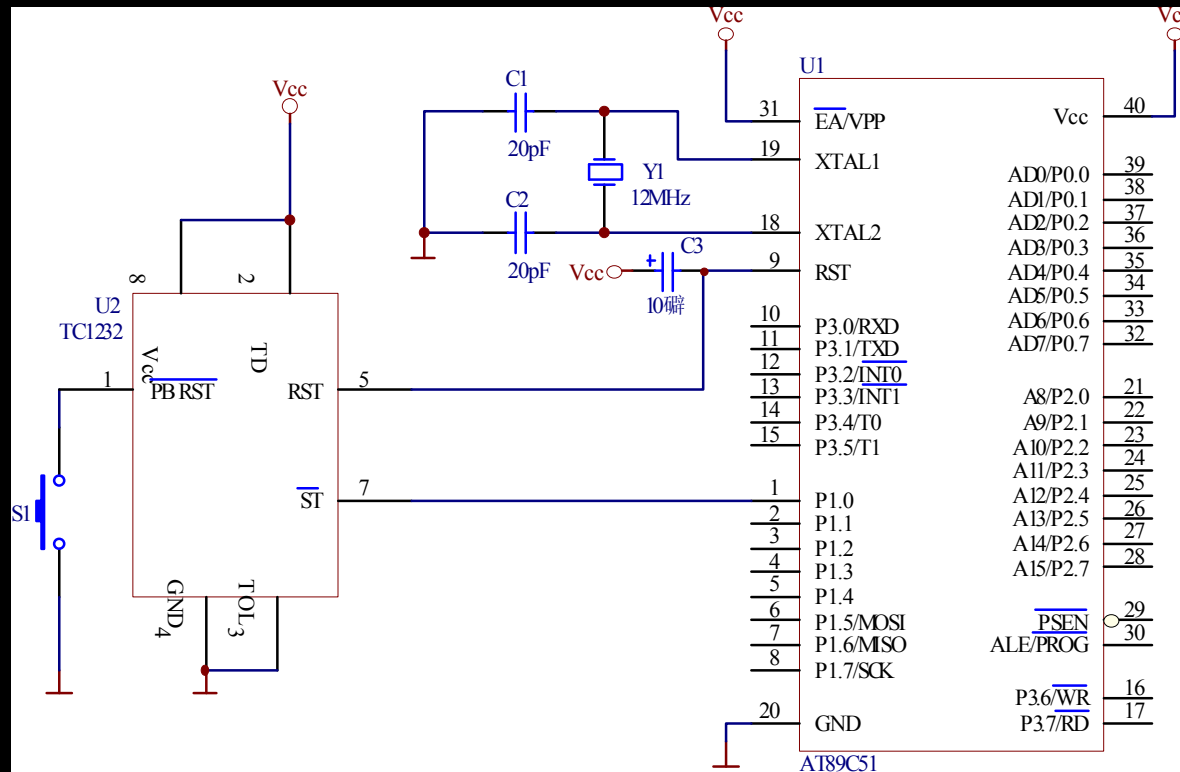
单片机系统中的看门狗具有以下两个特征：

- ① 看门狗定时器必须在一定时间内被刷新。如若不然，看门狗定时器就会溢出而导致单片机复位。
- ② 当看门狗定时器启动之后，程序可以控制看门狗定时器使单片机复位，或者等到看门狗定时器溢出时使单片机复位。后一种情况可以保证程序对导致看门狗定时器溢出的系统错误进行处理。

9.9 看门狗

• 9.9.2 外置看门狗

看门狗定时器能使单片机复位，看来看门狗定时器应该与单片机的复位管脚RST（第9脚）有直接的联系。



单片机系统与看门狗电路

9.9 看门狗

• 9.9.2 外置看门狗

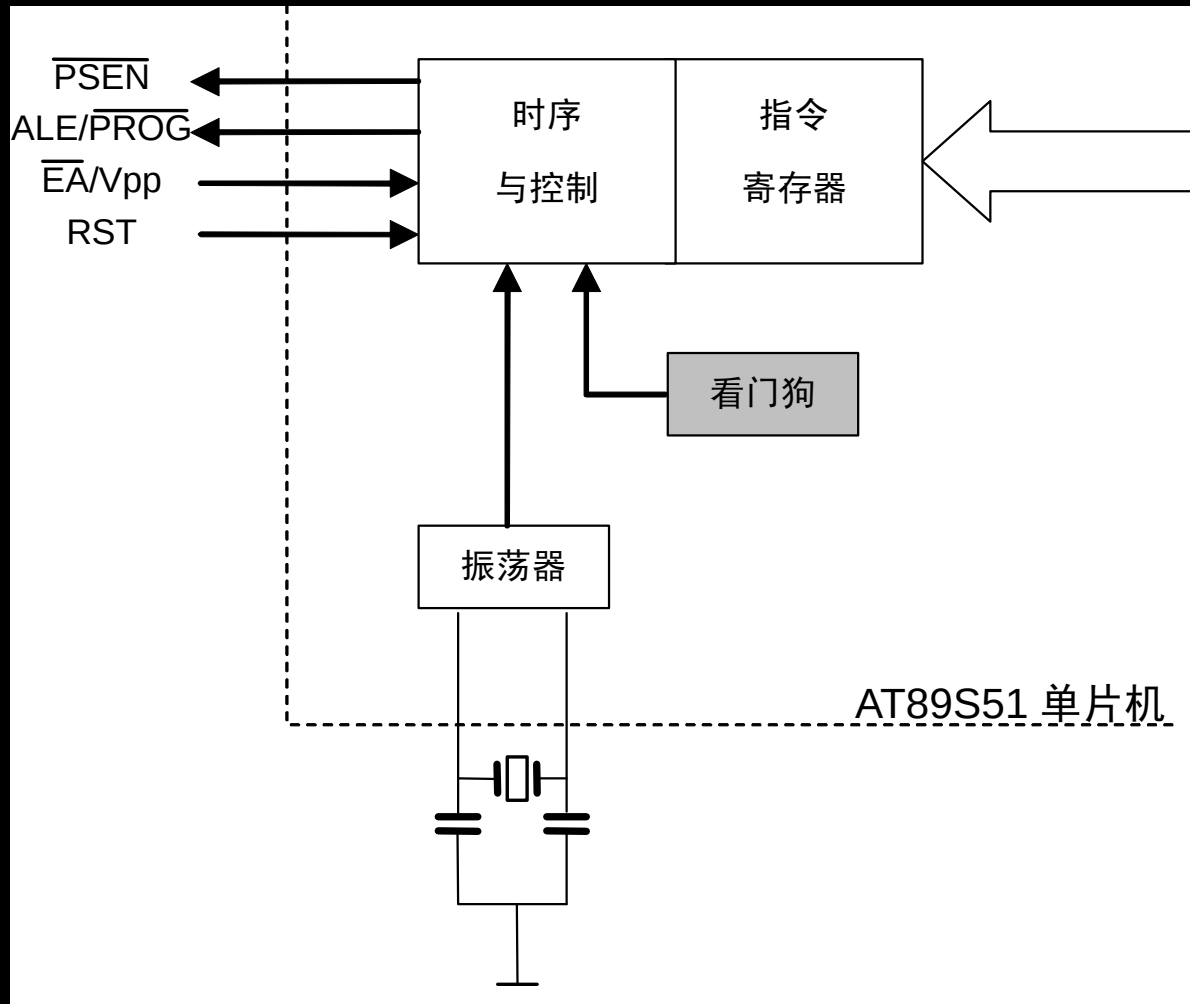
如果单片机的P1.0没有在看门狗定时器溢出时间之内向管脚输出一个下降沿，看门狗就会发挥作用，向单片机的复位端RST管脚输出一个复位信号使单片机复位。

条 件	看门狗定时器溢出时间		
	最 小 值	典 型 值	最 大 值
TD = GND	62.5ms	150ms	250ms
TD悬空	250ms	600ms	1000ms
TD = V _{cc}	500ms	1200ms	2000ms

TD管脚状态与看门狗定时器溢出时间

9.9 看门狗

• 9.9.3 内置看门狗



AT89S51单片机中的看门狗由一个14位的定时器和看门狗定时器复位寄存器WDTRST组成。WDTRST是特殊功能寄存器SFR中的一个，地址是A6H。

9.9 看门狗

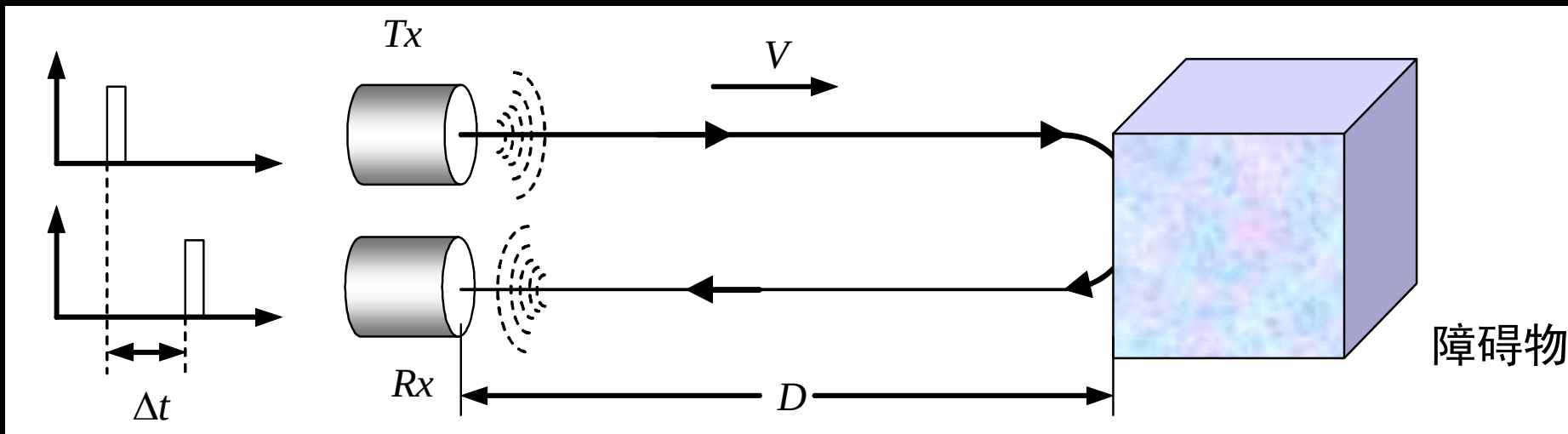
• 9.9.3 内置看门狗

- ✓ 当单片机上电复位时，默认看门狗功能被禁止。
- ✓ 要想启动看门狗功能，需要把立即数1EH和0E1H按顺序写入WDTRST寄存器中。
- ✓ 当看门狗启动后，其14位定时器的计数值每过1个机器周期增加1，直到当看门狗溢出时，它会使单片机的RST管脚电平被拉高从而促成单片机的复位。
- ✓ 当看门狗被启动后，程序是无法将其关闭的，而只有当单片机通过RST管脚重新复位或看门狗自己溢出导致单片机复位时看门狗才会关闭。

9.10 实例点拨——超声测距仪

• 9.10.1 什么是超声测距仪

超声测距仪是一种利用超声波来测量超声传感器到障碍物之间距离的仪器。



超声波用于距离测量 $D = (V \times \Delta t) / 2$ 。

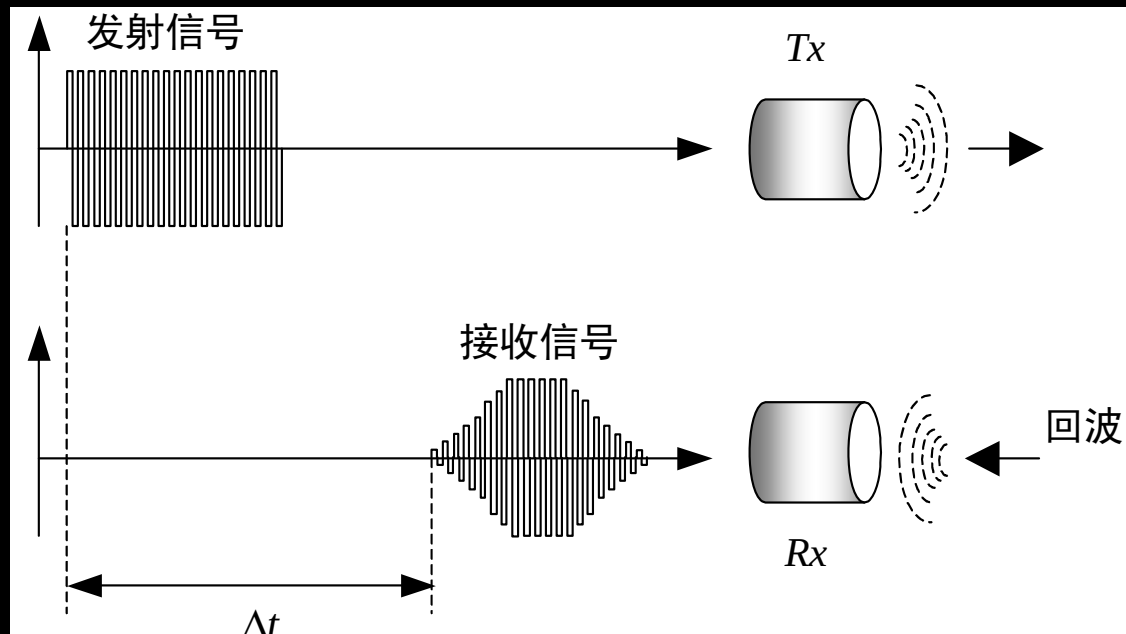
9.10 实例点拨——超声测距仪

• 9.10.2 明确单片机系统功能

- ✓ 从单片机的P1.0口输出周期=24 μ s的方波信号，该信号通过适当的驱动电路来推动超声波发射器Tx发射超声波（为了保证回波的质量，P1.0口连续输出20个方波作为Tx的发射脉冲信号）
- ✓ 发射出去的超声波遇到障碍物后被反射回来，并被超声波接收器检测到，而输出一个接收脉冲信号，该信号经过电路的调理后由单片机的I/O口检测到，随即停止Timer。

9.10 实例点拨——超声测距仪

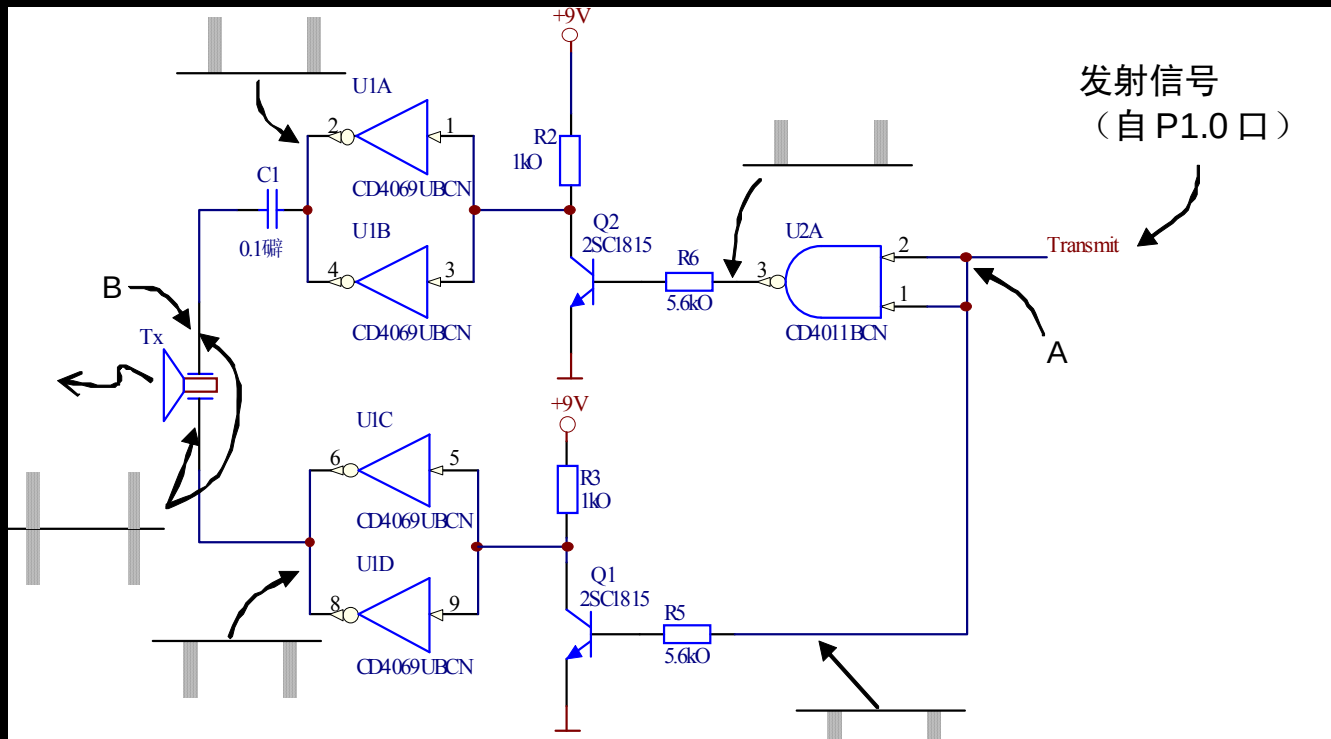
- 9.10.2 明确单片机系统功能
 - ✓ **Timer**中的当前计数就是超声波从发射到接收的时间差，单位为 μs （晶振频率=12MHz），利用程序计算出距离值并显示。
 - ✓ 显示完成后，单片机开始新一次的超声波脉冲发射与接收过程。



9.10 实例点拨——超声测距仪

• 9.10.3 硬件设计

硬件电路包括几个部分：超声波发射器及其驱动电路、超声波接收器及信号调理电路、单片机最简系统、两位七段数码管等。

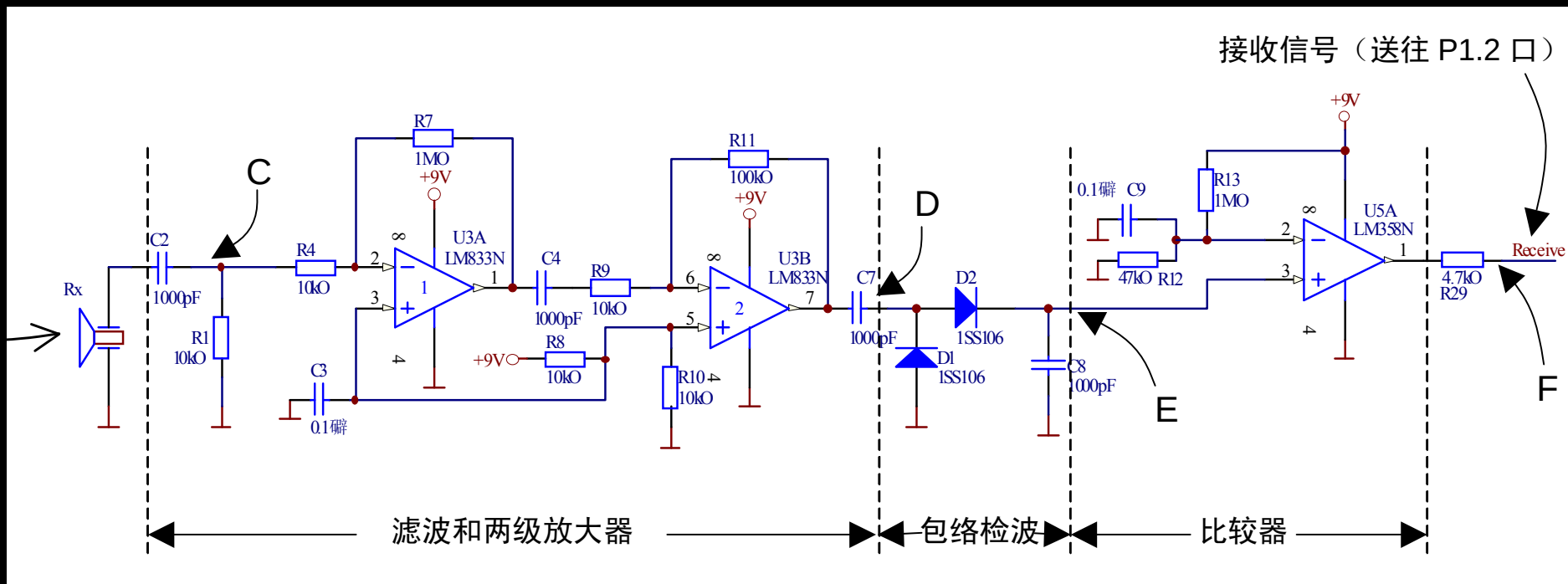


超声波发射器及其驱动电路

9.10 实例点拨——超声测距仪

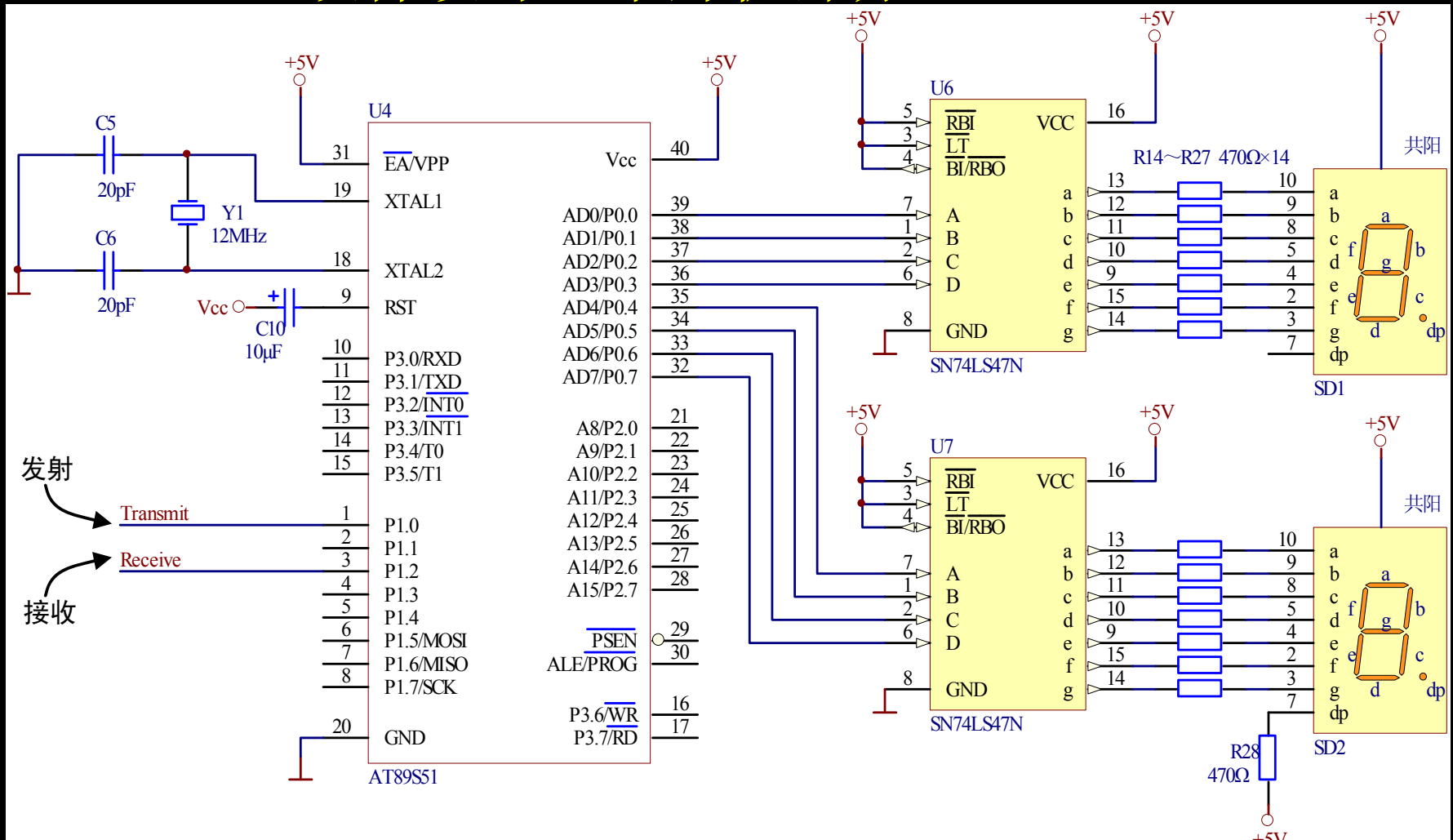
• 9.10.3 硬件设计

(超声波接收器及信号调理电路)



9.10 实例点拨——超声测距仪

• 9.10.3 硬件设计（单片机部分）



9.10 实例点拨——超声测距仪

• 9.10.4 软件设计

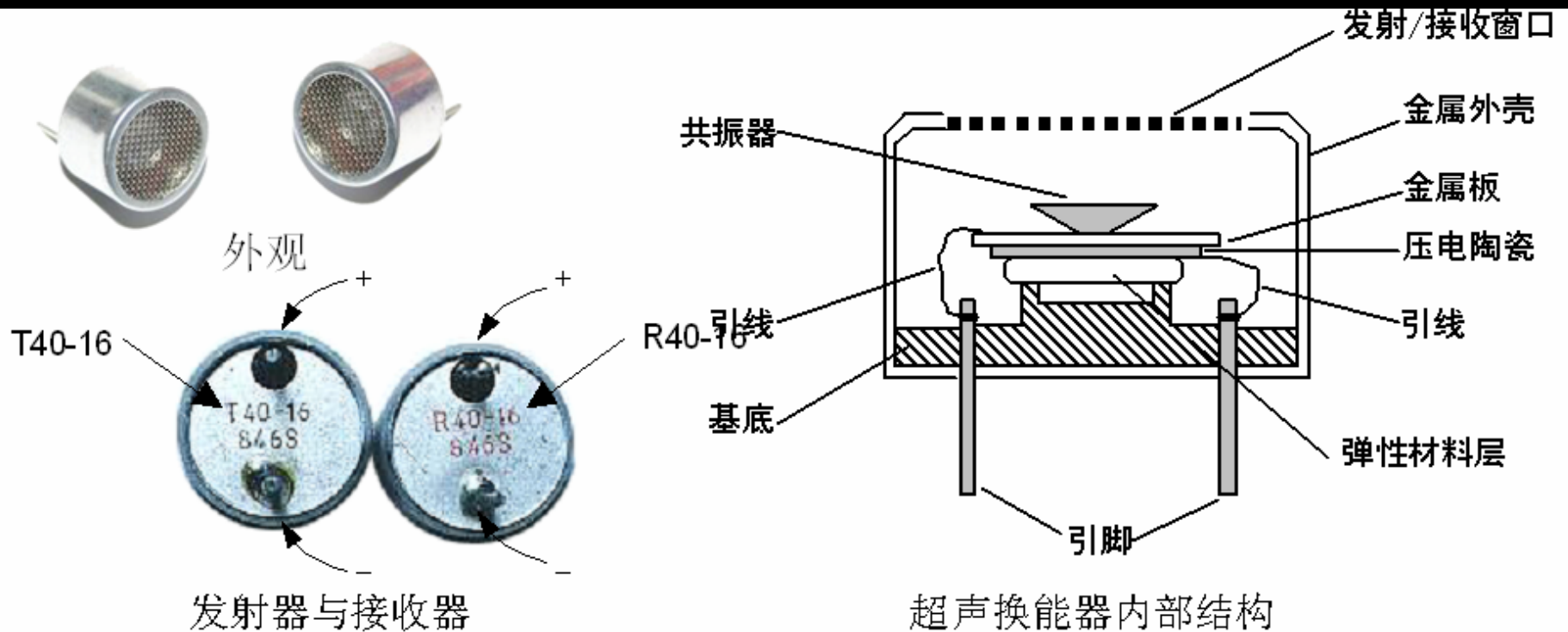
超声测距仪的程序有3个部分组成：产生发射脉冲信号、等待回波信号时的计时功能、计算并显示距离。

- ① 产生发射脉冲信号。可以使用**Timer 0**工作在模式2下（具有自动重新装载功能的8位定时器）进行延时而产生周期为**24 μ s**的脉冲信号。
- ② 等待回波信号时的计时功能。可以使用**Timer 1**工作在模式1下，**Timer**的计数值范围最大为**65536**。在**12MHz**晶振频率下，该超声测距仪的测量最大距离为**11.14m**。如果在**65.536ms**内没有接收到回波，说明测量距离大于**11.14m**或别的原因导致回波丢失，则**Timer 1**在**65.536ms**后溢出，程序跳回发射脉冲信号部分重新发射超声波。
- ③ 计算并显示距离。

器件介绍

• 1. 超声换能器

超声波发射器和超声波接收器具有电压和声波的转换功能，它们把一种形式的能量转换成另一种形式的能量，所以发射器和接收器统称为超声换能器（ultrasonic transducer）。



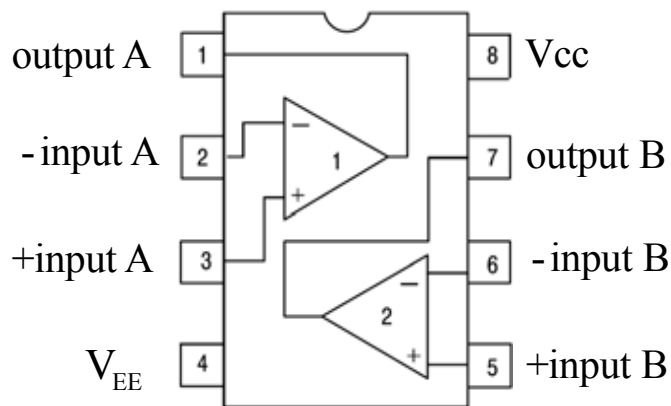
器件介绍

• 2.运算放大器

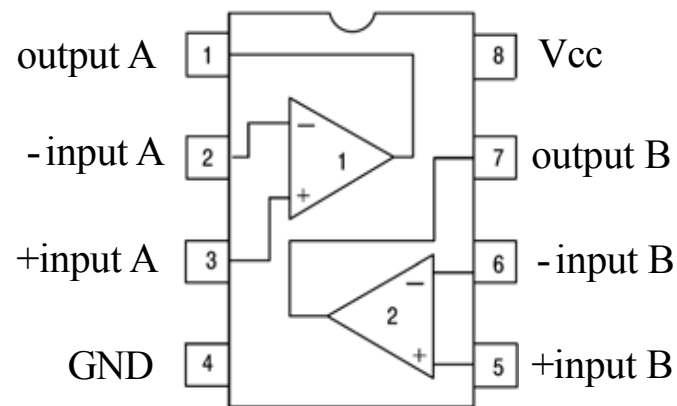
运算放大器（**operational amplifier**）简称运放，是一种高增益的集成型放大电路。以运算放大器为中心的应用电路很多，如放大器、有源滤波器、比较器等。



(a) 外观



(b) LM833内部结构图

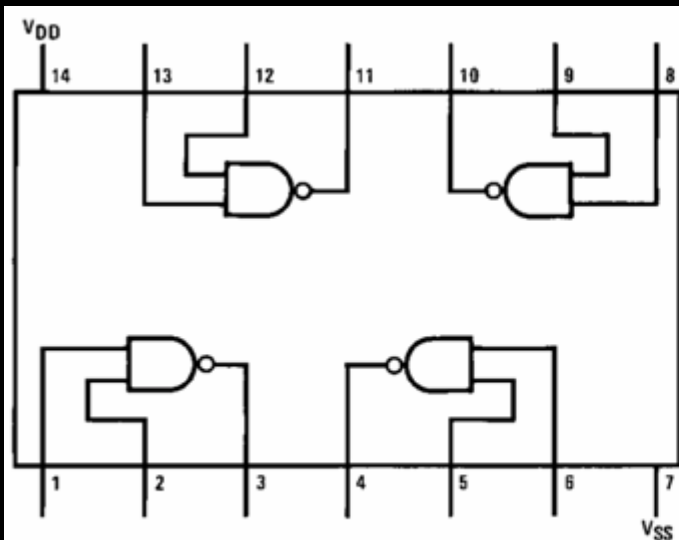


(c) LM358内部结构图

器件介绍

• 3. 与非门

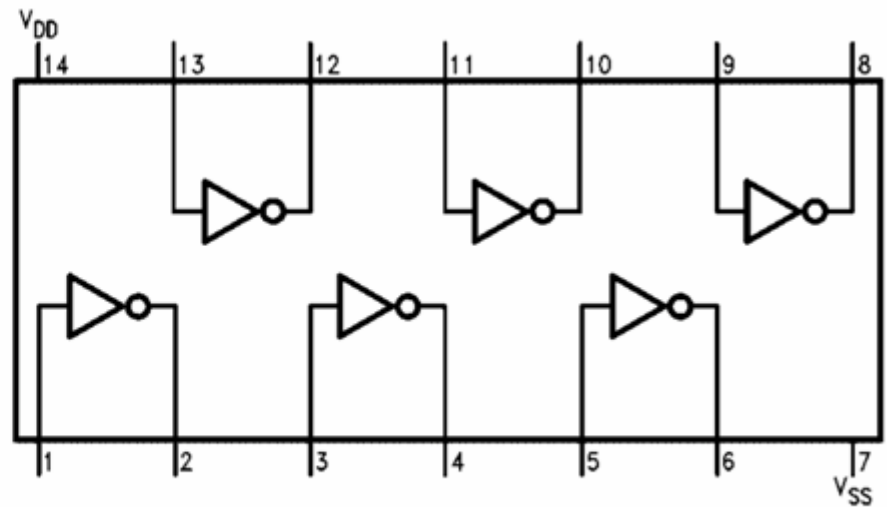
CD4011是一个与非门（NAND gate）集成电路，在系统中对单片机I/O口输出的方波信号进行反相。



与非门CD4011

• 4. 非门

CD4069是一个非门（Inverter）集成电路，它在系统中驱动超声波发射器。



非门CD4069

器件介绍

• 5. 二极管

二极管（**diode**）是最基础的电子器件之一，它的特点就是管中的**PN**结使其具有单向导电性。系统中使用的是肖特基二极管**1SS106**。二极管一端有一个白色的环代表对应的引脚为负极。由于该二极管具有良好的高频特性，所以应用在回波信号的包络检波器中。



第10章 与外界的沟通

——串行口通信

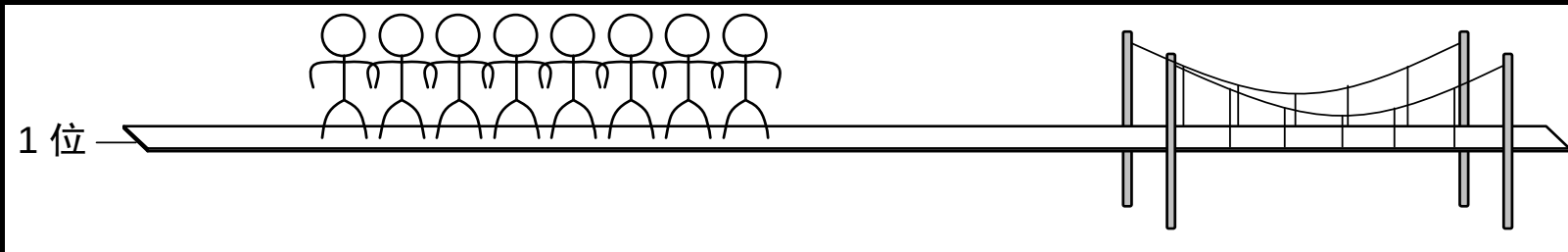
欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 10.1 串行与并行通信
- 10.2 单片机串行口如何发送与接收数据
- 10.3 单片机串行口的控制者
- 10.4 串行口工作模式及波特率
- 10.5 单片机之间的通信
- 10.6 计算机的串行口与控制
- 10.7 单片机与计算机之间的通信
- 10.8 实例点拨——智能交通灯

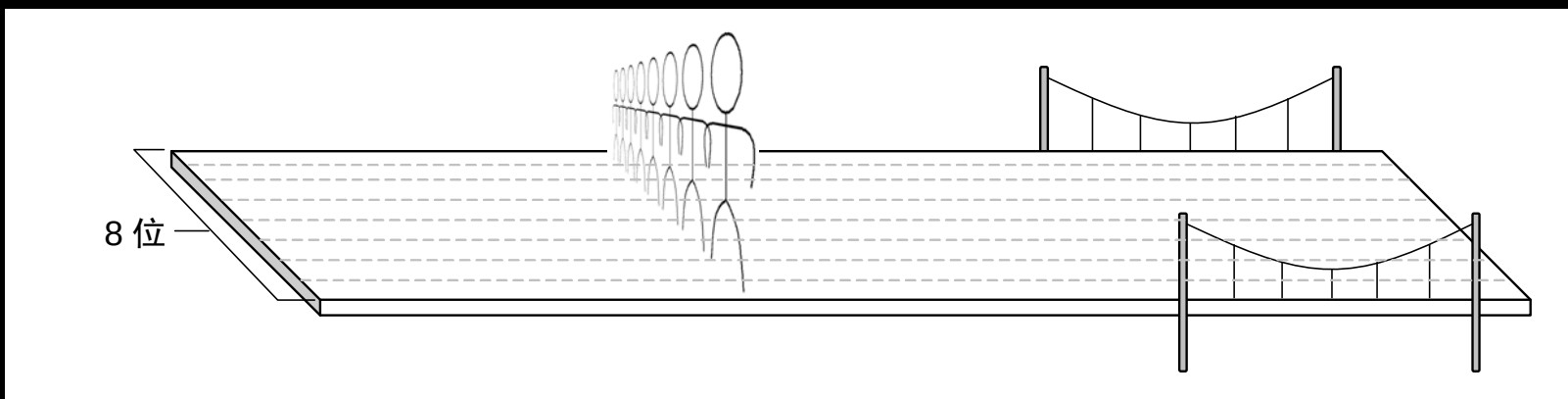
10.1 串行与并行通信

• 10.1.1 串行与并行通信的比喻



依次通过--串行通信--尽量少的数据线

串行方式：
一切USB
设备

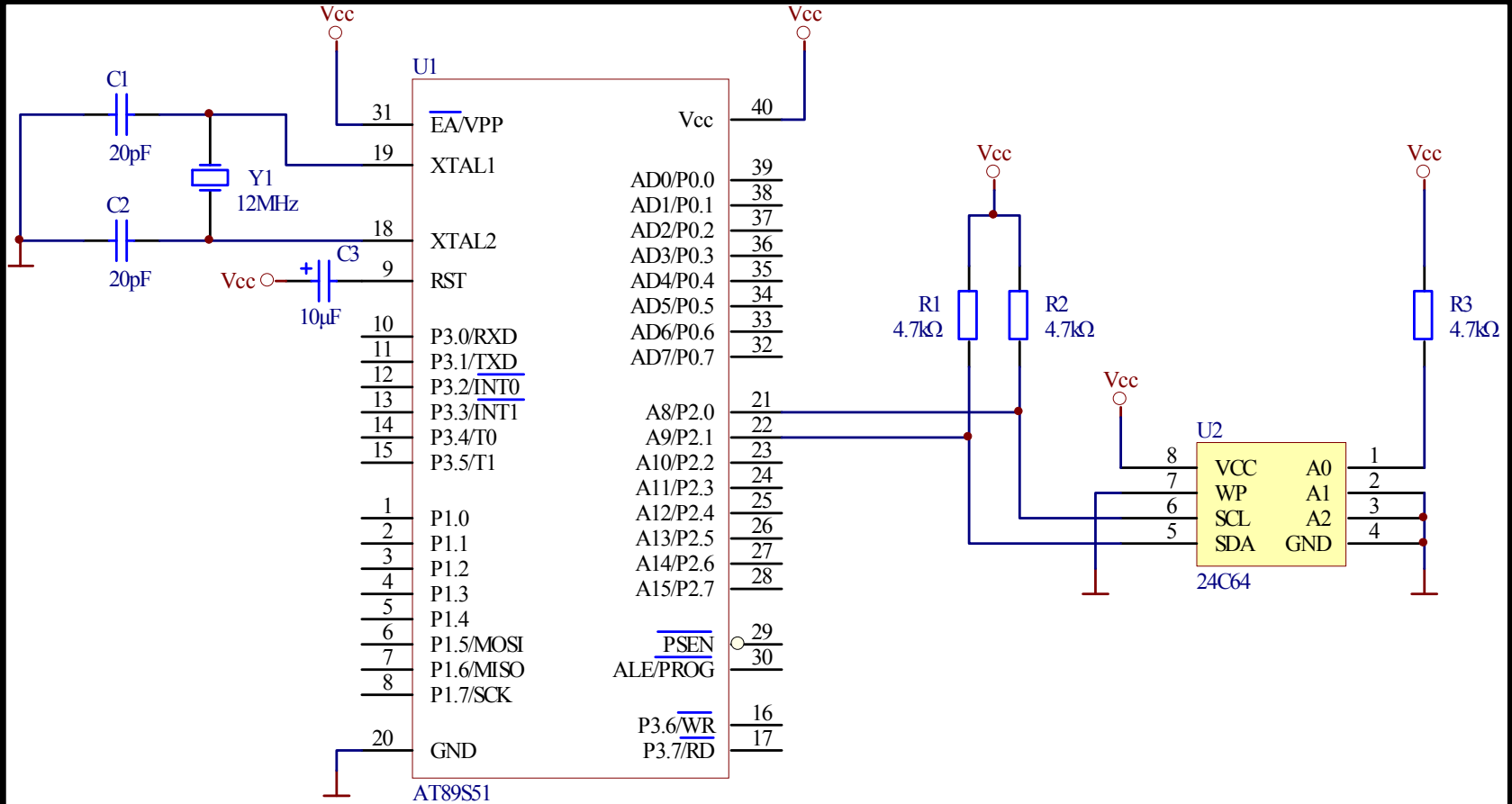


并排通过 --并行通信--更快的传输速度

并行方式：硬
盘；

10.1 串行与并行通信

• 10.1.2 串行与并行存储器



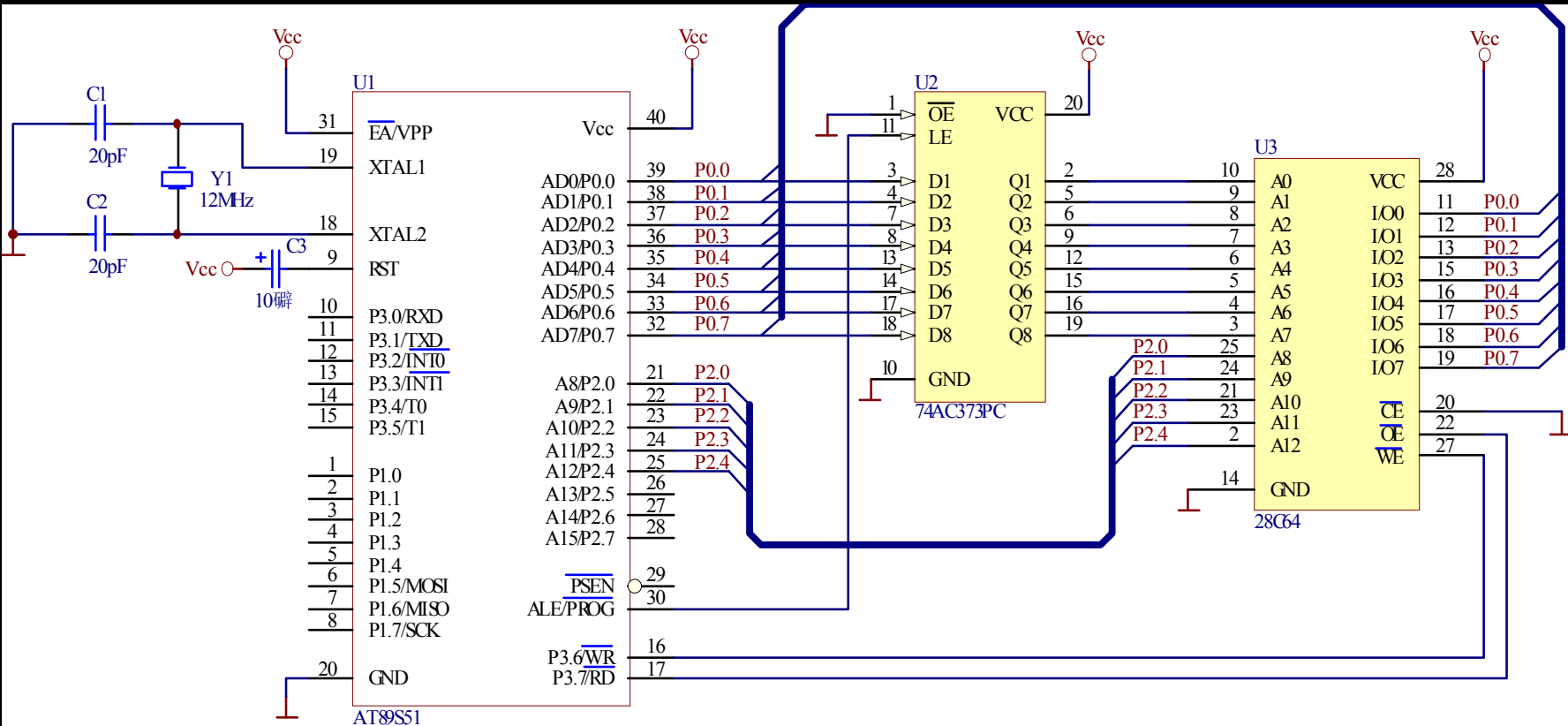
串行存储器的接口

串行存储器24C64只占用单片机的两个I/O口作为通信线

10.1 串行与并行通信

• 10.1.2 串行与并行存储器

并行存储器的接口



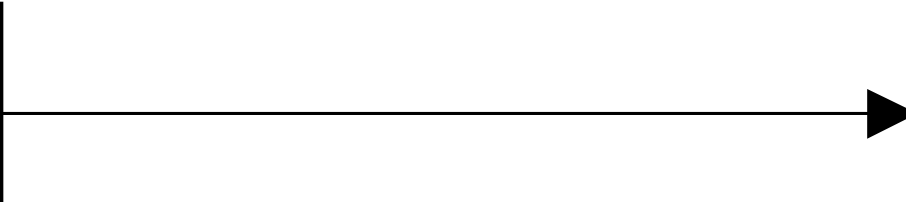
并行存储器28C64除占用13个I/O口外，还要使用74373锁存器作为地址锁存器

10.1 串行与并行通信

- 10.1.3 单工、半双工和全双工

在单工（**simplex**）下，发送方与接收方之间只有一条通信链路，数据只能从发送方向接收方传送，如广播电台和电视台发射的节目信号，只需要由接收方（收音机、电视机）接收。

发送方

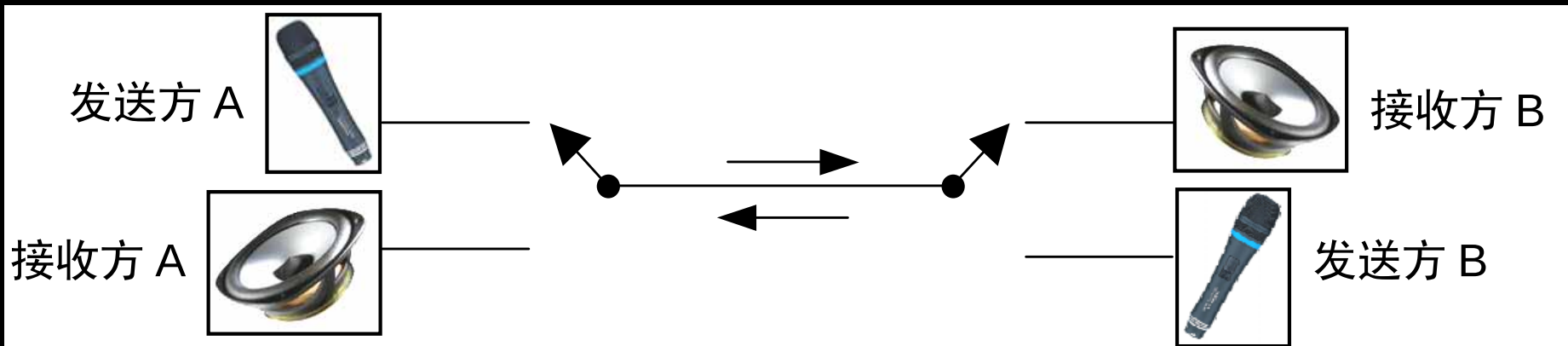


接收方

10.1 串行与并行通信

• 10.1.3 单工、半双工和全双工

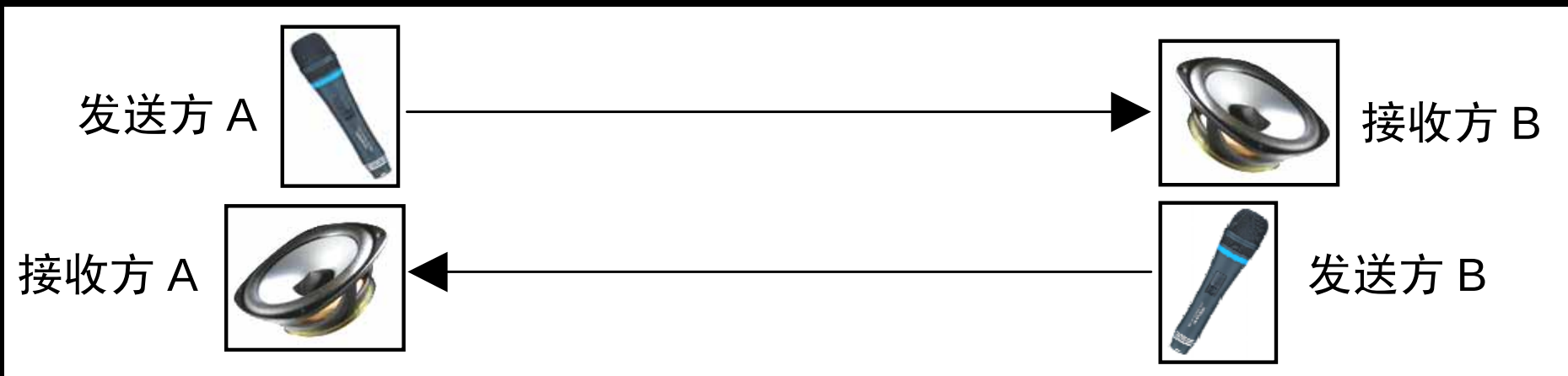
在半双工（**half duplex**）下，双方也只有一条通信链路，通过两个开关控制由哪一方发送，哪一方接收，A、B双方不能同时进行发送与接收。



10.1 串行与并行通信

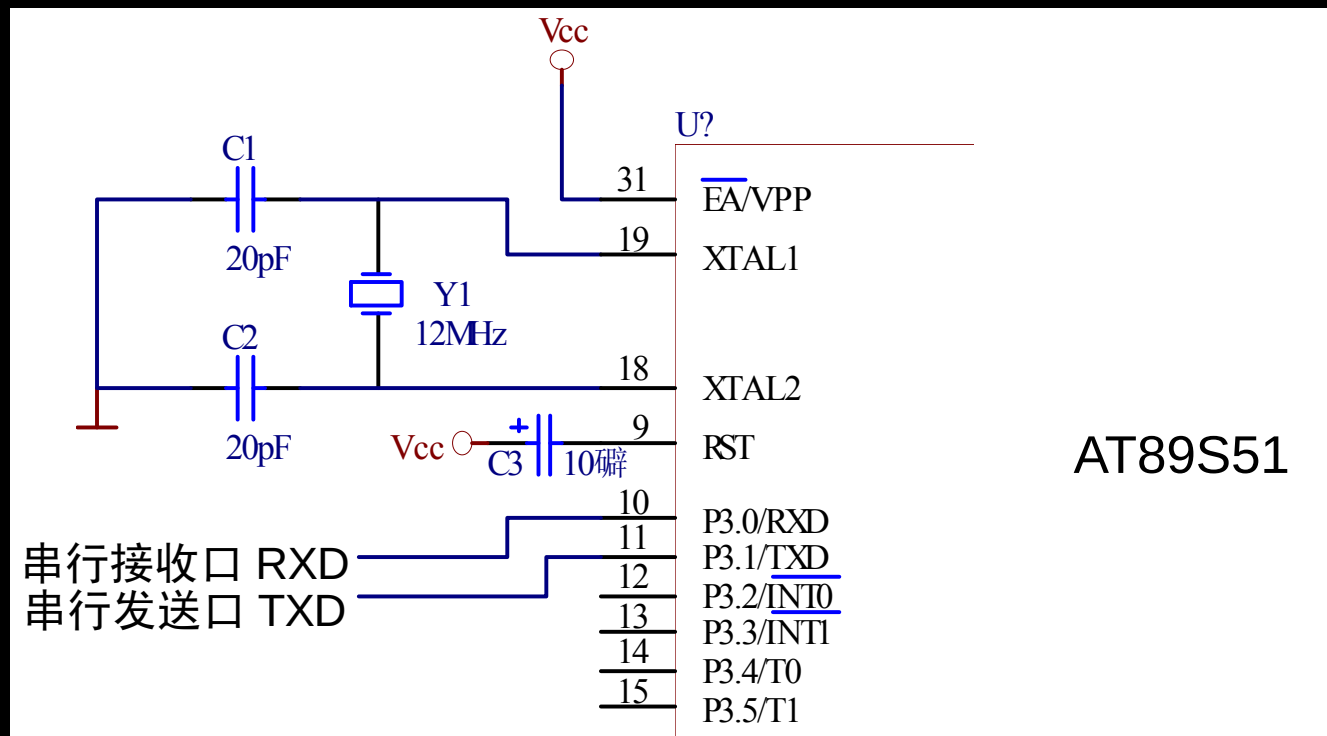
• 10.1.3 单工、半双工和全双工

在全双工（**duplex**）下，双方可以同时进行发送和接收，但它需要两条通信链路。



10.2 单片机串行口如何发送与接收数据

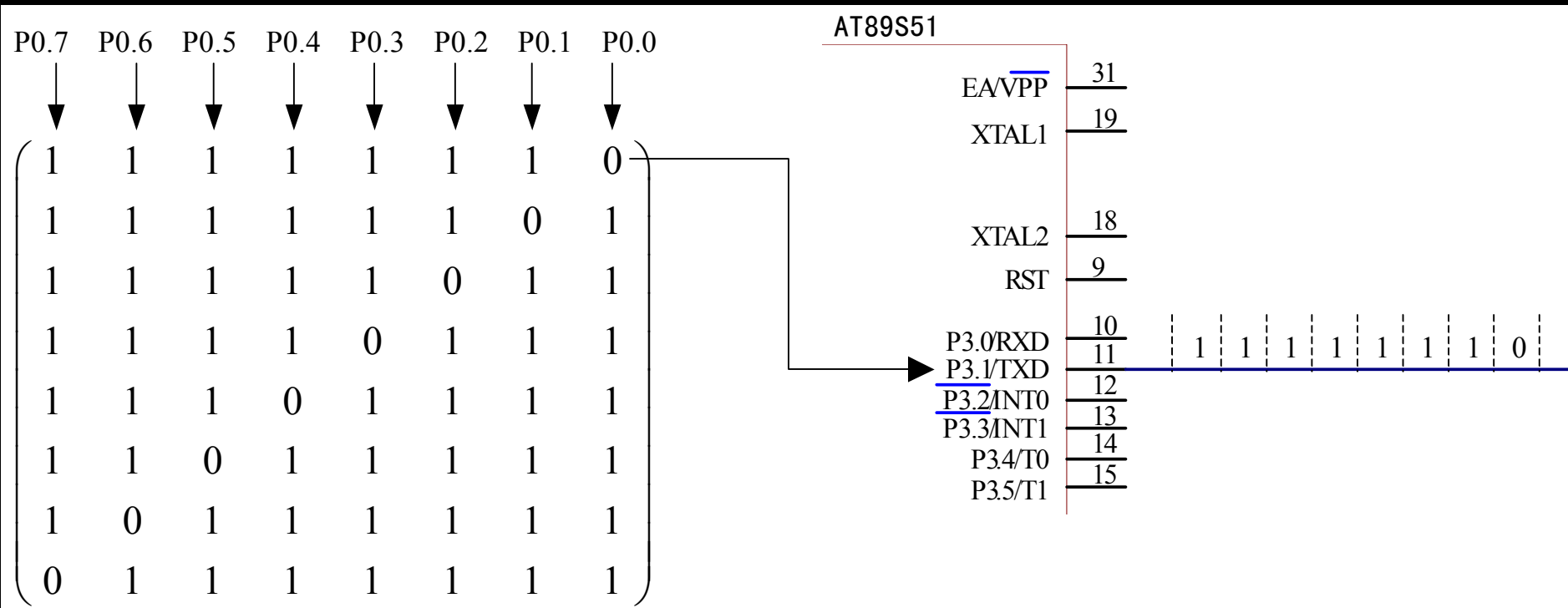
- AT89S51单片机的P3.0和P3.1除作为一般I/O口外，还分别在串行通信中充当接收口（RXD）和发送口（TXD）。



AT89S51单片机的串行口

10.2 单片机串行口如何发送与接收数据

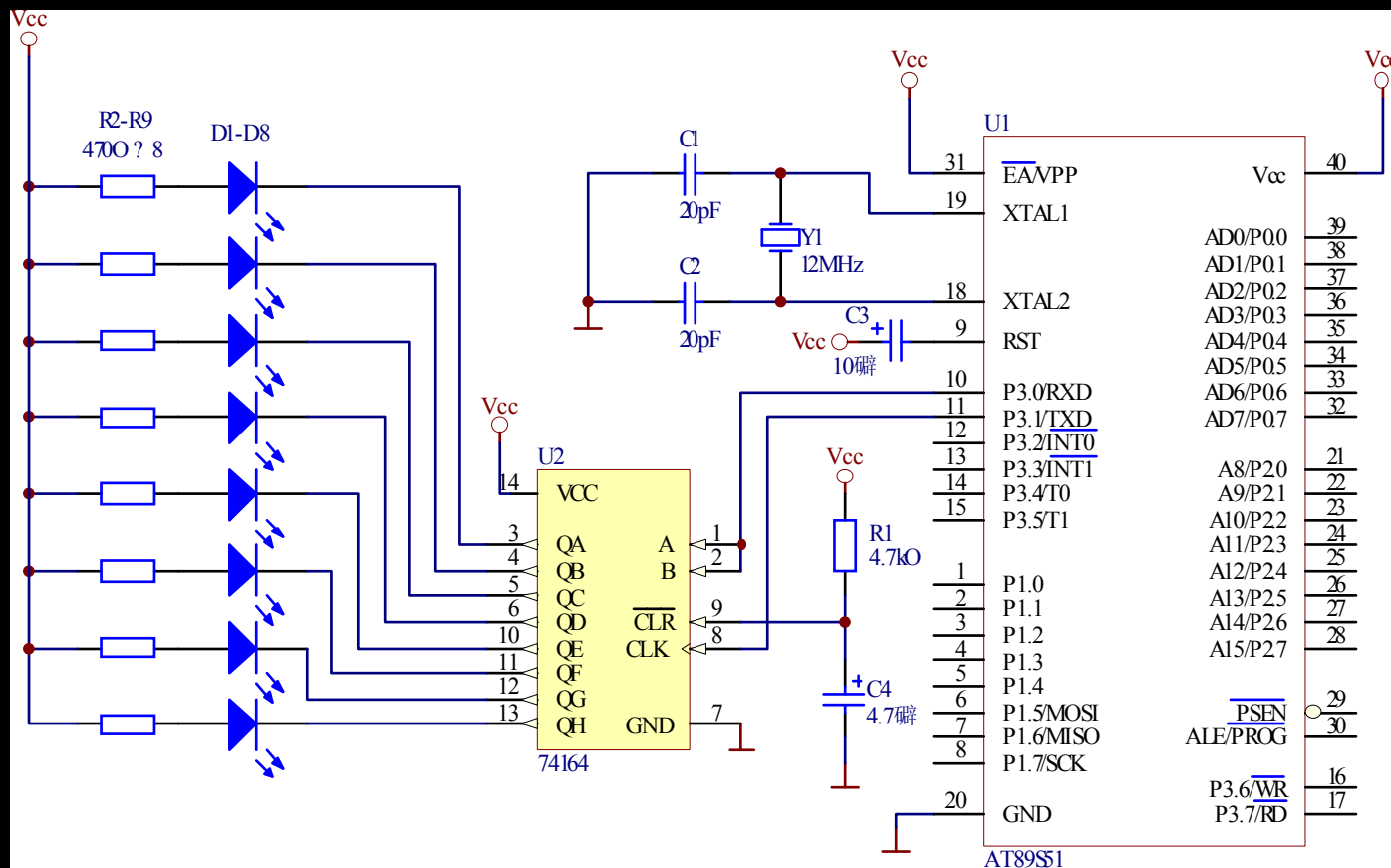
• 10.2.1 串行发送数据



从并行数据到串行数据

10.2 单片机串行口如何发送与接收数据

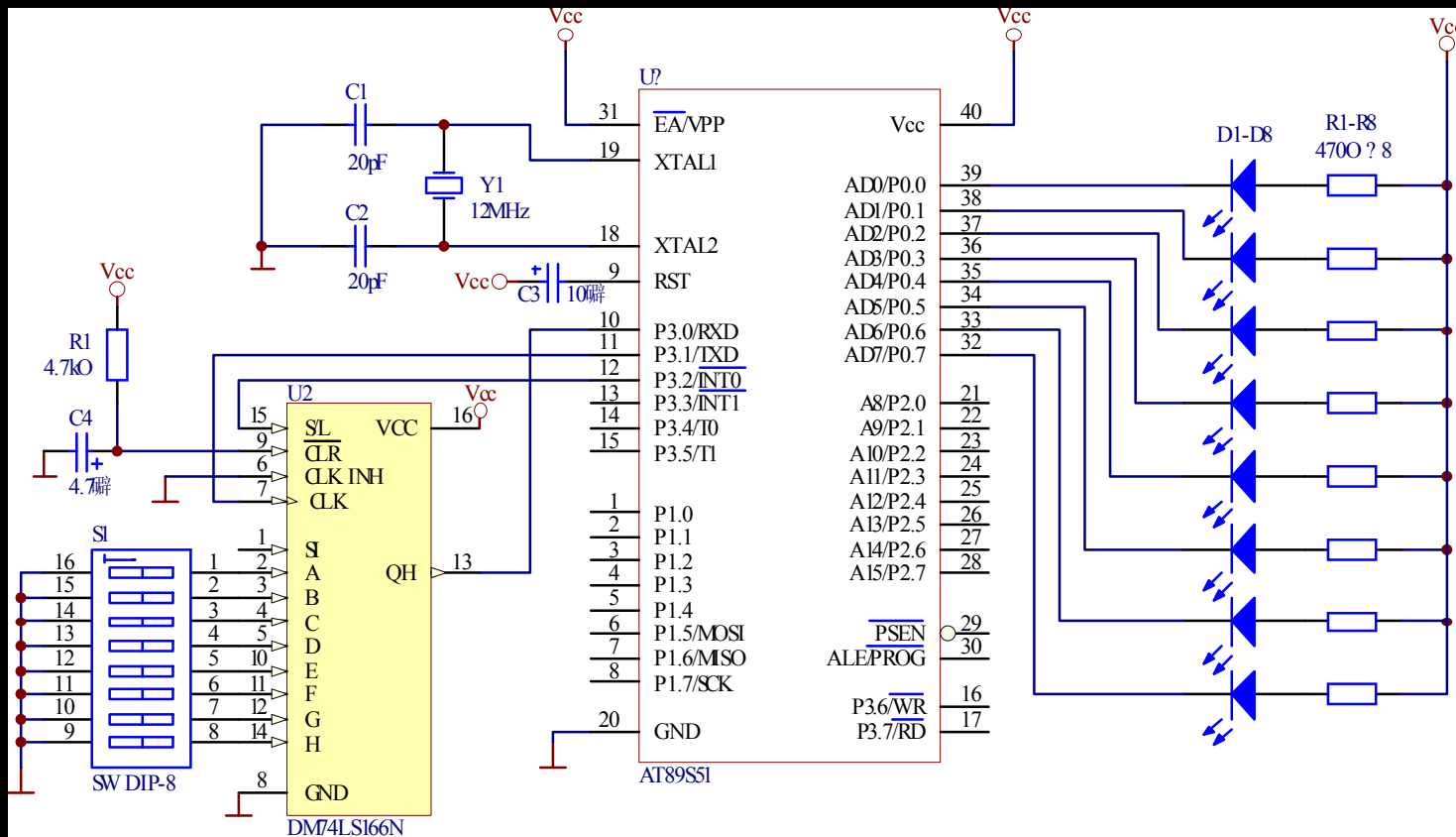
• 10.2.1 串行发送数据



单片机串行口构成的流水灯系统

10.2 单片机串行口如何发送与接收数据

• 10.2.2 串行接收数据



单片机串行口用于接收并行数据系统

10.3 单片机串行口的控制者

• 10.3.1 UART与波特率

- ✓ 在串行数据传输时，分为异步（asynchronous）和同步（synchronous）两种方式。AT89S51单片机的串行口采用的是异步传输方式，每次串行口通信时只发送或接收1个字节的数据。
- ✓ AT89S51单片机集成了一个叫做“通用异步发送-接收器”的功能结构，英文缩写是UART（universal asynchronous receiver-transmitter），我们使用SCON寄存器控制对象就是单片机的UART。
- ✓ 描述数据传输率：“字节/秒”（Bytes pre second），“位/秒”（bits per second），波特（baud）。

10.3 单片机串行口的控制者

• 10.3.2 串行口缓冲区SBUF

- ✓ 串行口缓冲区SBUF，是一个1个字节长度的寄存器，位于特殊功能寄存器区的99H上。
- ✓ 在发送数据时，只要把数据载入SBUF中，UART就会自动地将数据从串行口发送出去。在接收数据时，SBUF寄存器保持着从单片机串行口接收的数据，以供程序读取。

往SBUF中载入数据就自动发送；
接收的数据保存在SBUF中供读取。

10.3 单片机串行口的控制者

• 10.3.3 串行口控制寄存器SCON

串行口控制寄存器SCON是一个1个字节长度的寄存器，位于特殊功能寄存器区的98H，可对其进行位寻址。

7	6	5	4	3	2	1	0
FE/SM0	SM1	SM2	REN	TB8	RB8	TI	RI

FE/SM0 误帧位 / 串行口模式0位。

SM1 串行口模式1位。

SM2 多单片机通信使能位。

REN 接收使能位。

TB8 不常用

RB8 不常用

TI 发送中断标志位。

RI 接收中断标志位。

10.3 单片机串行口的控制者

• 10.3.3 串行口控制寄存器SCON

- ✓ FE/SM0位与SM1位——设置串行口工作模式(见后)。
- ✓ SM2——多单片机通信使能位。该位可以控制使能多个单片机之间的通信，。
- ✓ REN——接收使能位。当REN位置1时，单片机可以从串行口接收数据，如果REN清0，则接收功能被关闭。
- ✓ TI——发送中断标志位。TI与RI一样都是串行通信中重要的标志位。当单片机完成SBUF中的数据发送后，该位由硬件置1。
- ✓ RI——接收中断标志位。粗略的描述是当单片机从串行口接收完数据后该位被硬件置1。

10.4 串行口工作模式及波特率

- 由SCON寄存器中的SM0和SM1位设置单片机串行口工作在4种不同模式下，如表：

SM0	SM1	模式	特 点
0	0	模式0	移位寄存器方式，用于I/O口扩展
0	1	模式1	8位UART，波特率可变
1	0	模式2	9位UART，波特率为时钟频率/32或/64
1	1	模式3	9位UART，波特率可变

10.4 串行口工作模式及波特率

• 10.4.1 模式0

UART实际上是一个同步移位寄存器，该模式下只发送或接收8位数据。注意，此时数据由单片机的RXD管脚发送或接收，而TXD作为发送或接收的移位脉冲。发送或接收数据时，低位数据在前，高位数据在后。

- ① 模式0下发送。将要发送的8位数据载入SBUF，UART自动将SBUF中的数据转换成8位串行数据，并以晶振频率/12作为波特率从RXD发送出去。当数据发送完成后，TI会被置1。
- ② 模式0下接收。在UART接收串行数据之前需要设SCON中的RI=0、REN=1。UART就会启动接收过程，此时RXD为数据接收端，TXD仍然为移位脉冲输出。当1个字节的数据接收完毕后，UART将其载入SBUF，同时RI被置1。

$$\text{波特率} = \frac{\text{晶振频率}}{12} \quad \text{例如晶振频率}=12\text{MHz, 则波特率} = \frac{12 \times 10^6}{12} = 1 \text{ (MHz)}$$

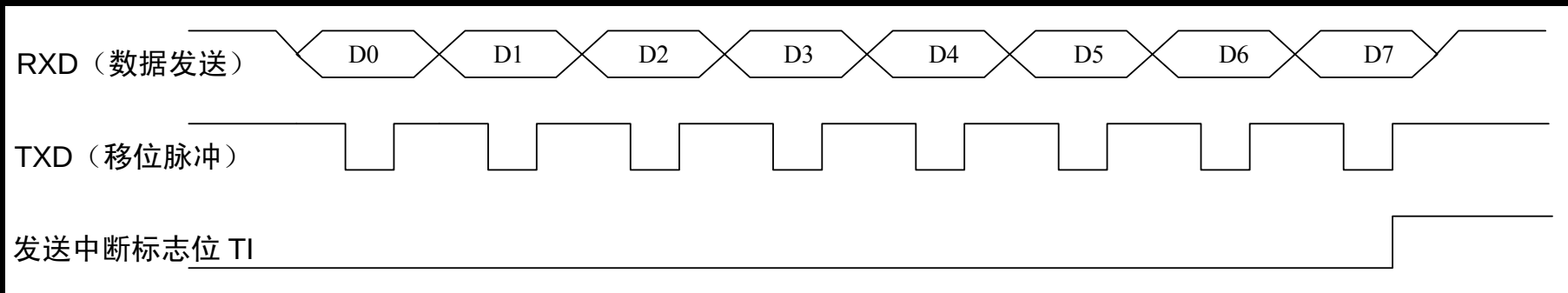
10.4 串行口工作模式及波特率

- 10.4.2 模式1

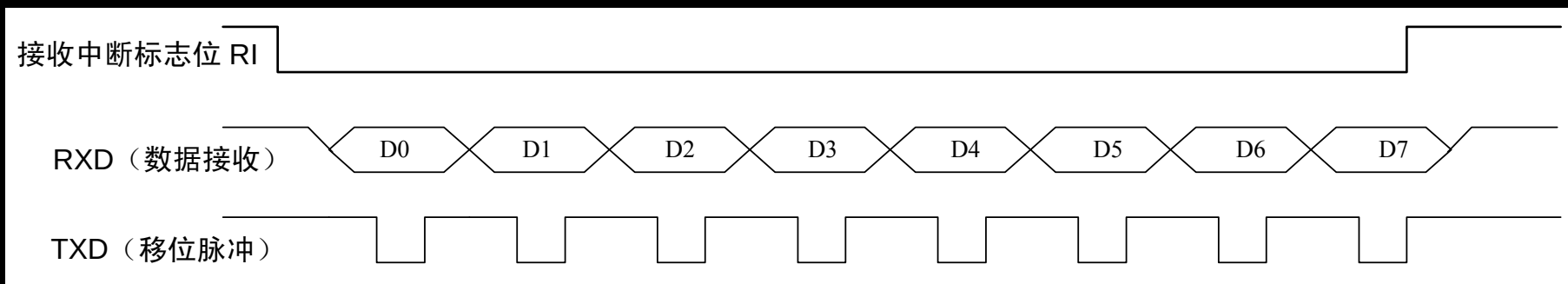
该模式下，UART作为异步通信口，每一帧发送或接收10位数据，这10个位分别是1个起始位“0”、8个数据位和1个停止位“1”。单片机的TXD为发送管脚，RXD为接收管脚。

10.4 串行口工作模式及波特率

• 10.4.2 模式1（模式1下收发时序）



模式1下发送时序



模式1下接收时序

10.4 串行口工作模式及波特率

- 10.4.2 模式1（模式1下的波特率）

该模式下通信的波特率是可变的，一般由Timer 1工作在模式2下，通过载入TH1和TL1的计数初始值来设置波特率。Timer工作在模式2下，是一个8位自动重新装载的定时器，装载时需要向TH1、TL1同时装载相同的计数初始值。单片机会自动根据Timer 1的设置情况使UART工作在特定的波特率下。

10.4 串行口工作模式及波特率

• 10.4.2 模式1（模式1下的波特率）

模式1下波特率和TH1（TL1）中载入计数初始值之间的关系如公式所示：

$$\text{波特率} = \frac{2^{\text{SMOD1}}}{32} \times \frac{\text{晶振频率}}{12 \times [256 - (\text{TH1})]}$$

SMOD1为PCON.7 {
 =0为单倍波特率
 =1为双倍波特率

TH1是Timer1寄存器。

7	6	5	4	3	2	1	0
SMOD1	SMOD0	保留	POF	GF1	GF0	PD	IDL

SMOD1在PCON中的位

10.4 串行口工作模式及波特率

• 10.4.2 模式1（模式1下的波特率实例）

例：使用单倍波特率，SMOD1=0，晶振频率=11.0592MHz，向TH1中载入F3H，即TH1=243，则波特率为：

$$\text{波特率} = \frac{2^{\text{SMOD1}}}{32} \times \frac{\text{晶振频率}}{12 \times [256 - (\text{TH1})]} = \frac{2^0}{32} \times \frac{11.0592 \times 10^6}{12 \times (256 - 243)} = 2400$$

① UART以模式1发送时，数据由TXD端送出。向SBUF载入数据后即自动发送。发送完一帧数据后，发送中断标志位TI=1，可用“JBC TI, CHECK”来检测TI位并将其清除为0。

② UART接收串行数据之前需将SCON中的REN位置1，UART就会启动接收过程。RXD为数据接收端。接收完1帧数据后，RI位被置1，可用“JB RI, CHECK”来检测RI位以判断接收完成情况。

10.4 串行口工作模式及波特率

- 10.4.2 模式1（模式1和模式3下波特率与TH1（=TL1）的计数初始值）

波特率	晶振频率 (MHz)	SMOD1 位	Timer 1工作在模式2下载入 TH1（=TL1）的计数初始值
600	12	0	0CCH
1200	12	1	0F6H
2400	12	0	0F3H
4800	12	1	0F3H
1200	11.0592	0	0E8H
2400	11.0592	0	0F4H
4800	11.0592	0	0FAH
9600	11.0592	0	0FDH
19200	11.0592	1	0FDH

10.4 串行口工作模式及波特率

• 10.4.3 模式2

该模式下UART为一个9位异步通信口，每一帧共发送或接收11位数据。这11位数据由1个起始位“0”、8个数据、第9位数据（TB8位，位于SCON内）和1个停止位“1”组成。模式2下的波特率为晶振频率的1/32或1/64，这取决于PCON中的SMOD1的设置，计算公式为：

$$\text{波特率} = \frac{2^{\text{SMOD1}}}{64} \times \text{晶振频率}$$

10.4 串行口工作模式及波特率

• 10.4.3 模式2

① UART以模式2发送时，数据由TXD端送出，数据中的第9位数据从SCON中的TB8位取得（可用指令“MOV TB8, bit”向TB8位载入数据）。向SBUF载入数据后即自动发送。发送完一帧数据后，发送中断标志位TI=1，可用指令“JBC TI, CHECK”来检测TI并将其清除为0。

② 在UART接收串行数据之前需要设SCON中的REN=1。UART就会启动接收过程。RXD为数据接收端，数据中的第9位载入SCON中的RB8位上。接收完1帧数据后，RI位被置1，可用指令“JB RI, CHECK”来检测RI位以判断接收完成情况。

10.4 串行口工作模式及波特率

• 10.4.4 模式3

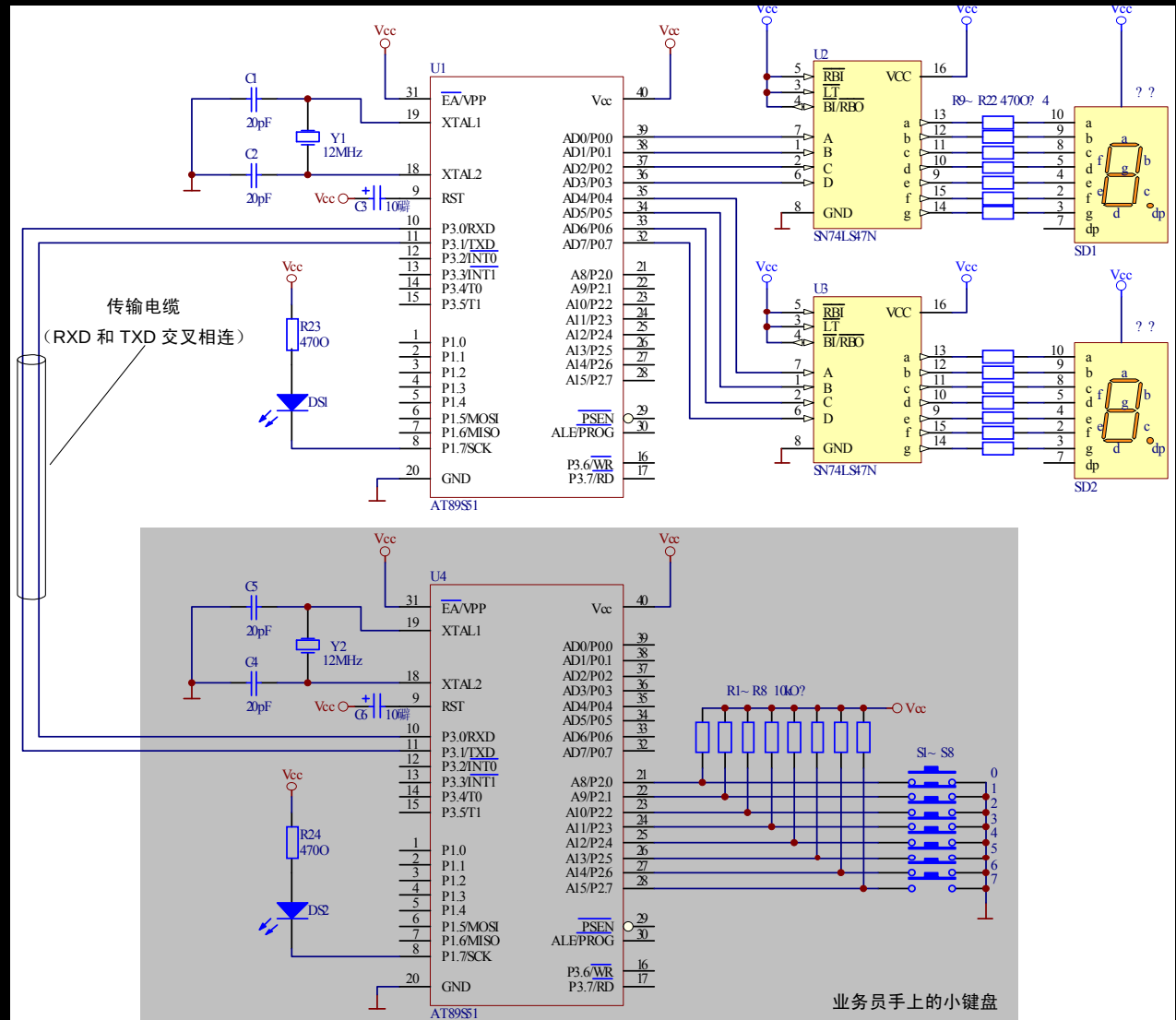
该模式与模式2的发送/接收过程和UART功能几乎完全一样，只是模式3的波特率可变，与模式1具有相同的计算公式，在实际应用中，可以参考下表来选择晶振和设置计数初始值。

波特率	晶振频率 (MHz)	SMOD1位	Timer 1工作在模式2下载入TH1 (=TL1) 的计数 初始值
600	12	0	0CCH
1200	12	1	0F6H
2400	12	0	0F3H
4800	12	1	0F3H
1200	11.0592	0	0E8H
2400	11.0592	0	0F4H
4800	11.0592	0	0FAH
9600	11.0592	0	0FDH
19200	11.0592	1	0FDH

10.5 单片机之间的通信

• 10.5.1 排队 呼叫器 ——两个单片机 之间的通信

排队呼叫器 系统电路



10.5 单片机之间的通信

- 10.5.1 排队呼叫器——两个单片机之间的通信
 - ✓ 两个单片机的RXD和TXD是交叉连接的。
 - ✓ 两个单片机的串行口直接相连时有一个问题，就是它们之间的传输电缆不能太长（应当在几米范围内），否则在传输过程中会因干扰和信号的损耗而产生误码。
 - ✓ 确定通信协议：

单片机U1和U4之间以2400的波特率进行串行口通信。

U4首先向U1发起通信请求，发送呼叫号99H。

U1接收到呼叫号后，向U4回送应答号01H。

通信建立，开始数据传输。

10.5 单片机之间的通信

- 两个单片机之间的通信程序

U4向U1发送显示数据 (1/2)

ORG 00H

START:

MOV SCON,#50H; ①设置串行口工作在模式1下，接收使能

MOV TMOD, #20H ; ②使用Timer 1工作在模式2下

MOV TH1, #0F3H ; ③波特率2400

MOV TL1, #0F3H

SETB TR1 ; ④启动Timer 1

DIAL:

MOV SBUF, #99H ; ⑤发送呼叫号99H

CHECK_99:

JBC TI, WAIT_RESPONSE ; ⑥判断是否发送完毕，完毕跳
转到接收应答

JMP CHECK_99

10.5 单片机之间的通信

- 两个单片机之间的通信程序

U4向U1发送显示数据 (2/2)

WAIT_RESPONSE:

JBC RI, CHECK_01; 串行口接收到数据则跳转到判断应答号

JMP WAIT_RESPONSE

CHECK_01:

MOV A, SBUF ; 将接收到的数据从SBUF载入ACC

CJNE A, #01H, DIAL ; 判断接收到的数据是否为应答号01H,
; 如果不是正确应答号, 就跳到DIAL重呼

SEND_NUM:

MOV A, 30H ; 将30H中的键盘按键值载入ACC

MOV SBUF, A ; 发送键盘按键值

CHECK_SEND:

JBC TI, FINISH ; 判断是否发送完毕

JMP CHECK_SEND

FINISH:.....

END

10.5 单片机之间的通信

- 两个单片机之间的通信程序

U1接收U4的显示数据 (1/2)

ORG 00H

START:

MOV SCON, #50H ; ①设置串行口工作在模式1下, 接收使能

MOV TMOD, #20H ; ②使用Timer 1工作在模式2下

MOV TH1, #0F3H ; ③波特率2400

MOV TL1, #0F3H

SETB TR1 ; ④启动Timer 1

WAIT_DIAL:

JBC RI, CHECK_99 ; ⑤判断是否接收到呼叫

JMP WAIT_DIAL

CHECK_99:

MOV A, SBUF ; ⑥接收到的数据载入ACC

CJNE A, #99H, WAIT_DIAL ; 判断是否为呼叫号99H

10.5 单片机之间的通信

- 两个单片机之间的通信程序

U1接收U4的显示数据 (2/2)

SEND_RESPONSE:

MOV SBUF, #01H ; 发送应答号01H

CHECK_01:

JBC TI, RECEIVE_NUM ; 判断应答号是否发送完毕

JMP CHECK_01

RECEIVE_NUM:

JBC RI, DISPLAY ; 等待接收显示数据

JMP RECEIVE_NUM

DISPLAY:

MOV A, SBUF ; 将接收到的显示数据载入ACC中

MOV P0, A ; 输出至P0口显示

CALL DELAY ; 延时

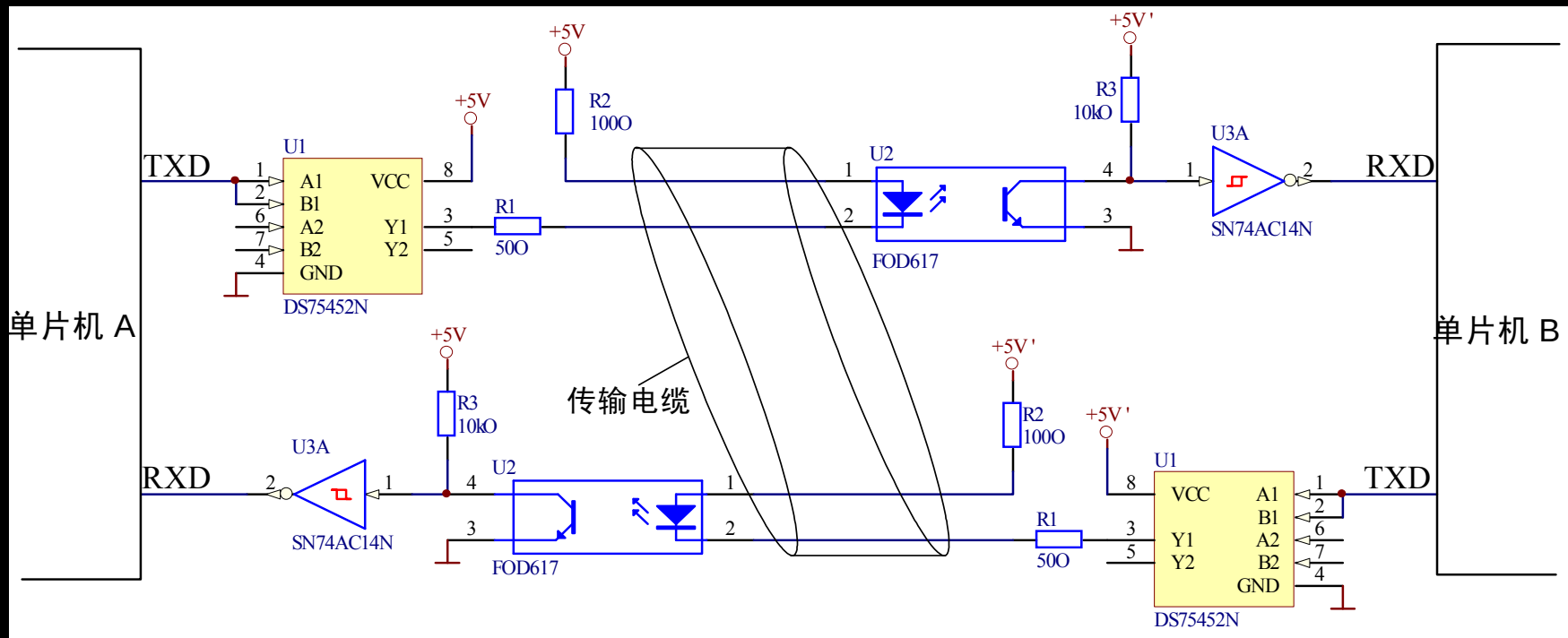
JMP START ; 循环

END

10.5 单片机之间的通信

• 10.5.2 远距离通信的解决方案

单片机串行口的电平比较低（5V左右），两个单片机的TXD和RXD对绞相连不适合直接进行远距离通信。解决的办法是提高信号的驱动能力后再由电缆进行传送。

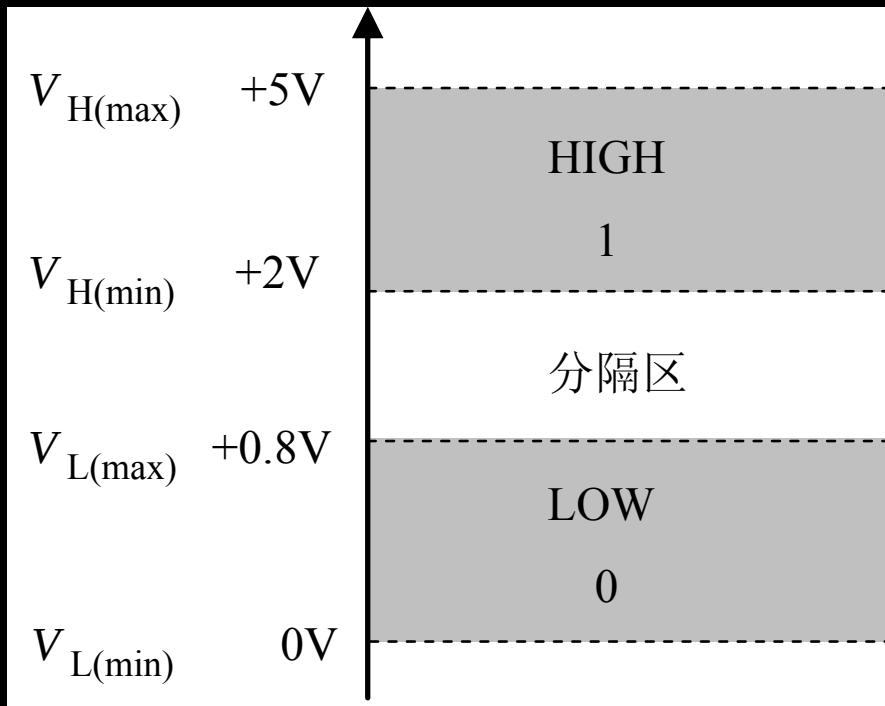


10.6 计算机的串行口与控制

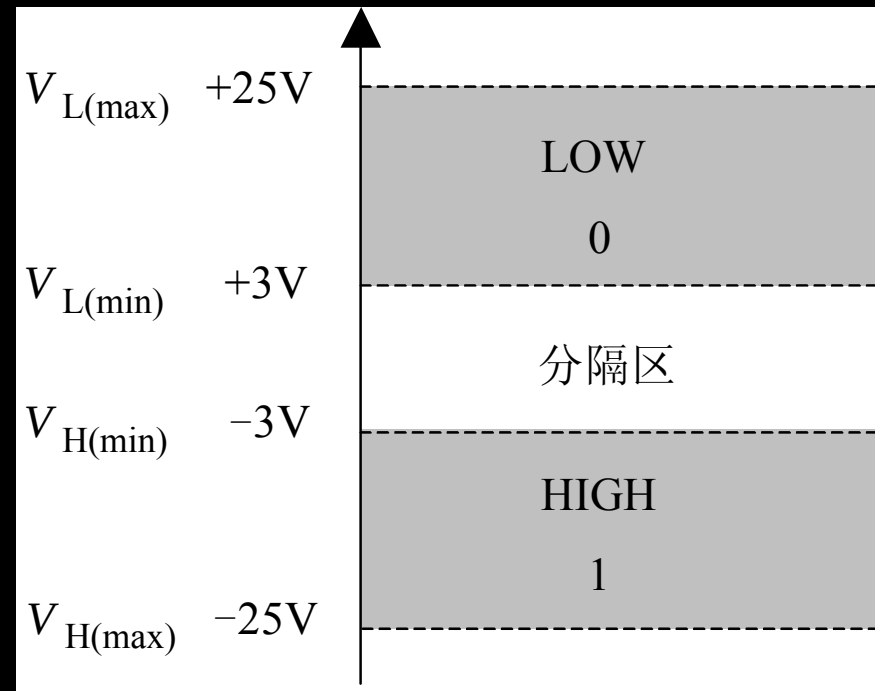
- 10.6.1 从RS-232标准到计算机串行口
- ✓ RS-232中RS是“RETMA Standard”的首字母缩写，RETMA（Radio-Electronics and Television Manufactures Association，无线电、电子、电视设备制造商协会）是RS-232标准的发布者。现在RETMA已经发展成为EIA（Electronic Industries Alliance，电子工业联合会）。所以RS-232标准与EIA-232标准是相同的。
- ✓ 由于RS-232标准早在1960年就已经被制订，而那时TTL逻辑电路还没有降生，所以它规定的逻辑电平比较奇怪，与今天的TTL电平不兼容。

10.6 计算机的串行口与控制

• 10.6.1 从RS-232标准到计算机串行口



(a) TTL逻辑电平

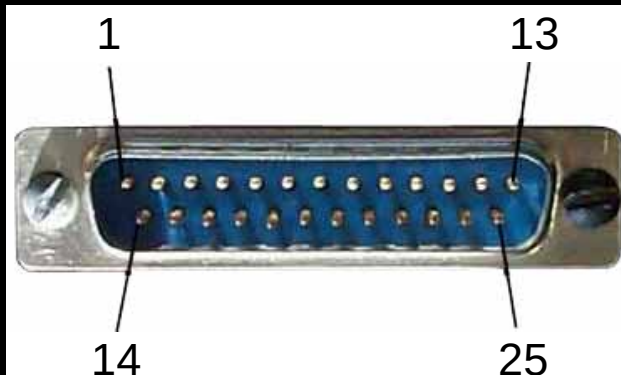


(b) RS-232逻辑电平

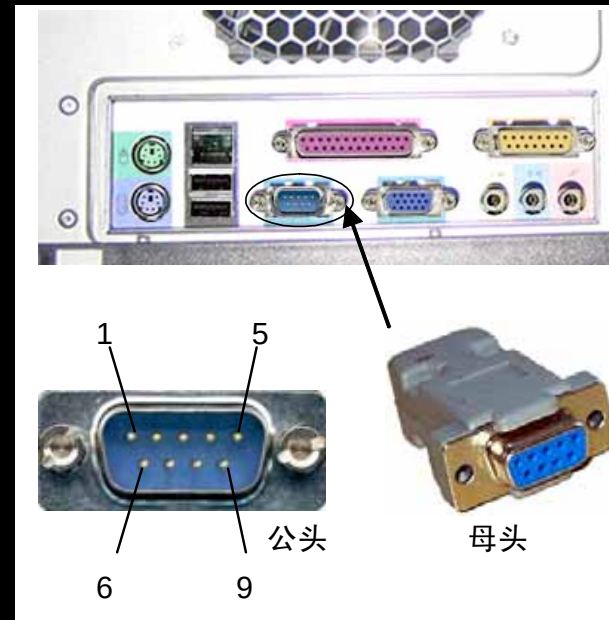
10.6 计算机的串行口与控制

• 10.6.1 从RS-232标准到计算机串行口

RS-232确定的是一种在数据终端设备（DTE）与数据运载设备（DCE）之间进行串行二进制数据交换的标准，它一定涉及接口器件的尺寸、规格等问题。



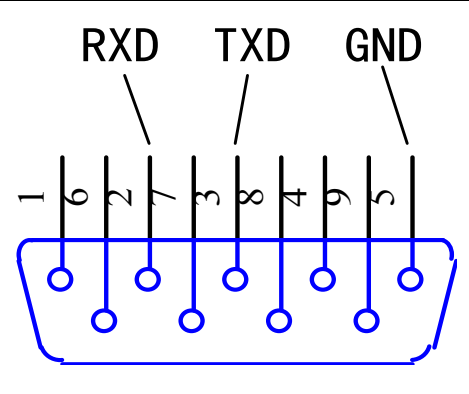
标准25芯DB-25接头



DB-9接头

10.6 计算机的串行口与控制

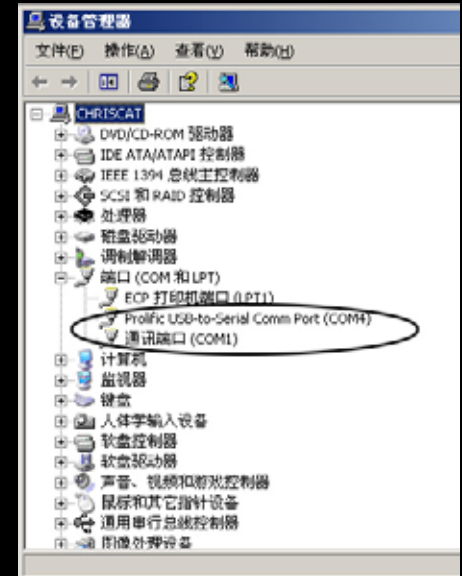
• 10.6.1 从RS-232标准到计算机串行口

串口示意图	序 号	名称及功能说明
	1	DCD: 载波检测位
	2	RXD: 接收数据位
	3	TXD: 发送数据位
	4	DTR: 数据终端准备信号位
	5	GND: 接地位
	6	DSR: 数据发送准备信号位
	7	RTS: 请求发送位
	8	CTS: 等待发送位
	9	RI: 响铃位

RS-232接口数据线说明

10.6 计算机的串行口与控制

• 10.6.1 从RS-232标准到计算机串行口(串口转换器)

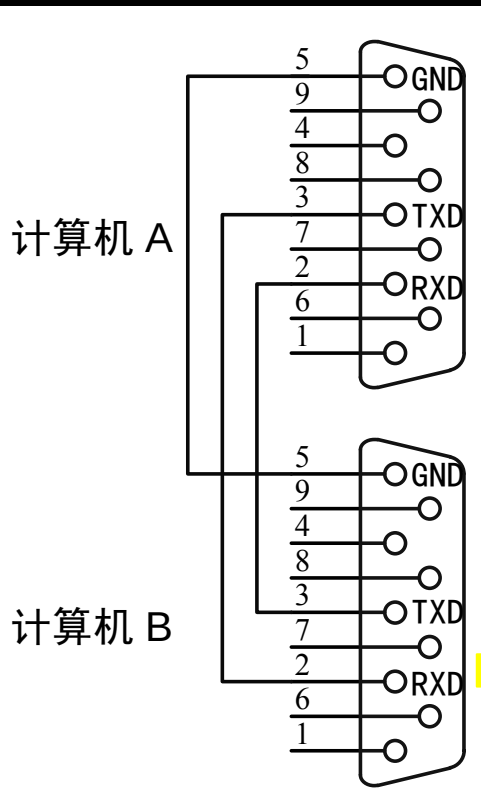


(a) USB转串口型 (b) PCMCIA转串口型

插上串口转换器并安装完成对应的驱动后，会在Windows的设备管理器中找到相应的硬件。“通信端口（COM1）”是计算机本身的串口，而名为“Prolific USB-to-Serial Comm Port（COM4）”才是USB转换器生成的串口。

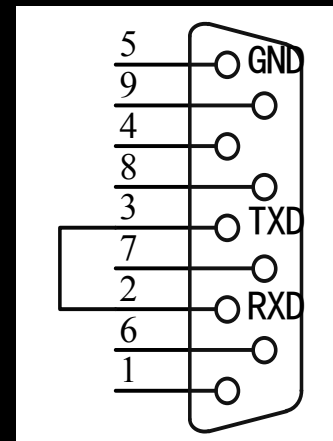
10.6 计算机的串行口与控制

• 10.6.1 从RS-232标准到计算机串行口(硬件连接)



(a) 两机通信

计算机A (GND) 与B (GND) 相连, 计算机A (RXD) 与B (TXD) 相连, 计算机A (TXD) 与B (RXD) 相连。



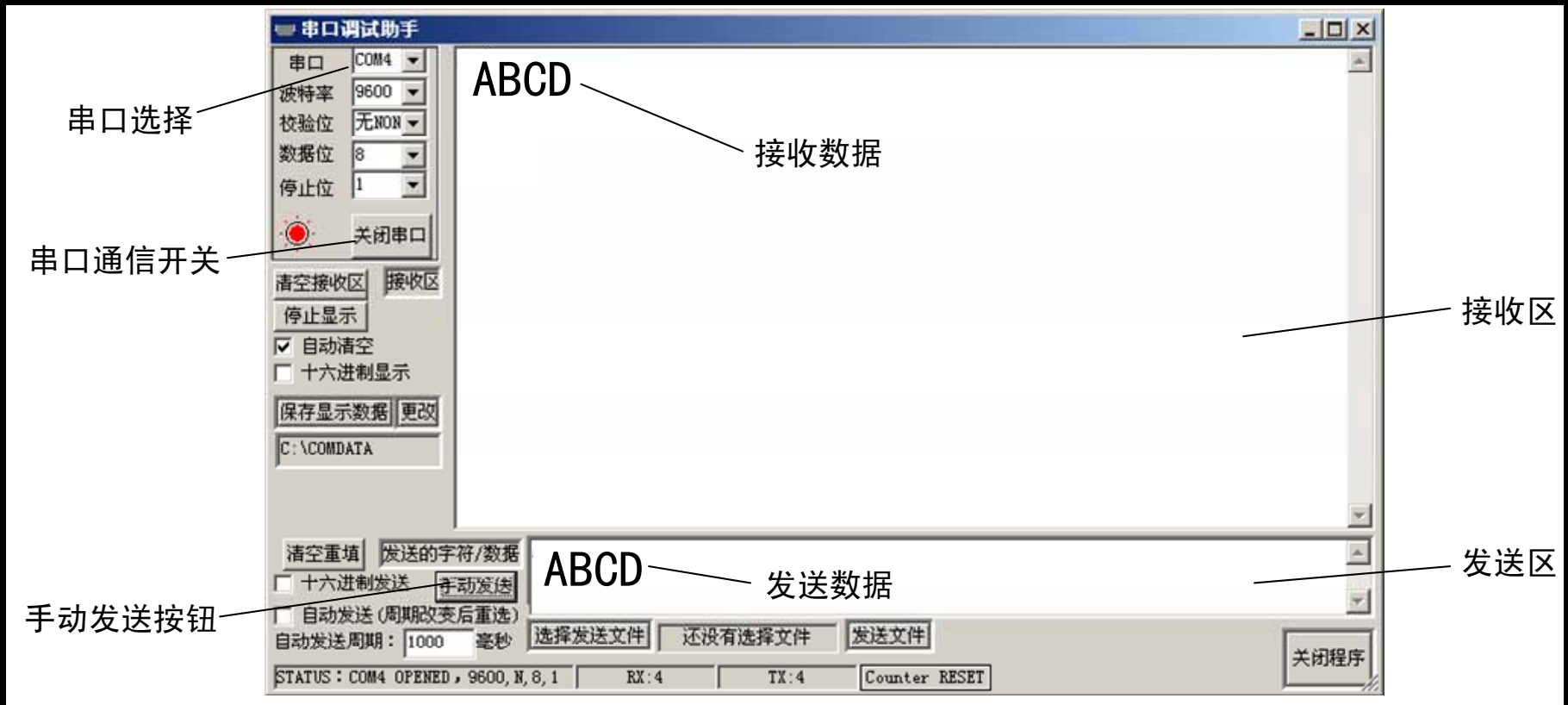
(b) 单机通信

计算机串口 TXD 与该口上 RXD 相连, 实现了本机发送的数据由本机接收的效果。

串口通信实验的硬件连接

10.6 计算机的串行口与控制

• 10.6.3 计算机串口通信实验——软件操作



注:如果接收不到数据或数据与发送的不一样,很有可能是串口号设置不正确,或串口的TXD和RXD没有连接上等问题造成的,或者试着把波特率调低。

10.6 计算机的串行口与控制

• 10.6.4 用Visual Basic编写一个串口通信程序

① “新建工程”对话框

新建一个“标准
EXE”工程

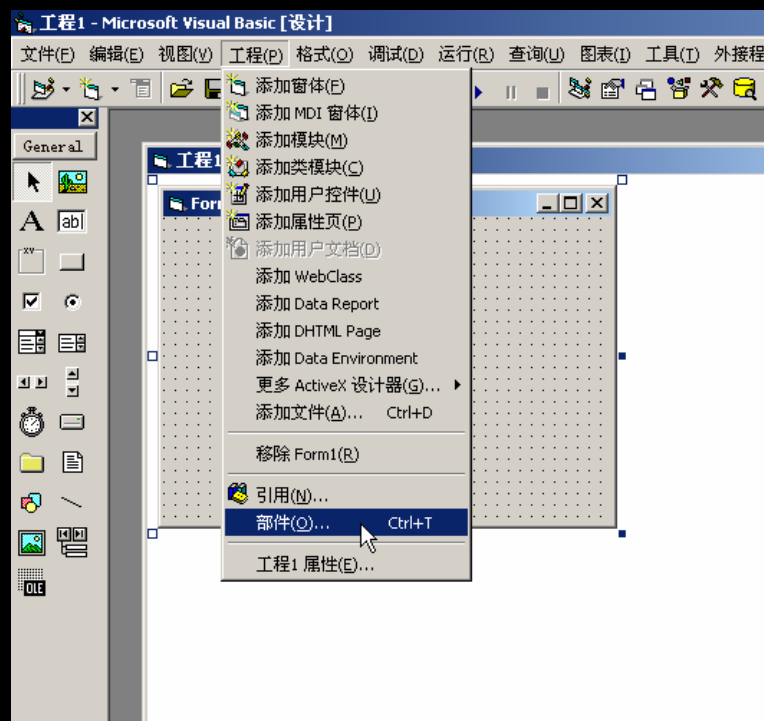


Open（打开）按钮

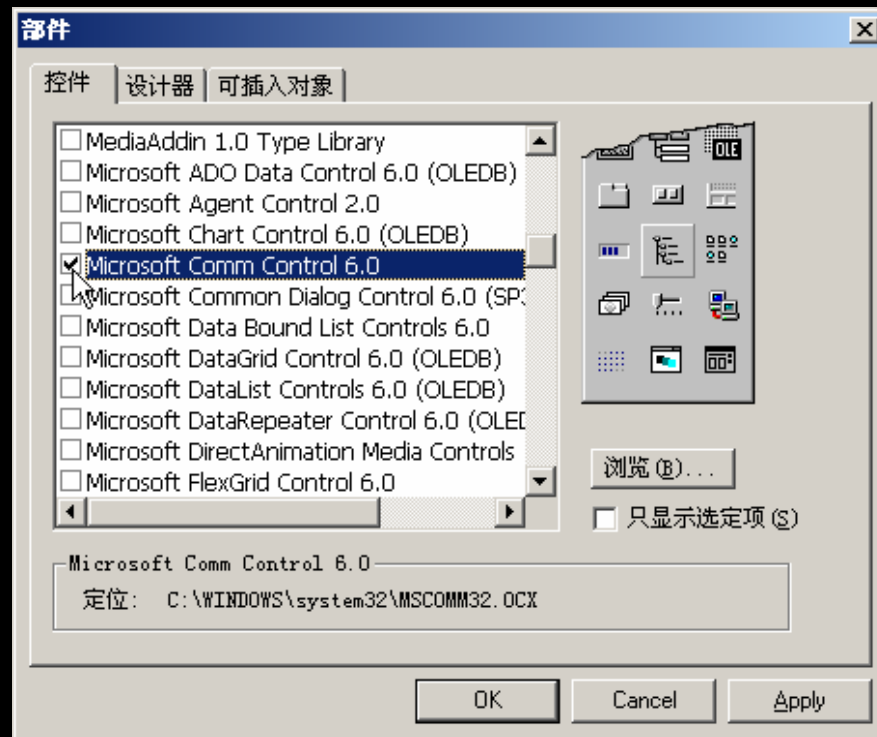
10.6 计算机的串行口与控制

• 10.6.4 用Visual Basic编写一个串口通信程序

② “部件”命令

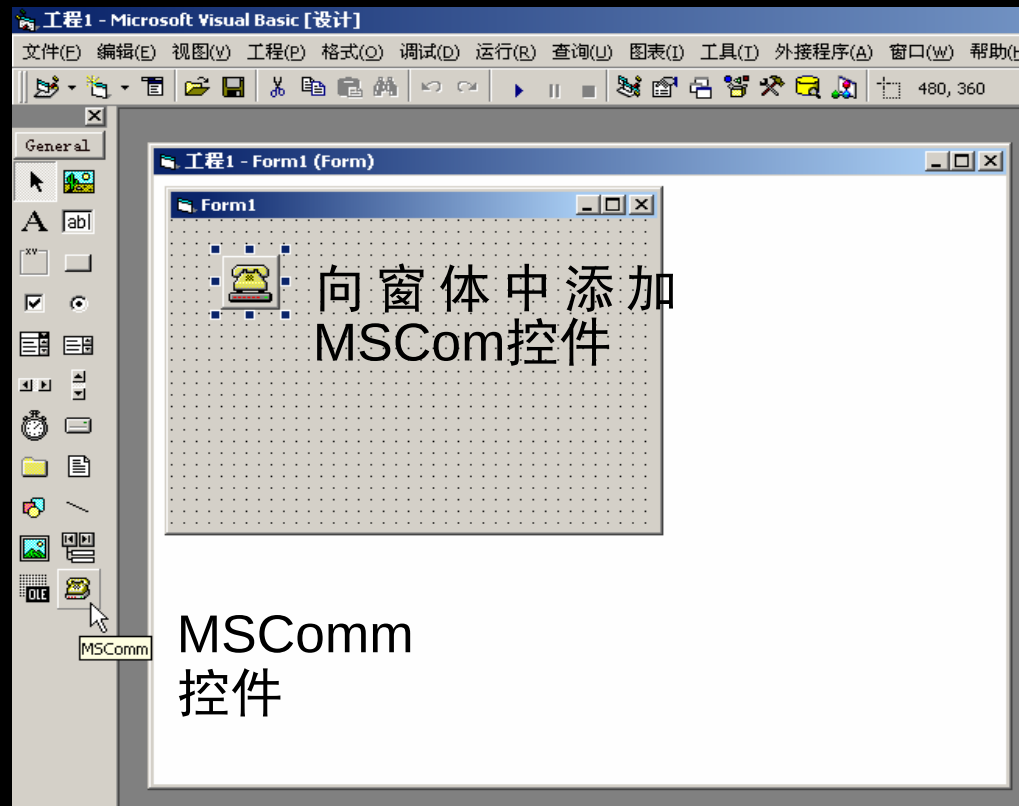


③ “部件”对话框



10.6 计算机的串行口与控制

- 10.6.4 用Visual Basic编写一个串口通信程序
- ④ MSComm控件及向窗体中添加



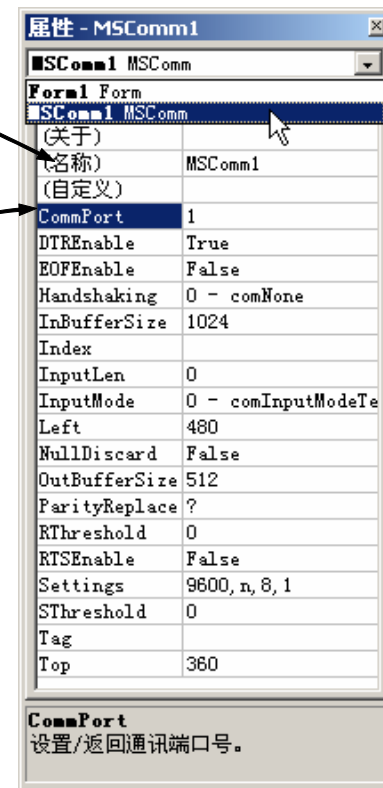
10.6 计算机的串行口与控制

• 10.6.4 用Visual Basic编写一个串口通信程序

⑤ MSComm1属性编辑栏

(名称): 该属性设置窗体中 MSComm 控件的名称, 在窗体代码中将使用这个名称来指向 MSComm 控件。

CommPort (串口号设置): CommPort 属性用于设置或返回通信连接串口号。程序在初始化时必须指定所要使用的串口号, 以便在程序运行期间对特定串口进行读写操作。同时, 程序也可借助此属性返回所使用的连接串口号。注意, 最大的串口号不能超过 16, 确定使用的串口号与图 10-21 中设备管理器中的显示串口号有关。例如使用 COM1, 就在 CommPort 属性中填写 “1”。



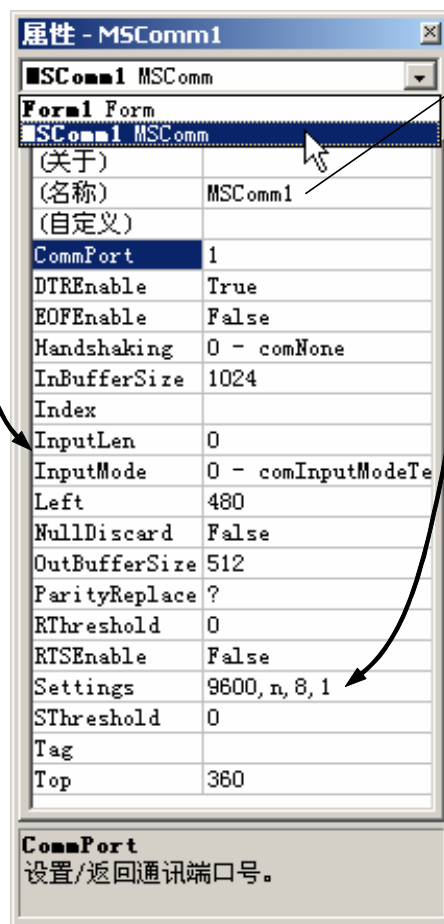
10.6 计算机的串行口与控制

• 10.6.4 用Visual Basic编写一个串口通信程序

⑤ MSComm1属性编辑栏

InputLen（接收字符数）：

InputLen 属性用于指定由串口读入的字符串长度或字节数，在需要指定读入输入缓冲区的字符长度时设置该属性。例如：InputLen=10，则当串口从RXD接收数据时，只会读取10个字符。如果在输入缓冲区中有55个字符，而设置InputLen=10，由于每一次的Input命令只能读10个字符，所以需要执行6次Input才能把缓冲区的数据读入完。

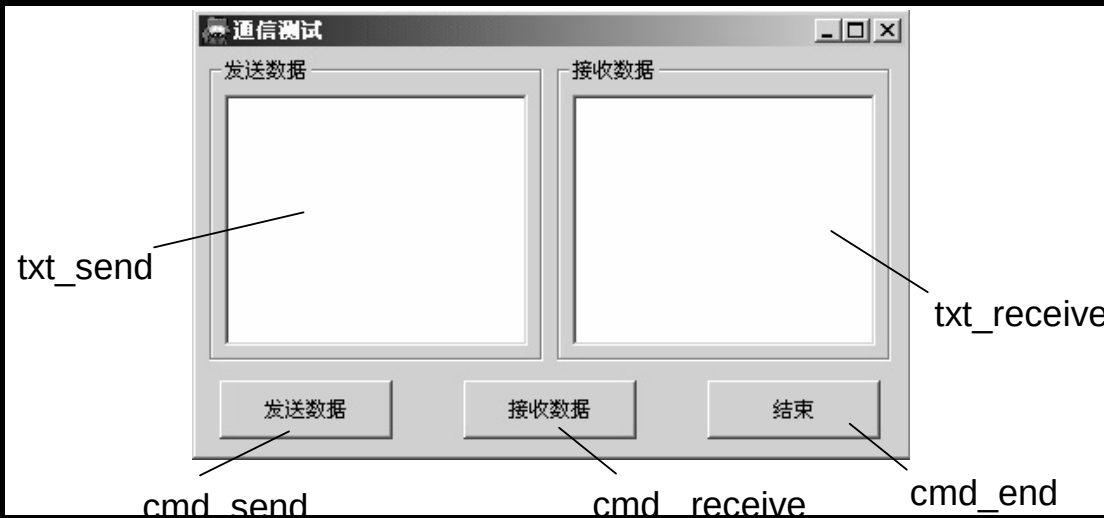


MSComm1 控件

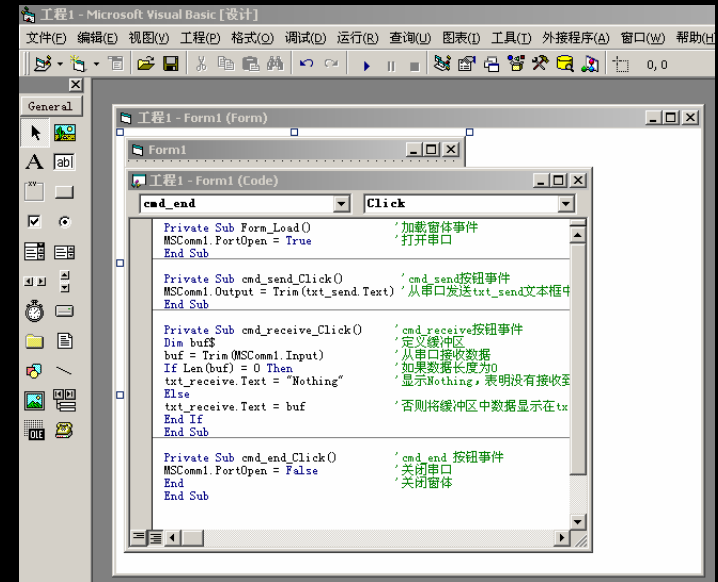
Settings（串口控制设置）：Settings 属性用于设置串口的初始化参数。默认值是“9600, n, 8, 1”。这个由逗号分隔的参数分别代表了波特率、奇偶校验位检查方式、数据位数和停止位数。波特率可选值包括 110、300、600、1200、2400、9600、14400、19200、28800 及 38400 等几种。在设置时需要注意与单片机 UART 设置的波特率一致（表 10-2 所示），否则无法正常接收和发送数据。Settings 默认值中的“n”是奇偶校验位，可以根据表 10-4 进行设置。默认值的“8”代表数据位数，可选择的值有 4、5、6、7 和 8。最后一位“1”是停止位数，可选择的值有 1、1.5 和 2。一般来说，这些值只需要修改波特率与单片机 UART 波特率相同即可正常工作。

10.6 计算机的串行口与控制

- 10.6.4 用Visual Basic编写一个串口通信程序
- ⑥ 串口通信实验窗体编辑与窗体代码编辑器



串口通信实验窗体编辑



窗体代码编辑器

10.6 计算机的串行口与控制

• 10.6.4 用Visual Basic编写一个串口通信程序

⑦ 控制串口的Visual Basic程序解析:

```
Private Sub Form_Load()      ;加载窗体事件
MSComm1.PortOpen=True      ;打开串口
End Sub
```

```
Private Sub cmd_send_Click() ;cmd_send按钮事件
MSComm1.Output=Trim(txt_send.Text)
                        ;从串口发送txt_send文本框中的数据
End Sub
```

```
Private Sub cmd_end_Click() ;cmd_end按钮事件
MSComm1.PortOpen=False    ;关闭串口
End                        ;关闭窗体
End Sub
```

10.6 计算机的串行口与控制

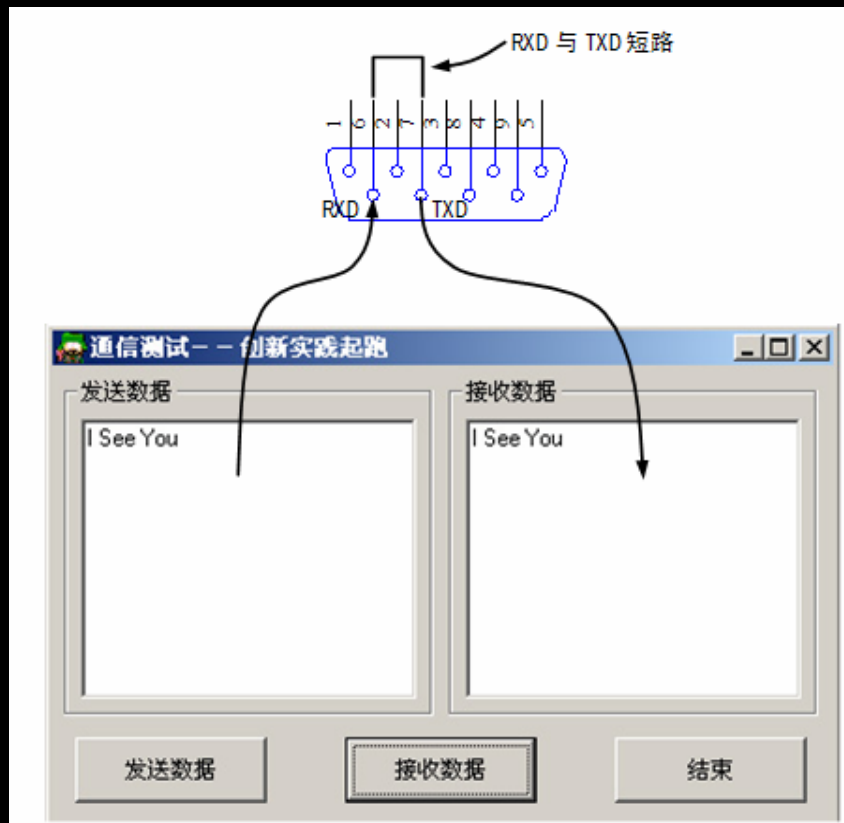
- 10.6.4 用Visual Basic编写一个串口通信程序

⑦ 控制串口的Visual Basic程序解析:

```
Private Sub cmd_receive_Click()      ;cmd_receive按钮事件
Dim buf$                            ;定义缓冲区
buf=Trim(MSComm1.Input)             ;从串口接收数据
If Len(buf)=0 Then                  ;如果数据长度为0
txt_receive.Text="Nothing"          ;显示Nothing，表明没有接收到任何数据
Else
txt_receive.Text=buf;否则将缓冲区中数据显示在txt_receive文本框中
End If
End Sub
```

10.6 计算机的串行口与控制

- 10.6.4 用Visual Basic编写一个串口通信程序
- ⑧ 单击按钮编译代码并运行

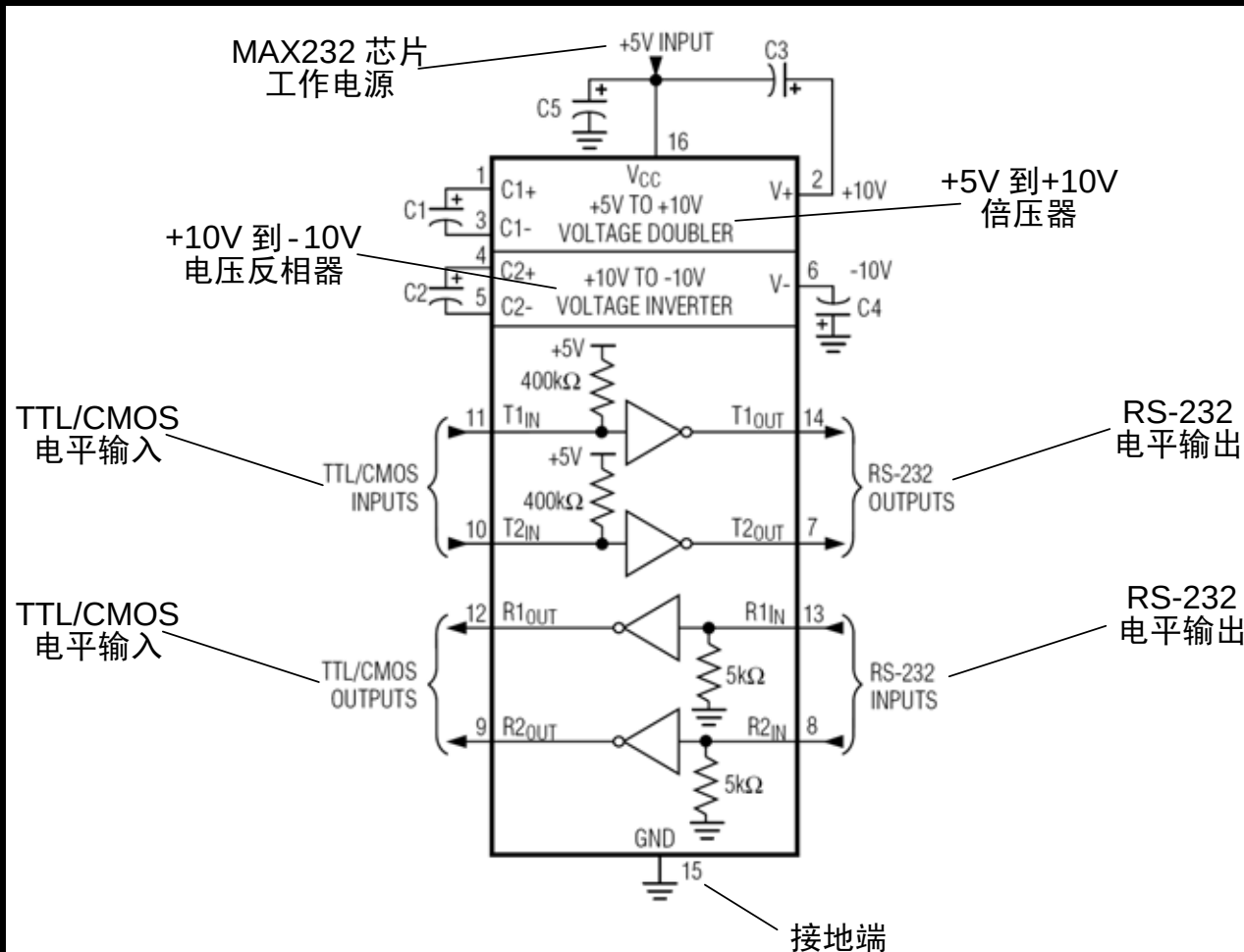


10.7 单片机与计算机之间的通信

- 10.7.1 单片机与计算机通信中的接口芯片
 - ✓ 计算机串口执行的是RS-232标准，高、低电平分别为-3~-25V和+3~+25V，与单片机的TTL逻辑中的+2~+5V和0~+0.8V不兼容。
 - ✓ 较为常用的接口芯片是MAX232。它能很好地完成TTL与RS-232的逻辑电平转换，工作电压与AT89S51单片机相同均为+5V，可以使用同一个电源。

10.7 单片机与计算机之间的通信

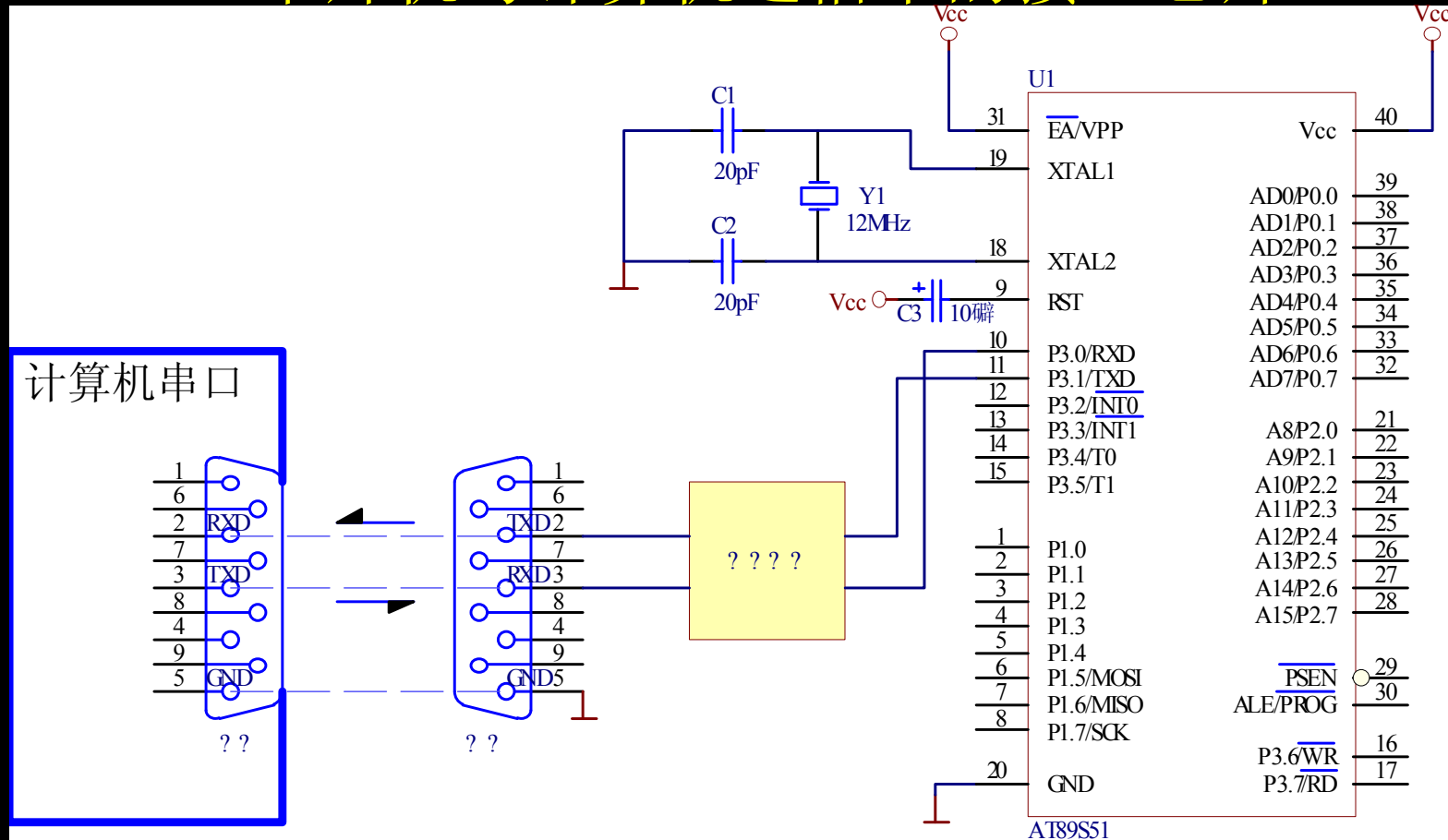
• 10.7.1 单片机与计算机通信中的接口芯片



MAX232集
成了两组电
平转换器。

10.7 单片机与计算机之间的通信

• 10.7.1 单片机与计算机通信中的接口芯片

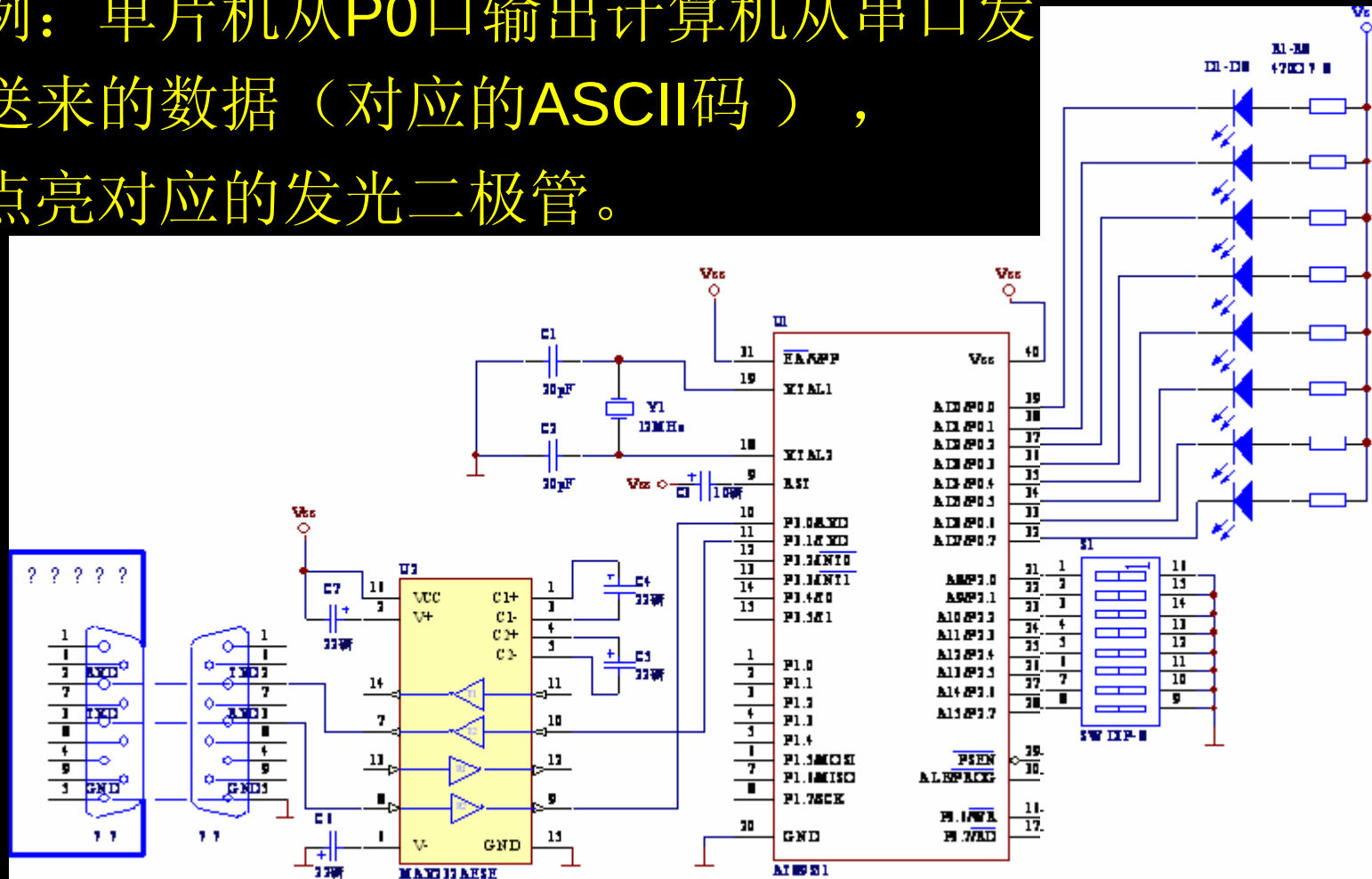


计算机与单片机串行口的接口

10.7 单片机与计算机之间的通信

• 10.7.2 单片机与计算机的串行通信

例：单片机从P0口输出计算机从串口发送来的数据（对应的ASCII码），
点亮对应的发光二极管。



10.7 单片机与计算机之间的通信

• 10.7.2 单片机与计算机的串行通信（软件）

单片机接收计算机串口发送的数据：

```

                ORG          00H
START:
    MOV  SCON,#50H ; ①设置串行口工作在模式1下，接收使能
    MOV  TMOD,#20H ; ②使用Timer 1工作在模式2下
    MOV          TH1, #0F3H ; ③波特率2400
    MOV          TL1, #0F3H
    SETB  TR1      ; ④启动Timer 1
WAIT:
    JBC  RI, DISPLAY ; ⑤判断是否接收到数据
    JMP          WAIT
DISPLAY:
    MOV          A, SBUF ; ⑥接收到的数据载入ACC
    MOV          P0, A   ; 将接收到的数据输出显示
    JMP          WAIT    ; 循环
END

```

10.7 单片机与计算机之间的通信

• 10.7.2 单片机与计算机的串行通信（软件）

单片机通过串口向计算机发送数据：

ORG 00H

START:

MOV SCON, #50H; ①设置串行口工作在模式1下，接收使

MOV TMOD, #20H; ②使用Timer 1工作在模式2下

MOV TH1, #0F3H ; ③波特率2400

MOV TL1, #0F3H

SETB TR1 ; ④启动Timer 1

SEND:

MOV A, P2 ; 把DIP开关的状态载入ACC中

MOV P0, A ; 显示DIP开关的状态

MOV SBUF, A ; ⑤发送数据

WAIT:

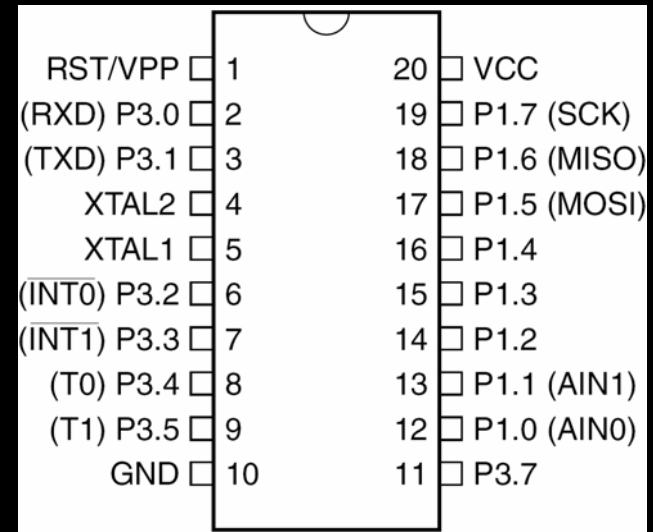
JBC TI, SEND ; ⑥判断是否发送完毕

JMP WAIT

END

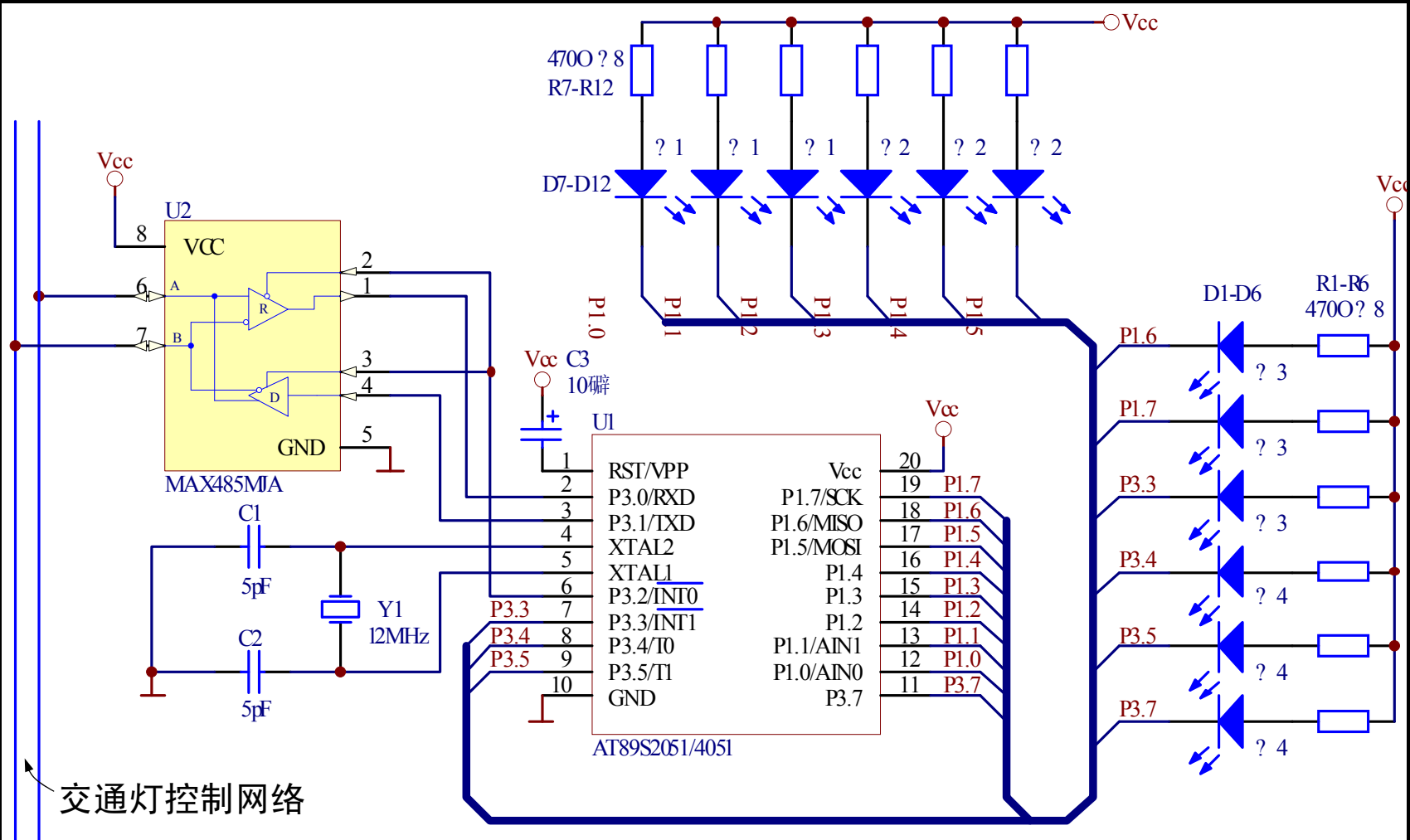
10.8 实例点拨——智能交通灯

- AT89S2051单片机简介：
 - ✓ 与51单片机指令、逻辑电平兼容。
 - ✓ 2K Bytes的可在线编程（ISP）Flash存储器（AT89S4051为4K字节）。
 - ✓ 256×8位片内RAM。
 - ✓ *可以在+2.7~+5.5V范围内工作。
 - ✓ 15个可编程的I/O口。
 - ✓ 两个16位定时/计数器。
 - ✓ 6个中断源。
 - ✓ 可编程串行口UART。
 - ✓ *具有一个可产生中断的模拟信号比较器。
 - ✓ *8位脉冲宽度调制（PWM）。
 - ✓ 低功耗的待机模式和休眠。
 - ✓ *电源电压走低时复位（brown out reset）。
 - ✓ *增强了串行口的错误帧检测及自动地址识别。



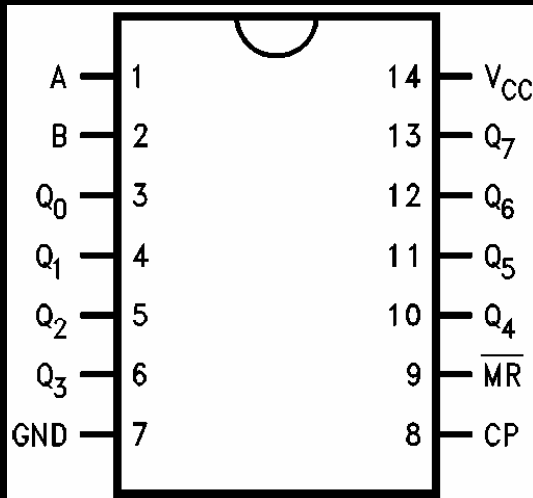
10.8 实例点拨——智能交通灯

- 基于AT89S2051单片机的交通灯

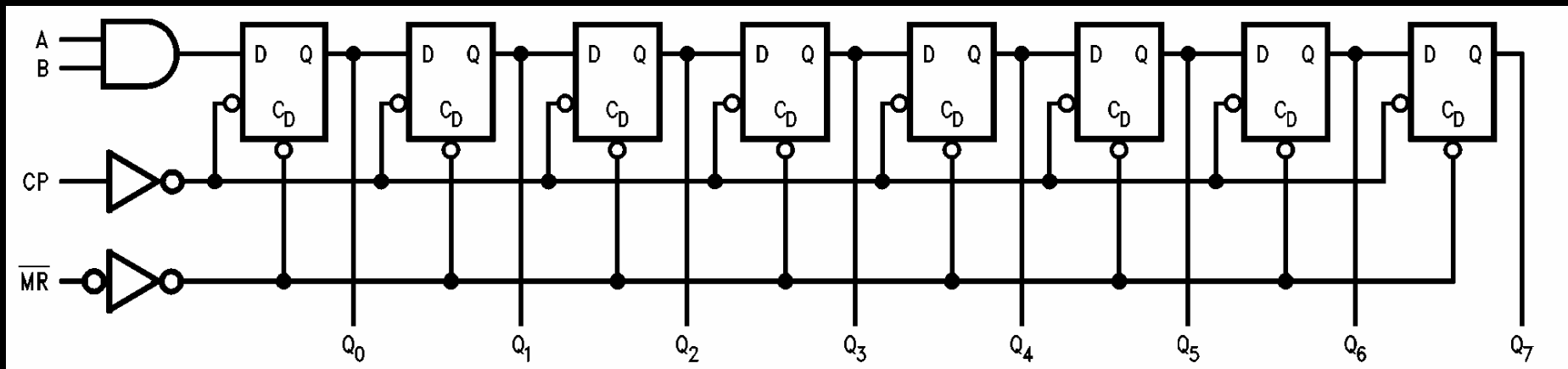


器件介绍

1. 8位串入/并出移位寄存器74164

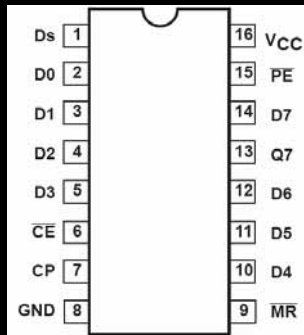


操作模式	输 入			输 出	
	$\overline{\text{MR}}$	A	B	Q_0	Q_1-Q_7
清0	L	X	X	L	L-L
移位	H	L	L	L	Q_0-Q_6
	H	L	H	L	Q_0-Q_6
	H	H	L	L	Q_0-Q_6
	H	H	H	H	Q_0-Q_6

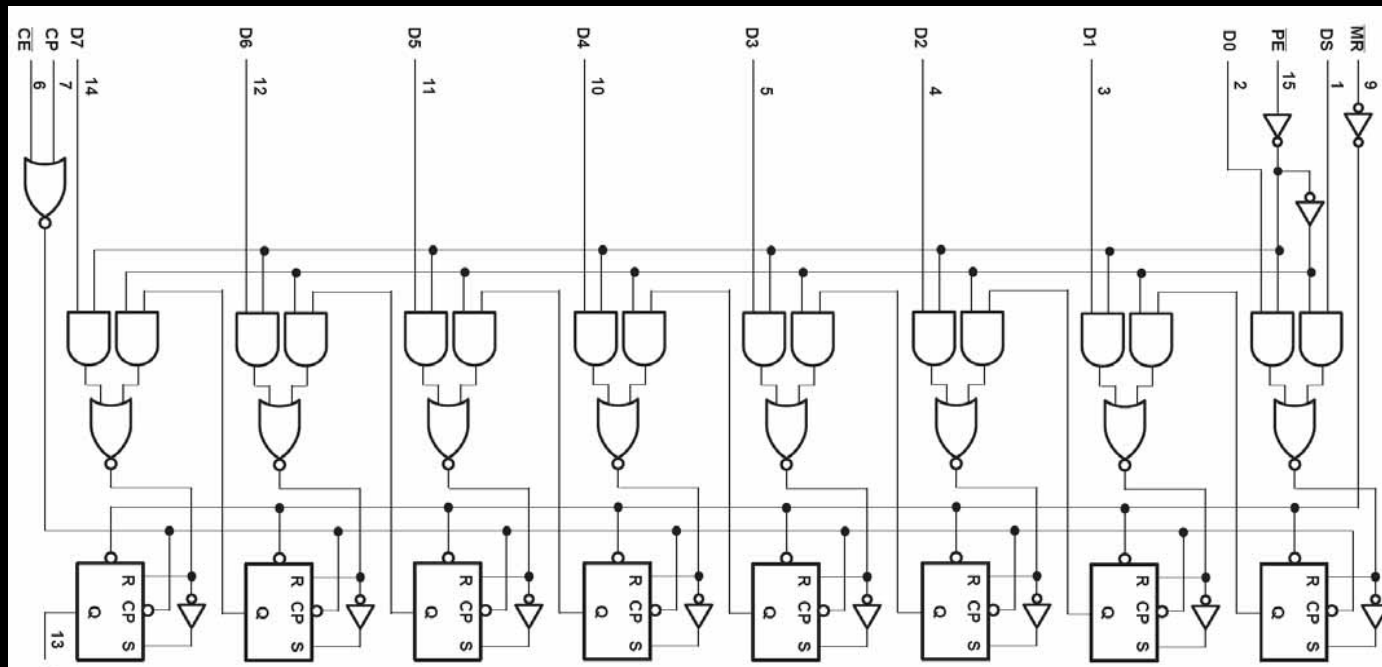


器件介绍

2. 8位并行装载移位寄存器74166



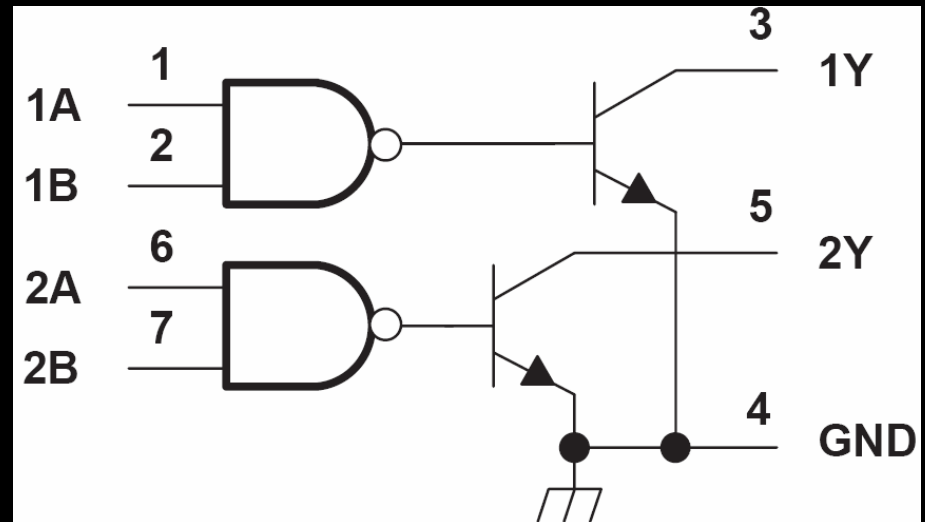
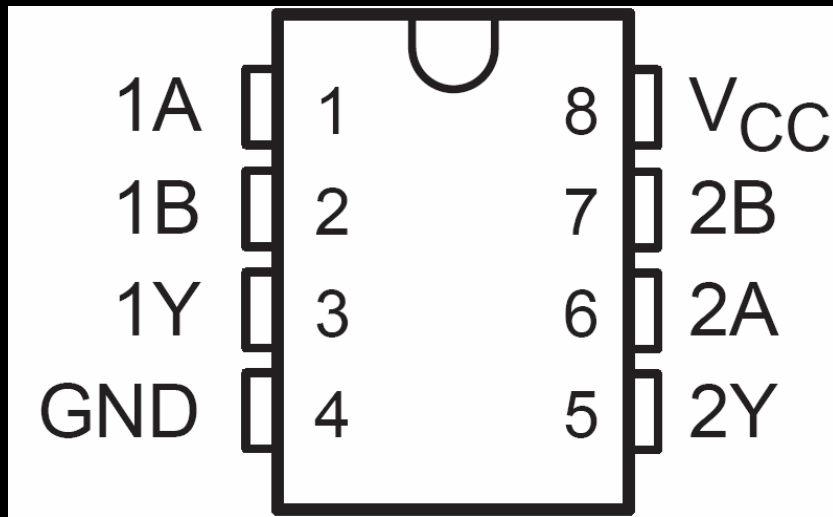
输入					输出			操作模式
PE	CE	CP	DS	D0-D7	Q0	Q1-Q6	Q7	
l	l	↑	X	l-l	L	L-L	L	并行输入
l	l	↑	X	h-h	H	H-H	H	
h	l	↑	l	X-X	L	q0-q5	q6	串行输出
h	l	↑	h	X-X	H	q0-q5	q6	
X	h	X	X	X-X	qn	q1-q6	q7	保持



器件介绍

3. 信号驱动DS75452

DS75452芯片可作为以TTL为逻辑的系统中的信号驱动。常用于高速逻辑缓冲器、功率驱动、继电器驱动、MOS驱动、总线驱动和存储器驱动等，



器件介绍

4. 光电耦合器FOD617

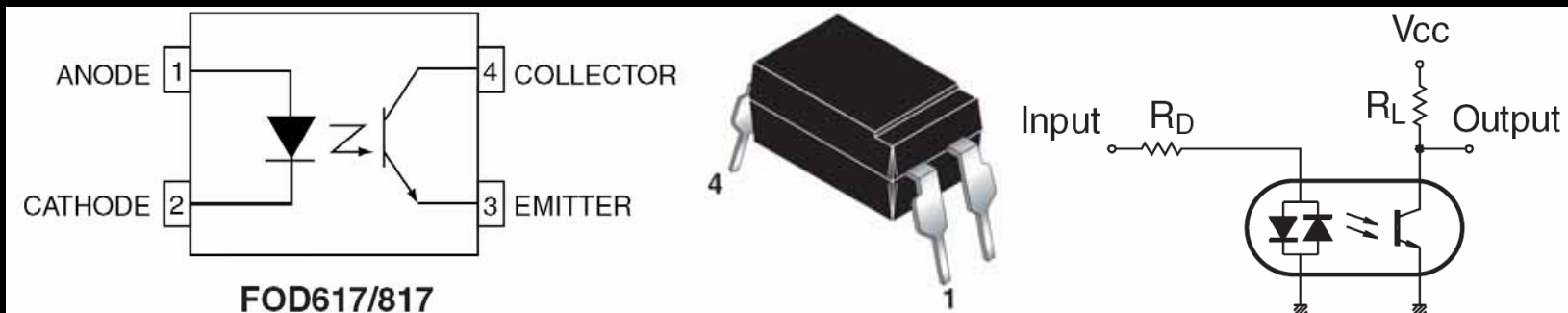
光电耦合器是以光为媒介传输电信号的一种电—光—电转换器件。它由发光源和受光器两部分组成。把发光源和受光器组装在同一密闭的壳体内，彼此间用透明绝缘体隔离。发光源的引脚为输入端，受光器的引脚为输出端。当发光源根据输入信号进行变化时，在受光器端能输出一个相应的信号。

常见的发光源为发光二极管，受光器为光敏二极管、光敏三极管等。

器件介绍

4. 光电耦合器FOD617

光电耦合器的种类较多，常见的有光电二极管型、光电三极管型、光敏电阻型、光控晶闸管型、光电达林顿型和集成电路型等。由于输入的电信号经过了电—光—电的转换，输出的电信号并不是直接的输入信号，而是具有相关性的“新”信号，所以光电耦合器常用于信号的隔离。



光电耦合器FOD617

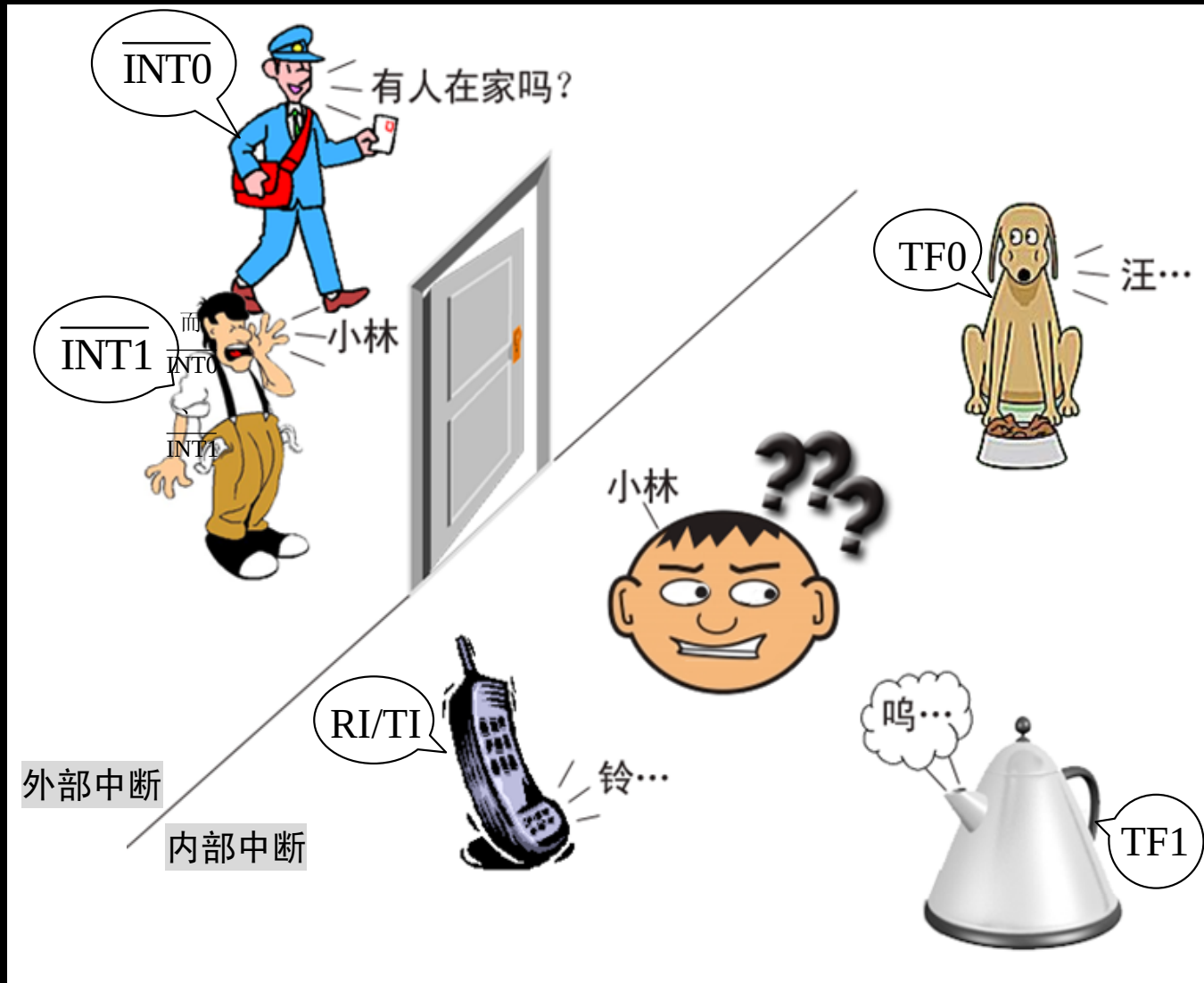
第11章 程序设计的魅力——中断控制

欢迎访问 电路飞翔网

<http://www.circuitfly.com> 获取更多信息

- 11.1 中断实例及中断的控制者
- 11.2 邮递员和朋友——外部中断
- 11.3 小狗和烧水壶——Timer 0和Timer 1中断
- 11.4 电话铃响——串行通信中断
- 11.5 中断的魅力和优先级控制
- 11.6 实例点拨——旋转的时钟

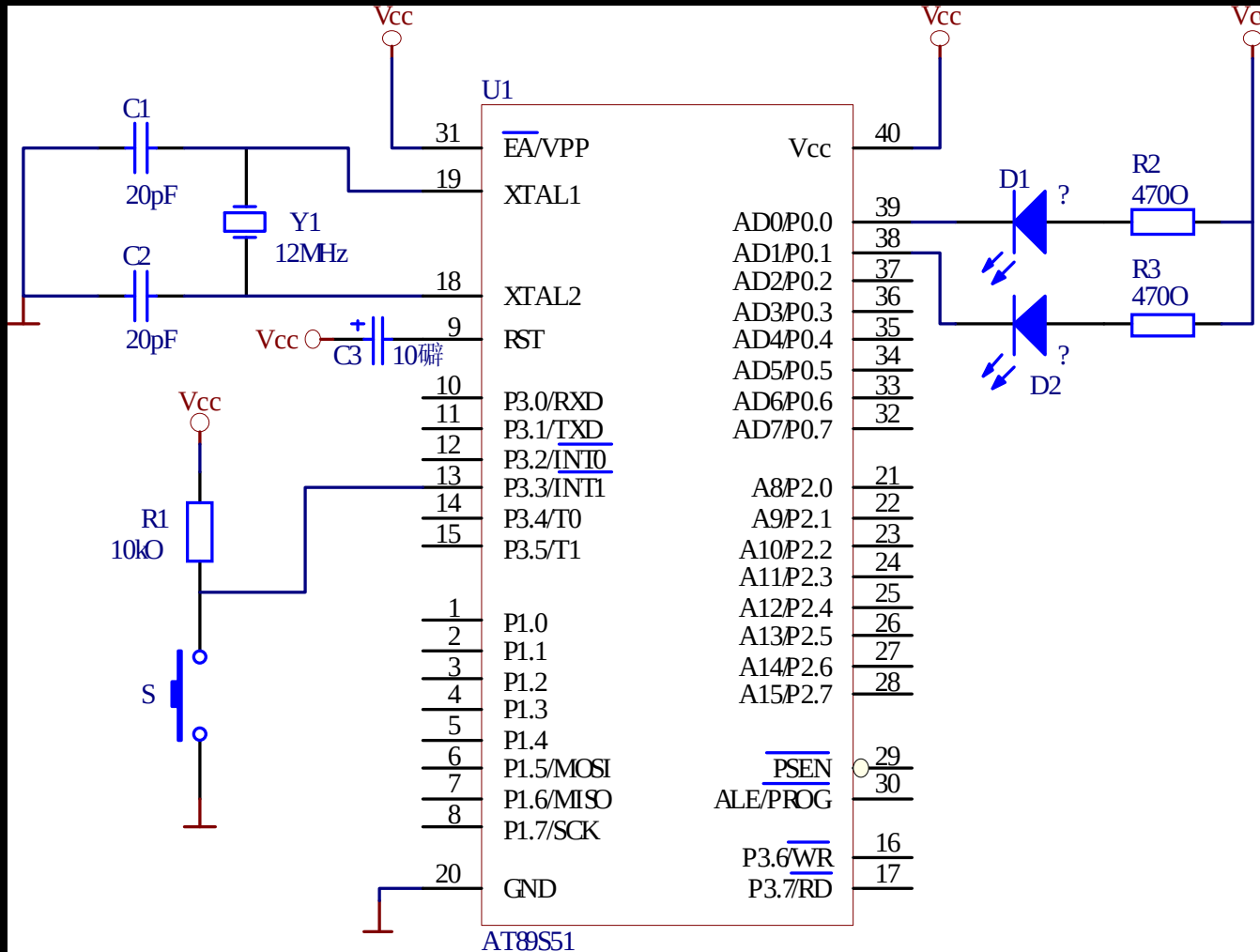
11.1 中断实例及中断的控制者



INT0、INT1、TF0、TF1、RI/TI这5个中断源在单片机中称为：
外部中断0、外部中断1、Timer 0中断、Timer 1中断、串行通信中断。

11.1 中断实例及中断的控制者

• 11.1.1 什么是中断



一个开关S和一个上拉电阻R1组成了一个外部中断信号发生器：当开关S闭合时，P3.3=0，外部中断产生。

11.1 中断实例及中断的控制者

- 11.1.1 什么是中断

例：开关中断产生后，出现绿灯闪一下，红灯闪一下

ORG 00H ; 起始地址00H

; 下面是主程序段，点亮绿色发光二极管D1

MAIN:

MOV IE, #84H ; 使能外部中断1

GREEN:

CLR P0.0 ; 点亮绿色发光二极管D1

JMP GREEN ; 循环

11.1 中断实例及中断的控制者

• 11.1.1 什么是中断(实例软件)

； 中断服务子程序段， 熄灭绿色发光二极管D1， 点亮红色D2

ORG 13H ; 中断服务子程序起始地址为13H

EXT1_RED:

SETB P0.0 ; 熄灭绿色发光二极管D1

CLR P0.1 ; 点亮红色D2

D1: MOV R4, #200 ; 延时程序100ms

D2: MOV R5, #248

DJNZ R5, \$

DJNZ R4, D2

SETB P0.1 ; 熄灭红色发光二极管D2

RETI ; 中断服务子程序结束

END ; 程序结束

11.1 中断实例及中断的控制者

- 11.1.2 中断服务的指挥官——中断向量表

中断向量表（interrupt vector table），向量即“取向”的意思，例如外部中断0发生时，单片机会到程序存储器的0003H中寻找中断服务子程序来执行。

中 断 源	向 量 地 址	中断标志位
外部中断（ $\overline{\text{INT0}}$ 管脚）	0003H	IE0
Timer 0中断	000BH	TF0
外部中断（ $\overline{\text{INT1}}$ 管脚）	0013H	IE1
Timer 1中断	001BH	TF1
串行通信中断	0023H	TI/RI

11.1 中断实例及中断的控制者

• 11.1.3 中断响应与处理过程

- ① 立即暂停正在执行的任何程序，并把下一条将要执行的指令地址压入堆栈中。
- ② 保存所有中断状态（但不在堆栈中）。
- ③ 根据中断向量表找到中断对应的地址。
- ④ 单片机到该地址上开始执行中断服务子程序，直到遇到指令**RETI**为止。
- ⑤ 遇到**RETI**指令后，从堆栈中弹出在中断产生时保存的将要执行的下一条指令的地址到程序计数器**PC**中，单片机开始从**PC**指示的地址继续执行程序。

11.1 中断实例及中断的控制者

• 11.1.3 中断响应与处理过程

问题：

每一个中断服务子程序的存放空间都非常有限，例如，外部中断0的中断向量为0003H，而Timer 0中断向量地址为000BH，可见外部中断0的中断服务子程序只有0003H~000AH这8个字节的空间来存放，这8个字节的实在也放不了几条指令。

如果单片机同时使能了多个中断，并且每一个中断服务子程序都按中断向量表中的地址开始存放，就很有可能出现一个中断服务子程序占据其他中断服务子程序空间的问题。

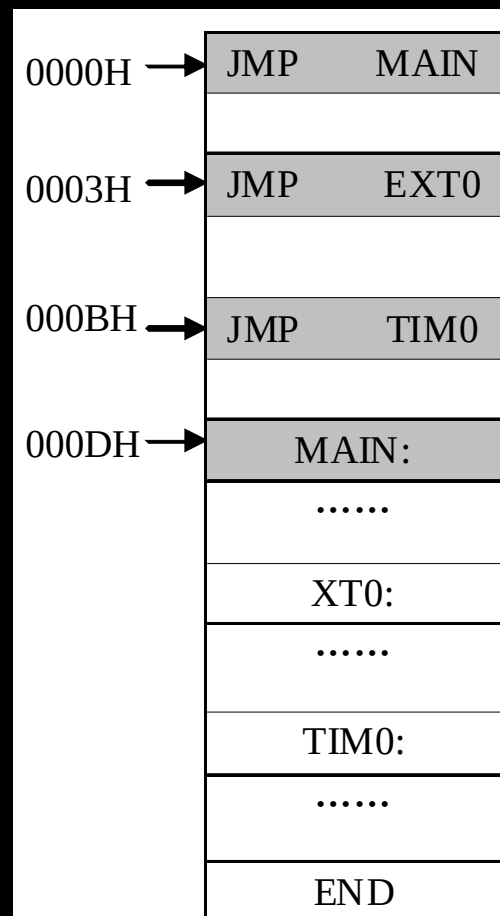
解决办法：JMP 指令

11.1 中断实例及中断的控制者

• 11.1.3 中断响应与处理过程

```

ORG 00H    ; 起始地址00H
JMP MAIN   ; 跳转到MAIN
ORG 03H    ; 外部中断0的起始地址03H
JMP EXT0   ; 跳转到EXT0
ORG 0BH    ; Timer 0中断的起始地址0BH
JMP TIM0   ; 跳转到TIM0
  
```



11.1 中断实例及中断的控制者

• 11.1.4 中断的控制者——中断使能寄存器IE

单片机上电复位之后，默认所有中断都被关闭。为了使单片机能对中断进行响应，需要对中断使能寄存器IE进行操作。IE的初始化过程即是使用MOV指令向其载入控制字。

7	6	5	4	3	2	1	0
EA	保留	保留	ES	ET1	EX1	ET0	EX0

EA 中断使能。

保留 不要访问该位。

ES 串行口中断使能。

ET1 Timer 1中断使能。

EX1 外部中断1使能。

ET0 Timer 0中断使能。

EX0 外部中断0使能。

11.2 邮递员和朋友——外部中断

• 11.2.1 外部中断 $\overline{\text{INT0}}$ 和 $\overline{\text{INT1}}$

(AT89S51单片机的两个外部中断触发端口)

外部中断 0 触发端口	10	P3.0/RXD	AD5/P0.5	33
	11	P3.1/TXD	AD6/P0.6	32
外部中断 1 触发端口	12	P3.2/ <u>INT0</u>	AD7/P0.7	
	13	P3.3/INT1	A8/P2.0	21
	14	P3.4/T0	A9/P2.1	22
	15	P3.5/T1	A10/P2.2	23
			A11/P2.3	24

这两个外部中断对应的中断向量的地址分别为0003H和0013H，分别由IE中的EX0和EX1位来使能/屏蔽。

11.2 邮递员和朋友——外部中断

- 11.2.2 外部中断的电平触发（ level-triggered or level-activated interrupt ）
 - ✓ 在使能外部中断后，如果在 $\overline{INT0}$ 或 $\overline{INT1}$ 口上出现低电平就会触发外部中断0或外部中断1，单片机会立即停止正在执行的程序，执行中断服务子程序，是单片机复位之后外部中断的默认触发方式。
 - ✓ 在单片机进入执行中断服务子程序时，在执行到中断服务子程序结束指令RETI之前，这个低电平必须撤走，否则将会使单片机执行完RETI指令后再次进入中断服务子程序。

11.2 邮递员和朋友——外部中断

- 11.2.2 外部中断的电平触发（ level-triggered or level-activated interrupt ）
 - ✓ 一般为了保证外部中断的产生，低电平的时长至少需要4个机器周期，若晶振式12MHz,则：

1 个机器周期

1 μ s

4 个机器周期

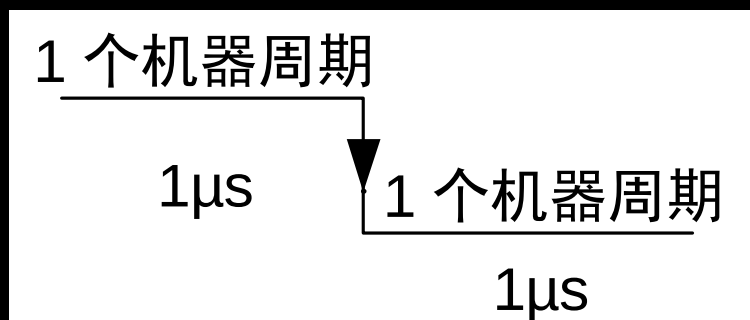
4 $\times$ 1 μ s

至 $\overline{\text{INT0}}$ 或 $\overline{\text{INT1}}$ 端口

低电平触发外部中断的保持时间

11.2 邮递员和朋友——外部中断

- 11.2.3 外部中断的边沿触发（edge-triggered or edge-activated interrupt）
 - ✓ 至少1个机器周期的高电平，且向低电平跳变之后至少保持1个机器周期的低电平，如果晶振频率为12MHz， 则：



11.2 邮递员和朋友——外部中断

- 11.2.3 外部中断的边沿触发（edge-triggered or edge-activated interrupt）

边沿触发方式需要指令来设置寄存器TCON。

7	6	5	4	3	2	1	0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

与定时/计数器有关

IE1外部中断1边沿标志位

IT1外部中断1方式控制位

（IT1清0电平触发/置1边沿）

IE0外部中断0边沿标志位

IT0外部中断0方式控制位

（IT1清0电平触发/置1边沿）

11.2 邮递员和朋友——外部中断

- 11.2.3 外部中断的边沿触发（edge-triggered or edge-activated interrupt）

① 电平触发方式下，若按下开关S不放，端口将不断获得低电平，中断不断产生，中断服务子程序被重复执行；边沿触发方式，只对端口的电平跳变有反应，若也不放开开关S，并不会连续地产生中断。

② 电平触发方式下，若单片机正在执行中断服务子程序时，外部中断再次发生，电平触发信号不会被锁存，所以单片机不会响应第2次中断。而在边沿触发方式下有所不同，单片机正在执行中断服务子程序，如果中断发生，电平跳变产生的中断触发信号保存在TCON寄存器中的IE0或IE1中，单片机再次检测IE0或IE1位，又进入中断服务程序中。

注意：在单片机执行中断服务子程序时，电平触发方式会“漏掉”新的中断，而边沿触发方式会“保留”新的中断。

11.3 小狗和烧水壶

——Timer 0和Timer 1中断

- 问题：前面通过检测Timer溢出标志位TFx来判断计时是否完成。在判断的过程中，单片机除了循环判断之外一般不做任何操作。如果在计时的同时又需要执行别的程序该怎么办呢？
 - 解决办法：当通过中断使能寄存器IE使能Timer中断，设置好Timer的工作参数，启动Timer之后单片机就可以去做别的操作。Timer溢出时，单片机将会自动停下正在执行的任何程序而进入Timer中断服务子程序中执行。
- 两个中断的中断向量地址分别为000BH和001BH。

11.3 小狗和烧水壶

——Timer 0和Timer 1中断

- 例：利用Timer 0中断产生50Hz矩形波

```
ORG      00H      ; 起始地址00H
```

```
JMP      MAIN     ; 跳转到MAIN
```

```
ORG 0BH          ; Timer 0中断服务子程序起始地址为0BH
```

```
JMP      TIM_LOW  ; 跳转到TIM_LOW
```

; 下面是主程序段，初始化

```
MAIN:
```

```
MOV      IE, #82H          ; 使能Timer 0中断
```

```
MOV      TMOD, #01H        ; 设置Timer 0工作在模式1下
```

```
TIMER0_LOAD:
```

```
MOV      TH0, #0B1H        ; 往TH0中载入计数初始值
```

```
MOV      TL0, #0E0H        ; 往TL0中载入计数初始值
```

```
SETB     TR0               ; 启动Timer 0
```

```
SET_HIGH:
```

```
SETB     P1.0              ; P1.0=1
```

```
JMP      SET_HIGH         ; 循环
```

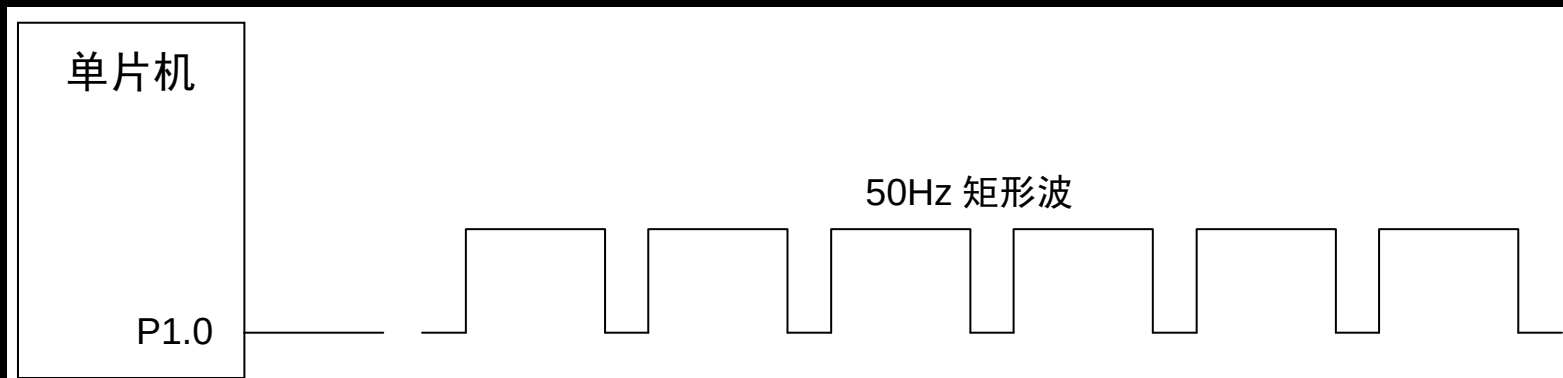
11.3 小狗和烧水壶

——Timer 0和Timer 1中断

; 下面是Timer中断服务子程序

TIM_LOW:

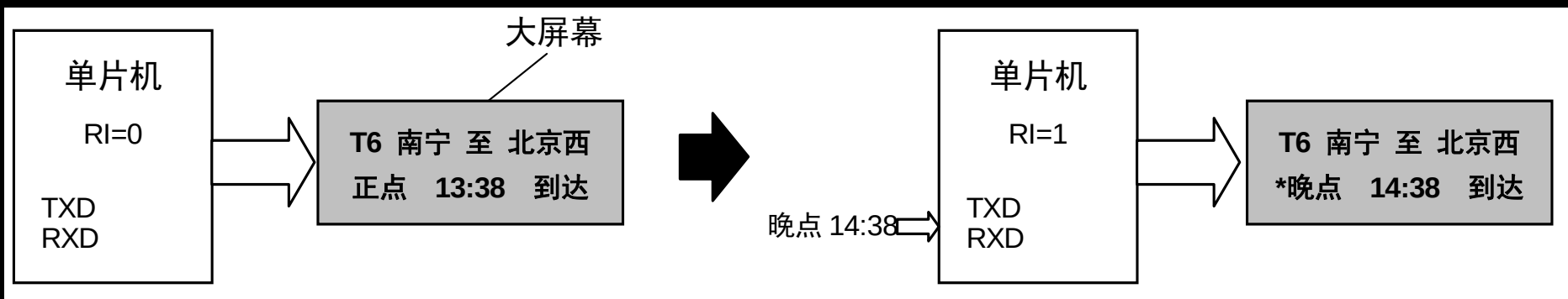
CLR	P1.0	; P1.0=0
MOV	TH0, #0B1H	; 往TH0中载入计数初始值
MOV	TL0, #0E0H	; 往TL0中载入计数初始值
RETI		
END		



11.4 电话铃响——串行通信中断

- ✓ 第十章中串行口发送和接收过程存在一个问题，就是单片机在判断TI和RI位时只能“一心一意”地进行而无法再进行其他操作。
- ✓ 串行通信中断的出现较好地解决了系统中既需要单片机响应串行通信的操作，也同时兼顾其他操作。
- ✓ 串行通信中断的中断向量地址为0023H。

11.4 电话铃响——串行通信中断

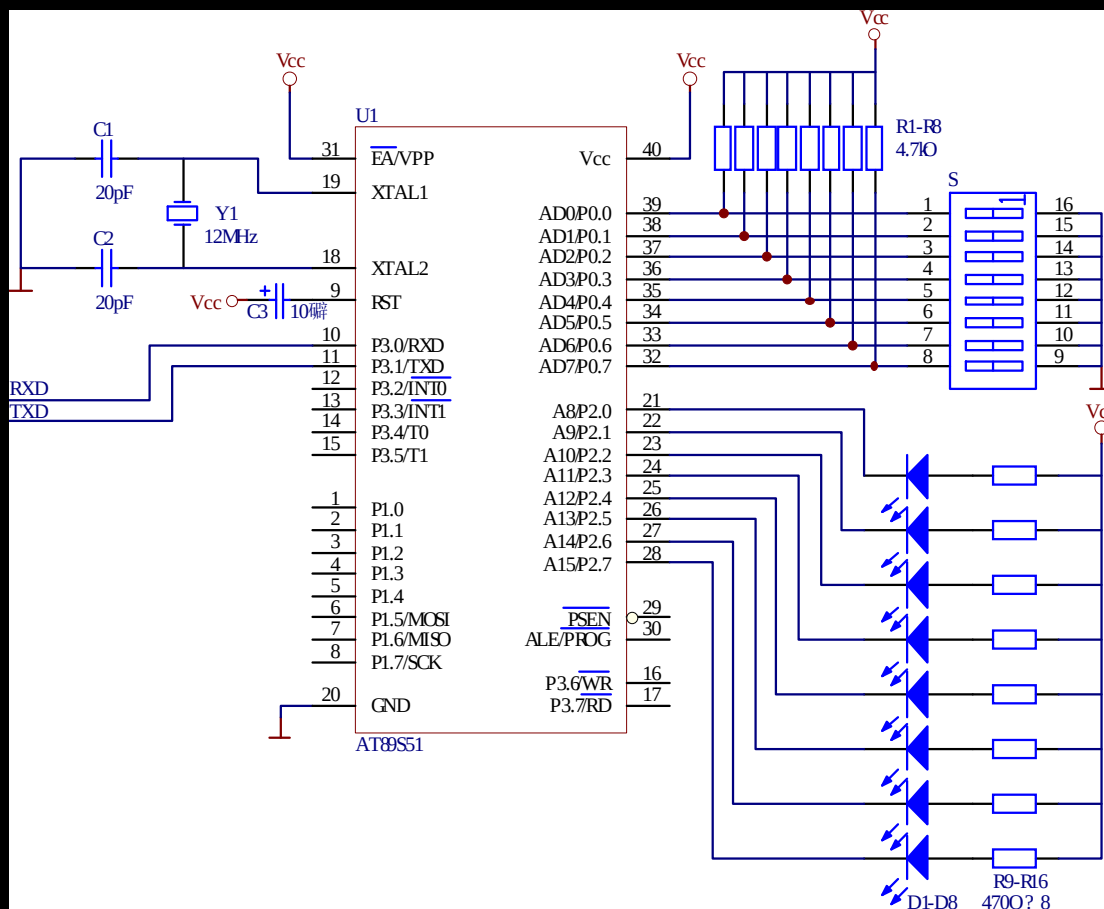


单片机串行通信中断（例）

- ✓ 单片机被设置工作在串行通信中断时，可以照常执行主程序。当接收或发送数据完成时，**RI**或**TI**会被硬件置**1**而产生串行通信中断，单片机则响应中断。
- ✓ 因为**RI**或**TI**的置**1**都会使单片机中断，程序要知道是发送数据完成的**TI=1**产生的中断或是串行口接收到数据的**RI=1**产生的就需要在中断服务子程序中先对**TI**和**RI**进行判断。

11.4 电话铃响——串行通信中断

- 例：将P0口上DIP开关产生的8位状态信号从串行口TXD发送出去，并由P2口上发光二极管显示。



串行通信中断的演示电路图

11.4 电话铃响——串行通信中断

- 例：串行通信中断的演示程序

```
ORG    00H    ; 起始地址00H
JMP                      MAIN ; 跳转到MAIN
ORG    23H    ; 串行中断服务子程序起始地址为23H
JMP                      SERIAL ; 跳转到TIM_LOW
; 下面是主程序段，初始化并发送数据
```

11.4 电话铃响——串行通信中断

- 例：串行通信中断的演示程序

MAIN:

```
MOV    IE, #90H           ; 使能串行中断
MOV    SCON, #50H         ; 设置串行口工作在模式1下
MOV    TMOD, #20H         ; 使用Timer 1工作在模式2下
MOV    TH1, #0F3H; 波特率2400（晶振频率=12MHz）
MOV    TL1, #0F3H
SETB   TR1                ; 启动Timer 1
```

SEND_DATA:

```
MOV    A, P0              ; 将P0口状态数据载入ACC
MOV    SBUF, A            ; 载入缓冲区以发送
MOV    P2, A              ; 在P2口上的发光二极管显示
JMP    SEND_DATA          ; 循环检测P2口状态并发送
```

11.4 电话铃响——串行通信中断

- 例：串行通信中断的演示程序

SERIAL: ; 这里是串行中断服务子程序

JB TI, SENT ; 如果TI=1, 说明是发送完成产生的中断,
; 跳到 SENT段将TI标志清0后返回主程序

MOV A, SBUF ; 如果TI=0, 说明是接收到数据产生的中断,
; 于是将缓冲区中的数据载入ACC中

MOV P2, A ; 在P2口上的发光二极管显示

CLR RI ; 清标志位RI

RETI ; 返回主程序

SENT:

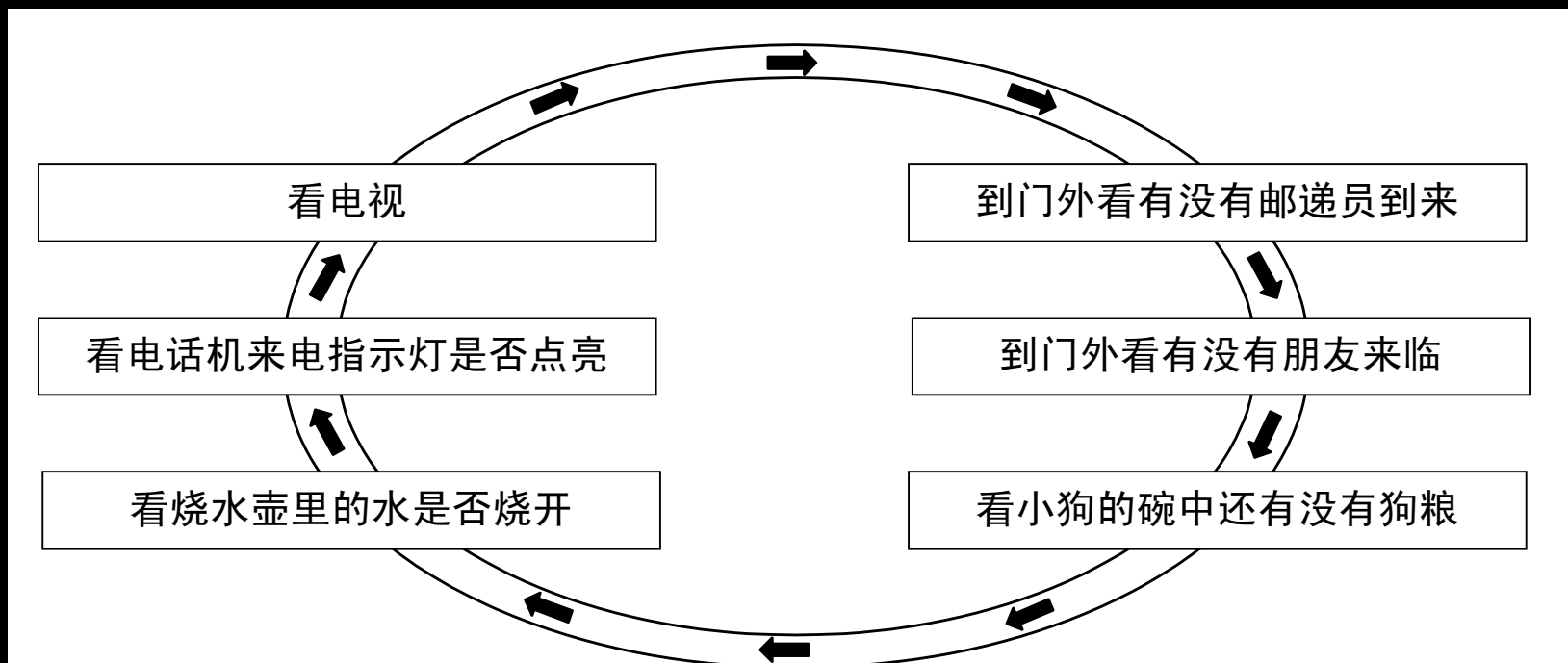
CLR TI ; 发送完成, 清标志位TI

RETI ; 返回主程序

END ; 程序结束

11.5 中断的魅力和优先级控制

• 11.5.1 中断的魅力——轮询与中断的比较



小林的循环关注

单片机的程序资源都耗费在访问设备的状态上，如果设备响应率比较低，单片机的程序执行效率将很低。

11.5 中断的魅力和优先级控制

- 11.5.1 中断的魅力——轮询与中断的比较
 - ✓ 在中断下，单片机同样能服务多个设备，等待某个设备产生中断时再去处理。在设备没有产生中断时单片机还可以专心地执行其他程序。
 - ✓ 中断的好处：
 - ① 单片机可以选择特定的中断进行响应和处理。可通过中断使能寄存器IE相应的位来使能系统希望响应和处理的的中断，而屏蔽那些不需要响应和处理的中断。轮询模式下，单片机必须响应和处理每一个设备。
 - ② 如果有多个中断同时发生，单片机可以根据程序的设定来优先响应和处理某一中断。而轮询模式下，单片机逐一对设备进行访问，没有优先顺序之分。

11.5 中断的魅力和优先级控制

• 11.5.2 中断的优先级控制

单片机能根据设置（中断优先级控制寄存器**IP**）来优先响应和处理某一中断，**IP**寄存器位于特殊功能寄存器区的**B8H**上。

7	6	5	4	3	2	1	0
保留	保留	保留	PS	PT1	PX1	PT0	PX0

置1则优先

保留 不要访问保留位，也不要往保留位写数据。

PS 串行口中断优先。 **PT1** **Timer 1**中断优先。

PX1 外部中断**1**优先。 **PT0** **Timer 0**中断优先。

PX0 外部中断**0**优先。

11.5 中断的魅力和优先级控制

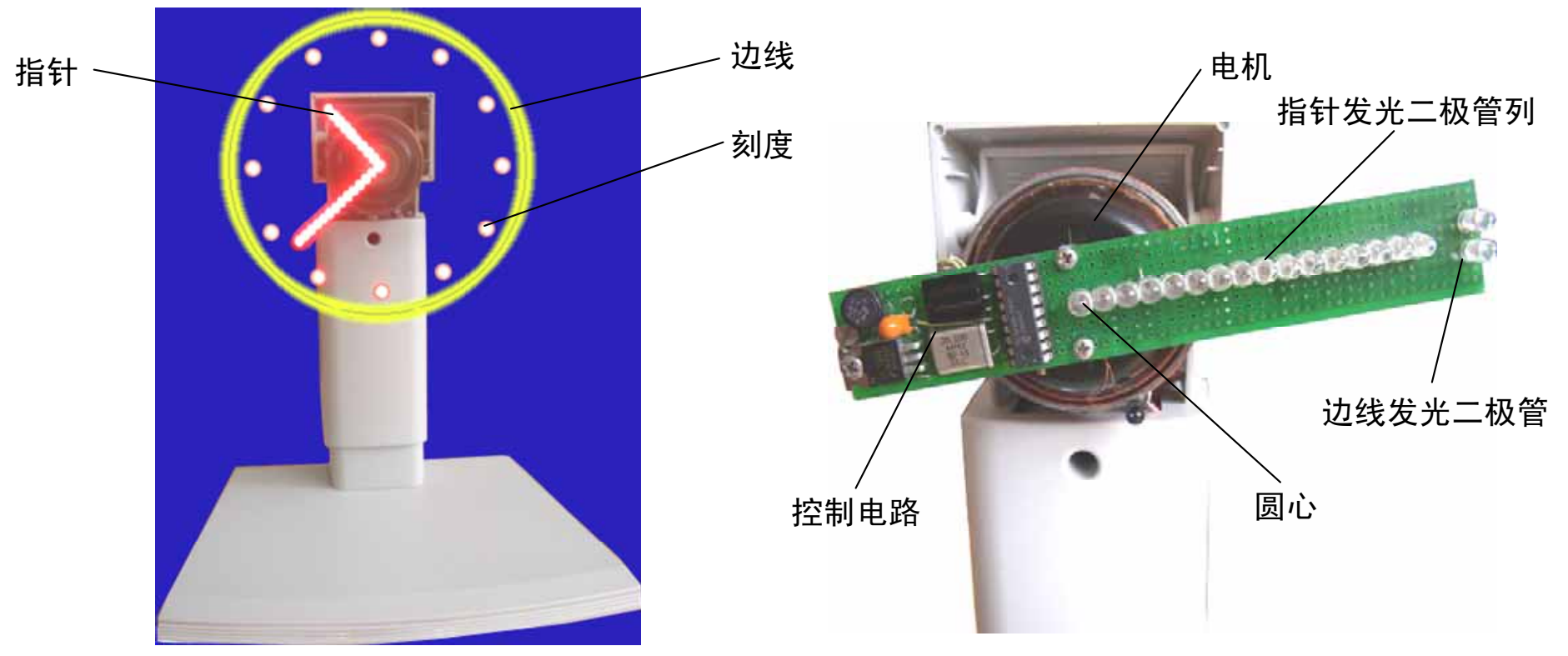
• 11.5.2 中断的优先级控制（默认的中断优先级）

优 先 级	中 断 源
最高  最低	外部中断 0 $\overline{\text{INT0}}$
	Timer 0 中断 TF0
	外部中断 1 $\overline{\text{INT1}}$
	Timer 1 中断 TF1
	串行通信中断 RI/TI

✓如果想调整表所示的默认中断优先级，可以将中断优先级控制寄存器IP中相应的位置1即可。

✓如果程序中把两个或两个以上的中断优先级位都置1，单片机又会根据表中描述的中断优先级来服务。

11.6 实例点拨——旋转的时钟



(a) 旋转的时钟

(b) 时钟只有一列发光二极管

11.6 实例点拨——旋转的时钟

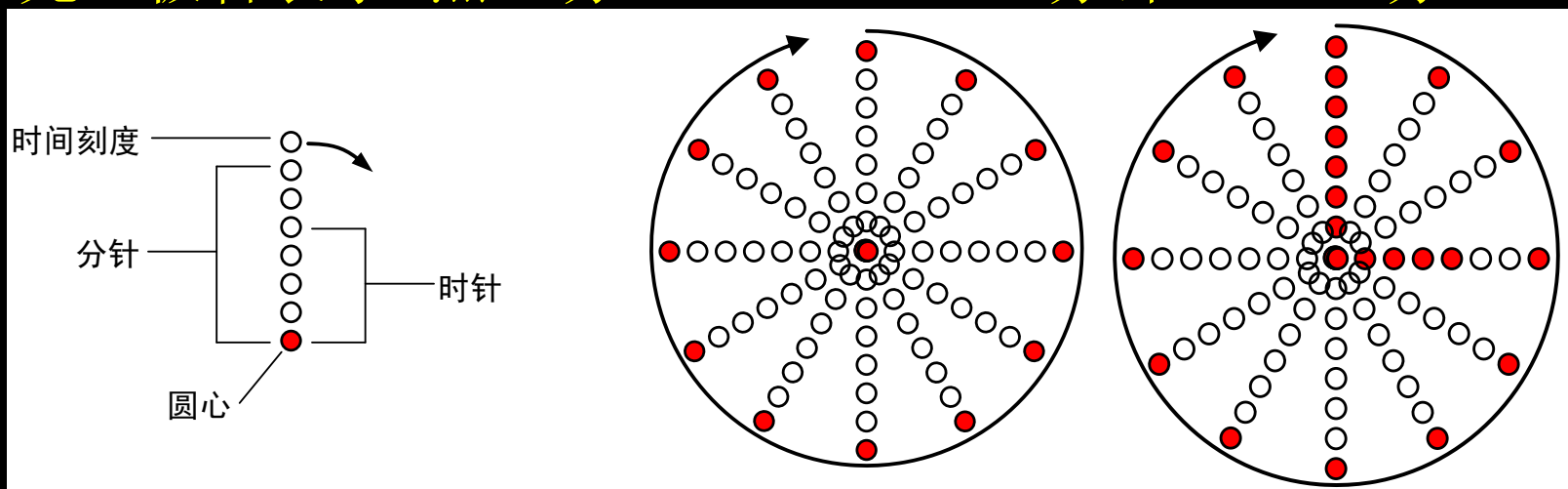
• 11.6.1 原理分析

- ✓ 人眼具有视觉暂留（**visual persistence**）的错觉，即无法区分间隔小于**0.1**秒的图像。
- ✓ 有**8**支发光二极管排成一行，最内侧的发光二极管为圆心，它们绕着圆心旋转，最外侧的发光二极管显示时间刻度，当时针在**12**点时我们假设角度为 **0°** ，则每个小时时针之间的角度为 **$30(360/12)$** 。于是当这一列发光二极管每旋转 **30°** ，最外侧的发光二极管就点亮一个瞬间以呈现出时间刻度。这样，如果在**0.1**秒之内这列发光二极管能旋转完一圈，则人眼就会产生错觉，而把先后产生的时间刻度连成**12**个完整的时间刻度。

11.6 实例点拨——旋转的时钟

• 11.6.1 原理分析

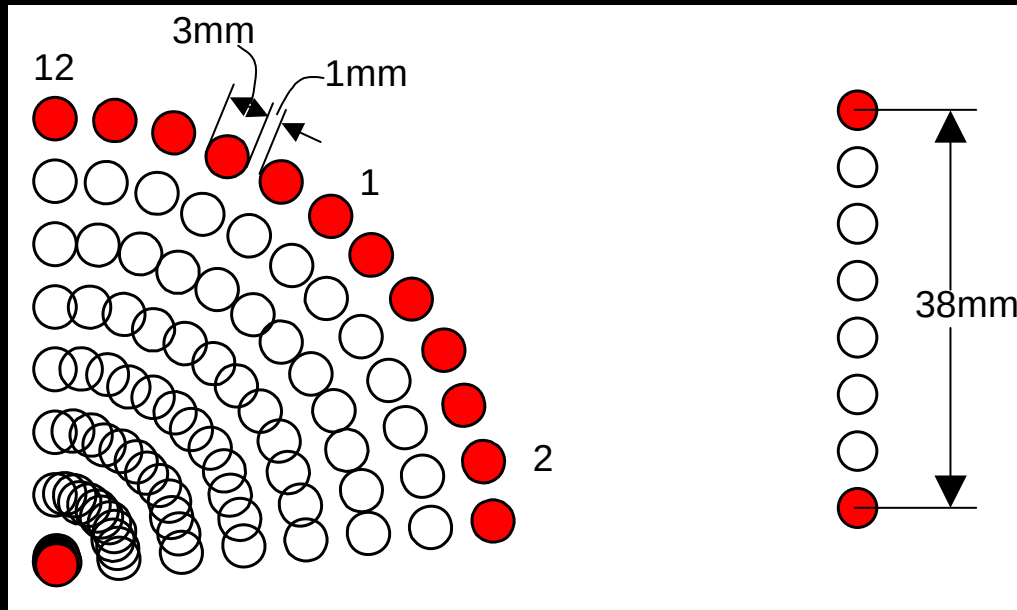
- ✓ 分针较时针长，假设分针用7支发光二极管来显示，时针用5支发光二极管来显示。要显示3点整，在时间刻度的显示基础上，控制在 0° 时点亮7支发光二极管以显示分针指在12点位置上；然后当发光二极管转过 90° 时再显示5支以显示时针指在3点位置上，由于发光二极管在不断地旋转，在0.1秒内重复点亮12点位置上的7支和3点位置上的5支发光二极管，直到1分钟以后在 6° 的位置上点亮7支发光二极管表示3点01分（ $360^\circ \div 60 \text{ (分钟)} = 6^\circ / \text{分}$ ）



11.6 实例点拨——旋转的时钟

• 11.6.1 原理分析（具体的设计过程）

- ✓ 计算旋转时钟的周长和半径。选用直径为3mm的发光二极管组成一列，这样最外侧的发光二极管旋转一周经过的显示位置应该有60个。为了在显示时每分钟之间有一个间隔，我们在每分钟之间引入一个1mm空隙，于是得到旋转的周长 $C=60 \times 3\text{mm} + 59 \times 1\text{mm} = 239\text{mm}$ ，可得半径 $R = C / 2\pi = 38\text{mm}$ ，将8支发光二极管按38mm平均间距排成一列。



11.6 实例点拨——旋转的时钟

- 11.6.1 原理分析（具体的设计过程）
 - ✓ 进行时间的计算。根据视觉暂留的原理，该列发光二极管在同一位置出现的时间间隔应该不大于0.1秒，即旋转周期 $T \leq 100\text{ms}$ 。为了计算方便，可取旋转周期 $T=60\text{ms}$ 。这样每分钟刻度上发光二极管的点亮和熄灭的时间 $t=60\text{ms}/60=1\text{ms}$ 。在这1ms中，可设计发光二极管点亮 0.7ms （ $700\mu\text{s}$ ），熄灭 0.3ms （ $300\mu\text{s}$ ）

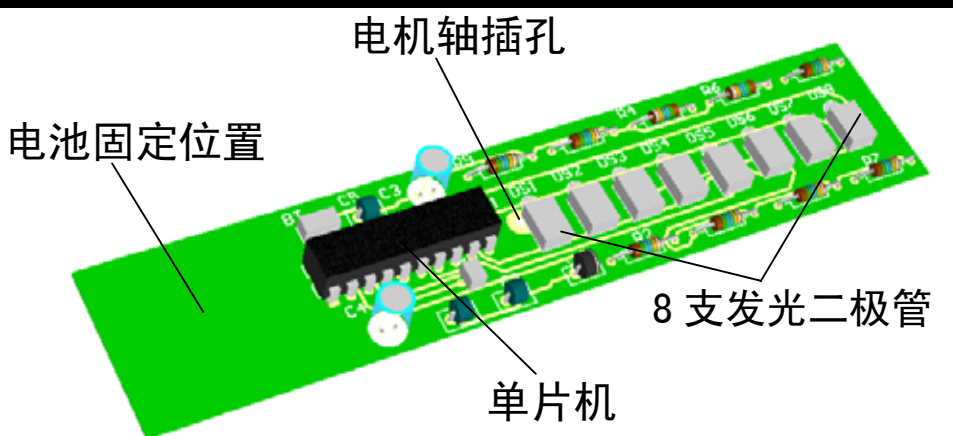
11.6 实例点拨——旋转的时钟

- 11.6.2 明确系统功能

电机带动一系列发光二极管绕轴旋转，单片机控制发光二极管在旋转过程中的相应位置上点亮以指示时间和刻度。与此同时，单片机内部进行时钟的计时操作，以控制旋转时钟显示正确的走时。

11.6 实例点拨——旋转的时钟

• 11.6.3 硬件设计

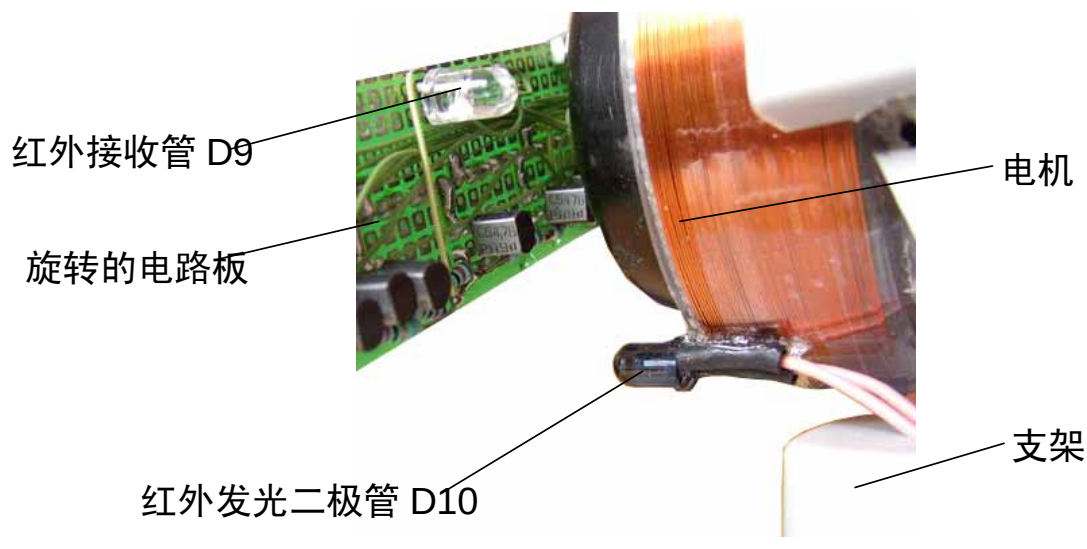


▷ 直流电机



△ 旋转时钟的电路板

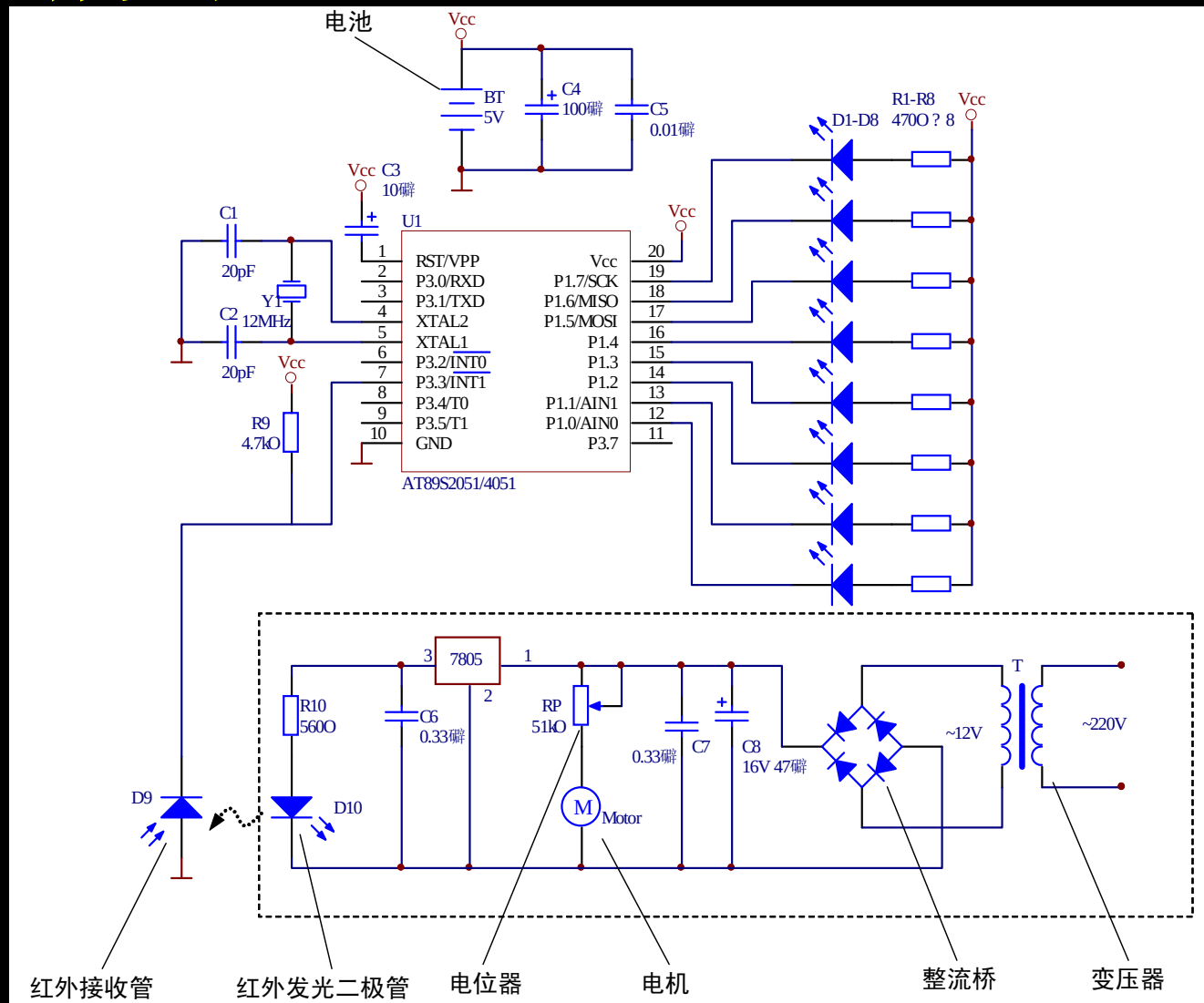
▷ 红外对管用于同步



11.6 实例点拨——旋转的时钟

• 11.6.3 硬件设计

旋转时钟系统电路图



11.6 实例点拨——旋转的时钟

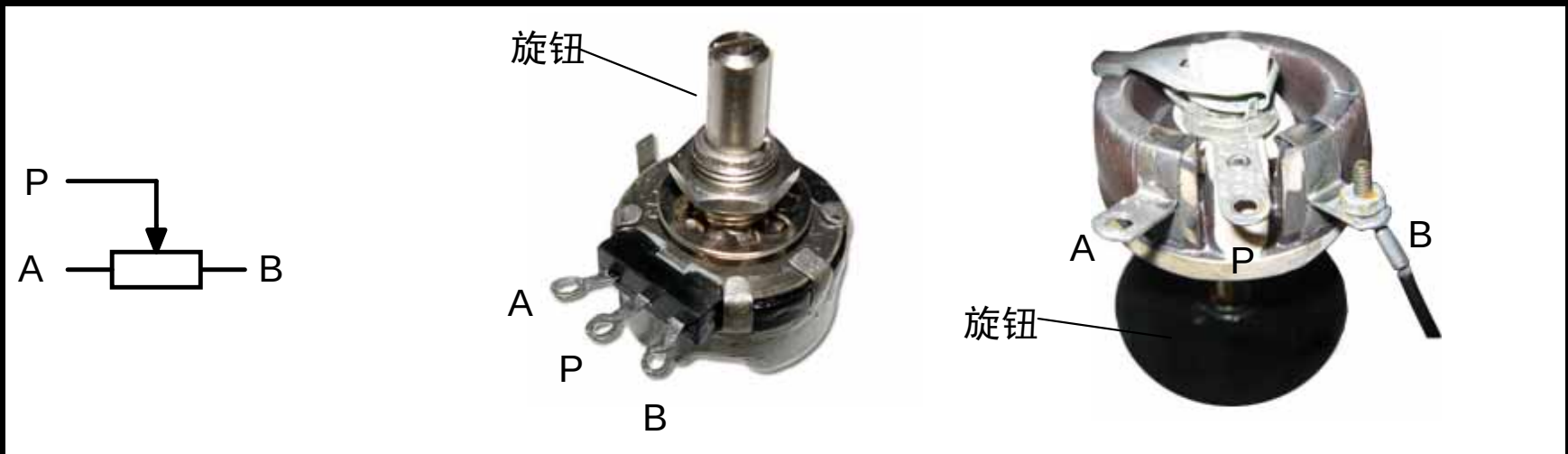
• 11.6.4 软件设计

- ✓ 使能单片机的两个中断——Timer 0中断和外部中断1，并设Timer 0的优先级较高。
- ✓ 启动Timer 0后，每50ms产生一次中断。
- ✓ 在Timer 0中断服务子程序TIM0中，R0计数器判断每50ms产生一次的中断是否到达20次，如果是计时1秒。R1是秒的计数器，如果R1计到60表明到达1分钟，则30H中的分钟值加1。
- ✓ 在计时的同时，外部中断1使能。每当红外对管对齐时，即每旋转一圈时中断服务子程序EXT1被执行。EXT1中将旋转一圈分成60份。根据60个不同的位置来判断是否显示时间刻度、分针和时针。
- ✓ (程序见书本)

器件介绍

1. 电位器:

A、B两端的阻值固定，等于电位器的标称值。中间引脚P与两端引脚之间的电阻值随着电位器的旋钮（**knob**）调节而改变。电位器外壳一般都会标注最大阻值和功率。大功率场合一般都使用绕线电位器。

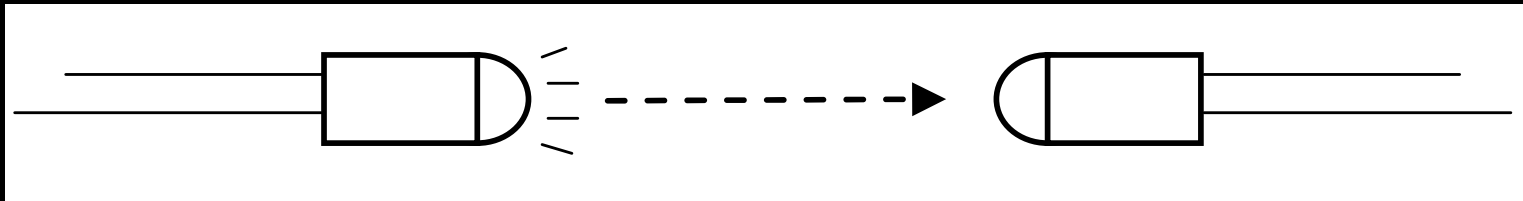


(a) 电路符号 (b) 一般电位器外观 (c) 绕线式电位器

器件介绍

2. 红外对管

红外对管（IR transmitter and receiver）是一种常用的光电器件。所谓对管就是由一支红外线发射管和红外线接收管组成。当发射管与接收管的发射与接收窗靠近对齐时，则接收管导通，一般来说，红外对管的有效距离为数米，如果想扩大感应距离可加装透镜。



红外对管的使用